In [1]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

NumPy Cheat Sheet — Most Used Functions

Creating arrays

Function	Purpose
np.array([...])	Create array
np.arange(start, stop, step)	Range array
np.linspace(start, stop, num)	Evenly spaced numbers
np.zeros((rows, cols))	Zero matrix
np.ones((rows, cols))	One matrix
np.eye(n)	Identity matrix

Random numbers

Function	Purpose
np.random.rand(n)	Uniform random (0–1)
np.random.randn(n)	Normal distribution
np.random.randint(a, b, size)	Random integers
np.random.normal(loc, scale, size)	Normal distribution

Array operations

Function	Purpose
np.sum()	Sum
np.mean()	Mean
np.median()	Median
np.std()	Standard deviation
np.var()	Variance
np.min(), np.max()	Min/Max
np.sort()	Sort array
np.unique()	Unique values
np.argmax(), np.argmin()	Index of max/min

Linear algebra

Function	Purpose
np.dot(A, B)	Matrix multiplication
np.matmul(A, B)	Same as dot
np.linalg.inv(A)	Inverse
np.linalg.det(A)	Determinant
np.linalg.eig(A)	Eigenvalues & eigenvectors

Reshaping & stacking

Function	Purpose
arr.reshape(r, c)	Reshape array
np.hstack([a, b])	Horizontal stack
np.vstack([a, b])	Vertical stack
np.concatenate([a, b])	Append arrays

Logical & conditional

Function	Purpose
np.where(cond)	Filter values
np.any(), np.all()	Check conditions
arr > 5	Boolean array

Math functions

Function	Purpose
np.sqrt()	Square root
np.exp()	Exponential
np.log()	Natural log
np.power(a, b)	a^b

Pandas vs NumPy — When to Use What

Task	Use Pandas	Use NumPy
DataFrame (rows/columns)	✔	✖
Labeled data	✔	✖
Missing values	✔	✖
Fast math operations	✔	✔✔✔
Big matrix calculations	✖	✔
EDA & cleaning	✔	✖

Quick Memory Trick

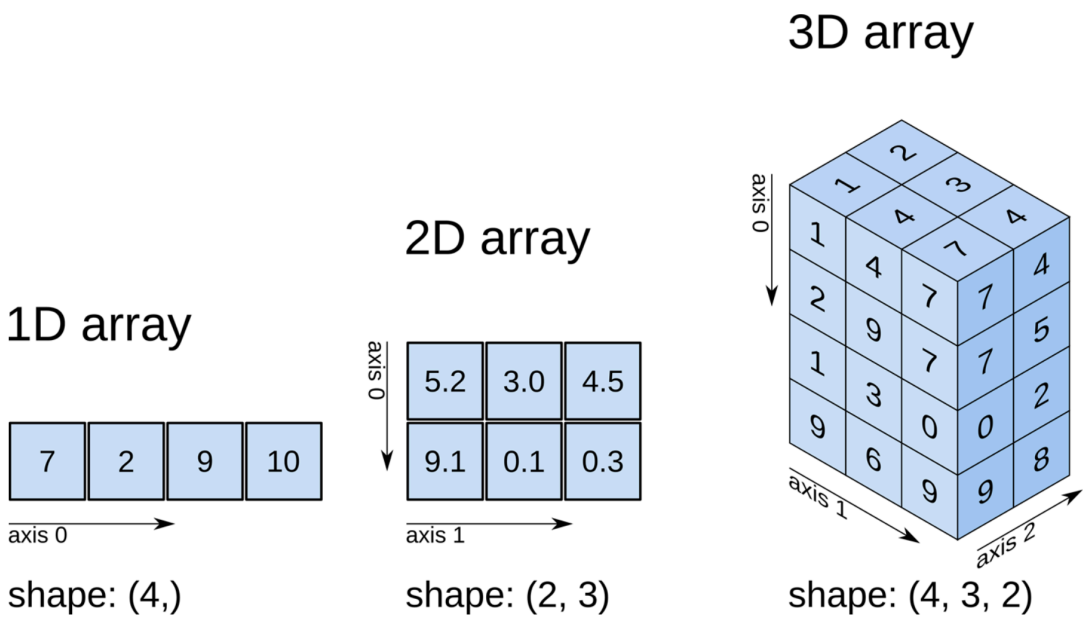
Pandas = tables with labels
NumPy = fast number crunching

In [368…

```
import os
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

In [369…
```
import numpy as np
```

In [370…
```
import pandas as pd
```



In [371…
```
a1 = np.array([4,5,6,7,8])
```

In [372…
```
type(a1)
```

Out[372…
```
numpy.ndarray
```

In [373…
```
a2 = np.array([3,4,3,2])
```

In [374…
```
a2
```

Out[374…
```
array([3, 4, 3, 2])
```

In [375…
```
a = np.concatenate((a1,a2))
```

In [376…
```
a
```

Out[376…
```
array([4, 5, 6, 7, 8, 3, 4, 3, 2])
```

In [377…
```
a1 = np.insert(a1,3,10)
```

In [378…
```
a1
```

Out[378…
```
array([ 4,  5,  6, 10,  7,  8])
```

In [379…
```
a1 = np.insert(a1,6,np.array([7,6,9]))
```

In [380…
```
a1
```

Out[380…
```
array([ 4,  5,  6, 10,  7,  8,  7,  6,  9])
```

In [381…
```
a1 = np.delete(a1,6)
```

In [382…
```
a1
```

Out[382…
```
array([ 4,  5,  6, 10,  7,  8,  6,  9])
```

In [383…
```
my_list = list(range(1,6))
```

In [384…
```
my_list
```

Out[384…
```
[1, 2, 3, 4, 5]
```

In [385…
```
a1 = np.arange(1,11)
```

In [386…
```
a1
```

Out[386…
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

In [387…
```
a1 = np.arange(1,26,3)
```

In [388…
```
a1
```

Out[388…
```
array([ 1,  4,  7, 10, 13, 16, 19, 22, 25])
```

In [389…
```
a1 = np.arange(40,7,-3)
```

In [390…
```
a1
```

Out[390…
```
array([40, 37, 34, 31, 28, 25, 22, 19, 16, 13, 10])
```

In [391…
```
x2 = list(a1)
```

In [392…
```
x2
```

Out[392…
```
[np.int64(40),
 np.int64(37),
 np.int64(34),
 np.int64(31),
 np.int64(28),
 np.int64(25),
 np.int64(22),
 np.int64(19),
 np.int64(16),
 np.int64(13),
 np.int64(10)]
```

In [393…
```
# list is just single dimensional
# while array could be multi-dimensional
```

In [394…
```
i = 34
a = i + x2[1]
print(a)
```

```
71
```

In [395…
```
a1 = np.array([3,6,5,8,6,4,3,2,8,7,8,9])
a1
```

Out[395…
```
array([3, 6, 5, 8, 6, 4, 3, 2, 8, 7, 8, 9])
```

In [396…
```
a2 = a1.reshape(3,4)    # two dimensional
```

In [397…
```
a2
```

Out[397…
```
array([[3, 6, 5, 8],
       [6, 4, 3, 2],
       [8, 7, 8, 9]])
```

In [398…
```
# array([[         two square brackets are means two dimensional array

#]])
```

In [399…
```
a3 = a1.reshape(3,2,2) # three dimensional arra    (read as 2*2 stacked on top
```

In [400…
```
a3
```

Out[400…
```
array([[[3, 6],
        [5, 8]],

       [[6, 4],
        [3, 2]],

       [[8, 7],
        [8, 9]]])
```

In [401…
```
k1 = [3,5,7,9]
k2 = [1,3,2,5]
k3 = [7,5,9,2]

k4 = np.array([k1,k2,k3])
```

In [402…
```
k4
```

Out[402…
```
array([[3, 5, 7, 9],
       [1, 3, 2, 5],
       [7, 5, 9, 2]])
```

In [403…
```
k4[1]
```

Out[403… array([1, 3, 2, 5])

In [404…
```
k4[[1,2]]
```

Out[404…
```
array([[1, 3, 2, 5],
       [7, 5, 9, 2]])
```

In [405…
```
k4 [:,[1,3]]
```

Out[405…
```
array([[5, 9],
       [3, 5],
       [5, 2]])
```

In [406…
```
b = k4[:,1]
```

In [407…
```
b
```

Out[407… array([5, 3, 5])

In [408…
```
b1 = b.reshape(-1,1)
b1
```

Out[408…
```
array([[5],
       [3],
       [5]])
```

In [409…
```
k4.flatten(order='c')
```

Out[409… array([3, 5, 7, 9, 1, 3, 2, 5, 7, 5, 9, 2])

In [410…
```
k4.flatten(order='f')
```

Out[410… array([3, 1, 7, 5, 3, 5, 7, 2, 9, 9, 5, 2])

In [411…
```
k4.transpose()
```

Out[411…
```
array([[3, 1, 7],
       [5, 3, 5],
       [7, 2, 9],
       [9, 5, 2]])
```

In [412…
```
newrow = np.array([1,5,7,6])
```

In [413…
```
a1 = np.vstack([k4, newrow])
```

In [414…
```
a1
```

Out[414…
```
array([[3, 5, 7, 9],
       [1, 3, 2, 5],
       [7, 5, 9, 2],
       [1, 5, 7, 6]])
```

In [415…
```
newcol = np.array([14,12,19])
```

In [416…
```
newcol
```

Out[416…
```
array([14, 12, 19])
```

In [417…
```
newcol = newcol.reshape(-1,1)
```

In [418…
```
newcol
```

Out[418…
```
array([[14],
       [12],
       [19]])
```

In [419…
```
a1 = np.hstack([k4, newcol])
```

In [420…
```
a1
```

Out[420…
```
array([[ 3,  5,  7,  9, 14],
       [ 1,  3,  2,  5, 12],
       [ 7,  5,  9,  2, 19]])
```

In [421…
```
a1 = a1.transpose()
```

In [422…
```
a1
```

Out[422…
```
array([[ 3,  1,  7],
       [ 5,  3,  5],
       [ 7,  2,  9],
       [ 9,  5,  2],
       [14, 12, 19]])
```

In [423…
```
a1 = np.insert(a1,2,[3,5,7], axis = 0)
a1
```

Out[423…
```
array([[ 3,  1,  7],
       [ 5,  3,  5],
       [ 3,  5,  7],
       [ 7,  2,  9],
       [ 9,  5,  2],
       [14, 12, 19]])
```

In [424…
```
# Insert a column with one value per row (a1 has 6 rows). The original list ha
a1 = np.insert(a1, 1, [3, 5, 7, 1, 3, 4], axis=1)
a1
```

Out[424…
```
array([[ 3,  3,  1,  7],
       [ 5,  5,  3,  5],
       [ 3,  7,  5,  7],
       [ 7,  1,  2,  9],
       [ 9,  3,  5,  2],
       [14,  4, 12, 19]])
```

In [425…
```
np.delete(a1, (0,2), axis = 0)
```

Out[425…
```
array([[ 5,  5,  3,  5],
       [ 7,  1,  2,  9],
       [ 9,  3,  5,  2],
       [14,  4, 12, 19]])
```

In [426…
```
np.delete(a1,(0,2), axis = 1)
```

Out[426…
```
array([[ 3,  7],
       [ 5,  5],
       [ 7,  7],
       [ 1,  9],
       [ 3,  2],
       [ 4, 19]])
```

In [427…
```
# broadcasting in array
```

In [428…
```
a1 = a1+2      # add 2 in every element in array
```

In [429…
```
a1
```

Out[429…
```
array([[ 5,  5,  3,  9],
       [ 7,  7,  5,  7],
       [ 5,  9,  7,  9],
       [ 9,  3,  4, 11],
       [11,  5,  7,  4],
       [16,  6, 14, 21]])
```

In [430…
```
a2 =  np.array([1,2,3,4,5,6,7])
```

In [431…
```
# column wise addition – ensure a2 length matches a1 rows
a1 + a2[:a1.shape[0]].reshape(-1,1)
```

Out[431…
```
array([[ 6,  6,  4, 10],
       [ 9,  9,  7,  9],
       [ 8, 12, 10, 12],
       [13,  7,  8, 15],
       [16, 10, 12,  9],
       [22, 12, 20, 27]])
```

In [432…
```
a2 = a2.reshape(-1,1)
```

In [433…
```
a2
```

Out[433…
```
array([[1],
       [2],
       [3],
       [4],
       [5],
       [6],
       [7]])
```

In [434…
```
b1 = np.array([1,2,3,4,5,6])
```

In [435…
```
b1 * 2
```

Out[435…
```
array([ 2,  4,  6,  8, 10, 12])
```

In [436…
```
d1 = np.array([1,2,4,5,61])
d0 = np.repeat(d1,[1,2,3,4,5])
```

In [437…
```
d0
```

Out[437…
```
array([ 1,  2,  2,  4,  4,  4,  5,  5,  5,  5, 61, 61, 61, 61, 61])
```

In [438…
```
np.sort(d0)
```

Out[438…
```
array([ 1,  2,  2,  4,  4,  4,  5,  5,  5,  5, 61, 61, 61, 61, 61])
```

In [439…
```
np.sort(b1)[::-1]
```

Out[439…
```
array([6, 5, 4, 3, 2, 1])
```

In [440…
```
np.argsort(b1)
```

Out[440…
```
array([0, 1, 2, 3, 4, 5])
```

In [441…
```
np.sort(a1, axis = 0)
```

Out[441…
```
array([[ 5,  3,  3,  4],
       [ 5,  5,  4,  7],
       [ 7,  5,  5,  9],
       [ 9,  6,  7,  9],
       [11,  7,  7, 11],
       [16,  9, 14, 21]])
```

In [442…
```
b1 = np.array([12,1,41,1,5,11,14,])
```

In [443…
```
np.where(b1>=4)
```

Out[443…
```
(array([0, 2, 4, 5, 6]),)
```

In [444…
```
t1 = np.where(b1>4)
```

In [445…
```
len(t1)
```

Out[445…
```
1
```

In [446…
```
len(t1[0])
```

Out[446…
```
5
```

In [447…
```
b1[[1,3,5]]
```

Out[447…
```
array([ 1,  1, 11])
```

In [448…
```
len(np.where(b1>=4)[0])
```

Out[448…
```
5
```

In [449…
```
(b1>=4).sum()
```

Out[449…
```
np.int64(5)
```

In [450…
```
t1
```

Out[450…
```
(array([0, 2, 4, 5, 6]),)
```

In [451…
```
len(t1[0])
```

Out[451…
```
5
```

In [452…
```
(a1>=4).sum()
```

Out[452…
```
np.int64(22)
```

In [453…
```
np.unique(b1)
```

Out[453…
```
array([ 1,  5, 11, 12, 14, 41])
```

In [454…
```
np.unique(b1, return_index=True)
```

Out[454…
```
(array([ 1,  5, 11, 12, 14, 41]), array([1, 4, 5, 0, 6, 2]))
```

In [455…
```
np.unique(b1, return_counts=True)
```

Out[455…
```
(array([ 1,  5, 11, 12, 14, 41]), array([2, 1, 1, 1, 1, 1]))
```

In [456…
```
np.unique(b1, return_inverse=True)          # for every unique element as refe
```

Out[456…
```
(array([ 1,  5, 11, 12, 14, 41]), array([3, 0, 5, 0, 1, 2, 4]))
```

In [457…
```
import numpy as np
x1 = np.array([1,2,3,2,1])
x2 = np.array([1,2,6,5,6])
```

In [458…
```
np.intersect1d(x1,x2)
```

Out[458…
```
array([1, 2])
```

In [459…
```
np.union1d(x1,x2)
```

Out[459… array([1, 2, 3, 5, 6])

In [460…
```
np.setdiff1d(x2,x1)
```

Out[460… array([5, 6])

In [461…
```
x3 = np.array([4,10,2,5])
```

In [462…
```
x3
```

Out[462… array([4, 10, 2, 5])

In [463…
```
x1
```

Out[463… array([1, 2, 3, 2, 1])

In [464…
```
np.in1d(x1,x3)
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_61606/2081356283.p
y:1: DeprecationWarning: `in1d` is deprecated. Use `np.isin` instead.
  np.in1d(x1,x3)
```

Out[464… array([False, True, False, True, False])

In [465…
```
np.where(np.in1d(x1,x3))
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_61606/573868696.p
y:1: DeprecationWarning: `in1d` is deprecated. Use `np.isin` instead.
  np.where(np.in1d(x1,x3))
```

Out[465… (array([1, 3]),)

In [466…
```
a1 = np.array([[1,4,7],[2,3,6]])
```

In [467…
```
a1
```

Out[467…
```
array([[1, 4, 7],
       [2, 3, 6]])
```

In [468…
```
a2  = np.array([[1,5],[0,2],[5,3]])
```

In [469…
```
a2
```

Out[469…
```
array([[1, 5],
       [0, 2],
       [5, 3]])
```

In [470…
```
a1.dot(a2)
```

Out[470…
```
array([[36, 34],
       [32, 34]])
```

In [471…
```
np.matmul(a1,a2)
```

Out[471…
```
array([[36, 34],
       [32, 34]])
```

In [472…
```
a3 = ([[1,2,3],[4,2,1],[3,2,6]])
```

In [473…
```
a3
```

Out[473…
```
[[1, 2, 3], [4, 2, 1], [3, 2, 6]]
```

In [474…
```
np.linalg.inv(a3)
```

Out[474…
```
array([[-0.38461538,  0.23076923,  0.15384615],
       [ 0.80769231,  0.11538462, -0.42307692],
       [-0.07692308, -0.15384615,  0.23076923]])
```

In [475…
```
a3 = np.array([[1,2,3],[4,2,1],[3,2,6]])
```

In [476…
```
a3
```

Out[476…
```
array([[1, 2, 3],
       [4, 2, 1],
       [3, 2, 6]])
```

In [477…
```
from numpy import random as rd
```

In [478…
```
rd.rand(10)
```

Out[478…
```
array([0.01181205, 0.93165956, 0.30563537, 0.18504695, 0.04156519,
       0.45092734, 0.51552185, 0.88986211, 0.96440635, 0.09142884])
```

In [479…
```
np.round(rd.rand(10),3)
```

Out[479…
```
array([0.834, 0.333, 0.473, 0.004, 0.564, 0.462, 0.045, 0.225, 0.133,
       0.793])
```

In [480…
```
np.round(rd.rand(40),3)
```

Out[480…
```
array([0.242, 0.589, 0.213, 0.14 , 0.156, 0.448, 0.288, 0.563, 0.408,
       0.504, 0.833, 0.405, 0.559, 0.083, 0.408, 0.903, 0.687, 0.034,
       0.079, 0.614, 0.315, 0.467, 0.57 , 0.573, 0.631, 0.666, 0.972,
       0.194, 0.802, 0.804, 0.887, 0.723, 0.894, 0.503, 0.655, 0.434,
       0.02 , 0.904, 0.969, 0.219])
```

In [481…
```
numbers_between_10_and_40 = np.arange(10, 41)
print(numbers_between_10_and_40)
```

```
[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40]
```

In [482…
```
np.random.randint(20,50)
```

Out[482… 43

In [483…
```
np.random.randint(20,50,10)
```

Out[483… array([35, 45, 21, 28, 22, 43, 24, 21, 23, 42])

In [3]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

Pandas Cheat Sheet — Most Used Functions

Basic inspection

Function	Purpose
df.head()	Show first rows
df.tail()	Show last rows
df.info()	Summary: datatypes + null counts
df.describe()	Numerical summary (mean, std, etc.)
df.shape	(rows, columns)
df.columns	List of column names
df.dtypes	Datatype of each column
df.index	Show index range

Handling missing values

Function	Purpose
df.isnull()	True/False for missing
df.isnull().sum()	Count missing values
df.dropna()	Remove rows with missing values
df.fillna(value)	Fill missing values

Selecting & filtering

Function	Purpose
df['col']	Select column
df[['col1','col2']]	Select multiple columns
df.loc[row_label, col_label]	Select by label
df.iloc[row_index, col_index]	Select by index
df[df.col > 10]	Conditional filtering
df.query("col > 10")	SQL-like filtering

Sorting

Function	Purpose
df.sort_values('col')	Sort by column
df.sort_values(['col1', 'col2'])	Sort by multiple columns

Handling duplicates

Function	Purpose
df.duplicated()	Identify duplicates
df.drop_duplicates()	Remove duplicates

Renaming & modifying columns

Function	Purpose
df.rename(columns={'old':'new'})	Rename columns
df.astype({'col':'int'})	Change datatype
df.drop(columns=['col'])	Drop a column

Combining data

Function	Purpose
pd.concat([df1, df2])	Vertical/Horizontal stacking

Function	Purpose
df.merge(df2, on='col')	SQL-style join
df.join(df2)	Join by index

GroupBy & Aggregation

Function	Purpose
df.groupby('col').mean()	Group & compute mean
df.groupby('col').agg(['mean','sum','count'])	Multi aggregation
df.groupby(['A','B']).size()	Count rows per group

Value counts & unique

Function	Purpose
df.col.value_counts()	Count of each category
df.col.nunique()	Number of unique values
df.col.unique()	List of unique values

Pivoting

Function	Purpose
df.pivot(index='A', columns='B', values='C')	Reshape table
df.pivot_table(index='A', columns='B', values='C', aggfunc='sum')	Pivot with aggregation

---

Statistics (EDA)

Function	Purpose
df.col.mean()	Mean
df.col.median()	Median
df.col.mode()	Mode
df.col.std()	Standard deviation
df.col.var()	Variance

Function	Purpose
df.col.min(), df.col.max()	Range
df.corr()	Correlation matrix
df.cov()	Covariance matrix

In [4]:
```
import pandas as pd
import os
import numpy as np
```

In [5]:
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

In [6]:
```
s1 = pd.Series([4,7,6,8,9])
```

In [7]:
```
s1
```

Out[7]:
```
0    4
1    7
2    6
3    8
4    9
dtype: int64
```

In [8]:
```
s1[2]
```

Out[8]:
```
np.int64(6)
```

In [9]:
```
s1[1:4]
```

Out[9]:
```
1    7
2    6
3    8
dtype: int64
```

In [10]:
```
s1 = pd.Series([4,7,6,8,9], index = ['A','B','C','D','E'])
```

In [11]:
```
s1
```

Out[11]:
```
A    4
B    7
C    6
D    8
E    9
dtype: int64
```

In [12]:
```
s1.iloc[2] # extracting from inbuild index =  index reference, [note - we can
```

Out[12]:
```
np.int64(6)
```

In [13]:
```
s1.loc['C'] # extracting from custom index = label reference which is custom


# also the custom labes could be duplicate such as index = ['A','B','A','B']
```

Out[13]:
```
np.int64(6)
```

In [14]:
```
s1.iloc[[1,3]]
```

Out[14]:
```
B    7
D    8
dtype: int64
```

In [15]:
```
s2 = s1.iloc[[1,3]]
```

In [16]:
```
s2
```

Out[16]:
```
B    7
D    8
dtype: int64
```

In [17]:
```
# we can also make the custom index and custom elements with the help of dicti
```

In [18]:
```
d1 = {'A':2, 'B':7,'C':8}
```

In [19]:
```
d1
```

Out[19]:
```
{'A': 2, 'B': 7, 'C': 8}
```

In [20]:
```
s2 = pd.Series(d1)
```

In [21]:
```
s2
```

Out[21]:
```
A    2
B    7
C    8
dtype: int64
```

In [22]:
```
s3 = pd.Series([5,3], index = ['D','E'])
```

In [23]:
```
s3
```

Out[23]:
```
D    5
E    3
dtype: int64
```

In [24]:
```
s2 = pd.concat([s2,s3])
```

In [25]:
```
s2
```

Out[25]:
```
A    2
B    7
C    8
D    5
E    3
dtype: int64
```

In [26]:
```
s3
```

Out[26]:
```
D    5
E    3
dtype: int64
```

In [27]:
```
s2.loc['B'] = 17
```

In [28]:
```
s2
```

Out[28]:
```
A     2
B    17
C     8
D     5
E     3
dtype: int64
```

In [29]:
```
s2.drop(s2.index[2])
```

Out[29]:
```
A     2
B    17
D     5
E     3
dtype: int64
```

In [30]:
```
s2.drop('C')
```

Out[30]:
```
A     2
B    17
D     5
E     3
dtype: int64
```

In [31]:
```
df = pd.DataFrame([['A',25,76], ['B',45, 90], ['C',26,58]], columns = ['Name',
```

In [32]:
```
df
```

Out[32]:

	Name	RollNo	Marks
0	A	25	76
1	B	45	90
2	C	26	58

In [33]:
```
d1 = {'PlayerName' :['Sachin', 'Dhoni', 'Gill', 'Surya'], 'Team':['MI','CSK',
```

In [34]:
```
d1
```

Out[34]:
```
{'PlayerName': ['Sachin', 'Dhoni', 'Gill', 'Surya'],
 'Team': ['MI', 'CSK', 'GT', 'MI']}
```

In [35]:
```
newdf = pd.DataFrame([['H',25, 68], ['R',26, 45]], columns=['XYZ','LMQ', 'PQR'
```

In [36]:
```
newdf
```

Out[36]:

	XYZ	LMQ	PQR
0	H	25	68
1	R	26	45

In [37]:
```
df = pd.concat([df, newdf])
```

In [38]:
```
newdf
```

Out[38]:

	XYZ	LMQ	PQR
0	H	25	68
1	R	26	45

In [39]:
```
df.iloc[[1,3]]
```

Out[39]:

	Name	RollNo	Marks	XYZ	LMQ	PQR
1	B	45.0	90.0	NaN	NaN	NaN
0	NaN	NaN	NaN	H	25.0	68.0

In [40]:
```
df2 = pd.concat([df, newdf]).reset_index(drop=True)
```

In [41]:
```
df2
```

Out[41]:

	Name	RollNo	Marks	XYZ	LMQ	PQR
0	A	25.0	76.0	NaN	NaN	NaN
1	B	45.0	90.0	NaN	NaN	NaN
2	C	26.0	58.0	NaN	NaN	NaN
3	NaN	NaN	NaN	H	25.0	68.0
4	NaN	NaN	NaN	R	26.0	45.0
5	NaN	NaN	NaN	H	25.0	68.0
6	NaN	NaN	NaN	R	26.0	45.0

In [42]:
```
df2 = pd.concat([df, newdf]).reset_index()  # add back index
```

In [43]:
```
df2
```

Out[43]:

	index	Name	RollNo	Marks	XYZ	LMQ	PQR
0	0	A	25.0	76.0	NaN	NaN	NaN
1	1	B	45.0	90.0	NaN	NaN	NaN
2	2	C	26.0	58.0	NaN	NaN	NaN
3	0	NaN	NaN	NaN	H	25.0	68.0
4	1	NaN	NaN	NaN	R	26.0	45.0
5	0	NaN	NaN	NaN	H	25.0	68.0
6	1	NaN	NaN	NaN	R	26.0	45.0

In [44]:
```
x1 = [45,68,76,89,2,34,1]
```

In [45]:
```
df2['marks2'] = x1
```

In [46]:
```
df2
```

Out[46]:

	index	Name	RollNo	Marks	XYZ	LMQ	PQR	marks2
0	0	A	25.0	76.0	NaN	NaN	NaN	45
1	1	B	45.0	90.0	NaN	NaN	NaN	68
2	2	C	26.0	58.0	NaN	NaN	NaN	76
3	0	NaN	NaN	NaN	H	25.0	68.0	89
4	1	NaN	NaN	NaN	R	26.0	45.0	2
5	0	NaN	NaN	NaN	H	25.0	68.0	34
6	1	NaN	NaN	NaN	R	26.0	45.0	1

In [47]:
```
df2.drop([0,2])
```

Out[47]:

	index	Name	RollNo	Marks	XYZ	LMQ	PQR	marks2
1	1	B	45.0	90.0	NaN	NaN	NaN	68
3	0	NaN	NaN	NaN	H	25.0	68.0	89
4	1	NaN	NaN	NaN	R	26.0	45.0	2
5	0	NaN	NaN	NaN	H	25.0	68.0	34
6	1	NaN	NaN	NaN	R	26.0	45.0	1

In [48]:
```
df2
```

Out[48]:

	index	Name	RollNo	Marks	XYZ	LMQ	PQR	marks2
0	0	A	25.0	76.0	NaN	NaN	NaN	45
1	1	B	45.0	90.0	NaN	NaN	NaN	68
2	2	C	26.0	58.0	NaN	NaN	NaN	76
3	0	NaN	NaN	NaN	H	25.0	68.0	89
4	1	NaN	NaN	NaN	R	26.0	45.0	2
5	0	NaN	NaN	NaN	H	25.0	68.0	34
6	1	NaN	NaN	NaN	R	26.0	45.0	1

In [49]:
```
df2 = df2.drop('index', axis=1)
```

In [50]:
```
df2
```

Out[50]:

	Name	RollNo	Marks	XYZ	LMQ	PQR	marks2
0	A	25.0	76.0	NaN	NaN	NaN	45
1	B	45.0	90.0	NaN	NaN	NaN	68
2	C	26.0	58.0	NaN	NaN	NaN	76
3	NaN	NaN	NaN	H	25.0	68.0	89
4	NaN	NaN	NaN	R	26.0	45.0	2
5	NaN	NaN	NaN	H	25.0	68.0	34
6	NaN	NaN	NaN	R	26.0	45.0	1

In [51]:
```
df2.drop(df2.columns[1], axis =1 )
```

Out[51]:

	Name	Marks	XYZ	LMQ	PQR	marks2
0	A	76.0	NaN	NaN	NaN	45
1	B	90.0	NaN	NaN	NaN	68
2	C	58.0	NaN	NaN	NaN	76
3	NaN	NaN	H	25.0	68.0	89
4	NaN	NaN	R	26.0	45.0	2
5	NaN	NaN	H	25.0	68.0	34
6	NaN	NaN	R	26.0	45.0	1

In [52]:
```
df2.iat[1,2] = 60
```

In [53]:
```
df2
```

Out[53]:

	Name	RollNo	Marks	XYZ	LMQ	PQR	marks2
0	A	25.0	76.0	NaN	NaN	NaN	45
1	B	45.0	60.0	NaN	NaN	NaN	68
2	C	26.0	58.0	NaN	NaN	NaN	76
3	NaN	NaN	NaN	H	25.0	68.0	89
4	NaN	NaN	NaN	R	26.0	45.0	2
5	NaN	NaN	NaN	H	25.0	68.0	34
6	NaN	NaN	NaN	R	26.0	45.0	1

In [54]:
```
df2.at[2,'Marks'] = 68
```

In [55]:
```
df2
```

Out[55]:

	Name	RollNo	Marks	XYZ	LMQ	PQR	marks2
0	A	25.0	76.0	NaN	NaN	NaN	45
1	B	45.0	60.0	NaN	NaN	NaN	68
2	C	26.0	68.0	NaN	NaN	NaN	76
3	NaN	NaN	NaN	H	25.0	68.0	89
4	NaN	NaN	NaN	R	26.0	45.0	2
5	NaN	NaN	NaN	H	25.0	68.0	34
6	NaN	NaN	NaN	R	26.0	45.0	1

In [56]:
```
erp = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='ERPData', engine='c
```

In [57]:
```
df.head()
```

Out[57]:

	Name	RollNo	Marks	XYZ	LMQ	PQR
0	A	25.0	76.0	NaN	NaN	NaN
1	B	45.0	90.0	NaN	NaN	NaN
2	C	26.0	58.0	NaN	NaN	NaN
0	NaN	NaN	NaN	H	25.0	68.0
1	NaN	NaN	NaN	R	26.0	45.0

In [58]:
```
df.shape
```

Out[58]:
```
(5, 6)
```

In [59]:
```
df.tail()
```

Out[59]:

	Name	RollNo	Marks	XYZ	LMQ	PQR
0	A	25.0	76.0	NaN	NaN	NaN
1	B	45.0	90.0	NaN	NaN	NaN
2	C	26.0	58.0	NaN	NaN	NaN
0	NaN	NaN	NaN	H	25.0	68.0
1	NaN	NaN	NaN	R	26.0	45.0

In [60]:
```
df.columns
```

Out[60]:
```
Index(['Name', 'RollNo', 'Marks', 'XYZ', 'LMQ', 'PQR'], dtype='object')
```

In [61]:
```
list(df.columns)
```

Out[61]:
```
['Name', 'RollNo', 'Marks', 'XYZ', 'LMQ', 'PQR']
```

In [62]:
```
df.head()
```

Out[62]:

	Name	RollNo	Marks	XYZ	LMQ	PQR
0	A	25.0	76.0	NaN	NaN	NaN
1	B	45.0	90.0	NaN	NaN	NaN
2	C	26.0	58.0	NaN	NaN	NaN
0	NaN	NaN	NaN	H	25.0	68.0
1	NaN	NaN	NaN	R	26.0	45.0

In [63]:
```
df.columns.values[1] = 'Location'
```

In [64]:
```
erp.head()
```

Out[64]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
1	AXCP-78	MWH-1	67
2	LXCV-21	MWH-2	27
3	AXCP-78	MWH-5	65
4	AXCP-78	MWH-4	36

In [65]:
```
ind = np.where(erp.Location == 'MWH-2')
```

In [66]:
```
ind
```

Out[66]:
```
(array([ 2,  7,  8, 15, 35, 36, 42, 45, 46]),)
```

In [67]:
```
df1 = erp.iloc[ind]
```

In [68]:
```
df1
```

Out[68]:

	MaterialID	Location	Quantity
2	LXCV-21	MWH-2	27
7	TMI-43T	MWH-2	29
8	GCVB-79	MWH-2	10
15	GCVB-79	MWH-2	87
35	GCVB-79	MWH-2	31
36	GCVB-79	MWH-2	28
42	SDRT-67	MWH-2	31
45	DDBN-89	MWH-2	69
46	AXCP-78	MWH-2	85

In [69]:
```
ind = np.where(((erp.Location == 'MWH-2') | (erp.Location == 'MWH-4')) & (erp.
```

In [70]:
```
ind
```

Out[70]:
```
(array([ 5, 12, 15, 46]),)
```

In [71]:
```
ind = np.where((erp.Location.isin(['MWH-2', 'MWH-4'])) & (erp.Quantity > 75))
```

In [72]:
```
df2 = erp.iloc[ind].reset_index(drop= True)
```

In [73]:
```
df2
```

Out[73]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	78
1	TMI-43T	MWH-4	76
2	GCVB-79	MWH-2	87
3	AXCP-78	MWH-2	85

In [74]:
```
min(erp.Quantity)
```

Out[74]:
```
10
```

In [75]:
```
ind = np.where(erp.Quantity<=40)
```

In [76]:
```
ind[0]
```

Out[76]:
```
array([ 0,  2,  4,  6,  7,  8, 10, 13, 16, 18, 20, 28, 30, 34, 35, 36, 40,
       42, 43, 47, 49])
```

In [77]:
```
df1 = erp.drop(ind[0])
```

In [78]:
```
min(df1.Quantity)
```

Out[78]:
```
42
```

In [79]:
```
max(df1.Quantity)
```

Out[79]:
```
152
```

In [80]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='Pune', engine='openp
```

In [81]:
```
df.head()
```

Out[81]:

	Name	Subject	Grade
0	Rakesh	Python	A
1	Manoj	MLPython	C
2	Vaibhav	Statistics	B
3	Hitesh	CommSkills	A
4	Suyash	ProjectMgmt	B

In [82]:
```
loc = df.where((df.Subject == 'Python') | (df.Subject == 'MLPython'))
```

In [83]:
```
# `ind` is a tuple returned by np.where, use the first element (array of row i
rows = ind[0]
erp.iloc[rows]
```

Out[83]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
2	LXCV-21	MWH-2	27
4	AXCP-78	MWH-4	36
6	TMI-43T	MWH-4	31
7	TMI-43T	MWH-2	29
8	GCVB-79	MWH-2	10
10	SDRT-67	MWH-1	34
13	TMI-43T	MWH-4	32
16	SDRT-67	MWH-5	12
18	TMI-43T	MWH-4	34
20	TMI-43T	MWH-4	39
28	LXCV-21	MWH-5	34
30	DDBN-89	MWH-4	27
34	GCVB-79	MWH-5	26
35	GCVB-79	MWH-2	31
36	GCVB-79	MWH-2	28
40	DDBN-89	MWH-1	34
42	SDRT-67	MWH-2	31
43	DDBN-89	MWH-3	38
47	AXCP-78	MWH-3	29
49	DDBN-89	MWH-1	39

In [84]:
```
df1 = loc.dropna(how='all').reset_index(drop=True)
```

In [85]:
```
df1
```

Out[85]:

	Name	Subject	Grade
0	Rakesh	Python	A
1	Manoj	MLPython	C

In [86]:
```
erp.head()
```

Out[86]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
1	AXCP-78	MWH-1	67
2	LXCV-21	MWH-2	27
3	AXCP-78	MWH-5	65
4	AXCP-78	MWH-4	36

In [87]:
```
min_q = erp.Quantity.min()
ind = np.where(erp.Quantity == min_q)
```

In [88]:
```
ind
```

Out[88]:
```
(array([8]),)
```

In [89]:
```
# `ind` is a tuple returned by np.where, e.g. (array([8]),)
# use the first element (array of row indices) with iloc.
rows = ind[0]

# If you want all matching rows' first column:
df1 = erp.iloc[rows, 0]

# If you want only the first matching value (scalar), uncomment:
# df1 = erp.iloc[rows[0], 0]
```

In [90]:
```
df1
```

Out[90]:
```
8    GCVB-79
Name: MaterialID, dtype: object
```

In [91]:
```
erp
```

Out[91]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
1	AXCP-78	MWH-1	67
2	LXCV-21	MWH-2	27
3	AXCP-78	MWH-5	65
4	AXCP-78	MWH-4	36
5	TMI-43T	MWH-4	78
6	TMI-43T	MWH-4	31
7	TMI-43T	MWH-2	29
8	GCVB-79	MWH-2	10
9	TMI-43T	MWH-1	120
10	SDRT-67	MWH-1	34
11	SDRT-67	MWH-4	58
12	TMI-43T	MWH-4	76
13	TMI-43T	MWH-4	32
14	TMI-43T	MWH-3	65
15	GCVB-79	MWH-2	87
16	SDRT-67	MWH-5	12
17	SDRT-67	MWH-5	102
18	TMI-43T	MWH-4	34
19	TMI-43T	MWH-4	52
20	TMI-43T	MWH-4	39
21	TMI-43T	MWH-4	75
22	DDBN-89	MWH-4	48
23	DDBN-89	MWH-5	71
24	AXCP-78	MWH-1	152
25	AXCP-78	MWH-1	109
26	DDBN-89	MWH-1	57
27	LXCV-21	MWH-5	83

	MaterialID	Location	Quantity
28	LXCV-21	MWH-5	34
29	LXCV-21	MWH-5	57
30	DDBN-89	MWH-4	27
31	TMI-43T	MWH-4	65
32	TMI-43T	MWH-4	43
33	TMI-43T	MWH-5	112
34	GCVB-79	MWH-5	26
35	GCVB-79	MWH-2	31
36	GCVB-79	MWH-2	28
37	AXCP-78	MWH-5	65
38	AXCP-78	MWH-5	70
39	AXCP-78	MWH-1	145
40	DDBN-89	MWH-1	34
41	SDRT-67	MWH-1	57
42	SDRT-67	MWH-2	31
43	DDBN-89	MWH-3	38
44	LXCV-21	MWH-3	42
45	DDBN-89	MWH-2	69
46	AXCP-78	MWH-2	85
47	AXCP-78	MWH-3	29
48	AXCP-78	MWH-1	75
49	DDBN-89	MWH-1	39

In [92]:
```
ind = np.where(erp.Quantity==min(erp.Quantity))
```

In [93]:
```
import pandas as pd
```

In [94]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='EmpInfo')
```

In [95]:
```
import pandas as pd
import numpy as np
```

In [96]:
```
df.head(7)
```

Out[96]:

	Name	EmpID	Deptt	Passport
0	Sudeep	A342	Quality	DF453278
1	Ruchika	C578	Sales	CA567657
2	NaN	J436	Admin	DF453278
3	Arjun	A342	NaN	RF453432
4	Rishi	R475	Prodn	DF453278
5	Chetan	NaN	NaN	ER534867
6	Deepak	NaN	Quality	RF453432

In [97]:
```
ind = np.where(df.Deptt.isnull())
```

In [98]:
```
ind
```

Out[98]:
```
(array([3, 5]),)
```

In [99]:
```
ind2 = np.where((df.EmpID.isnull()| df.Deptt.isnull()))
```

In [100…
```
ind2
```

Out[100…
```
(array([3, 5, 6]),)
```

In [101…
```
df1 = df.iloc[ind2[0]]
```

In [102…
```
df1
```

Out[102…

	Name	EmpID	Deptt	Passport
3	Arjun	A342	NaN	RF453432
5	Chetan	NaN	NaN	ER534867
6	Deepak	NaN	Quality	RF453432

In [103…
```
ind2 = not(np.where(df.EmpID.isnull()))

# where empid is not null
```

In [104…
```
ind2
```

Out[104…
```
False
```

In [105…
```
newdf = df.dropna()
```

In [106…
```
newdf
```

Out[106…

	Name	EmpID	Deptt	Passport
0	Sudeep	A342	Quality	DF453278
1	Ruchika	C578	Sales	CA567657
4	Rishi	R475	Prodn	DF453278

In [107…
```
df.isnull().sum()
```

Out[107…
```
Name        1
EmpID       2
Deptt       2
Passport    0
dtype: int64
```

In [108…
```
df.Deptt.fillna('Overhead')
```

Out[108…
```
0     Quality
1       Sales
2       Admin
3    Overhead
4       Prodn
5    Overhead
6     Quality
Name: Deptt, dtype: object
```

In [109…
```
df
```

Out[109…

	Name	EmpID	Deptt	Passport
0	Sudeep	A342	Quality	DF453278
1	Ruchika	C578	Sales	CA567657
2	NaN	J436	Admin	DF453278
3	Arjun	A342	NaN	RF453432
4	Rishi	R475	Prodn	DF453278
5	Chetan	NaN	NaN	ER534867
6	Deepak	NaN	Quality	RF453432

In [110…
```
df['New Col'] = df.Deptt.fillna('Overhead')
```

In [111…
```
df.Deptt.fillna('Overhead', inplace = True)
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_65633/969436898.p
y:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Ser
ies through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work bec
ause the intermediate object on which we are setting values always behaves as a
copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.me
thod({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, t
o perform the operation inplace on the original object.


  df.Deptt.fillna('Overhead', inplace = True)
```

In [1]:
```
# in case of numeric column, nan is median values for that column
```

In [113…
```
x1 = np.array([1,2,3,np.nan,5,np.nan,7])
```

In [114…
```
df['Marks'] = x1
```

In [115…
```
df
```

Out[115…

	Name	EmpID	Deptt	Passport	New Col	Marks
0	Sudeep	A342	Quality	DF453278	Quality	1.0
1	Ruchika	C578	Sales	CA567657	Sales	2.0
2	NaN	J436	Admin	DF453278	Admin	3.0
3	Arjun	A342	Overhead	RF453432	Overhead	NaN
4	Rishi	R475	Prodn	DF453278	Prodn	5.0
5	Chetan	NaN	Overhead	ER534867	Overhead	NaN
6	Deepak	NaN	Quality	RF453432	Quality	7.0

In [116…
```
df['Marks'] = df.Marks.fillna(df.Marks.median())
```

In [117…
```
df
```

Out[117…

	Name	EmpID	Deptt	Passport	New Col	Marks
0	Sudeep	A342	Quality	DF453278	Quality	1.0
1	Ruchika	C578	Sales	CA567657	Sales	2.0
2	NaN	J436	Admin	DF453278	Admin	3.0
3	Arjun	A342	Overhead	RF453432	Overhead	3.0
4	Rishi	R475	Prodn	DF453278	Prodn	5.0
5	Chetan	NaN	Overhead	ER534867	Overhead	3.0
6	Deepak	NaN	Quality	RF453432	Quality	7.0

In [118…
```
df.Deptt.fillna(method='ffill')
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_65633/536854854.p
y:1: FutureWarning: Series.fillna with 'method' is deprecated and will raise in
a future version. Use obj.ffill() or obj.bfill() instead.
  df.Deptt.fillna(method='ffill')
```

Out[118…
```
0     Quality
1       Sales
2       Admin
3    Overhead
4       Prodn
5    Overhead
6     Quality
Name: Deptt, dtype: object
```

In [119…
```
df['Marks'] = df.Marks.fillna(df.Marks.mode())
```

In [120…
```
df
```

Out[120…

	Name	EmpID	Deptt	Passport	New Col	Marks
0	Sudeep	A342	Quality	DF453278	Quality	1.0
1	Ruchika	C578	Sales	CA567657	Sales	2.0
2	NaN	J436	Admin	DF453278	Admin	3.0
3	Arjun	A342	Overhead	RF453432	Overhead	3.0
4	Rishi	R475	Prodn	DF453278	Prodn	5.0
5	Chetan	NaN	Overhead	ER534867	Overhead	3.0
6	Deepak	NaN	Quality	RF453432	Quality	7.0

In [121…
```
df.head()
```

Out[121…

	Name	EmpID	Deptt	Passport	New Col	Marks
0	Sudeep	A342	Quality	DF453278	Quality	1.0
1	Ruchika	C578	Sales	CA567657	Sales	2.0
2	NaN	J436	Admin	DF453278	Admin	3.0
3	Arjun	A342	Overhead	RF453432	Overhead	3.0
4	Rishi	R475	Prodn	DF453278	Prodn	5.0

In [122…
```
df
```

Out[122…

	Name	EmpID	Deptt	Passport	New Col	Marks
0	Sudeep	A342	Quality	DF453278	Quality	1.0
1	Ruchika	C578	Sales	CA567657	Sales	2.0
2	NaN	J436	Admin	DF453278	Admin	3.0
3	Arjun	A342	Overhead	RF453432	Overhead	3.0
4	Rishi	R475	Prodn	DF453278	Prodn	5.0
5	Chetan	NaN	Overhead	ER534867	Overhead	3.0
6	Deepak	NaN	Quality	RF453432	Quality	7.0

In [123…
```
df.duplicated()
```

Out[123…
```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
dtype: bool
```

In [124…
```
df.tail()
```

Out[124…

	Name	EmpID	Deptt	Passport	New Col	Marks
2	NaN	J436	Admin	DF453278	Admin	3.0
3	Arjun	A342	Overhead	RF453432	Overhead	3.0
4	Rishi	R475	Prodn	DF453278	Prodn	5.0
5	Chetan	NaN	Overhead	ER534867	Overhead	3.0
6	Deepak	NaN	Quality	RF453432	Quality	7.0

In [125…
```
df.duplicated('Passport')
```

Out[125…
```
0    False
1    False
2     True
3    False
4     True
5    False
6     True
dtype: bool
```

In [126…
```
i= np.where(df.duplicated('Passport'))
```

In [127…
```
i
```

Out[127…
```
(array([2, 4, 6]),)
```

In [128…
```
df = df.iloc[i]
```

In [129…
```
df
```

Out[129…

	Name	EmpID	Deptt	Passport	New Col	Marks
2	NaN	J436	Admin	DF453278	Admin	3.0
4	Rishi	R475	Prodn	DF453278	Prodn	5.0
6	Deepak	NaN	Quality	RF453432	Quality	7.0

In [130…
```
np.where(df.Passport.duplicated(keep=False))
```

Out[130…
```
(array([0, 1]),)
```

In [131…
```
np.where(df.Passport.duplicated(keep='first')) # except first
```

Out[131…
```
(array([1]),)
```

In [132…
```
np.where(df.Passport.duplicated(keep='last')) # except last
```

Out[132… (array([0]),)

In [133…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 3 entries, 2 to 6
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   Name      2 non-null      object 
 1   EmpID     2 non-null      object 
 2   Deptt     3 non-null      object 
 3   Passport  3 non-null      object 
 4   New Col   3 non-null      object 
 5   Marks     3 non-null      float64
dtypes: float64(1), object(5)
memory usage: 168.0+ bytes
```

In [134…
```
df.shape
```

Out[134… (3, 6)

In [135…
```
df.columns
```

Out[135…
```
Index(['Name', 'EmpID', 'Deptt', 'Passport', 'New Col', 'Marks'], dtype='obje
ct')
```

In [136…
```
df.index
```

Out[136… Index([2, 4, 6], dtype='int64')

In [137…
```
df['Name']
```

Out[137…
```
2       NaN
4     Rishi
6    Deepak
Name: Name, dtype: object
```

In [138…
```
df[['Name', 'Marks']]
```

Out[138…

	Name	Marks
2	NaN	3.0
4	Rishi	5.0
6	Deepak	7.0

In [139…
```
df.iloc[1:5]
```

Out[139…

	Name	EmpID	Deptt	Passport	New Col	Marks
4	Rishi	R475	Prodn	DF453278	Prodn	5.0
6	Deepak	NaN	Quality	RF453432	Quality	7.0

In [140…
```
df
```

Out[140…

	Name	EmpID	Deptt	Passport	New Col	Marks
2	NaN	J436	Admin	DF453278	Admin	3.0
4	Rishi	R475	Prodn	DF453278	Prodn	5.0
6	Deepak	NaN	Quality	RF453432	Quality	7.0

In [141…
```
df[df['Marks']>1]
```

Out[141…

	Name	EmpID	Deptt	Passport	New Col	Marks
2	NaN	J436	Admin	DF453278	Admin	3.0
4	Rishi	R475	Prodn	DF453278	Prodn	5.0
6	Deepak	NaN	Quality	RF453432	Quality	7.0

In [142…
```
eg = pd.read_excel('data/subject.xlsx')
```

In [143…
```
eg.Subject.isnull().sum()
```

Out[143… `np.int64(22)`

In [144…
```
eg1=eg.Subject.value_counts().reset_index()
```

In [145…
```
eg1['prop'] = eg1['count']/sum(eg1['count'])
```

In [146…
```
eg1['miss'] = np.round(eg1.prop*22)
```

In [147…
```
ind = np.where(eg.Subject.isnull())
```

In [ ]:
```
import random

random.shuffle(ind[0])

random.shuffle(ind[0])
```

In [151…
```
eg.iloc[ind[0][:11]] = 'Maths'
eg.iloc[ind[0][11:17]] = 'Science'
eg.iloc[ind[0][17:]] = 'Arts'
```

In [152…

```
eg.head()
```

Out[152…

	Subject
0	Maths
1	Science
2	Maths
3	Maths
4	Maths

In [1]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

In [2]:
```
import pandas as pd
import numpy as np
import os
```

In [3]:
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

In [4]:
```
os.getcwd()
```

Out[4]:
```
'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-stats'
```

In [5]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='ERPData')
```

In [6]:
```
df.head()
```

Out[6]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
1	AXCP-78	MWH-1	67
2	LXCV-21	MWH-2	27
3	AXCP-78	MWH-5	65
4	AXCP-78	MWH-4	36

In [7]:
```
grp1 = df.groupby('Location')
```

In [8]:
```
grp1.get_group('MWH-2')
```

Out[8]:

	MaterialID	Location	Quantity
2	LXCV-21	MWH-2	27
7	TMI-43T	MWH-2	29
8	GCVB-79	MWH-2	10
15	GCVB-79	MWH-2	87
35	GCVB-79	MWH-2	31
36	GCVB-79	MWH-2	28
42	SDRT-67	MWH-2	31
45	DDBN-89	MWH-2	69
46	AXCP-78	MWH-2	85

In [9]:
```
grp1.agg(np.sum).Quantity
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_32758/429834943.p
y:1: FutureWarning: The provided callable <function sum at 0x106b9c5e0> is curr
ently using DataFrameGroupBy.sum. In a future version of pandas, the provided c
allable will be used directly. To keep current behavior pass the string "sum" i
nstead.
  grp1.agg(np.sum).Quantity
```

Out[9]:
```
Location
MWH-1    889
MWH-2    397
MWH-3    174
MWH-4    728
MWH-5    697
Name: Quantity, dtype: int64
```

In [10]:
```
grp1.Quantity.agg(['sum','size'])
```

Out[10]:

	sum	size
Location		
MWH-1	889	11
MWH-2	397	9
MWH-3	174	4
MWH-4	728	15
MWH-5	697	11

In [11]:
```
df2 = grp1.agg(np.sum).Quantity
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_32758/1664439342.p
y:1: FutureWarning: The provided callable <function sum at 0x106b9c5e0> is curr
ently using DataFrameGroupBy.sum. In a future version of pandas, the provided c
allable will be used directly. To keep current behavior pass the string "sum" i
nstead.
  df2 = grp1.agg(np.sum).Quantity
```

In [12]:
```
df2
```

Out[12]:
```
Location
MWH-1    889
MWH-2    397
MWH-3    174
MWH-4    728
MWH-5    697
Name: Quantity, dtype: int64
```

In [13]:
```
df2.shape
```

Out[13]:
```
(5,)
```

In [14]:
```
type(df2)
```

Out[14]:
```
pandas.core.series.Series
```

In [15]:
```
df2 = grp1.agg(np.sum).Quantity.reset_index().rename_axis(columns=None)


# index got renamed it will be removed by rename_axis
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_32758/1983690337.p
y:1: FutureWarning: The provided callable <function sum at 0x106b9c5e0> is curr
ently using DataFrameGroupBy.sum. In a future version of pandas, the provided c
allable will be used directly. To keep current behavior pass the string "sum" i
nstead.
  df2 = grp1.agg(np.sum).Quantity.reset_index().rename_axis(columns=None)
```

In [16]:
```
df2
```

Out[16]:

	Location	Quantity
0	MWH-1	889
1	MWH-2	397
2	MWH-3	174
3	MWH-4	728
4	MWH-5	697

In [17]:
```
type(df2)
```

Out[17]: pandas.core.frame.DataFrame

In [18]:
```
grp1.Quantity.count().reset_index()
```

Out[18]:

	Location	Quantity
0	MWH-1	11
1	MWH-2	9
2	MWH-3	4
3	MWH-4	15
4	MWH-5	11

In [19]:
```
grp1.Quantity.sum().reset_index()
```

Out[19]:

	Location	Quantity
0	MWH-1	889
1	MWH-2	397
2	MWH-3	174
3	MWH-4	728
4	MWH-5	697

In [20]:
```
grp1.Quantity.mean().reset_index()
```

Out[20]:

	Location	Quantity
0	MWH-1	80.818182
1	MWH-2	44.111111
2	MWH-3	43.500000
3	MWH-4	48.533333
4	MWH-5	63.363636

In [21]:
```
grp2 = df.groupby(['MaterialID', 'Location'])
```

In [22]:
```
grp2.groups

# read as (axcp78, mwh2) combo is occuring in [1, 24, 25, 39, 48] rows
```

Out[22]:
```
{('AXCP-78', 'MWH-1'): [1, 24, 25, 39, 48], ('AXCP-78', 'MWH-2'): [46], ('AXC
P-78', 'MWH-3'): [47], ('AXCP-78', 'MWH-4'): [4], ('AXCP-78', 'MWH-5'): [3, 3
7, 38], ('DDBN-89', 'MWH-1'): [26, 40, 49], ('DDBN-89', 'MWH-2'): [45], ('DDB
N-89', 'MWH-3'): [43], ('DDBN-89', 'MWH-4'): [22, 30], ('DDBN-89', 'MWH-5'):
[23], ('GCVB-79', 'MWH-2'): [8, 15, 35, 36], ('GCVB-79', 'MWH-5'): [34], ('LX
CV-21', 'MWH-2'): [2], ('LXCV-21', 'MWH-3'): [44], ('LXCV-21', 'MWH-5'): [27,
28, 29], ('SDRT-67', 'MWH-1'): [10, 41], ('SDRT-67', 'MWH-2'): [42], ('SDRT-6
7', 'MWH-4'): [11], ('SDRT-67', 'MWH-5'): [16, 17], ('TMI-43T', 'MWH-1'):
[9], ('TMI-43T', 'MWH-2'): [7], ('TMI-43T', 'MWH-3'): [14], ('TMI-43T', 'MW
H-4'): [0, 5, 6, 12, 13, 18, 19, 20, 21, 31, 32], ('TMI-43T', 'MWH-5'): [33]}
```

In [23]:
```
grp2.agg([np.sum,np.size])


# axcp78 at mwh5 is total of 200 and at 3 different consignment
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_32758/3651602171.p
y:1: FutureWarning: The provided callable <function sum at 0x106b9c5e0> is curr
ently using SeriesGroupBy.sum. In a future version of pandas, the provided call
able will be used directly. To keep current behavior pass the string "sum" inst
ead.
  grp2.agg([np.sum,np.size])
```

Out[23]:

		Quantity	
		sum	size
MaterialID	**Location**		
AXCP-78	**MWH-1**	548	5
	MWH-2	85	1
	MWH-3	29	1
	MWH-4	36	1
	MWH-5	200	3
DDBN-89	**MWH-1**	130	3
	MWH-2	69	1
	MWH-3	38	1
	MWH-4	75	2
	MWH-5	71	1
GCVB-79	**MWH-2**	156	4
	MWH-5	26	1
LXCV-21	**MWH-2**	27	1
	MWH-3	42	1
	MWH-5	174	3
SDRT-67	**MWH-1**	91	2
	MWH-2	31	1
	MWH-4	58	1
	MWH-5	114	2
TMI-43T	**MWH-1**	120	1
	MWH-2	29	1
	MWH-3	65	1
	MWH-4	559	11
	MWH-5	112	1

In [24]:
```
newdf = grp2.agg([np.sum, np.size])
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_32758/4092300133.p
y:1: FutureWarning: The provided callable <function sum at 0x106b9c5e0> is curr
ently using SeriesGroupBy.sum. In a future version of pandas, the provided call
able will be used directly. To keep current behavior pass the string "sum" inst
ead.
  newdf = grp2.agg([np.sum, np.size])
```

In [25]:
```
newdf
```

Out[25]:

		Quantity	
		sum	size
MaterialID	**Location**		
AXCP-78	**MWH-1**	548	5
	MWH-2	85	1
	MWH-3	29	1
	MWH-4	36	1
	MWH-5	200	3
DDBN-89	**MWH-1**	130	3
	MWH-2	69	1
	MWH-3	38	1
	MWH-4	75	2
	MWH-5	71	1
GCVB-79	**MWH-2**	156	4
	MWH-5	26	1
LXCV-21	**MWH-2**	27	1
	MWH-3	42	1
	MWH-5	174	3
SDRT-67	**MWH-1**	91	2
	MWH-2	31	1
	MWH-4	58	1
	MWH-5	114	2
TMI-43T	**MWH-1**	120	1
	MWH-2	29	1
	MWH-3	65	1
	MWH-4	559	11
	MWH-5	112	1

In [26]:
```
newdf.loc['AXCP-78']
```

Out[26]:

	Quantity	
	sum	size
Location		
MWH-1	548	5
MWH-2	85	1
MWH-3	29	1
MWH-4	36	1
MWH-5	200	3

In [27]:
```
newdf.loc[[('LXCV-21', 'MWH-3'),('TMI-43T', 'MWH-2')]]

# This selects and displays the aggregated rows for:
# - MaterialID 'LXCV-21' at Location 'MWH-3'
# - MaterialID 'TMI-43T' at Location 'MWH-2'
# The result shows the sum and size (count) for each numeric column for these
```

Out[27]:

		Quantity	
		sum	size
MaterialID	**Location**		
LXCV-21	**MWH-3**	42	1
TMI-43T	**MWH-2**	29	1

In [28]:
```
newdf.loc[['AXCP-78', 'TMI-43T']]
```

Out[28]:

		Quantity	
		sum	size
MaterialID	**Location**		
AXCP-78	**MWH-1**	548	5
	MWH-2	85	1
	MWH-3	29	1
	MWH-4	36	1
	MWH-5	200	3
TMI-43T	**MWH-1**	120	1
	MWH-2	29	1
	MWH-3	65	1
	MWH-4	559	11
	MWH-5	112	1

In [29]:
```
df2 = df.sort_values(['Quantity'], ascending=True)

# single column based sorting
```

In [30]:
```
df2.head().reset_index()
```

Out[30]:

	index	MaterialID	Location	Quantity
0	8	GCVB-79	MWH-2	10
1	16	SDRT-67	MWH-5	12
2	34	GCVB-79	MWH-5	26
3	2	LXCV-21	MWH-2	27
4	30	DDBN-89	MWH-4	27

In [31]:
```
df2 = df.sort_values(['MaterialID', 'Quantity'], ascending=True)

# two preference sorting
```

In [32]:
```
df2.head()
```

Out[32]:

	MaterialID	Location	Quantity
47	AXCP-78	MWH-3	29
4	AXCP-78	MWH-4	36
3	AXCP-78	MWH-5	65
37	AXCP-78	MWH-5	65
1	AXCP-78	MWH-1	67

In [33]:
```
df2 = df.sort_values(['Location','MaterialID','Quantity'], ascending=[False, F

# multi column sorting with ordering preference

# location = desc, materialid = desc, quantity = asc
```

In [34]:
```
df2.head()
```

Out[34]:

	MaterialID	Location	Quantity
33	TMI-43T	MWH-5	112
16	SDRT-67	MWH-5	12
17	SDRT-67	MWH-5	102
28	LXCV-21	MWH-5	34
29	LXCV-21	MWH-5	57

In [35]:
```
b1 = [9,40,80,120,160]
p1 = ['A','B','C','D']
df['Grade'] = pd.cut(df.Quantity, bins=b1, labels=p1)

# This categorizes the 'Quantity' values into bins defined by b1.
# Each bin is labeled with the corresponding value from p1.
# The intervals are:
#   (9, 40]    -> 'A'
#   (40, 80]   -> 'B'
#   (80, 120]  -> 'C'
#   (120, 160] -> 'D'
# Note: The left edge is open and the right edge is closed by default (i.e., 4
```

In [36]:
```
df.head()
```

Out[36]:

	MaterialID	Location	Quantity	Grade
0	TMI-43T	MWH-4	34	A
1	AXCP-78	MWH-1	67	B
2	LXCV-21	MWH-2	27	A
3	AXCP-78	MWH-5	65	B
4	AXCP-78	MWH-4	36	A

In [37]:
```
tz = df.sort_values(['MaterialID', 'Quantity'])
```

In [38]:
```
tz.head()
```

Out[38]:

	MaterialID	Location	Quantity	Grade
47	AXCP-78	MWH-3	29	A
4	AXCP-78	MWH-4	36	A
3	AXCP-78	MWH-5	65	B
37	AXCP-78	MWH-5	65	B
1	AXCP-78	MWH-1	67	B

In [39]:
```
pune = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='Pune')
mumbai = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='Mumbai')
```

In [40]:
```
pune.head()
```

Out[40]:

	Name	Subject	Grade
0	Rakesh	Python	A
1	Manoj	MLPython	C
2	Vaibhav	Statistics	B
3	Hitesh	CommSkills	A
4	Suyash	ProjectMgmt	B

In [41]:
```
mumbai.head()
```

Out[41]:

	Name	Subject	Grade
0	Vaibhav	Six Sigma	A
1	Deepika	Statistics	B
2	Arjun	CommSkills	A
3	Chetan	Python	A
4	Abhishek	MLPython	B

In [42]:
```
new = pd.merge(pune, mumbai, on='Subject', how='inner')

# by default how = 'inner' (which only gives matching records on subject), we
# so in this case ( on = 'subject )  is good for essentially comparing the mar
```

In [43]:
```
new = pd.merge(pune, mumbai, on=['Subject','Grade'])

# so basically the combination of both subject and grade must be common for ou
```

In [44]:
```
new = pd.merge(pune, mumbai, on=['Subject'], how = 'left')

# this means that the left dataframe shouldn't contain any null records, and c
```

In [45]:
```
new
```

Out[45]:

	Name_x	Subject	Grade_x	Name_y	Grade_y
0	Rakesh	Python	A	Chetan	A
1	Manoj	MLPython	C	Abhishek	B
2	Vaibhav	Statistics	B	Deepika	B
3	Hitesh	CommSkills	A	Arjun	A
4	Suyash	ProjectMgmt	B	NaN	NaN

In [46]:
```
new
```

Out[46]:

	Name_x	Subject	Grade_x	Name_y	Grade_y
0	Rakesh	Python	A	Chetan	A
1	Manoj	MLPython	C	Abhishek	B
2	Vaibhav	Statistics	B	Deepika	B
3	Hitesh	CommSkills	A	Arjun	A
4	Suyash	ProjectMgmt	B	NaN	NaN

In [47]:
```
new = pd.merge(pune, mumbai, on=['Subject'], how='outer')

# combines both pune and mumbai dataframes, keeping all rows from both, and ma
```

In [48]:
```
new
```

Out[48]:

	Name_x	Subject	Grade_x	Name_y	Grade_y
0	Hitesh	CommSkills	A	Arjun	A
1	Manoj	MLPython	C	Abhishek	B
2	Suyash	ProjectMgmt	B	NaN	NaN
3	Rakesh	Python	A	Chetan	A
4	NaN	Six Sigma	NaN	Vaibhav	A
5	Vaibhav	Statistics	B	Deepika	B

In [49]:
```
d1 = pd.DataFrame({'ID': ['A','B','D'], "Val1":[34,78,67]})
d2 = pd.DataFrame({'ID':['A','B','C','D','E'], "Val2":[30,40,80,20,90]})
```

In [50]:
```
pd.merge(d1,d2, how='left')
```

Out[50]:

	ID	Val1	Val2
0	A	34	30
1	B	78	40
2	D	67	20

In [51]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name = 'Health')
```

In [52]:
```
df.head()
```

Out[52]:

	Country	Disease	2019	2020	2021
0	India	Cancer	12	94	35
1	UK	Cancer	95	19	84
2	US	Cancer	98	19	53
3	Japan	Cancer	89	52	14
4	India	Covid	38	19	62

In [53]:
```
df2 = df.melt(id_vars=['Country', 'Disease'], var_name = 'Year',  value_name =

# id_vars are the columns that we want to keep intact
```

```
# var_name = columns we are melting
# value_name = organizing of the melted columns valuesinto the new column (her


# so here values are reorganized by changing the order
```

In [54]:
```
df3 = df2.pivot_table(columns='Disease', values='Cases',index=['Country', 'Yea

# pivot_table is for widening / boradening the table, its exactly opposite for

# so basically we are putting the every unique value that comes under the dise

# remember the values in cases comes from the disease column
```

In [55]:
```
df3
```

Out[55]:

	Disease	Cancer	Covid	Heart Attack
Country	**Year**			
India	**2019**	12.0	38.0	84.0
	2020	94.0	19.0	10.0
	2021	35.0	62.0	98.0
Japan	**2019**	89.0	14.0	11.0
	2020	52.0	12.0	43.0
	2021	14.0	90.0	75.0
UK	**2019**	95.0	69.0	44.0
	2020	19.0	73.0	17.0
	2021	84.0	35.0	24.0
US	**2019**	98.0	87.0	34.0
	2020	19.0	97.0	40.0
	2021	53.0	35.0	54.0

In [56]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='ERPData')
```

In [57]:
```
df.head()
```

Out[57]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
1	AXCP-78	MWH-1	67
2	LXCV-21	MWH-2	27
3	AXCP-78	MWH-5	65
4	AXCP-78	MWH-4	36

In [58]:
```
df1 = df.pivot_table(columns = 'Location', values='Quantity', index='MaterialI
```

In [59]:
```
df1.head()
```

Out[59]:

	MaterialID	MWH-1	MWH-2	MWH-3	MWH-4	MWH-5
0	AXCP-78	109.600000	85.0	29.0	36.0	66.666667
1	DDBN-89	43.333333	69.0	38.0	37.5	71.000000
2	GCVB-79	NaN	39.0	NaN	NaN	26.000000
3	LXCV-21	NaN	27.0	42.0	NaN	58.000000
4	SDRT-67	45.500000	31.0	NaN	58.0	57.000000

In [60]:
```
df1 = df.pivot_table(columns = 'Location', values='Quantity', index='MaterialI

# here aggfunc says count the sums appearances of record, default is mean if n
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_32758/1238229603.p
y:1: FutureWarning: The provided callable <function sum at 0x106b9c5e0> is curr
ently using DataFrameGroupBy.sum. In a future version of pandas, the provided c
allable will be used directly. To keep current behavior pass the string "sum" i
nstead.
  df1 = df.pivot_table(columns = 'Location', values='Quantity', index='Material
ID', aggfunc=np.sum).reset_index().rename_axis(columns=None)
```

In [61]:
```
df1.head()
```

Out[61]:

	MaterialID	MWH-1	MWH-2	MWH-3	MWH-4	MWH-5
0	AXCP-78	548.0	85.0	29.0	36.0	200.0
1	DDBN-89	130.0	69.0	38.0	75.0	71.0
2	GCVB-79	NaN	156.0	NaN	NaN	26.0
3	LXCV-21	NaN	27.0	42.0	NaN	174.0
4	SDRT-67	91.0	31.0	NaN	58.0	114.0

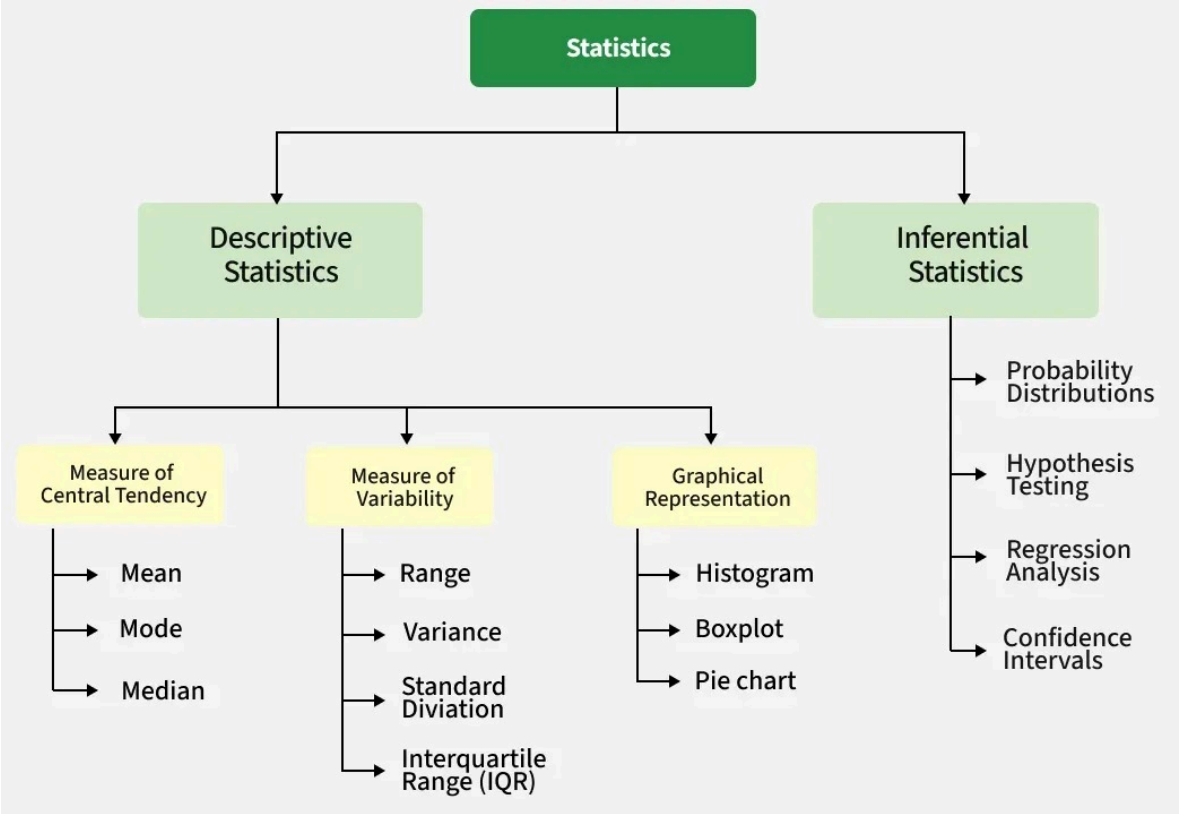
In [62]:
```
df1 = df.pivot_table(columns = 'Location', values='Quantity', index='MaterialI

# here aggfunc says count the total appearances of record
```

In [63]:
```
df1.head()
```

Out[63]:

	MaterialID	MWH-1	MWH-2	MWH-3	MWH-4	MWH-5
0	AXCP-78	5.0	1.0	1.0	1.0	3.0
1	DDBN-89	3.0	1.0	1.0	2.0	1.0
2	GCVB-79	NaN	4.0	NaN	NaN	1.0
3	LXCV-21	NaN	1.0	1.0	NaN	3.0
4	SDRT-67	2.0	1.0	NaN	1.0	2.0

In [93]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```



Types of Data

`Continuous`

- Result of a measurement, Decimals are possible (true value can be fractional)
- Requires an **instrument** to measure

`Discrete`

- Result of a **count**
- No decimals, whole numbers only

`Categorical (Attribute / Qualitative)`

- Describes **qualities**, **categories**, **labels**, not numeric values
- **Examples:** gender, color, brand, type of car, yes/no, low/medium/high

  categorical data types:

  - **Nominal** (no natural order) → color, city, fruit
  - **Ordinal** (natural order exists) → rating scales, education level, ranking

Descriptive statistics

In [2]:
```
import pandas as pd
import numpy as np
import warnings
import os
```

In [3]:
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

In [4]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='ERPData')
```

In [5]:
```
df.head()
```

Out[5]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
1	AXCP-78	MWH-1	67
2	LXCV-21	MWH-2	27
3	AXCP-78	MWH-5	65
4	AXCP-78	MWH-4	36

In [6]:
```
df.describe()
```

Out[6]:

	Quantity
count	50.000000
mean	57.700000
std	31.887078
min	10.000000
25%	34.000000
50%	54.500000
75%	74.000000
max	152.000000

In [7]:
```
np.unique(df.MaterialID)
```

Out[7]:
```
array(['AXCP-78', 'DDBN-89', 'GCVB-79', 'LXCV-21', 'SDRT-67', 'TMI-43T'],
      dtype=object)
```

In [8]:
```
len(np.unique(df.MaterialID))
```

Out[8]:
```
6
```

In [9]:
```
len(df.MaterialID)
```

Out[9]:
```
50
```

In [10]:
```
np.unique(df.MaterialID, return_counts=True)
```

Out[10]:
```
(array(['AXCP-78', 'DDBN-89', 'GCVB-79', 'LXCV-21', 'SDRT-67', 'TMI-43T'],
       dtype=object),
 array([11,  8,  5,  5,  6, 15]))
```

In [11]:
```
mat_count = df.MaterialID.value_counts().reset_index()
```

In [12]:
```
mat_count
```

Out[12]:

	MaterialID	count
0	TMI-43T	15
1	AXCP-78	11
2	DDBN-89	8
3	SDRT-67	6
4	LXCV-21	5
5	GCVB-79	5

In [13]:
```
mat_count['prop'] = mat_count['count']/sum(mat_count['count'])*100

# This calculates the percentage (proportion) of each MaterialID's count out c
# and stores it in a new column 'prop'. Each value shows what percent of the t
```

In [14]:
```
mat_count
```

Out[14]:

	MaterialID	count	prop
0	TMI-43T	15	30.0
1	AXCP-78	11	22.0
2	DDBN-89	8	16.0
3	SDRT-67	6	12.0
4	LXCV-21	5	10.0
5	GCVB-79	5	10.0

In [15]:
```
grades = np.array(['A','B','C'])
```

In [16]:
```
grades
```

Out[16]:
```
array(['A', 'B', 'C'], dtype='<U1')
```

In [17]:
```
df = pd.read_excel('data/covid_cases.xlsx')
```

In [18]:
```
df.head()
```

Out[18]:

	State	Cases
0	UP	250
1	KA	350
2	MH	650
3	KL	400
4	MP	350

In [19]:
```
df = df.sort_values('Cases', ascending=False)
```

In [20]:
```
df.head()
```

Out[20]:

	State	Cases
2	MH	650
3	KL	400
1	KA	350
4	MP	350
0	UP	250

In [21]:
```
df['prop'] = df ['Cases']/sum(df['Cases'])*100
```

In [22]:
```
df.head()
```

Out[22]:

	State	Cases	prop
2	MH	650	27.083333
3	KL	400	16.666667
1	KA	350	14.583333
4	MP	350	14.583333
0	UP	250	10.416667

In [23]:
```
round(np.cumsum(df.prop),2)
```

Out[23]:
```
2     27.08
3     43.75
1     58.33
4     72.92
0     83.33
5     89.58
7     95.83
6    100.00
Name: prop, dtype: float64
```

In [24]:
```
np.cumsum(df.prop).iloc[1]
```

Out[24]:
```
np.float64(43.75)
```

Descriptive Statistics

Central Tendency

- **Mean** $\rightarrow$ average
- **Median** $\rightarrow$ sorted middle value (if n is odd n+1/2)
- **Mode** $\rightarrow$ frequency

Variation or Dispersion

- `Range` $\rightarrow$ max val - min val
- `IQR` $\rightarrow$ middle 50% of data
- `Variance` (σ^2 or s^2): $\rightarrow$ The average of the **squared** differences from the Mean.
    - Formula (Population): $\sigma^2 = \frac{\sum (x_i - \mu)^2}{N}$
    - Formula (Sample): $s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$
- `Standard Deviation` (σ or s)`: The square root of the variance. It is the most commonly used measure of spread and is expressed in the same units as the data.
    - Formula (Population): $\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$
    - Formula (Sample): $s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n-1}}$

- **Skewness** - measure of the **asymmetry** of the distribution about its mean.
- **Kurtosis** - measure of the **tailedness** and **peakedness** of the distribution.

In [27]:
```
grades=np.array(['A','B','C','B','C','A','A','A','B','C','A','A','A','B'])
```

In [28]:
```
np.unique(grades,return_counts=True)
```

Out[28]:
```
(array(['A', 'B', 'C'], dtype='<U1'), array([7, 4, 3]))
```

In [29]:
```
p1=np.array([8,3,3])
```

In [30]:
```
np.cumsum(p1)     # sum of previous number and number itselt
```

Out[30]:
```
array([ 8, 11, 14])
```

In [31]:
```
np.unique(grades,return_counts=True)[1]
```

Out[31]:
```
array([7, 4, 3])
```

In [32]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='faithful')
```

In [33]:
```
df.head()
```

Out[33]:

	eruptions	waiting
0	3.600	79
1	1.800	54
2	3.333	74
3	2.283	62
4	4.533	85

In [34]:
```
erupt = df['eruptions']
```

In [35]:
```
len(erupt)
```

Out[35]:
```
272
```

In [36]:
```
sum(erupt)
```

Out[36]:
```
948.677
```

In [37]:
```
m = sum(erupt)/len(erupt)
```

In [38]:
```
m
```

Out[38]:
```
3.487783088235294
```

In [39]:
```
np.mean(erupt)
```

Out[39]: np.float64(3.4877830882352936)

In [40]:
```
df.eruptions.mean()
```

Out[40]: np.float64(3.4877830882352936)

In [41]:
```
x1 = np.sort(erupt)
```

In [42]:
```
x1[135]
```

Out[42]: np.float64(4.0)

In [43]:
```
x1[136]
```

Out[43]: np.float64(4.0)

In [44]:
```
(x1[135] + x1[136])/2
```

Out[44]: np.float64(4.0)

In [45]:
```
np.median(erupt)
```

Out[45]: np.float64(4.0)

In [46]:
```
students = [20,35,15]
total_marks = [8,9,10]
```

In [47]:
```
nums = np.array([20,35,15])
marks = np.array([8,9,10])
```

In [48]:
```
sum(nums*marks)/sum(nums)
```

Out[48]: np.float64(8.928571428571429)

In [49]:
```
len(np.where(erupt==1.867)[0])
```

Out[49]: 8

In [50]:
```
np.where(erupt==1.867)
```

Out[50]: (array([36, 92, 98, 105, 130, 203, 212, 220]),)

don't calcualate mean if the values are highly volatile, calculate median instead median can be calculated at any situation

calculation graphs should prefer higher number of records like salary
[1,2,3,4,3,1,2,90,91] -> so the graph should be favoring the less salary people in viz.

- mode = most frequent element

per dataset there could be only 1 mean and 1 median, but modes could be more (bimodal, multimodal)

In [51]:
```
np.ptp(erupt)                                    # calculate range inbuild
```

Out[51]:
```
np.float64(3.4999999999999996)
```

In [ ]:
```
max(erupt)-min(erupt)                            # calculate range manual

# note - range shouldn't be used if the dataset is with extreme values
```

Out[ ]:
```
3.4999999999999996
```

In [53]:
```
# there should be consistency while using the range, lots of organizations hav

x1 = [4,5,6,3,4,2,3,4,4,5,1,9] # -> range is 8
x2 = [6,6,6,6,6,6,6,6,6,6,1,9] # -> range is 8


# here in both cases the range is 8 but the x2 is more consistent
```

In [54]:
```
np.percentile(erupt,40)
```

Out[54]:
```
np.float64(3.6)
```

In [55]:
```
np.percentile(erupt,40)
np.percentile(erupt,80)
```

Out[55]:
```
np.float64(4.533)
```

In [56]:
```
max(erupt)
```

Out[56]:
```
5.1
```

manual vs inbuilt method to calculate percentile, and understand percentile

In [57]:
```
t1=[1,1,2,2,3,3,3,3,4,4,4,4,4,4,5,5,5,5,5,6,6,6,6,6,7,7,7,7,7,7]
```

In [58]:
```
1 + (len(t1)-1) * 0.25
```

Out[58]:
```
8.25
```

In [59]:
```
3 * 0.25 + 4 * 0.75
```

Out[59]:
```
3.75
```

In [60]:
```
np.percentile(t1,25)
```

Out[60]:
```
np.float64(3.25)
```

In [61]:
```
3 * 0.75 + 4* 0.25
```

Out[61]:
```
3.25
```

In [62]:
```
1 + (len(t1)-1) * 0.7
```

Out[62]:
```
21.299999999999997
```

In [63]:
```
t1[20]
```

Out[63]:
```
6
```

In [64]:
```
t1[21]
```

Out[64]:
```
6
```

In [65]:
```
1 + (len(t1) + 1) * 0.8
```

Out[65]:
```
25.8
```

In [66]:
```
t1[23]
```

Out[66]:
```
6
```

In [67]:
```
t1[24]
```

Out[67]:
```
7
```

In [68]:
```
df.head()
```

Out[68]:

	eruptions	waiting
0	3.600	79
1	1.800	54
2	3.333	74
3	2.283	62
4	4.533	85

In [69]:
```
df.describe()
```

Out[69]:

	eruptions	waiting
count	272.000000	272.000000
mean	3.487783	70.897059
std	1.141371	13.594974
min	1.600000	43.000000
25%	2.162750	58.000000
50%	4.000000	76.000000
75%	4.454250	82.000000
max	5.100000	96.000000

In [70]:
```
np.percentile(erupt, 75) - np.percentile(erupt, 25)
```

Out[70]:
```
np.float64(2.2915)
```

In [71]:
```
np.percentile(x2, 75) - np.percentile(x2, 25)
```

Out[71]:
```
np.float64(0.0)
```

In [72]:
```
x2
```

Out[72]:
```
[6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 1, 9]
```

In [73]:
```
np.var(erupt)        #population variance
```

Out[73]:
```
np.float64(1.2979388904492861)
```

In [74]:
```
np.var(erupt, ddof=1)    # sample variance
```

Out[74]:
```
np.float64(1.302728332849468)
```

In [75]:
```
np.std(erupt)             # population standard variance
```

Out[75]:
```
np.float64(1.139271210225768)
```

In [76]:
```
np.std(erupt, ddof=1)      # sample standard deviation
```

Out[76]:
```
np.float64(1.141371251105208)
```

In [77]:
```
len(erupt - np.mean(erupt))
```

Out[77]:
```
272
```

In [78]:
```
sum(erupt - np.mean(erupt))
```

Out[78]: 1.4854784069484595e-13

In [79]:
```
ss = sum((erupt - np.mean(erupt))**2)  # sum of squared deviation from mean
```

In [80]:
```
ss/len(erupt)
```

Out[80]: 1.2979388904492863

In [81]:
```
np.var(erupt)
```

Out[81]: np.float64(1.2979388904492861)

In [82]:
```
(ss/len(erupt) ** 0.5)
```

Out[82]: 21.406156563677587

In [83]:
```
np.std(erupt)
```

Out[83]: np.float64(1.139271210225768)

In [84]:
```
ss/(len(erupt)-1)
```

Out[84]: 1.3027283328494683

In [85]:
```
np.var(erupt, ddof=1)
```

Out[85]: np.float64(1.302728332849468)

In [86]:
```
ss/(len(erupt)-1) ** 0.5
```

Out[86]: 21.445614950277655

In [96]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

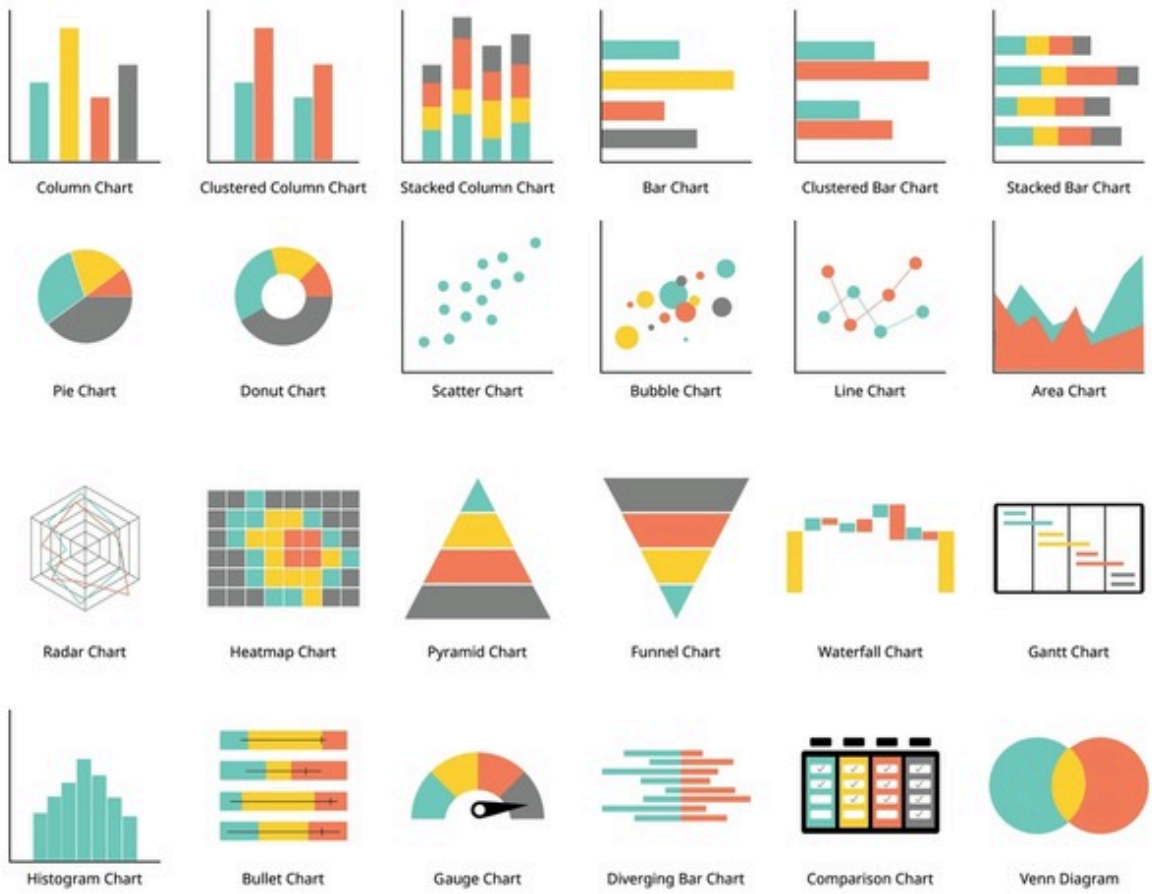
In [2]:
```
import scipy
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib
from matplotlib import pyplot as plt
import os
```

In [3]:
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

different charts and usecases

Chart Type	What It Shows	When to Use	Example Use-Case
Line Chart	Trends over time	Time-series data	Stock prices, sales over months
Bar Chart	Comparing categories	Categorical comparisons	Sales by product, revenue by region
Histogram	Distribution of numeric data	Checking shape (normal, skewed)	Age distribution, exam scores
Box Plot	Spread, median, outliers	Comparing distributions across groups	Salaries by department
Scatter Plot	Relationship between 2 variables	Correlation analysis	Height vs weight
Pie Chart	Percentage share	Simple proportion comparisons	Market share of companies
Heatmap	Intensity / correlation matrix	Pattern visualization	Correlation between features
Count Plot	Frequency of	Category counts	Number of students per

Chart Type	What It Shows	When to Use	Example Use-Case
	categories		grade
Violin Plot	Distribution + density	Understanding spread & shape	Comparing income groups
Area Chart	Cumulative trend	Stacked data over time	Sales distribution by category
Pair Plot	Multiple scatter plots together	Multivariate analysis	EDA with many features
Bubble Chart	Scatter + size dimension	3-variable comparison	GDP vs population vs CO_2
Barh Chart	Horizontal comparison	Long category names	Expenses by category
Stem Plot	Discrete data points	Small numeric sequences	Signal processing, sequences
Radar Chart	Multi-metric comparison	Profile of attributes	Comparing skill scores

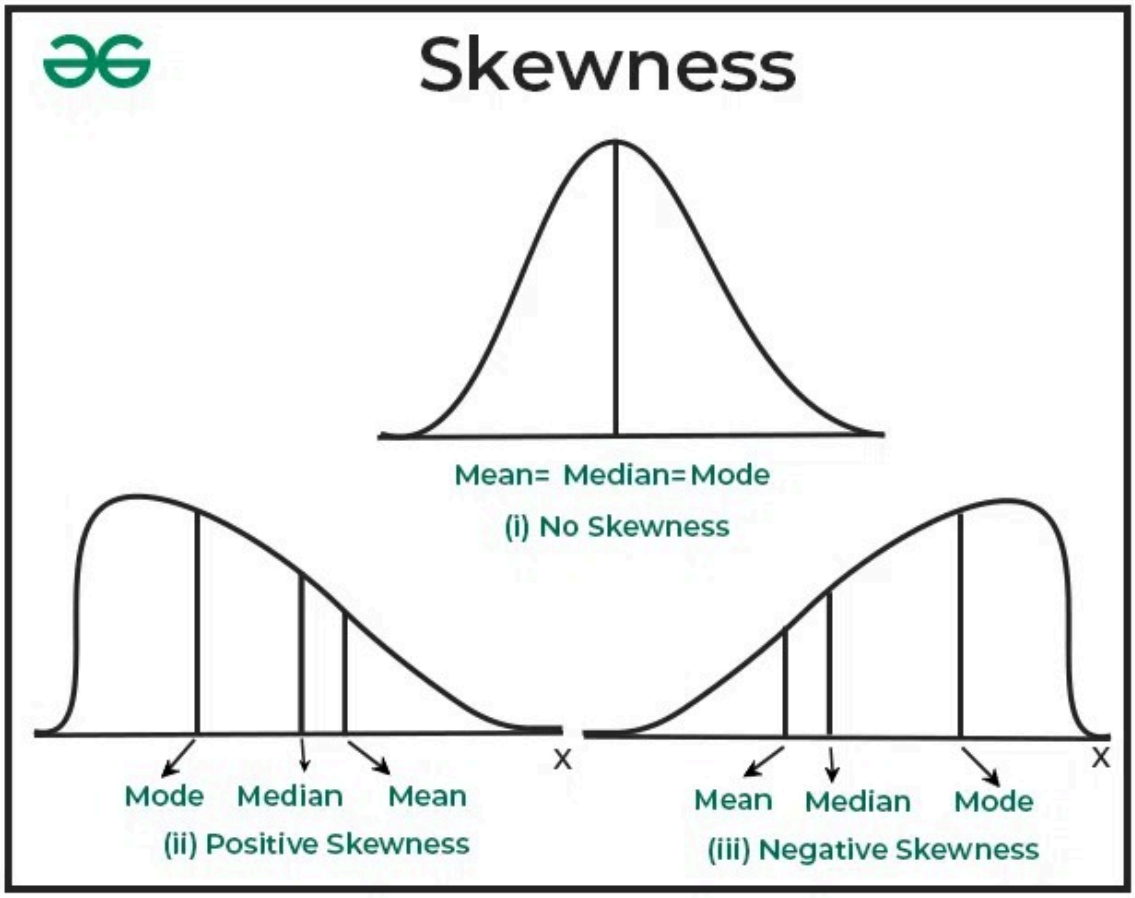


In [92]:
```
# * SKEWNESS *
# RIGHT SIDE SKEW(PEAK ON THE LEFT)
x1 = np.repeat([1,2,3,4,5,6,7,8,9,10],[5,10,150,140,120,100,80,60,40,20])
```

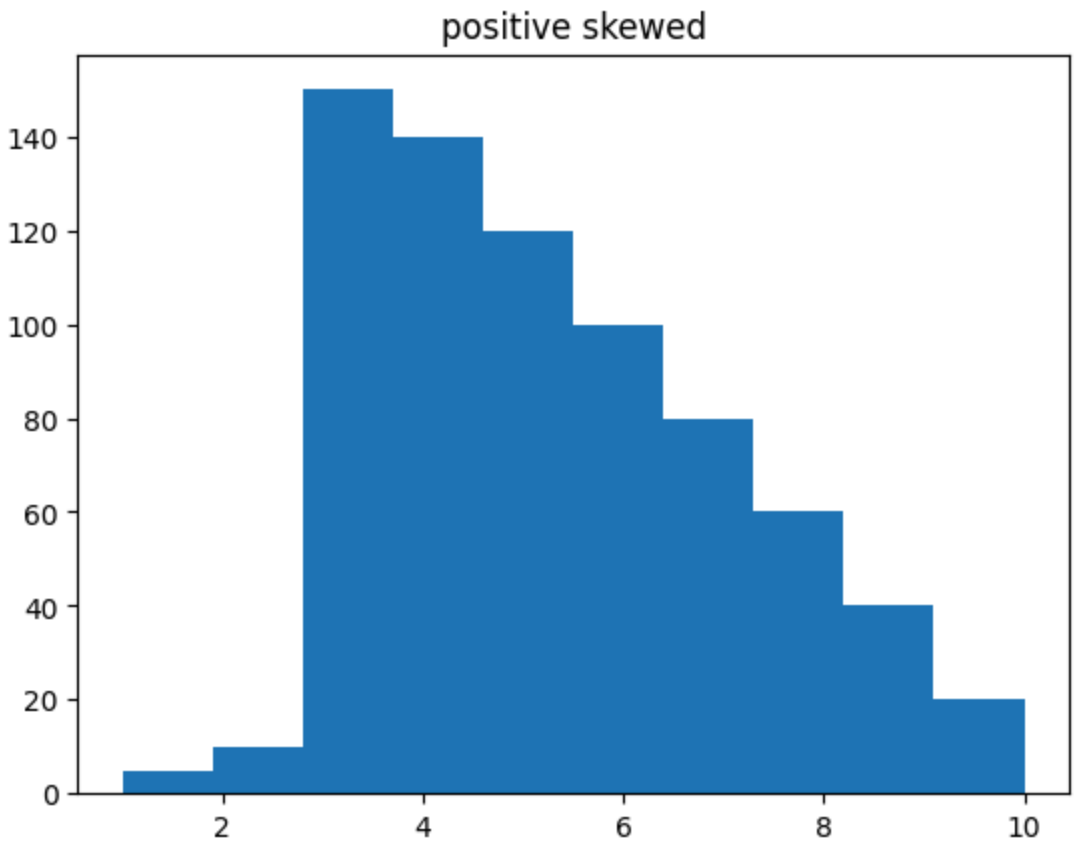
In [95]:
```
scipy.stats.skew(x1)

# 1. no skewness = 0
# 2. positive skewness > 0
# 3. negative skewness < 0
```

Out[95]:
```
np.float64(0.48916996679169206)
```

skewness value tell us how much the peak is away from the centre



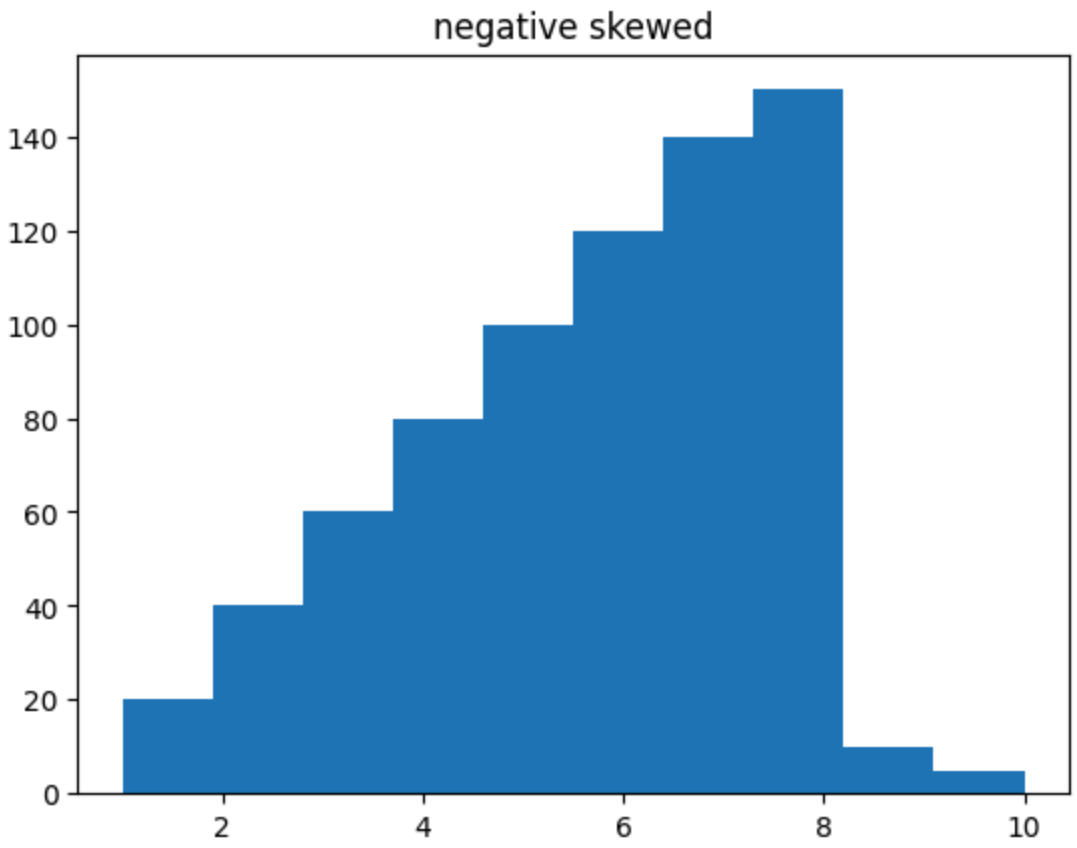
In [98]:
```
plt.hist(x1)
plt.title('positive skewed')
plt.show()
```



In [6]:
```
scipy.stats.skew(x1)
```

Out[6]:
```
np.float64(0.48916996679169206)
```

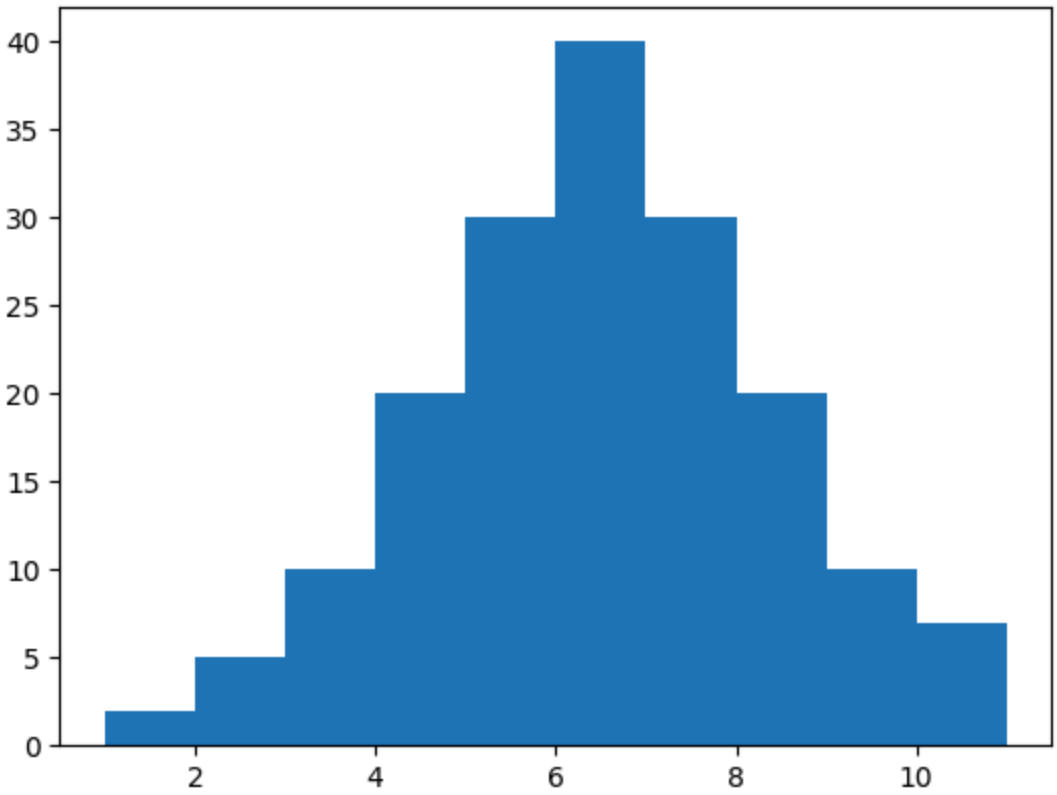
In [101…
```
# LEFT SIDE SKEW(PEAK ON THE RIGHT)
x2 = np.repeat([1,2,3,4,5,6,7,8,9,10],[20,40,60,80,100,120,140,150,10,5])

plt.hist(x2)
plt.title('negative skewed')
plt.show()
```



In [8]:
```
scipy.stats.skew(x2)
# => np.float64(-0.48916996679169217)
```

Out[8]:
```
np.float64(-0.48916996679169217)
```

In [9]:
```
# PEAK AT THE CENTRE
x3 = np.repeat([1,2,3,4,5,6,7,8,9,10,11],[2,5,10,20,30,40,30,20,10,5,2])

plt.hist(x3)
plt.show()
```



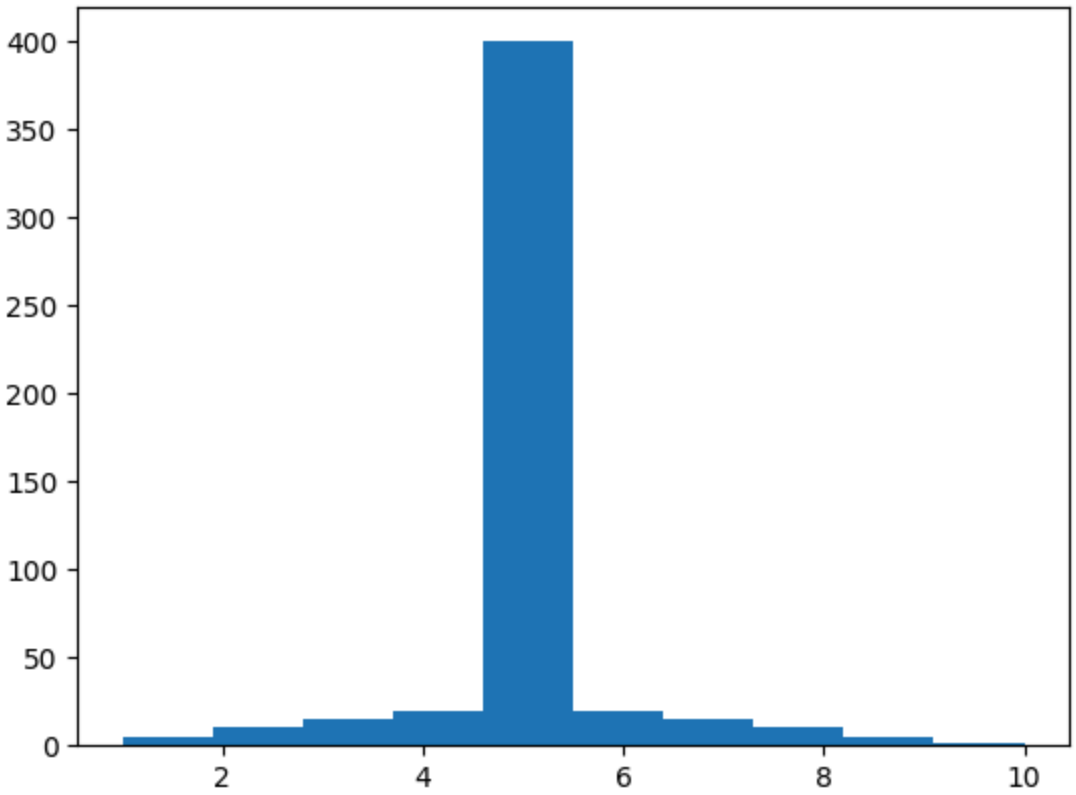
In [10]:
```
scipy.stats.skew(x3) # skewness = 0
```

Out[10]:
```
np.float64(0.0)
```

kurtosis - sharp peak

In [11]:
```
# * KURTOSIS *
# SHARP PEAK
x1 = np.repeat([1,2,3,4,5,6,7,8,9,10],[5,10,15,20,400,20,15,10,5,2])
```

In [12]:
```
plt.hist(x1)
plt.show()

scipy.stats.kurtosis(x1)
```

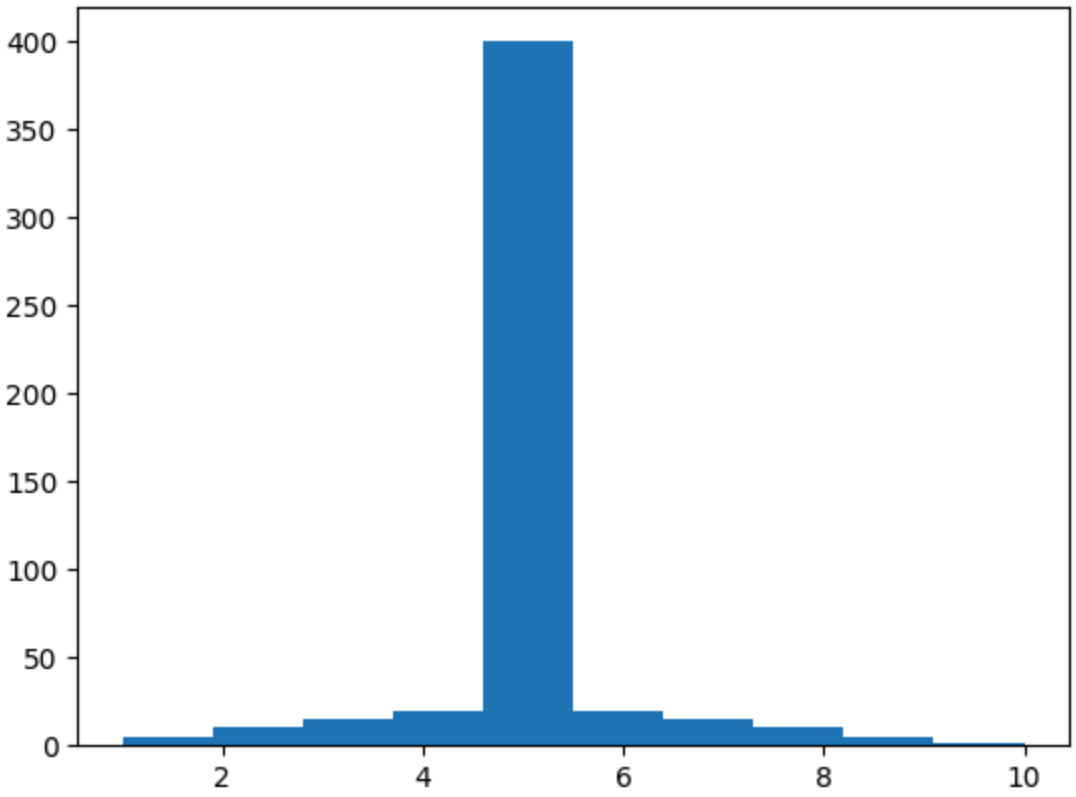


Out[12]:
```
np.float64(6.850298487712665)
```

In [13]:
```
x1 = np.repeat([1,2,3,4,5,6,7,8,9,10],[5,10,15,20,400,20,15,10,5,2])
```

In [14]:
```
plt.hist(x1) #sharp peak
```

Out[14]:
```
(array([  5.,  10.,  15.,  20., 400.,  20.,  15.,  10.,   5.,   2.]),
 array([ 1. ,  1.9,  2.8,  3.7,  4.6,  5.5,  6.4,  7.3,  8.2,  9.1, 10. ]),
 <BarContainer object of 10 artists>)
```



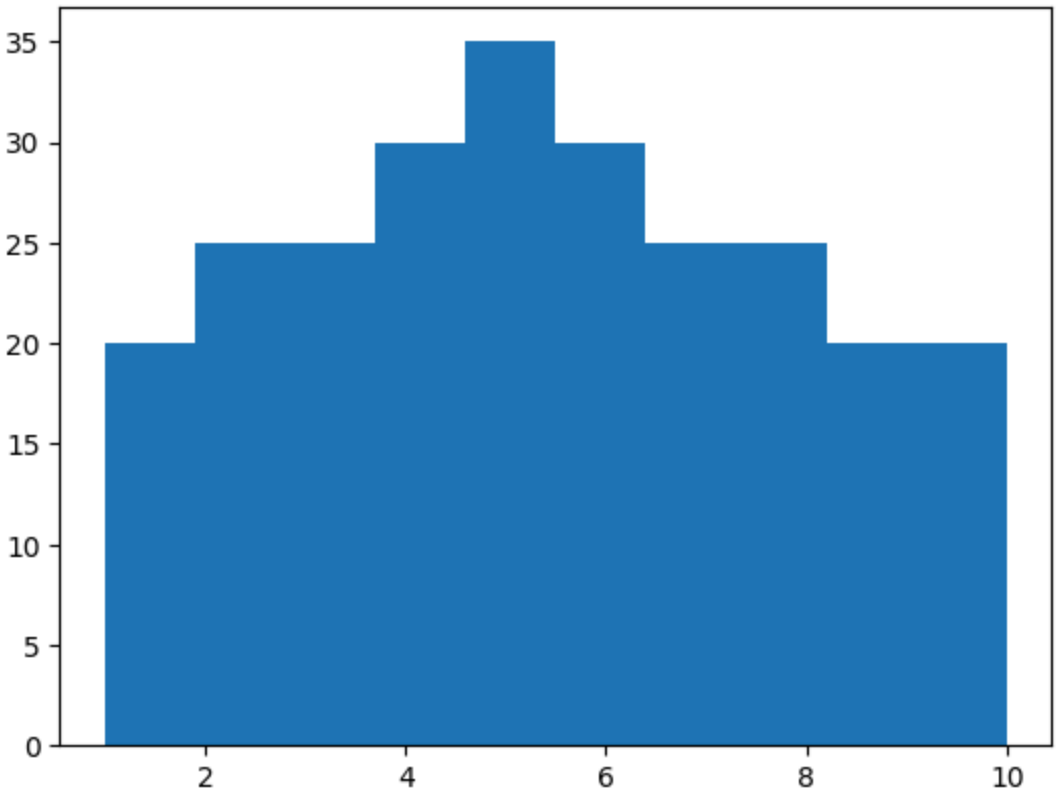
In [15]:
```
scipy.stats.kurtosis(x1)
```

Out[15]:
```
np.float64(6.850298487712665)
```

In [16]:
```
x2 = np.repeat([1,2,3,4,5,6,7,8,9,10],[20,25,25,30,35,30,25,25,20,20])
```

In [17]:
```
plt.hist(x2) #flat peak
```

Out[17]:
```
(array([20., 25., 25., 30., 35., 30., 25., 25., 20., 20.]),
 array([ 1. ,  1.9,  2.8,  3.7,  4.6,  5.5,  6.4,  7.3,  8.2,  9.1, 10. ]),
 <BarContainer object of 10 artists>)
```



In [18]:
```
scipy.stats.kurtosis(x2)
```

Out[18]:
```
np.float64(-1.0233152049490384)
```

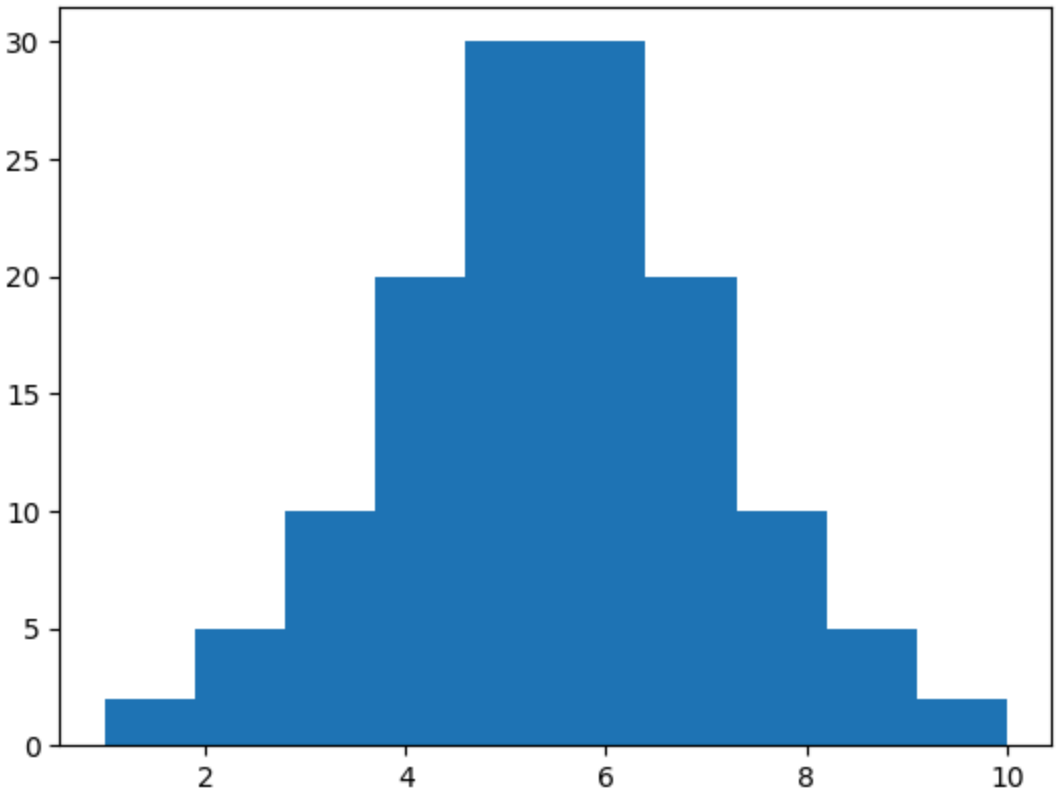
In [19]:
```
x3 = np.repeat([1,2,3,4,5,6,7,8,9,10],[2,5,10,20,30,30,20,10,5,2])
```

In [20]:
```
scipy.stats.kurtosis(x3)
```

Out[20]:
```
np.float64(-0.05621739841876128)
```

In [21]:
```
plt.hist(x3) #
```

Out[21]:
```
(array([ 2.,  5., 10., 20., 30., 30., 20., 10.,  5.,  2.]),
 array([ 1. ,  1.9,  2.8,  3.7,  4.6,  5.5,  6.4,  7.3,  8.2,  9.1, 10. ]),
 <BarContainer object of 10 artists>)
```



---

visualization

In [22]:
```
import numpy as np
from numpy import random
import pandas as pd
import scipy
from scipy import stats

import matplotlib
from matplotlib import pyplot as plt
import seaborn as sns
import pylab
from pylab import legend
from pylab import plot, show, title, xlabel, ylabel
```

---

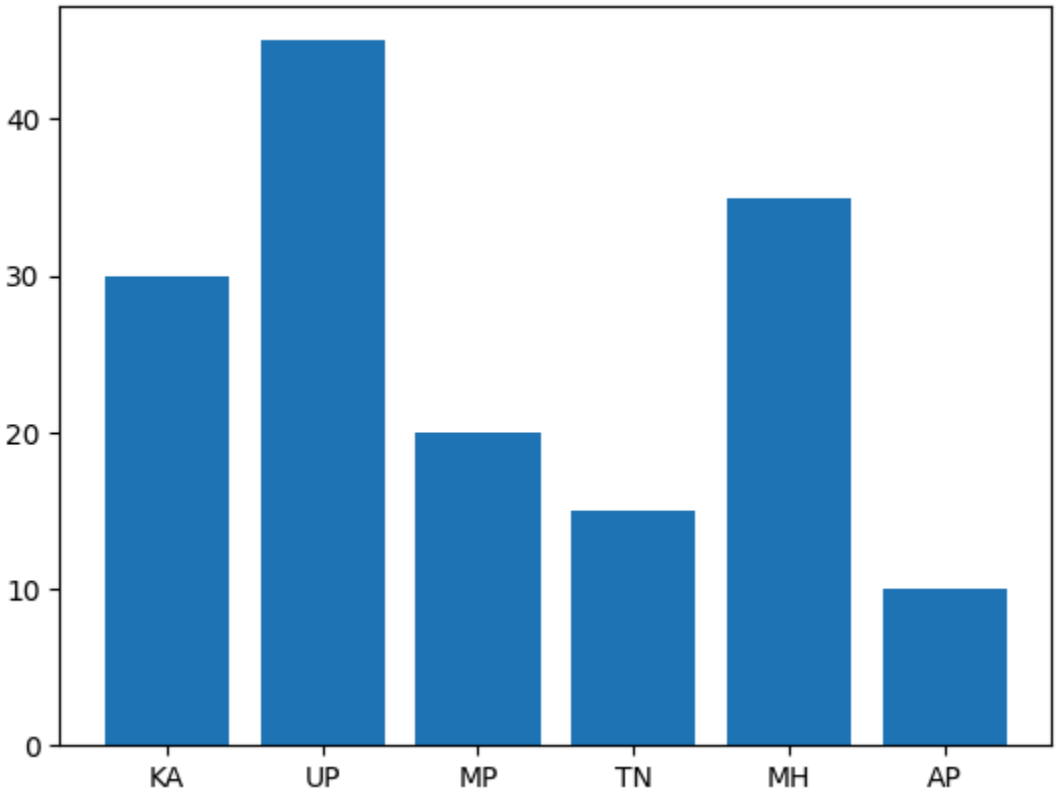
barcharts

we use when comparing the categories in data

In [23]:
```
states = ['KA', 'UP', 'MP', 'TN', 'MH', 'AP']
counts = [30,45,20,15,35,10]
```

In [24]:
```
plt.bar(states, counts)
```

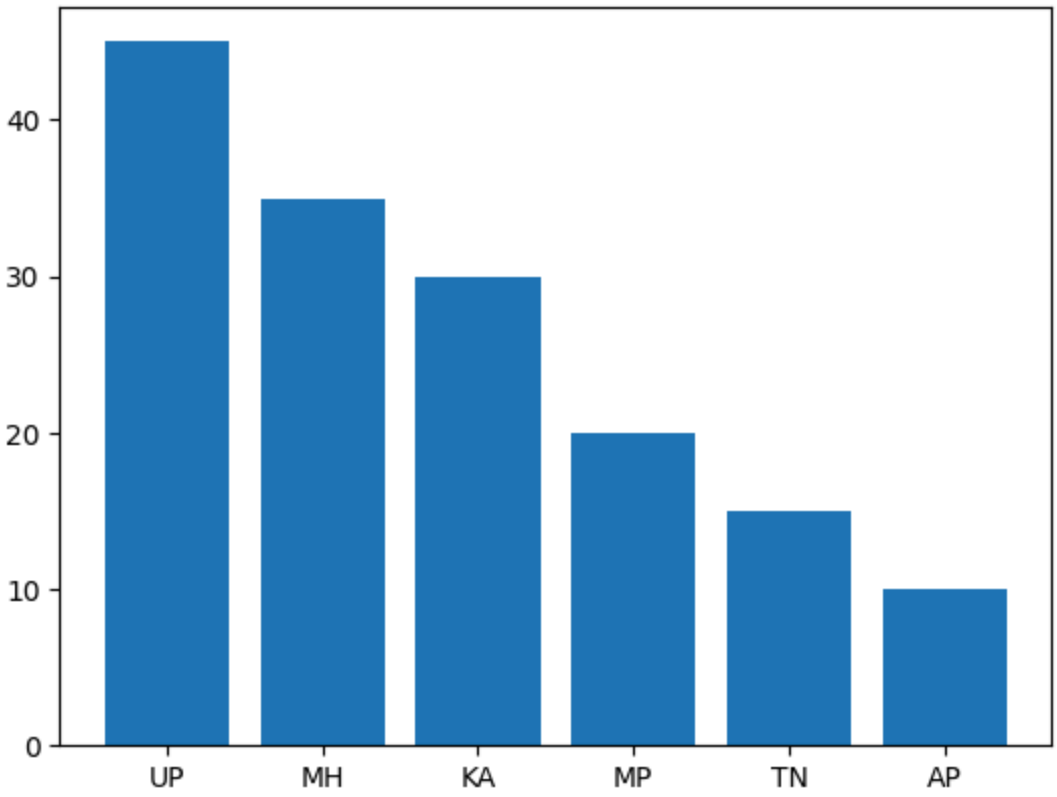
Out[24]: <BarContainer object of 6 artists>



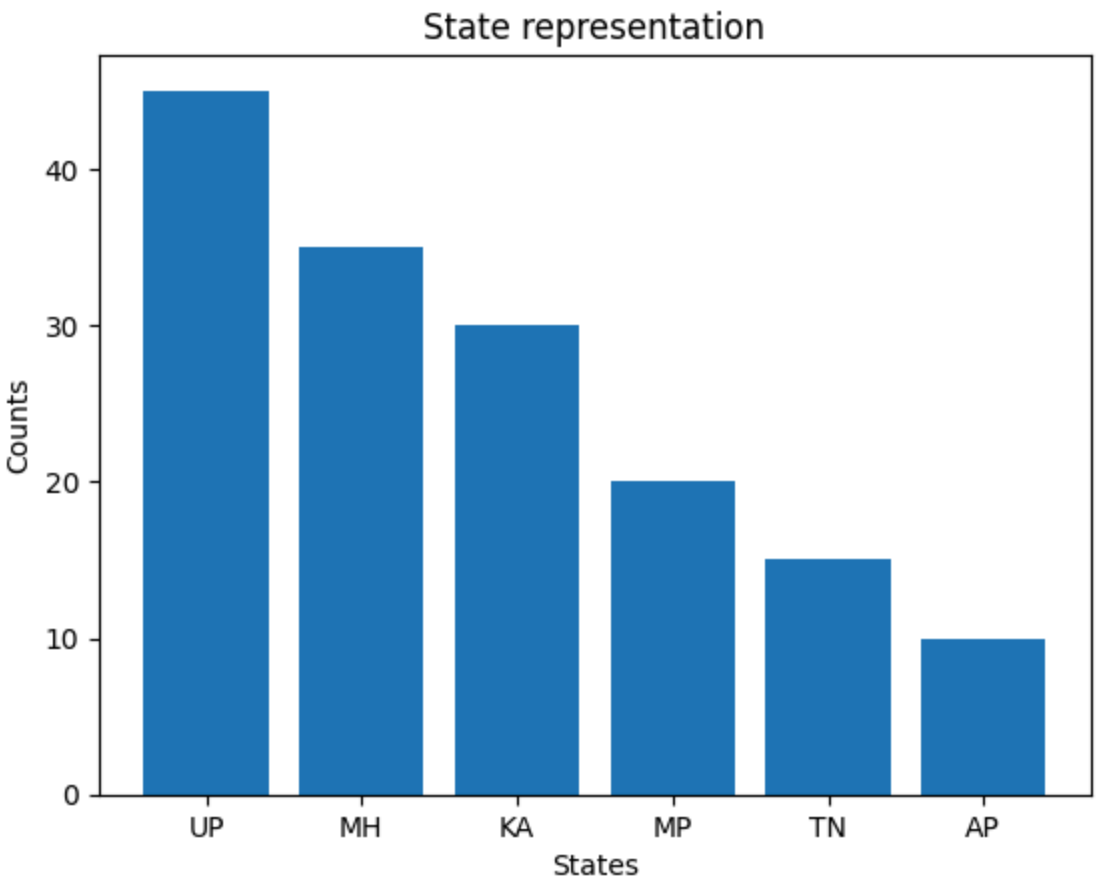
In [25]:
```
df = pd.DataFrame(zip(states,counts), columns=['State', 'Count'])
df = df.sort_values(['Count'], ascending=False)
```

In [26]:
```
plt.bar(df.State, df.Count)
```

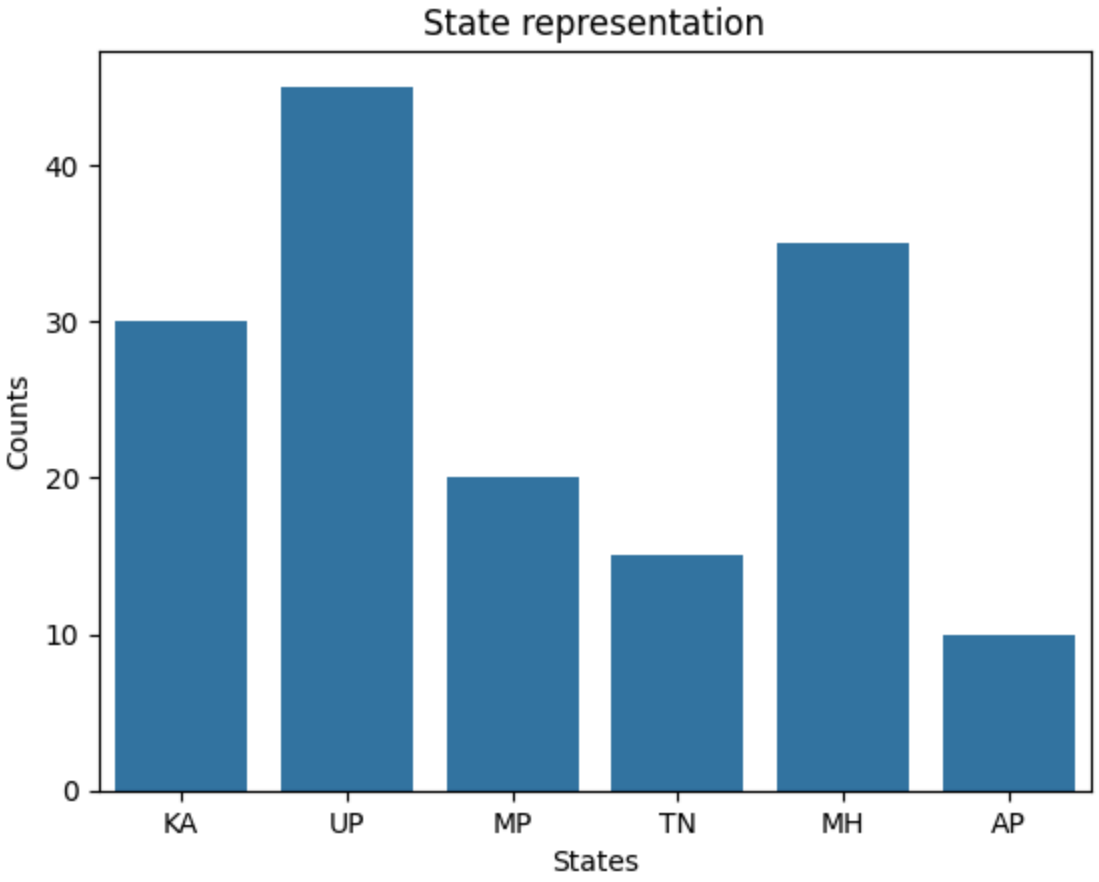
Out[26]: <BarContainer object of 6 artists>



In [27]:
```
plt.bar(df['State'], df['Count'], width=0.8)
plt.title('State representation')
plt.xlabel('States')
plt.ylabel('Counts')
plt.show()
```



In [28]:
```
sns.barplot(x=states, y=counts)
plt.title('State representation')
plt.xlabel('States')
plt.ylabel('Counts')
plt.show()
```



In [29]:
```
import os
os.getcwd()
os.chdir('/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-sta
```

In [30]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='ERPData')
```

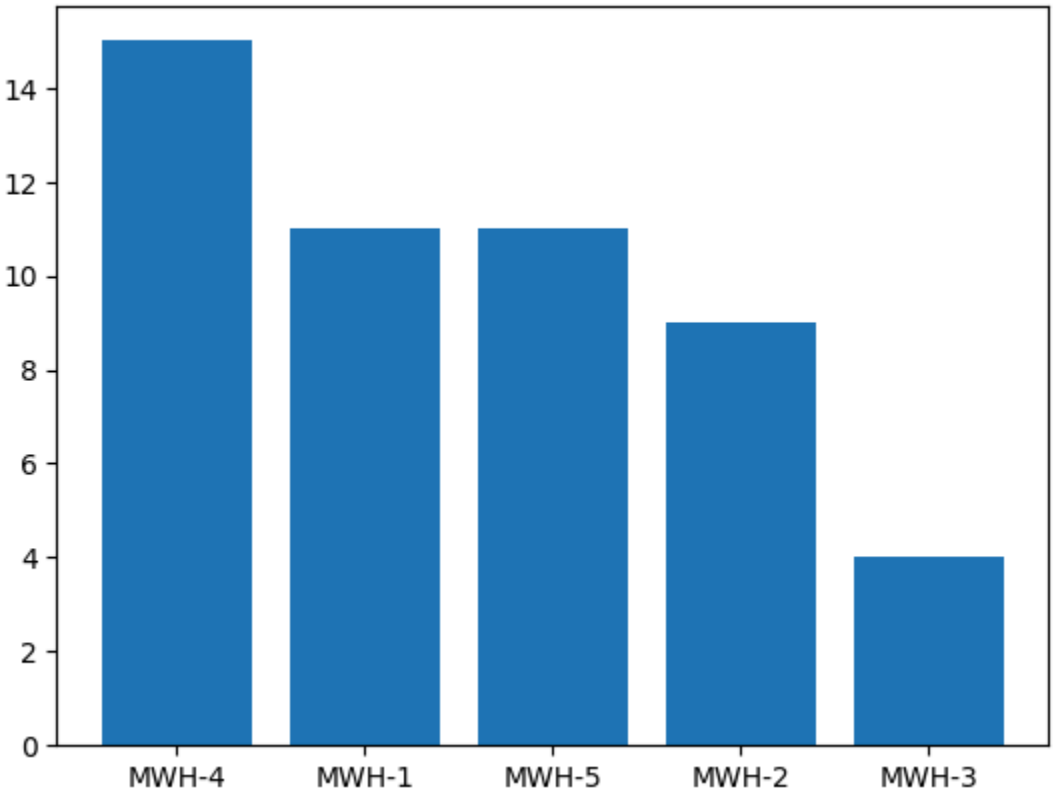
In [31]:
```
df.head()
```

Out[31]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
1	AXCP-78	MWH-1	67
2	LXCV-21	MWH-2	27
3	AXCP-78	MWH-5	65
4	AXCP-78	MWH-4	36

In [32]:
```
# This code counts the number of occurrences of each unique value in the 'Loca
# using value_counts(), which returns a Series with locations as the index and
# Then, plt.bar() creates a bar chart with the location names on the x-axis an
# Finally, plt.show() displays the plot.
```

```
s1 = df.Location.value_counts()
plt.bar(s1.index, s1.values)
plt.show()
```

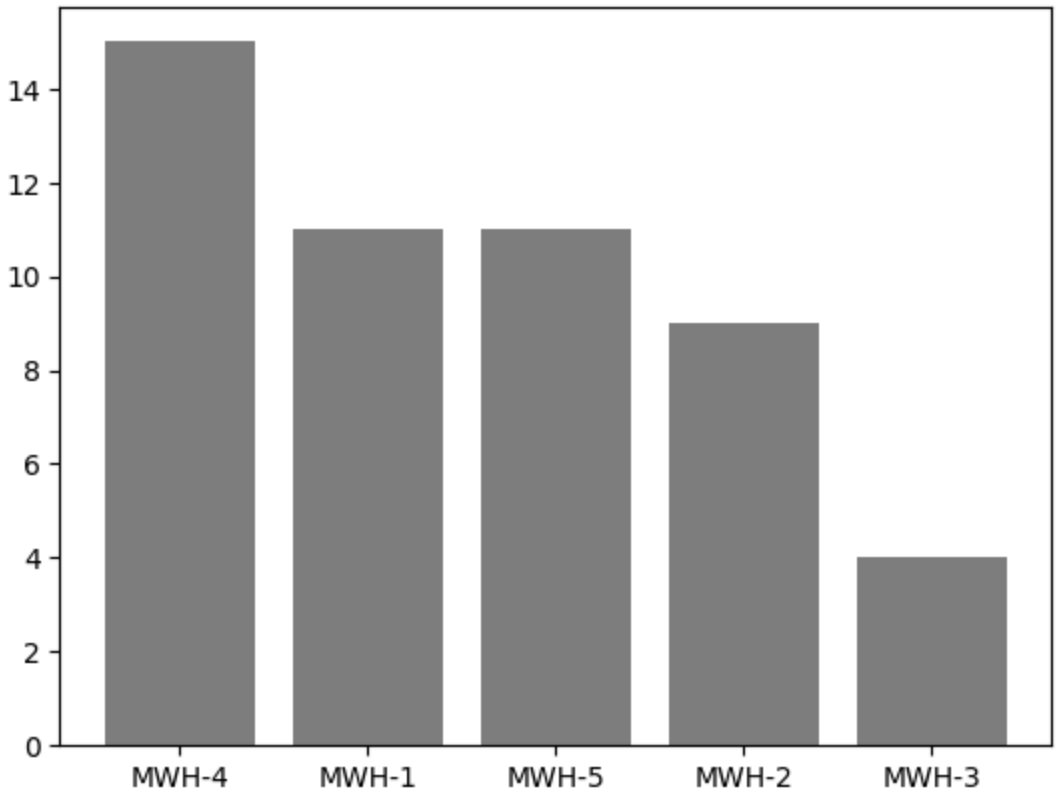


In [33]:
```
df1 = df.Location.value_counts().reset_index()
df1
```

Out[33]:

	Location	count
0	MWH-4	15
1	MWH-1	11
2	MWH-5	11
3	MWH-2	9
4	MWH-3	4

In [34]:
```
plt.bar(df1['Location'], df1['count'], color='grey')
plt.show()
```



In [35]:
```
df1
```

Out[35]:

	Location	count
0	MWH-4	15
1	MWH-1	11
2	MWH-5	11
3	MWH-2	9
4	MWH-3	4

In [36]:
```
min(df['Quantity'])
```

Out[36]: 10

In [ ]:
```
```

In [37]:
```
print(counts)
```

```
[30, 45, 20, 15, 35, 10]
```

In [38]:
```
df.head()
```

Out[38]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
1	AXCP-78	MWH-1	67
2	LXCV-21	MWH-2	27
3	AXCP-78	MWH-5	65
4	AXCP-78	MWH-4	36

In [39]:
```
df.head()
```

Out[39]:

	MaterialID	Location	Quantity
0	TMI-43T	MWH-4	34
1	AXCP-78	MWH-1	67
2	LXCV-21	MWH-2	27
3	AXCP-78	MWH-5	65
4	AXCP-78	MWH-4	36

In [40]:
```
b1 = [9,40,80,120,160]
p1 = ['A','B','C','D']
```

In [41]:
```
df['cat'] = pd.cut(df.Quantity, bins=b1, labels=p1)
```

In [42]:
```
df.head()
```

Out[42]:

	MaterialID	Location	Quantity	cat
0	TMI-43T	MWH-4	34	A
1	AXCP-78	MWH-1	67	B
2	LXCV-21	MWH-2	27	A
3	AXCP-78	MWH-5	65	B
4	AXCP-78	MWH-4	36	A

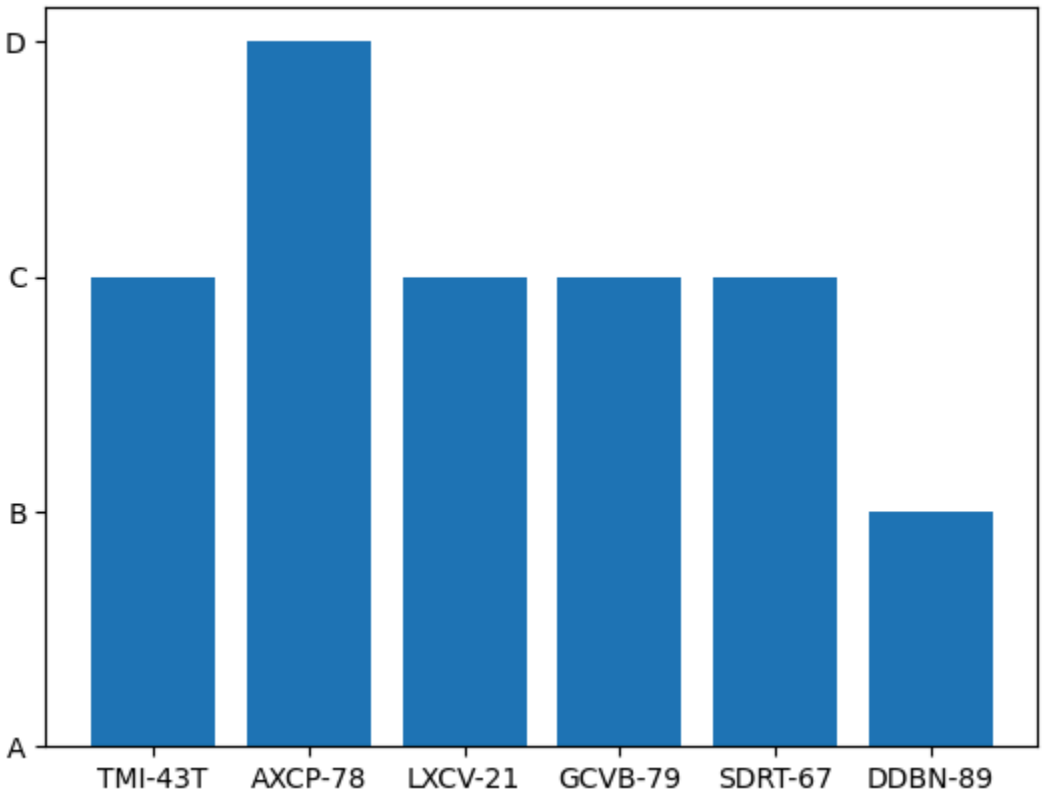
In [43]:
```
df.head()
```

Out[43]:

	MaterialID	Location	Quantity	cat
0	TMI-43T	MWH-4	34	A
1	AXCP-78	MWH-1	67	B
2	LXCV-21	MWH-2	27	A
3	AXCP-78	MWH-5	65	B
4	AXCP-78	MWH-4	36	A

In [44]:
```
plt.bar(df['MaterialID'],df['cat'])
plt.show
```

Out[44]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



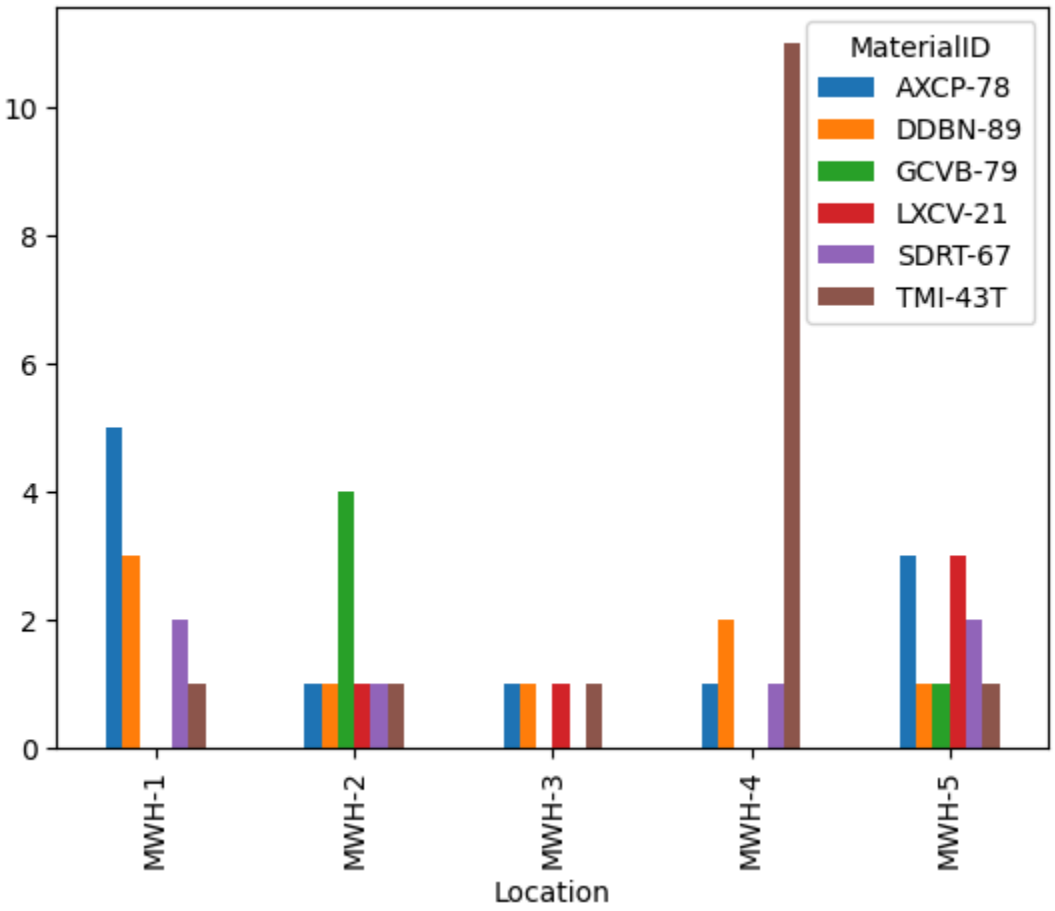
In [45]:
```
df2 = df.pivot_table(columns='MaterialID', index='Location',values='Quantity',
```

In [46]:
```
df2
```

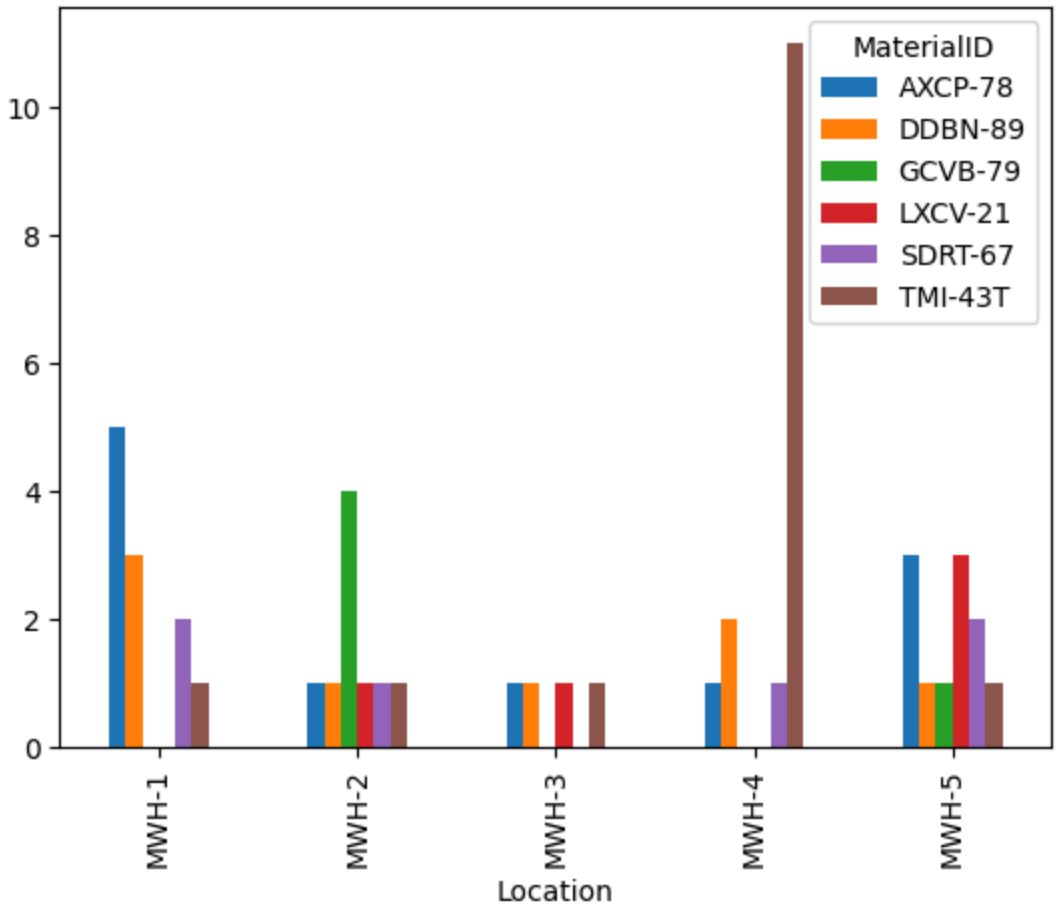
Out[46]:

MaterialID	AXCP-78	DDBN-89	GCVB-79	LXCV-21	SDRT-67	TMI-43T
Location						
MWH-1	5.0	3.0	NaN	NaN	2.0	1.0
MWH-2	1.0	1.0	4.0	1.0	1.0	1.0
MWH-3	1.0	1.0	NaN	1.0	NaN	1.0
MWH-4	1.0	2.0	NaN	NaN	1.0	11.0
MWH-5	3.0	1.0	1.0	3.0	2.0	1.0

In [47]:
```
# auto plot with multiple labels without specifying x and y

df2.plot(kind = 'bar')
plt.show()
```

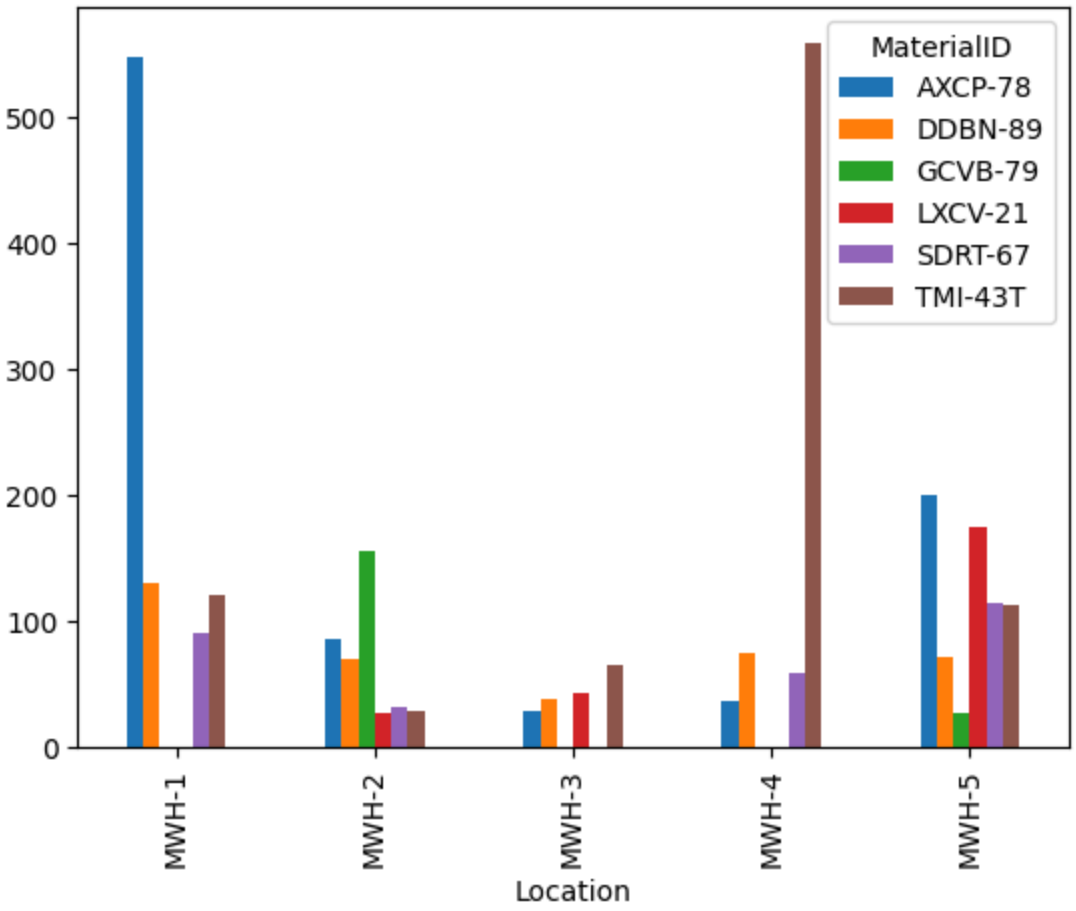


In [48]:
```
# or the otherway around without using pivot table

pd.crosstab(df.Location, df.MaterialID).plot(kind='bar')
plt.show()
```

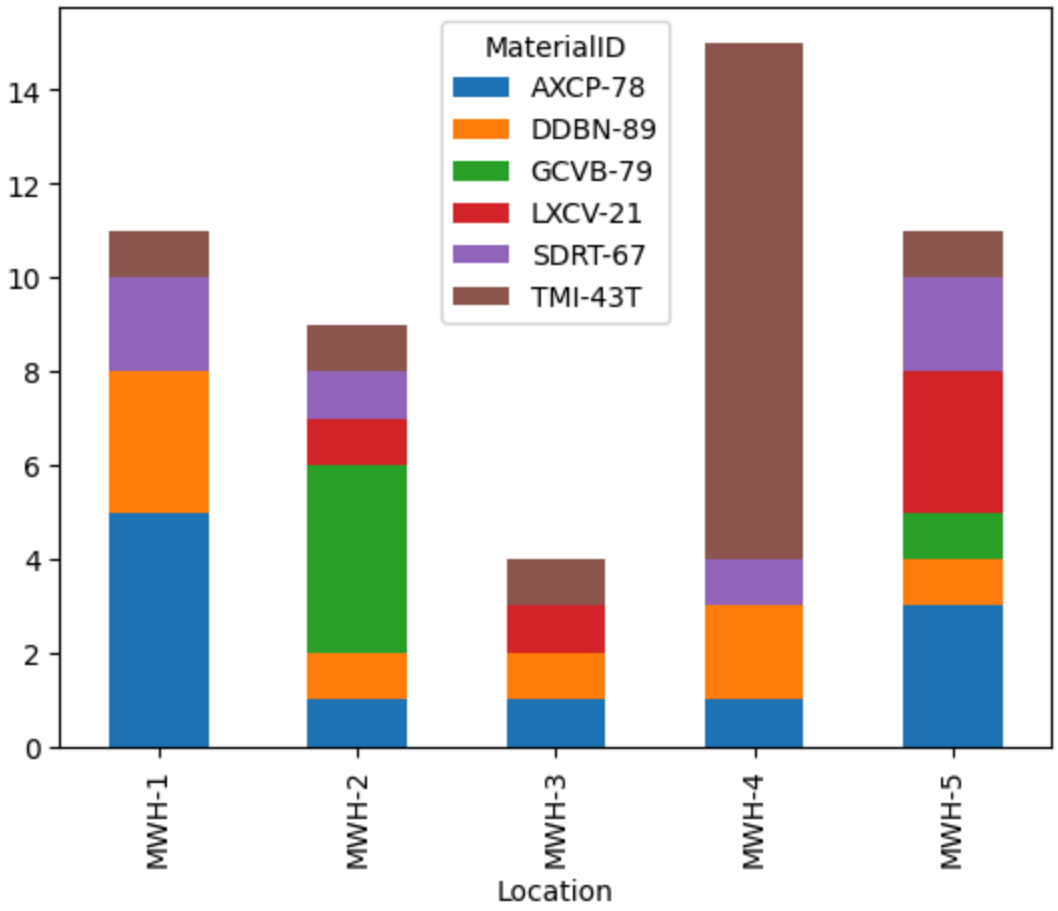


In [49]:
```
pd.crosstab(df.Location, df.MaterialID, values=df.Quantity, aggfunc=np.sum).pl
plt.show()
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_32637/1166014858.p
y:1: FutureWarning: The provided callable <function sum at 0x104df9ee0> is curr
ently using DataFrameGroupBy.sum. In a future version of pandas, the provided c
allable will be used directly. To keep current behavior pass the string "sum" i
nstead.
  pd.crosstab(df.Location, df.MaterialID, values=df.Quantity, aggfunc=np.sum).p
lot(kind='bar')
```

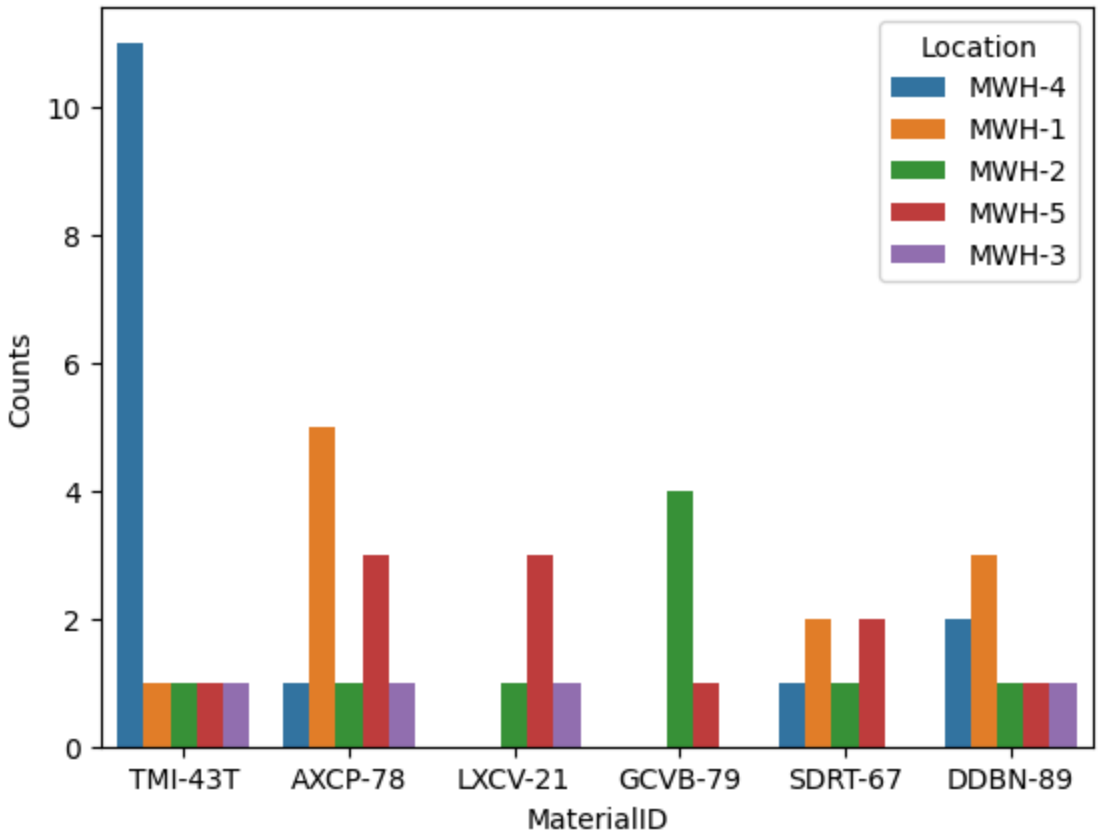


In [50]:
```
pd.crosstab(df.Location, df.MaterialID).plot(kind='bar', stacked=True)
plt.show()
```



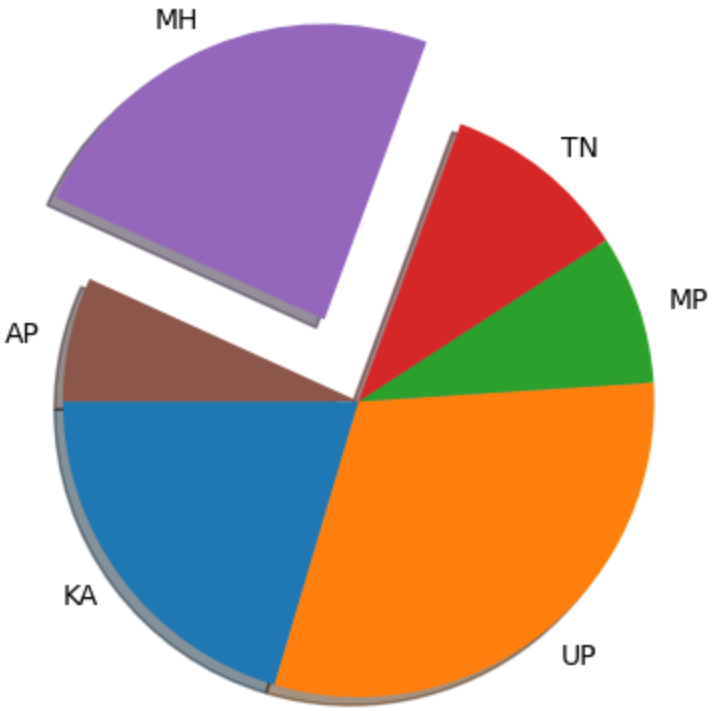
In [51]:
```
sns.barplot(x='MaterialID', y='Quantity', hue='Location', data=df, ci=None, es
plt.ylabel('Counts')
plt.show()

# ci = confidence interval
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_32637/3058818635.p
y:1: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

  sns.barplot(x='MaterialID', y='Quantity', hue='Location', data=df, ci=None, e
stimator=np.size)
```



In [52]:
```
states = ['KA', 'UP', 'MP', 'TN', 'MH', 'AP']
counts = [30,45,12,15,35,10]
```

In [53]:
```
explode_values = [0,0,0.,0,0.3,0]
plt.pie(x=counts, labels=states, explode =explode_values, shadow=True, startar
plt.show()



# startangle = rotate the graph
```



In [54]:
```
df = pd.DataFrame(zip(states,counts), columns=['State', 'Count'])
```

In [55]:
```
df
```

Out[55]:

	State	Count
0	KA	30
1	UP	45
2	MP	12
3	TN	15
4	MH	35
5	AP	10

In [56]:
```
df['angle'] = [i/sum(df.Count)*360 for i in df.Count]      # angle produced by
```

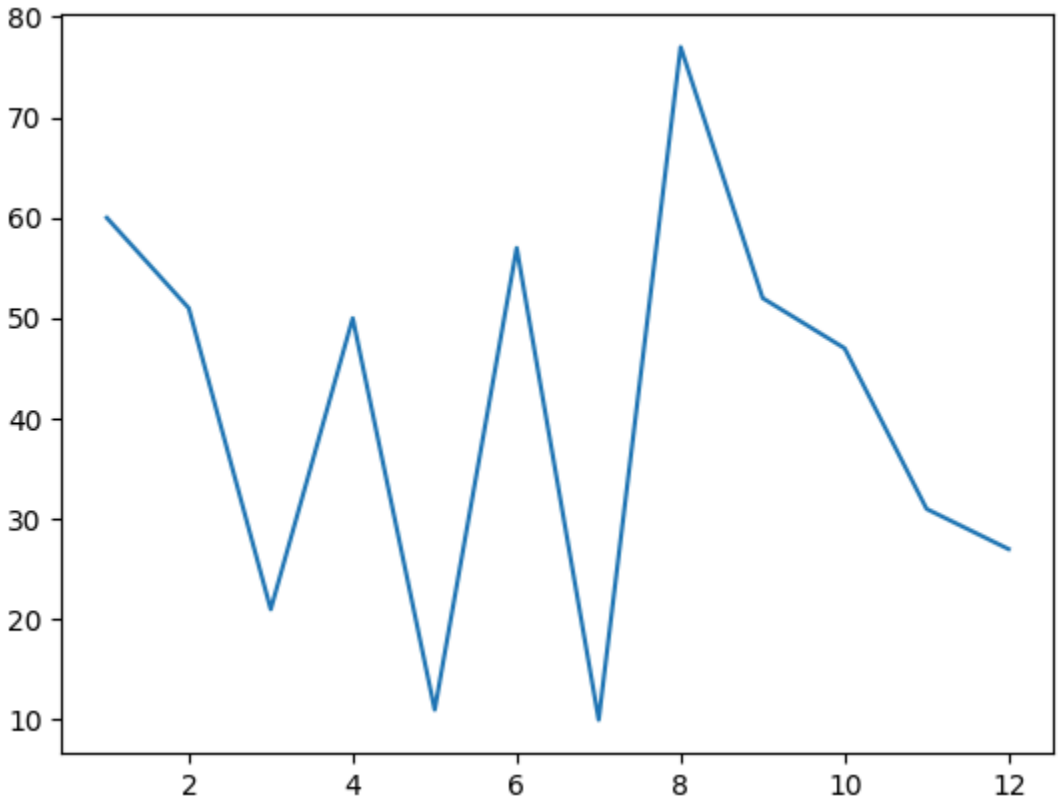
In [57]:
```
df.head()
```

Out[57]:

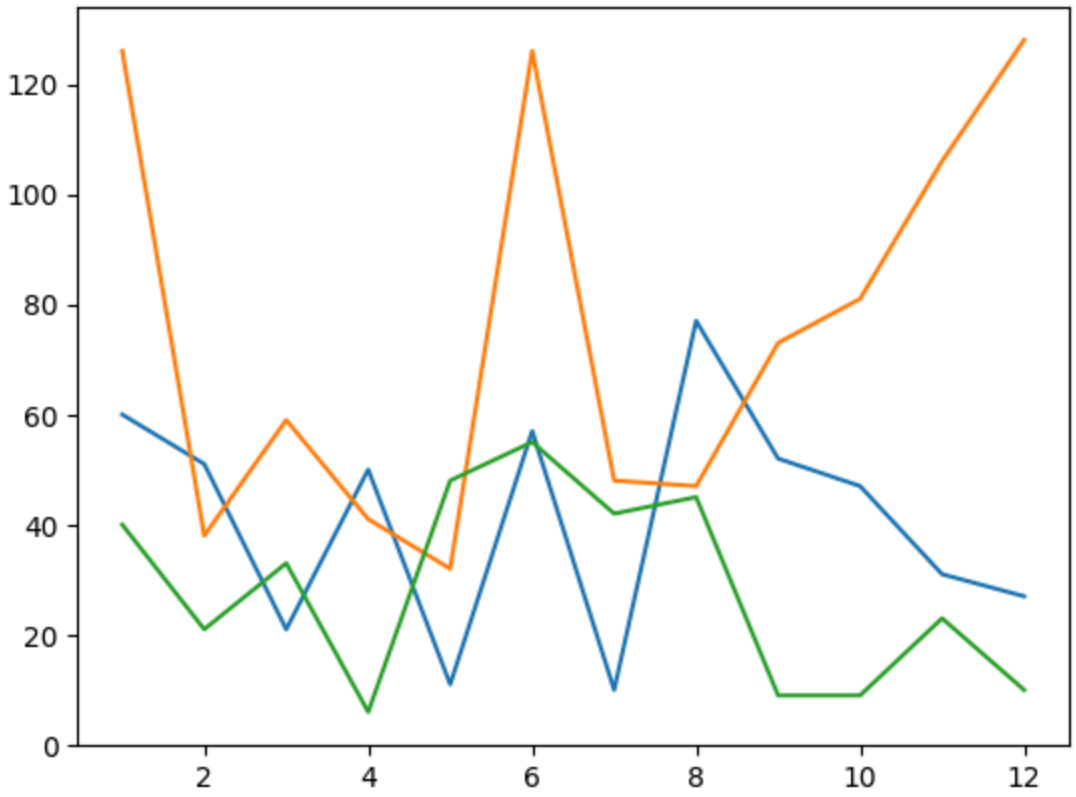
	State	Count	angle
0	KA	30	73.469388
1	UP	45	110.204082
2	MP	12	29.387755
3	TN	15	36.734694
4	MH	35	85.714286

In [58]:
```
x1 = np.random.randint(10,80,12)      # start: end: how many      -> end is ex
months = range(1,13)
plot(months, x1)


# NOTE
# plt.plot() → returns Line2D object        -> [<matplotlib.lines.Line2D at 0x
# plt.show() → actually displays the graph
```

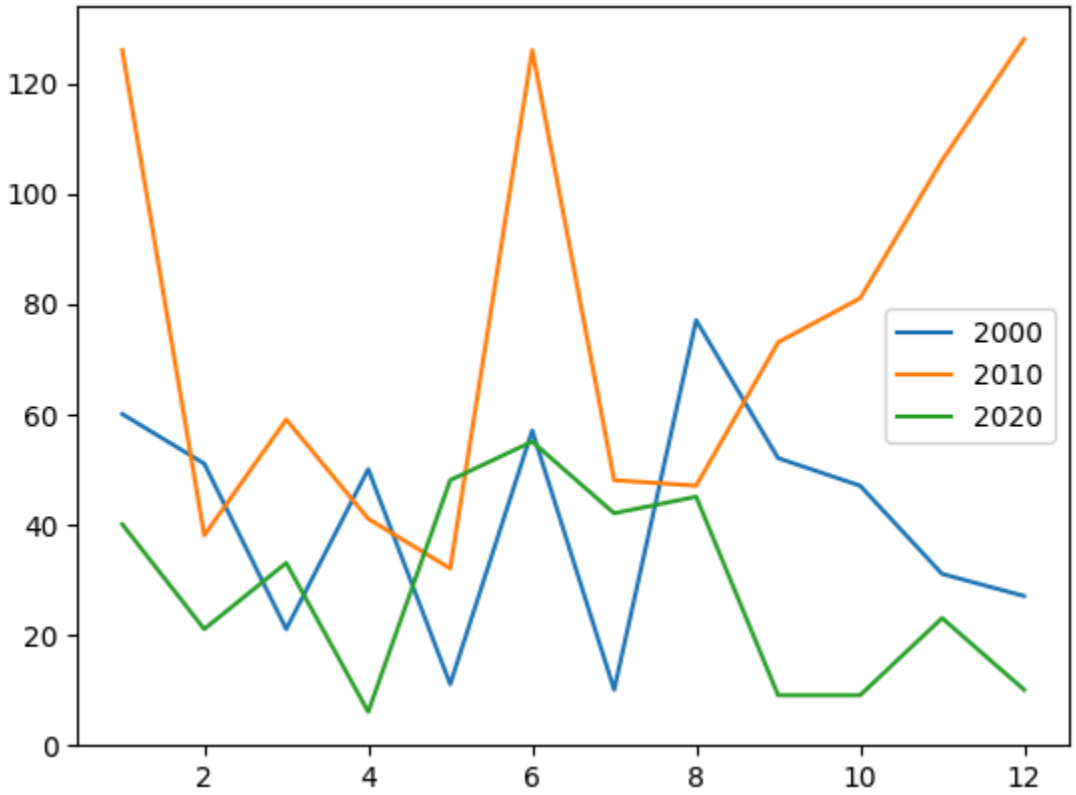
Out[58]:
```
[<matplotlib.lines.Line2D at 0x1520c0e10>]
```



In [59]:
```
x2 = np.random.randint(20,130,12)
x3 = np.random.randint(5,60,12)
plot(months, x1, months,x2, months, x3)
plt.show()
```



In [60]:
```
plot(months, x1, months, x2, months, x3)
plt.legend([2000,2010,2020])
plt.show()
```



In [61]:
```
df = sns.load_dataset('fmri')
```

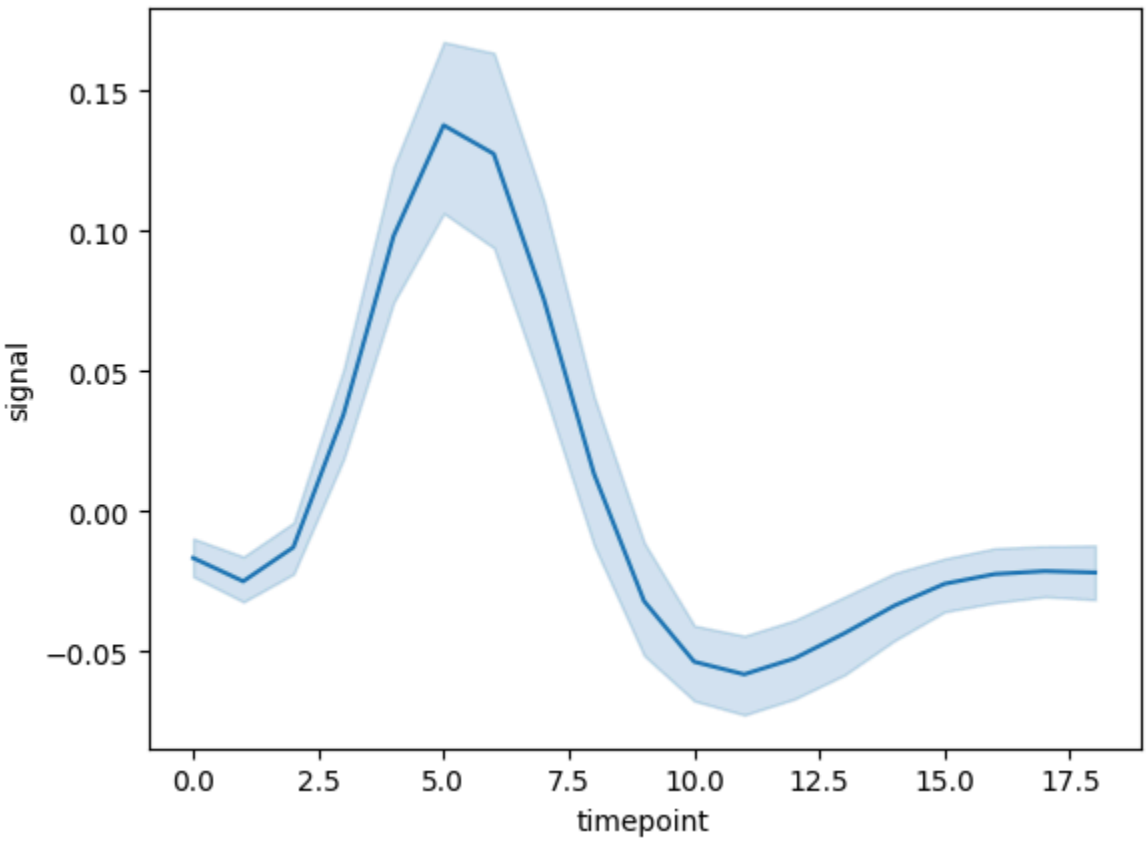
In [62]:
```
df.head()
```

Out[62]:

	subject	timepoint	event	region	signal
0	s13	18	stim	parietal	-0.017552
1	s5	14	stim	parietal	-0.080883
2	s12	18	stim	parietal	-0.081033
3	s11	18	stim	parietal	-0.046134
4	s10	18	stim	parietal	-0.037970

In [63]:
```
sns.lineplot(data=df, x='timepoint', y='signal')
```

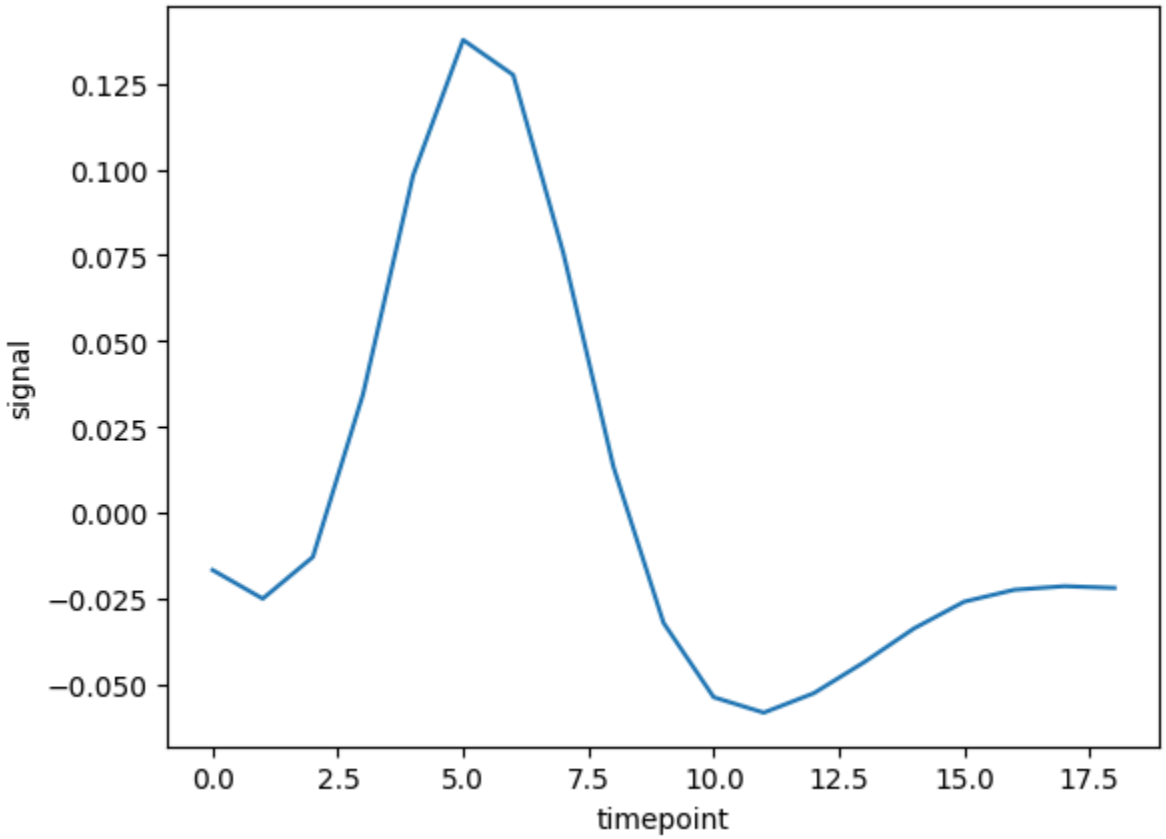
Out[63]:
```
<Axes: xlabel='timepoint', ylabel='signal'>
```



In [64]:
```
sns.lineplot(data=df, x='timepoint', y='signal', ci=None)


# ci (confidence interval) controls the shaded uncertainty region around the l
# By default, seaborn shows a 95% confidence interval.
# Setting ci=None removes this shading and plots only the main line for cleane
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_32637/2639816696.p
y:1: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

  sns.lineplot(data=df, x='timepoint', y='signal', ci=None)
```

Out[64]: <Axes: xlabel='timepoint', ylabel='signal'>



In [65]:
```
len(df)
```

Out[65]: 1064

In [66]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name = 'faithful')
```

In [67]:
```
df.head()
```

Out[67]:

	eruptions	waiting
0	3.600	79
1	1.800	54
2	3.333	74
3	2.283	62
4	4.533	85

In [68]:
```
len(df)
```

Out[68]:
```
272
```

In [69]:
```
erupt = df.eruptions
```

In [70]:
```
min(erupt)
```

Out[70]:
```
1.6
```

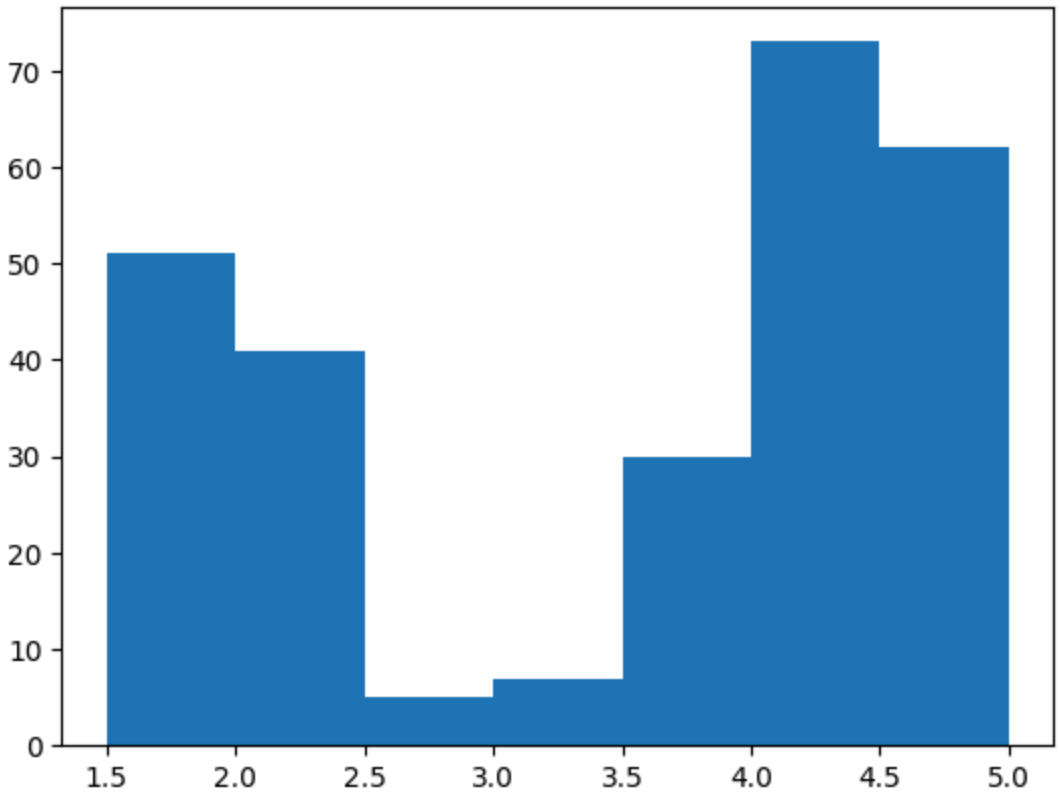
In [71]:
```
max(erupt)
```

Out[71]:
```
5.1
```

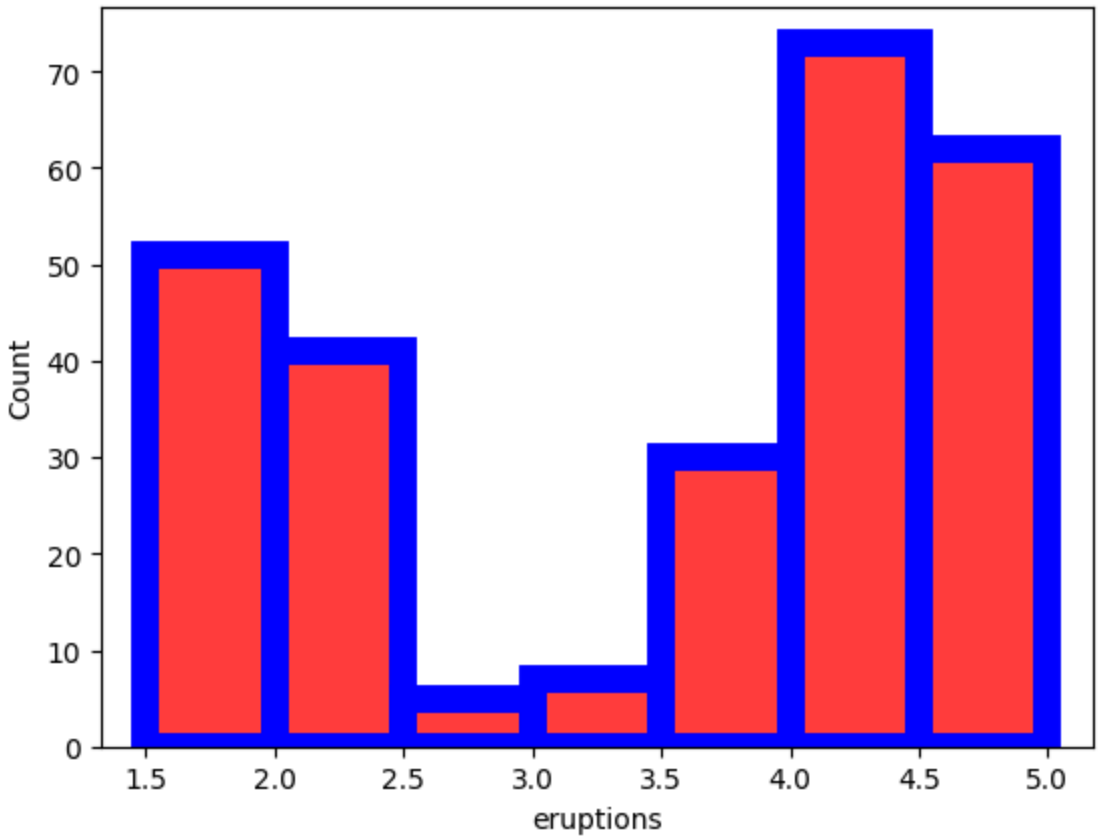
In [72]:
```
# use numpy.arange for float steps  cuz (range() requires integer step)

bins = []
for i in np.arange(1.5, 5.2, 0.5):
    bins.append(i)
```

In [73]:
```
plt.hist(erupt, bins= bins)
```

Out[73]:
```
(array([51., 41.,  5.,  7., 30., 73., 62.]),
 array([1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5, 5. ]),
 <BarContainer object of 7 artists>)
```



In [74]:
```
sns.histplot(erupt, bins=bins, color='red', edgecolor='blue', linewidth=10)
plt.show()


# linewidth = border thickness
# edgecolor = border
```



In [75]:
```
group1= np.random.normal(loc=10, scale =2, size=100)
```

In [76]:
```
np.mean(group1)
```

Out[76]:
```
np.float64(9.847471252713213)
```

In [77]:
```
group2 = np.random.normal(loc=12, scale =2.5, size=100)
```

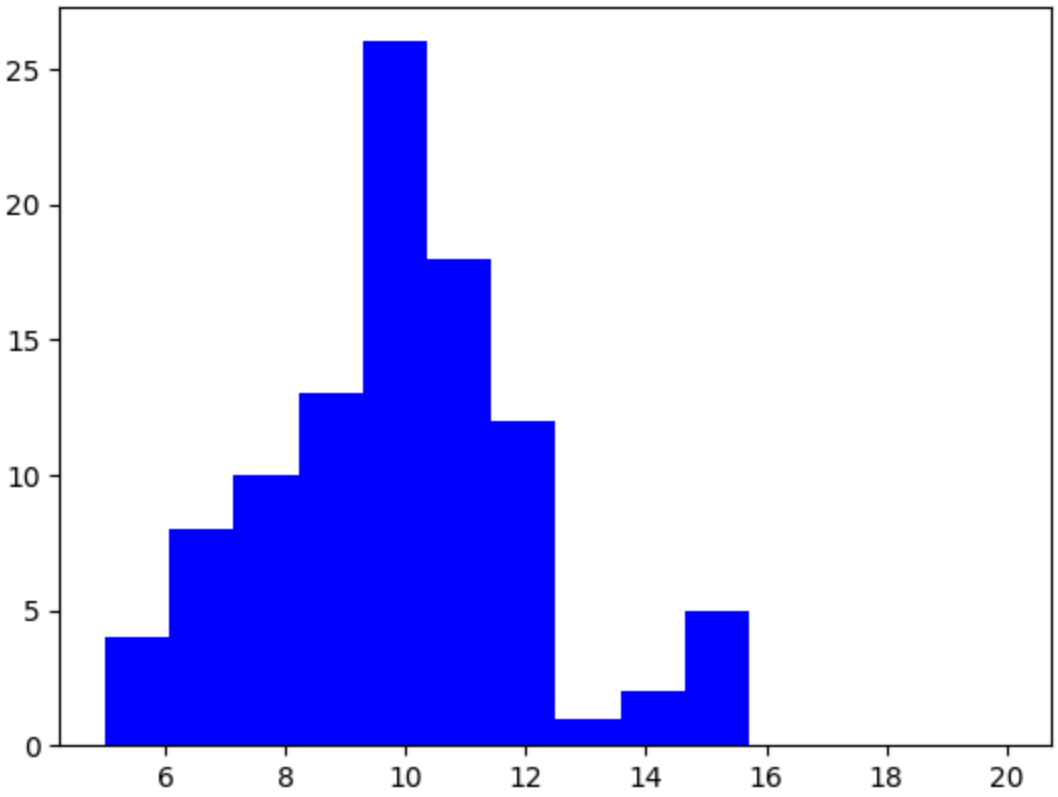
In [78]:
```
np.mean(group2)
```

Out[78]:
```
np.float64(11.864745032312545)
```

In [79]:
```
bins=np.linspace(5,20,15)
```

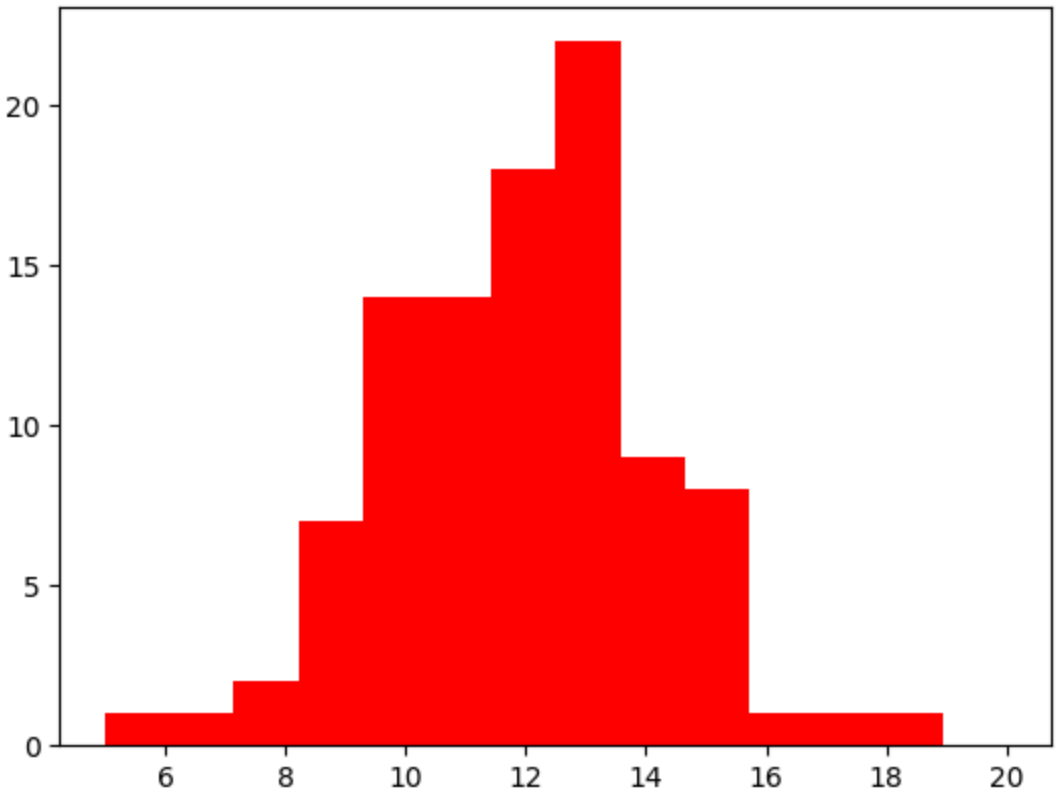
In [80]:
```
plt.hist(group1, bins=bins, color='blue')
```

Out[80]:
```
(array([ 4.,  8., 10., 13., 26., 18., 12.,  1.,  2.,  5.,  0.,  0.,  0.,
         0.]),
 array([ 5.        ,  6.07142857,  7.14285714,  8.21428571,  9.28571429,
        10.35714286, 11.42857143, 12.5       , 13.57142857, 14.64285714,
        15.71428571, 16.78571429, 17.85714286, 18.92857143, 20.        ]),
 <BarContainer object of 14 artists>)
```

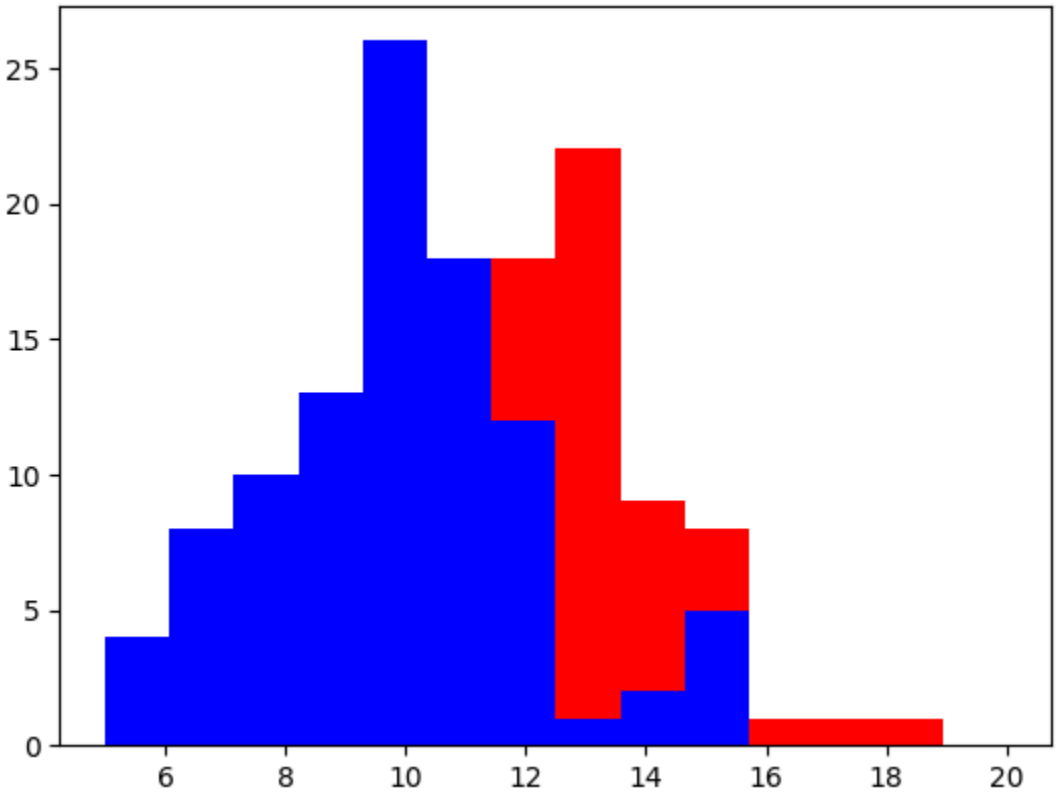


In [81]:
```
plt.hist(group2, bins =bins, color='red')
```

Out[81]:
```
(array([ 1.,  1.,  2.,  7., 14., 14., 18., 22.,  9.,  8.,  1.,  1.,  1.,
         0.]),
 array([ 5.        ,  6.07142857,  7.14285714,  8.21428571,  9.28571429,
        10.35714286, 11.42857143, 12.5       , 13.57142857, 14.64285714,
        15.71428571, 16.78571429, 17.85714286, 18.92857143, 20.        ]),
 <BarContainer object of 14 artists>)
```



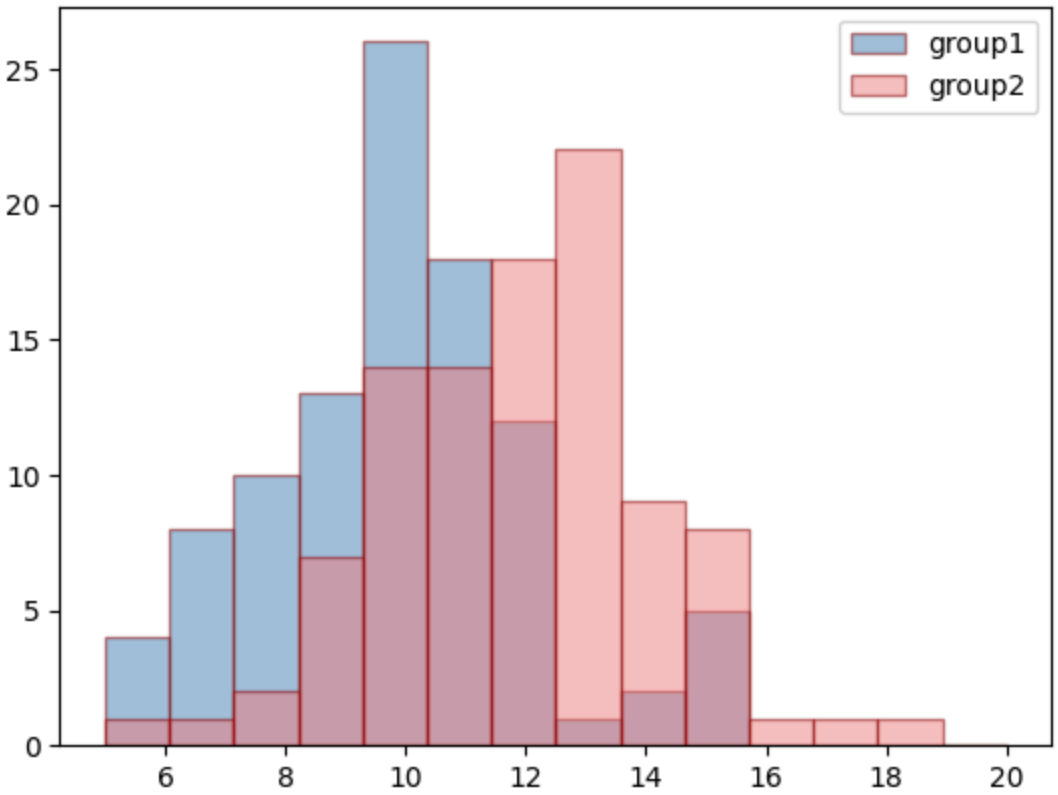
In [82]:
```
plt.hist(group2, bins =bins, color='red')
plt.hist(group1, bins=bins, color='blue')
plt.show()
```



In [83]:
```
np.random.normal(loc=25, scale=8, size =20)


# np.random.normal() generates random numbers from a normal (Gaussian) distrib
# loc  = mean of the distribution (center point), this is mean
# scale = standard deviation (spread of the values)
# size = how many random values to generate
# So this creates 20 values normally distributed around mean=25 with SD=8.
```

Out[83]:
```
array([24.32545557, 18.60374749, 29.88961433, 36.019364  , 24.61032745,
       23.8579139 , 20.47827778, 28.02011485, 28.6023405 , 32.38670572,
       23.09790352, 26.73713239, 26.95214201, 18.73323501, 28.59912291,
       17.32736196, 35.30162664, 31.4107352 , 20.53663117, 19.17495956])
```

In [84]:
```
plt.hist(group1, bins=bins, color='steelblue', edgecolor='darkred', alpha=0.5,
plt.hist(group2, bins=bins, color='lightcoral', edgecolor='darkred', alpha=0.5

plt.legend()
plt.show()



# Plotting overlapping histograms for comparing two groups.


# bins      = number of intervals/buckets.
# color     = fill color of the bars.
# edgecolor = outline color of each bar.
# alpha     = transparency (0 = invisible, 1 = solid) → helps both histograms
# label     = name shown in the legend.
```



In [85]:
```
df
```

Out[85]:

	eruptions	waiting
0	3.600	79
1	1.800	54
2	3.333	74
3	2.283	62
4	4.533	85
...	...	...
267	4.117	81
268	2.150	46
269	4.417	90
270	1.817	46
271	4.467	74

272 rows × 2 columns

In [86]:
```
wait = df.waiting
```

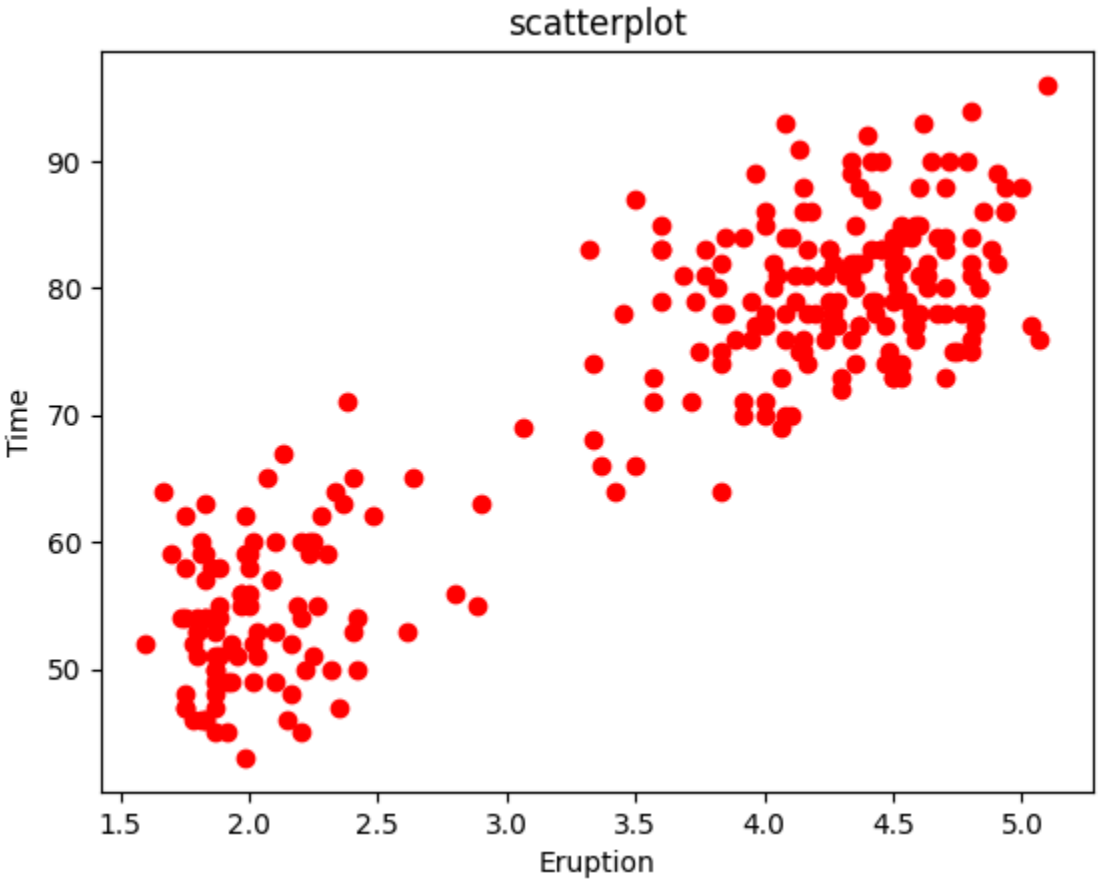
```
erupt = df.eruptions
```

In [87]:
```
df.head()
```

Out[87]:

	eruptions	waiting
0	3.600	79
1	1.800	54
2	3.333	74
3	2.283	62
4	4.533	85

In [88]:
```
plt.scatter(erupt, wait, color='red')
plt.title('scatterplot')
plt.xlabel('Eruption')
plt.ylabel('Time')
plt.show()
```



In [89]:
```
df.head()
```

Out[89]:

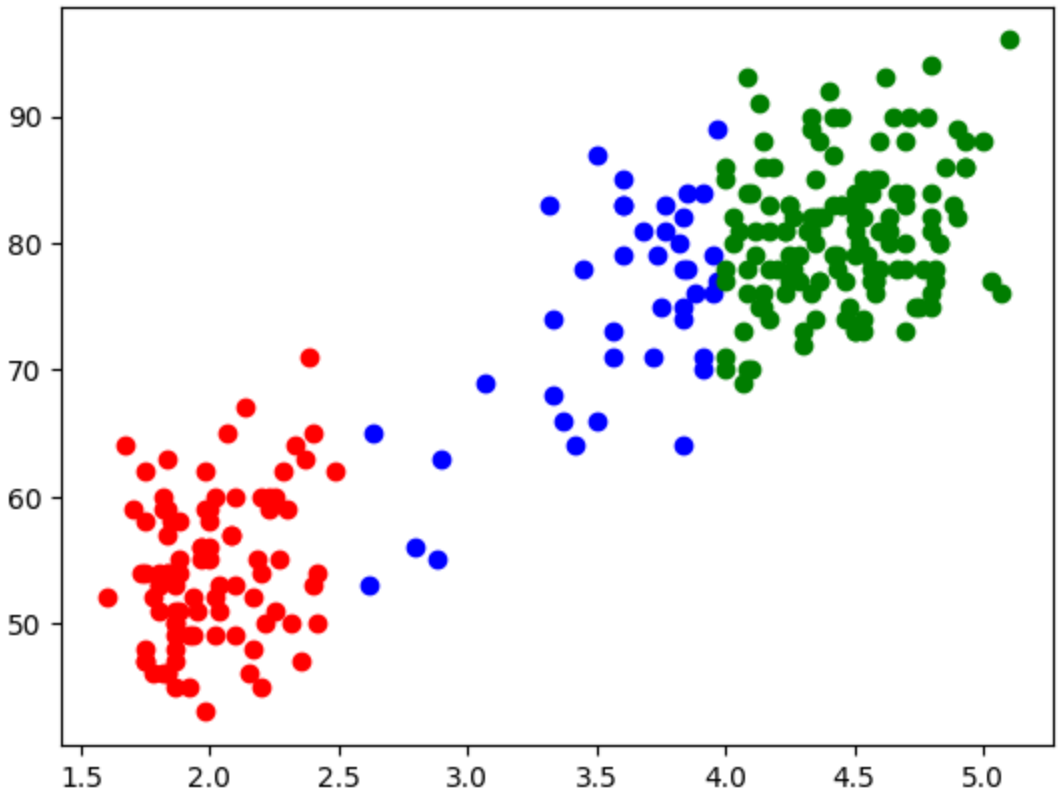
	eruptions	waiting
0	3.600	79
1	1.800	54
2	3.333	74
3	2.283	62
4	4.533	85

In [90]:

```
grp = []
for ctr in erupt:
    if ctr<2.5:
        grp.append('A')
    elif ctr<4:
        grp.append('B')
    else:
        grp.append('C')
df['Group']=grp

dfA = df[df.Group == 'A']
dfB = df[df.Group == 'B']
dfC = df[df.Group == 'C']

plt.scatter(dfA.eruptions,dfA.waiting, c= 'red')
plt.scatter(dfB.eruptions,dfB.waiting, c= 'blue')
plt.scatter(dfC.eruptions,dfC.waiting, c= 'green')
```

Out[90]:

```
<matplotlib.collections.PathCollection at 0x1520fa5d0>
```


90
80
70
60
50
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

In [ ]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

In [ ]:
```
import scipy
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib
from matplotlib import pyplot as plt
import seaborn as sns
import os
from scipy.stats import *
from scipy.stats import norm, t, binom, poisson, expon
import numpy as np
from numpy import random
from matplotlib import pyplot as plt
```

population vs sample

population

- data collected from ALL possible sources
- example: all people voting in general election

sample

- data collected from subset of population
- example: people contacted for survey (always a subset)

parameter

- the variable for which data is collected

data

- numbers collected for parameters

population parameter (PP)

- summary of data from entire population
- example: general election result

sample statistic (SS)

- summary of data from sample
- example: exit poll result

relationship:

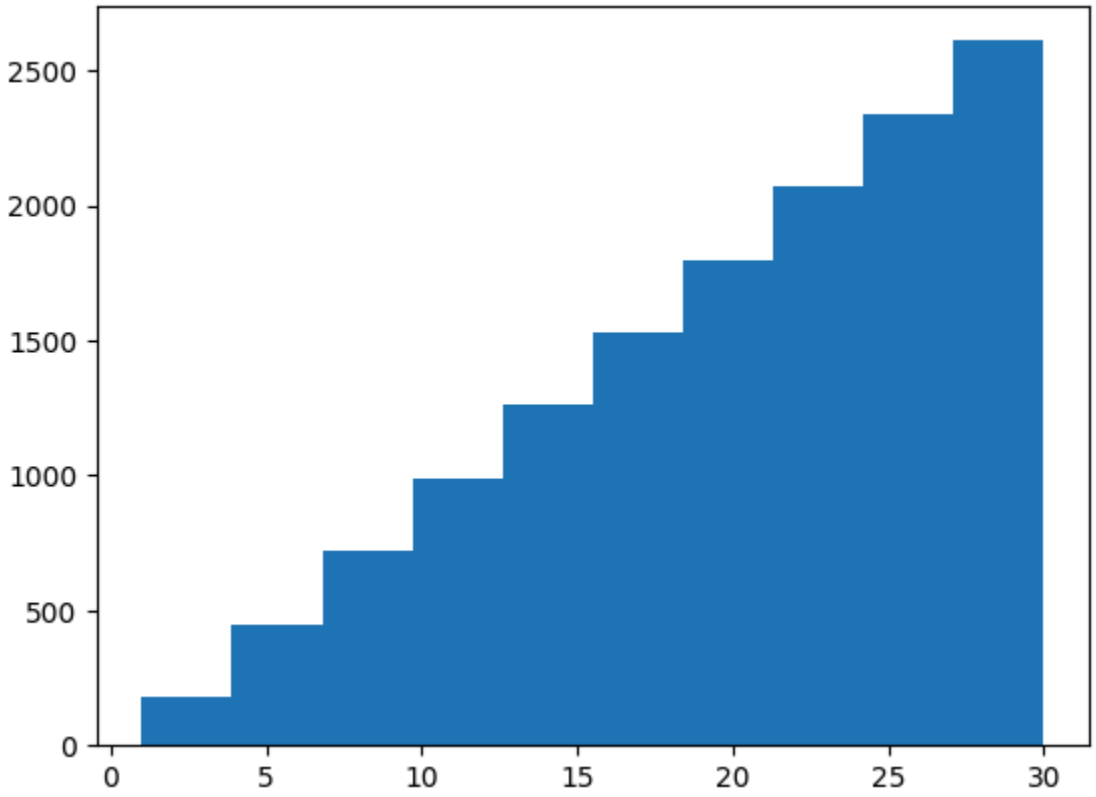
$$SS = PP + \text{Bias} + \text{Chance Variation}$$

- SS = PP → claim (ideal but unrealistic)
- SS ≠ PP → reality (what actually happens)

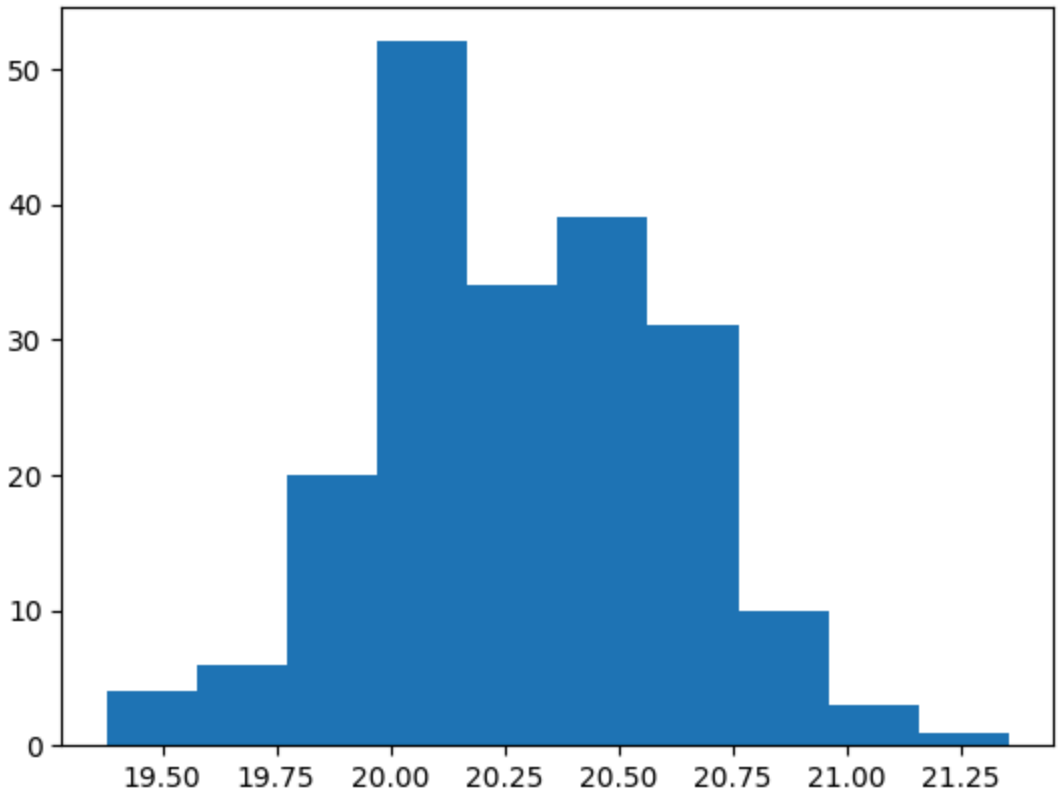
note: chance variation can only be eliminated when sample size = population size (which rarely happens in practice)

In [ ]:
```
# demonstration of central limit theorem

# create skewed population distribution
x1 = np.repeat(np.arange(1, 31), np.arange(10, 310, 10))
x1 = np.repeat(x1, 3)

# plot original population (skewed)
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.hist(x1, bins=30, edgecolor='black', alpha=0.7)
plt.title('original population distribution (skewed)')
plt.xlabel('value')
plt.ylabel('frequency')

# draw 200 samples, each of size 500
m = []
for ctr1 in range(1, 201):
    val = np.random.choice(x1, 500)
    m.append(val)

# calculate mean of each sample
mean_m = []
for ctr1 in range(len(m)):
    mean_m.append(np.mean(m[ctr1]))

# plot sampling distribution of means (normal)
plt.subplot(1, 2, 2)
plt.hist(mean_m, bins=30, edgecolor='black', alpha=0.7, color='green')
plt.title('sampling distribution of means (normal)')
plt.xlabel('sample mean')
plt.ylabel('frequency')
```

```
plt.tight_layout()
plt.show()

print(f'population is skewed, but sample means are normally distributed')
```




```
20.28853
0.3335889492102158
```



central limit theorem (CLT)

as the number of samples increases, the distribution of sample means approaches a normal distribution. this is true regardless of the original population distribution.

key insight: even if the population is skewed, the sample means will be normally distributed when you have enough samples.

In [ ]:
```
# summary statistics of sampling distribution
print(f'mean of sample means: {np.mean(mean_m):.4f}')
print(f'std of sample means: {np.std(mean_m, ddof=1):.4f}')
print(f'\nthis std is called standard error (SE)')
```

```
20.28853
0.3335889492102158
```

The `ddof` parameter in `np.std()` stands for **Delta Degrees of Freedom**.

- By default, `np.std(..., ddof=0)` calculates the **population standard deviation** (divides by N).
- If you set `ddof=1`, it calculates the **sample standard deviation** (divides by N-1), which is an unbiased estimator for the standard deviation of a population from a sample.

In summary:

- `np.std(mean_m, ddof=0)` → population std (divide by N)
- `np.std(mean_m, ddof=1)` → sample std (divide by N-1, unbiased)

Why use `ddof=1`? When you have a sample and want to estimate the population standard deviation, use `ddof=1`.

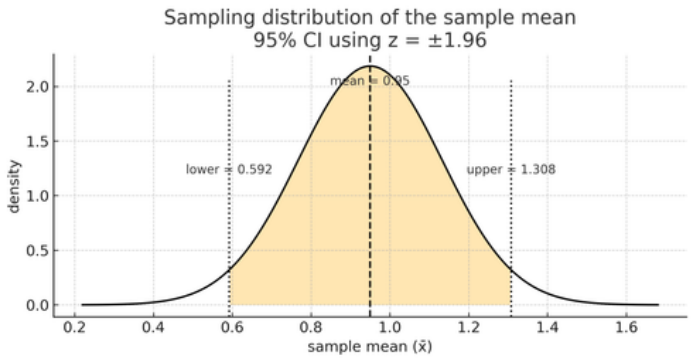
Central Limit Theorem (Sample Mean) and the 95% Z-Interval

Concept

When the sample size is reasonably large, the sampling distribution of the sample mean ($\bar{x}$) becomes approximately normal with:

- Mean μ (use the symbol directly, e.g. μ = 0.95)
- Standard error SE given by the formula below.

$$ \text{SE} = \frac{\sigma}{\sqrt{n}} $$



Standard Normal Inequality for 95% Confidence

The 95% standard normal interval is:

$$ -1.96 \le Z \le +1.96 $$

where

$$ Z = \frac{\bar{X} - \mu}{\text{SE}} $$

Converting Z-Interval to $\bar{x}$-Interval

Start with:

$$ -1.96 \le \frac{\bar{X} - \mu}{\text{SE}} \le 1.96 $$

Multiply by SE and add μ:

$$ \mu - 1.96\cdot\text{SE} \le \bar{X} \le \mu + 1.96\cdot\text{SE} $$

This is the 95% confidence interval for the sample mean.

Numerical Example (use direct signs for single-line values)

- Mean μ = 0.95
- Population standard deviation σ = 1.0 (assumed)
- Sample size n = 30

Compute SE:

$$ \text{SE} = \frac{1}{\sqrt{30}} \approx 0.1826 $$

Lower bound:

$$ 0.95 - 1.96 \times 0.1826 \approx 0.592 $$

Upper bound:

$$ 0.95 + 1.96 \times 0.1826 \approx 1.308 $$

So the 95% confidence interval for the sample mean is:

$$ 0.592 \le \bar{X} \le 1.308 $$

Interpretation

- Curve: sampling distribution of the sample mean under CLT.
- Center line: mean μ = 0.95 (written with the μ symbol directly).
- Shaded region: central 95% of possible sample means.
- Vertical dotted lines: lower and upper bounds of the 95% interval.

Meaning: if you repeatedly take samples of size n = 30, about 95% of the sample means will lie between 0.592 and 1.308.

confidence = 95%

significance becomes rest - 5%

Intuition (single-line symbols)

- z critical values: za = -1.96, zb = +1.96
- SE = σ / √n
- CI for x̄: μ ± 1.96·SE

In [ ]:
```
# confidence interval using normal distribution (z-interval)
# formula: x¯ ± z * (σ/√n)

n = 100  # sample size
sm = 45.7  # sample mean
pop_sd = 6.8  # population standard deviation
cl = 0.95  # 95% confidence level

# find z-critical values for 95% CI
za = norm.ppf(0.025)  # lower tail (2.5%)
zb = norm.ppf(0.975)  # upper tail (97.5%)

# calculate confidence interval bounds
se = pop_sd / n**0.5  # standard error
low_est = sm + za * se
upper_est = sm + zb * se

print(f'sample mean = {sm}')
print(f'standard error = {se:.4f}')
print(f'z-critical values: {za:.4f}, {zb:.4f}')
print(f'95% CI: [{low_est:.4f}, {upper_est:.4f}]')
```

In [ ]:
```
low_est  # lower bound: 2.5% probability of being less than this value
```

Out[ ]:
```
np.float64(44.36722449051277)
```

In [ ]:
```
upper_est  # upper bound: 2.5% probability of being more than this value
```

Out[ ]:
```
np.float64(47.03277550948724)
```

In [ ]:
```
# margin of error (MOE) = upper_est - lower_est = 2 * z * σ/√n
moe = upper_est - low_est
print(f'margin of error = {moe:.4f}')
print(f'also equals: 2 * {zb:.4f} * {se:.4f} = {2 * zb * se:.4f}')
```

factors affecting margin of error (MOE)

the margin of error depends on three main factors:

1. sample size (n)

- as sample size increases → MOE decreases
- relationship: MOE ∝ 1/√n

2. standard deviation (σ)

- as data becomes more consistent (σ decreases) → MOE decreases
- relationship: MOE ∝ σ

3. confidence level

- as confidence level increases → z increases → MOE increases
- common z-values:
    - 90% CI: z = 1.645
    - 95% CI: z = 1.96
    - 99% CI: z = 2.576

formula: MOE = z × σ/√n

In [ ]:
```
# using scipy built-in function for t-distribution CI
# note: slight difference from z-interval because uses t-distribution
scipy.stats.t.interval(cl, n-1, sm, pop_sd/n**0.5)
```

Out[ ]:
```
(np.float64(44.3507324729741), np.float64(47.049267527025904))
```

In [ ]:
```
# confidence interval using t-distribution (t-interval)
# use t-distribution when population σ is unknown or n < 30
# formula: x¯ ± t * (s/√n)

n = 100
sm = 45.7
pop_sd = 6.8
cl = 0.95
df = n - 1  # degrees of freedom

# find t-critical values
za = t.ppf((1-cl)/2, df)  # lower tail
zb = t.ppf(cl + (1-cl)/2, df)  # upper tail

# calculate CI
se = pop_sd / n**0.5
low_est = sm + za * se
upper_est = sm + zb * se

print(f't-critical values: {za:.4f}, {zb:.4f}')
print(f'95% t-interval: [{low_est:.4f}, {upper_est:.4f}]')
```

In [ ]:
```
low_est  # lower bound of 95% t-interval
```

Out[ ]: np.float64(44.3507324729741)

In [ ]:
```
upper_est  # upper bound of 95% t-interval
```

Out[ ]: np.float64(47.049267527025904)

In [ ]:
```
# confidence interval for proportion
# formula: p̂ ± z * √(p̂(1-p̂)/n)

n = 500  # total population surveyed
r = 200  # total positive responses
p = r / n  # sample proportion

# standard deviation for proportion
sd = (p * (1 - p)) ** 0.5

# 95% CI for proportion
z_critical = 1.96
se = sd / n**0.5
low_est = p - z_critical * se
upper_est = p + z_critical * se

print(f'sample proportion p̂ = {p:.4f} ({p*100:.1f}%)')
print(f'standard deviation = {sd:.4f}')
print(f'standard error = {se:.4f}')
print(f'95% CI: [{low_est:.4f}, {upper_est:.4f}]')
print(f'interpretation: true proportion lies between {low_est*100:.2f}% and {u
```

```
0.4898979485566356
```

In [ ]:
```
(p * (1 - p)) ** 0.5  # standard deviation for proportion formula
```

Out[ ]: 0.4898979485566356

In [ ]:
```
low_est  # lower bound of proportion CI
```

Out[ ]: 0.357058551491595

In [ ]:
```
upper_est  # upper bound of proportion CI
```

Out[ ]: 0.44294144850840506

In [ ]:
```
from scipy.stats import rv_discrete  # for discrete random variables
```

In [ ]:
```
# confidence interval for poisson distribution
# when count data follows poisson distribution with mean λ = 20
# formula: uses ppf to find bounds

lambda_val = 20
ci = scipy.stats.poisson.interval(0.95, lambda_val)
print(f'95% CI for poisson(λ={lambda_val}): {ci}')
print(f'interpretation: 95% of counts will be between {ci[0]} and {ci[1]}')
```

Out[ ]: (np.float64(12.0), np.float64(29.0))

In [ ]:
```
poisson.ppf(0.025, 20)  # lower 2.5% percentile
```

Out[ ]: np.float64(12.0)

In [ ]:
```
poisson.ppf(0.975, 20)  # upper 97.5% percentile
```

Out[ ]: np.float64(29.0)

---

hypothesis testing

hypothesis testing is a statistical method to make decisions about population parameters based on sample data.

key concepts:

- null hypothesis (H_0): the claim we assume is true (status quo)
- alternative hypothesis (H_1 or H_a): what we want to prove
- p-value: probability of getting observed results if H_0 is true
- significance level (α): maximum acceptable probability of type I error

type I and type II errors

reducing both errors:

- the only way to decrease both types of errors is to increase the sample size
- as sample size → population size, errors → 0
- theoretically, there are no errors when sample size = population size (but this is impractical in real-world scenarios)

understanding the two types of errors

type I error: reject true H_0 (false positive)

- scenario: bad sample but good population
- probability = α (significance level)
- example: rejecting a good product because of a bad sample

type II error: fail to reject false H_0 (false negative)

- scenario: good sample but bad population
- probability = β (power = 1 - β)

- example: accepting a bad product because the sample happened to be good

steps in hypothesis testing

step 1: establish null (H_0) and alternative (H_1) hypotheses

the null hypothesis represents the status quo - if found true, no action/decision/changes are required.

three types of tests based on alternative hypothesis:

case 1: left tail test

- example: salary analysis
- H_0: salary >= 10 LPA
- H_1: salary < 10 LPA
- rejection region: left side (lower tail)

case 2: right tail test

- example: delivery time
- H_0: time <= 30 min
- H_1: time > 30 min
- rejection region: right side (upper tail)

case 3: two tail test

- example: salt content in food
- H_0: salt = 2 grams
- H_1: salt ≠ 2 grams
- rejection region: both sides

---

step 2: establish confidence level and significance level (α)

- significance level (α) = 100% - confidence level
- common values:
    - α = 0.05 (95% confidence)
    - α = 0.01 (99% confidence)

---

step 3: choose appropriate statistical test

- **z-test:** large sample, known σ

- **t-test:** small sample, unknown σ
- **chi-square:** categorical data
- **ANOVA:** compare multiple groups

---

step 4: calculate test statistic and p-value

use python functions to compute these values.

---

step 5: make decision

- if p-value < α: reject H_0
- if p-value >= α: fail to reject H_0

---

interpretation of p-value and α

α (level of significance): maximum allowed probability of type I error

p-value: actual probability of type I error given the data

---

practical analogy: covid safety decision

imagine deciding whether to go out during covid:

- **H_0:** stay at home (safe option)
- **H_1:** move out
- **p-value:** probability of infection if moving out (from tracking app)
- **α:** maximum acceptable infection risk (recommended by doctor) = 0.05

scenario 1: p-value = 0.20

- p-value > α, so **do not reject H_0** (stay home)
- interpretation: 20% risk > 5% acceptable risk → too risky to go out

scenario 2: p-value = 0.02

- p-value < α, so **reject H_0** (can move out)
- interpretation: 2% risk < 5% acceptable risk → safe enough to go out

In [147…

```
# z-critical value for left tail test at α = 0.10
z_crit = norm.ppf(0.1)
print(f'z-critical = {z_crit:.4f}')
print(f'if test statistic < {z_crit:.4f}, reject H₀')
```

Out[147… `np.float64(-1.2815515655446004)`

types of null hypotheses and tests

1. one-sample tests

- test a single population parameter
- examples:
    - H_0: $\mu = \mu_0$
    - H_0: $\mu >= \mu_0$
    - H_0: $\mu <= \mu_0$

2. two-sample tests

- compare two populations
- examples:
    - H_0: $\mu_1 = \mu_2$
    - H_0: $\mu_1 - \mu_2 = 0$

3. paired tests

- compare before/after measurements on same subjects
- H_0: $\mu\_diff = 0$

4. proportion tests

- test population proportion
- H_0: $p = p_0$

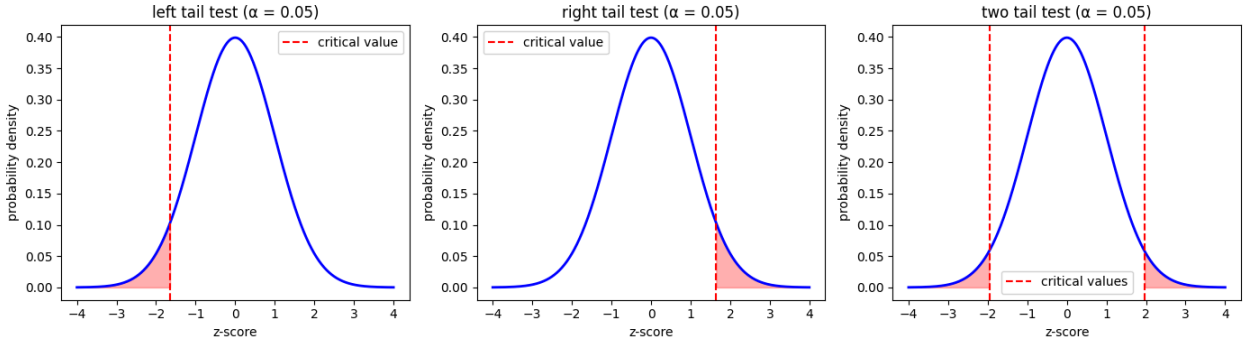
5. variance tests

- test population variance
- H_0: $\sigma^2 = \sigma_0^2$

In [149…

```
# visualization of hypothesis testing regions
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import norm

x = np.linspace(-4, 4, 1000)
y = norm.pdf(x)

plt.figure(figsize=(14, 4))

# left tail test
plt.subplot(1, 3, 1)
plt.plot(x, y, 'b-', linewidth=2)
```

```
plt.fill_between(x[x < -1.645], y[x < -1.645], alpha=0.3, color='red')
plt.axvline(-1.645, color='red', linestyle='--', label='critical value')
plt.title('left tail test (α = 0.05)')
plt.xlabel('z-score')
plt.ylabel('probability density')
plt.legend()

# right tail test
plt.subplot(1, 3, 2)
plt.plot(x, y, 'b-', linewidth=2)
plt.fill_between(x[x > 1.645], y[x > 1.645], alpha=0.3, color='red')
plt.axvline(1.645, color='red', linestyle='--', label='critical value')
plt.title('right tail test (α = 0.05)')
plt.xlabel('z-score')
plt.ylabel('probability density')
plt.legend()

# two tail test
plt.subplot(1, 3, 3)
plt.plot(x, y, 'b-', linewidth=2)
plt.fill_between(x[x < -1.96], y[x < -1.96], alpha=0.3, color='red')
plt.fill_between(x[x > 1.96], y[x > 1.96], alpha=0.3, color='red')
plt.axvline(-1.96, color='red', linestyle='--', label='critical values')
plt.axvline(1.96, color='red', linestyle='--')
plt.title('two tail test (α = 0.05)')
plt.xlabel('z-score')
plt.ylabel('probability density')
plt.legend()

plt.tight_layout()
plt.show()
```



In [ ]:

In [8]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

In [9]:
```
# one-sample z-test / one-sample t-test

# purpose: compare sample mean against a claim value
# determine if difference is statistically significant

# when to use:
# - z-test: large sample (n >= 30), known population σ
# - t-test: small sample (n < 30), unknown population σ

# formula for test statistic:
# z = (x¯ - μ₀) / (σ / √n)
# t = (x¯ - μ₀) / (s / √n)

# where:
# x¯ = sample mean
# μ₀ = claimed/hypothesized population mean
# σ = population standard deviation (z-test)
# s = sample standard deviation (t-test)
# n = sample size
```

In [10]:
```
import scipy
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib
from matplotlib import pyplot as plt
import seaborn as sns
import os
from scipy.stats import *
from scipy.stats import norm, t, binom, poisson, expon
import statsmodels
from statsmodels import stats
from statsmodels.stats import weightstats as sswa
```

In [11]:
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

In [12]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name = 'faithful')
```

In [13]:
```
df.head()
```

Out[13]:

	eruptions	waiting
0	3.600	79
1	1.800	54
2	3.333	74
3	2.283	62
4	4.533	85

In [14]:
```
erupt = df.eruptions
```

In [15]:
```
# example: test if mean eruption time is at least 3.6 minutes

# claim: mean is at least 3.6
# H₀: μ >= 3.6 (null hypothesis - what we assume is true)
# H₁: μ < 3.6 (alternative - what we want to prove)

# this is a left tail test
# we use one-sample z-test to compare sample mean against claim value 3.6
```

In [16]:
```
# One-sample Z-test (test if mean < 3.6)
# Returns (z_statistic, p_value)
z_stat, p_value = sswa.ztest(erupt, value=3.6, alternative='smaller')
print('z-statistic =', z_stat)
print('p-value =', p_value)
# Interpretation:
# H0: population mean >= 3.6
# H1: population mean < 3.6
# For alternative='smaller', p_value = P(Z <= z_stat).
# Small p-value (e.g. < 0.05) => reject H0 in favor of H1 (mean < 3.6).
# Large p-value => insufficient evidence to conclude mean < 3.6.
# Example: if sample mean = 3.48 and the 95% one-sided upper confidence bound
# then 3.6 ≤ 3.602 so the data do not provide strong evidence that the true me
```

```
z-statistic = -1.621495827018525
p-value = 0.05245567210775938
```

In [17]:
```
# manual calculation of z-statistic
# formula: z = (x¯ - μ₀) / (s / √n)

sample_mean = np.mean(erupt)
claim_value = 3.6
sample_std = np.std(erupt, ddof=1)
n = len(erupt)

z_manual = (sample_mean - claim_value) / (sample_std / n**0.5)
```

```
print(f'manual z-statistic = {z_manual:.4f}')
```

```
manual z-statistic = -1.6215
```

In [18]:
```
# t-critical value for α = 0.05, df = 271 (left tail)
t_crit = t.ppf(0.05, 271)
print(f't-critical value = {t_crit:.4f}')
print(f'if t-statistic < {t_crit:.4f}, reject H₀')
```

```
t-critical value = -1.6505
if t-statistic < -1.6505, reject H₀
```

In [19]:
```
# z-critical value for α = 0.05 (left tail)
z_crit = norm.ppf(0.05)
print(f'z-critical value = {z_crit:.4f}')
```

```
z-critical value = -1.6449
```

In [20]:
```
# 95% upper confidence bound (one-sided)
# formula: x̄ + z_critical * (s / √n)

upper_bound = np.mean(erupt) + 1.650495778817526 * np.std(erupt, ddof=1) / ler
print(f'95% upper confidence bound = {upper_bound:.4f}')
print(f'sample mean = {np.mean(erupt):.4f}')
print(f'claim value = 3.6')
print(f'\nif claim value <= upper bound, do not reject H₀')
```

```
95% upper confidence bound = 3.6020
sample mean = 3.4878
claim value = 3.6

if claim value <= upper bound, do not reject H₀
```

In [21]:
```
# right tail test example
# H₀: μ <= claim_value
# H₁: μ > claim_value
# test if mean is significantly greater than claim
```

In [22]:
```
# right tail test: is mean > 3.42?
z_stat_right, p_val_right = sswa.ztest(erupt, value=3.42, alternative='larger'
print(f'z-statistic = {z_stat_right:.4f}')
print(f'p-value = {p_val_right:.4f}')
print(f'\nif p-value < 0.05, reject H₀ (mean > 3.42)')
```

```
z-statistic = 0.9794
p-value = 0.1637

if p-value < 0.05, reject H₀ (mean > 3.42)
```

In [23]:
```
# 95% lower confidence bound (one-sided)
# formula: x̄ - z_critical * (s / √n)

lower_bound = np.mean(erupt) - 1.650495778817526 * np.std(erupt, ddof=1) / ler
print(f'95% lower confidence bound = {lower_bound:.4f}')
print(f'sample mean = {np.mean(erupt):.4f}')
```

```
95% lower confidence bound = 3.3736
sample mean = 3.4878
```

In [24]:
```python
# practice example: is population mean > 6?

my_list = [3, 6, 5, 4, 6, 7, 9, 6, 2]

# claim: mean > 6
# H₀: μ <= 6 (null hypothesis)
# H₁: μ > 6 (alternative - we want to prove this)
# this is a right tail test
```

In [25]:
```python
# note: testing wrong direction here (smaller instead of larger)
# for H₁: μ > 6, should use alternative='larger'
z_stat_wrong, p_val_wrong = sswa.ztest(my_list, value=6, alternative='smaller'
print(f'testing if mean < 6 (wrong direction)')
print(f'z-statistic = {z_stat_wrong:.4f}')
print(f'p-value = {p_val_wrong:.4f}')
```

```
testing if mean < 6 (wrong direction)
z-statistic = -0.9428
p-value = 0.1729
```

In [26]:
```python
# 95% upper confidence bound for my_list
upper_ci = np.mean(my_list) + 1.650495778817526 * np.std(my_list, ddof=1) / le
print(f'95% upper CI bound = {upper_ci:.4f}')
```

```
95% upper CI bound = 6.5004
```

In [27]:
```python
# sample mean of my_list
sample_mean = np.mean(my_list)
print(f'sample mean = {sample_mean:.4f}')
print(f'claim value = 6')
print(f'difference = {sample_mean - 6:.4f}')
```

```
sample mean = 5.3333
claim value = 6
difference = -0.6667
```

In [28]:
```python
# t-critical value for α = 0.05, df = n-1 (right tail)
df = len(my_list) - 1
t_crit_right = t.ppf(0.05, df)
print(f't-critical value (left tail) = {t_crit_right:.4f}')
print(f'for right tail test, use positive value = {-t_crit_right:.4f}')
```

```
t-critical value (left tail) = -1.8595
for right tail test, use positive value = 1.8595
```

In [29]:
```python
# 95% upper confidence bound using t-distribution
t_val = 1.8595480375228428
upper_bound_t = np.mean(my_list) + t_val * np.std(my_list, ddof=1) / len(my_li
print(f'95% upper bound = {upper_bound_t:.4f}')
print(f'interpretation: with 95% confidence, true mean <= {upper_bound_t:.4f}'
```

```
95% upper bound = 6.6482
interpretation: with 95% confidence, true mean <= 6.6482
```

In [30]:
```
# manual t-statistic calculation
# formula: t = (x̄ - μ₀) / (s / √n)

t_manual = (np.mean(my_list) - 6) / (np.std(my_list, ddof=1) / (len(my_list)**
print(f't-statistic = {t_manual:.4f}')
print(f'interpretation: how many standard errors is sample mean from claimed v
```

```
t-statistic = -0.9428
interpretation: how many standard errors is sample mean from claimed value?
```

In [31]:
```
# p-value using normal distribution
p_norm = norm.cdf(-0.9428090415820639)
print(f'p-value (normal) = {p_norm:.4f}')
print(f'this is P(Z < {-0.9428090415820639:.4f})')
```

```
p-value (normal) = 0.1729
this is P(Z < -0.9428)
```

In [32]:
```
# p-value using t-distribution (more accurate for small samples)
p_t = t.cdf(-0.9428090415820639, len(my_list) - 1)
print(f'p-value (t-distribution) = {p_t:.4f}')
print(f'df = {len(my_list) - 1}')
print(f'\nfor right tail: p-value = 1 - {p_t:.4f} = {1 - p_t:.4f}')
print(f'since p-value > 0.05, fail to reject H₀')
print(f'conclusion: insufficient evidence that mean > 6')
```

```
p-value (t-distribution) = 0.1867
df = 8

for right tail: p-value = 1 - 0.1867 = 0.8133
since p-value > 0.05, fail to reject H₀
conclusion: insufficient evidence that mean > 6
```

In [33]:
```
# two-sample z-test / two-sample t-test

# purpose: compare means of two independent samples
# determine if difference between means is statistically significant

# formula for test statistic:
# z = (x̄₁ - x̄₂ - d₀) / SE
# where SE = √(σ₁²/n₁ + σ₂²/n₂)

# for t-test:
# t = (x̄₁ - x̄₂ - d₀) / SE
# where SE = √(s₁²/n₁ + s₂²/n₂)

# d₀ is the claimed difference (often 0)

# use cases:
# - compare treatment vs control groups
# - compare before vs after
# - compare two populations
```

In [34]:
```
# example: test if average weight difference is at least 7 kg
```

```
# claim: avg(men) - avg(women) >= 7
# H₀: μ_men - μ_women >= 7 (null hypothesis)
# H₁: μ_men - μ_women < 7 (alternative)
# this is a left tail test
```

In [35]:
```
males = [76,56,62,65,54,70]
females = [54,57,48,64,57]
```

In [36]:
```
# two-sample z-test: is difference < 7?
z_stat_2samp, p_val_2samp = sswa.ztest(males, females, value=7, alternative='s
print(f'sample means:')
print(f'  males: {np.mean(males):.2f} kg')
print(f'  females: {np.mean(females):.2f} kg')
print(f'  difference: {np.mean(males) - np.mean(females):.2f} kg')
print(f'\ntest results:')
print(f'  z-statistic = {z_stat_2samp:.4f}')
print(f'  p-value = {p_val_2samp:.4f}')
print(f'\nif p-value < 0.05, reject H₀ (difference < 7 kg)')
```

```
sample means:
  males: 63.83 kg
  females: 56.00 kg
  difference: 7.83 kg

test results:
  z-statistic = 0.1879
  p-value = 0.5745

if p-value < 0.05, reject H₀ (difference < 7 kg)
```

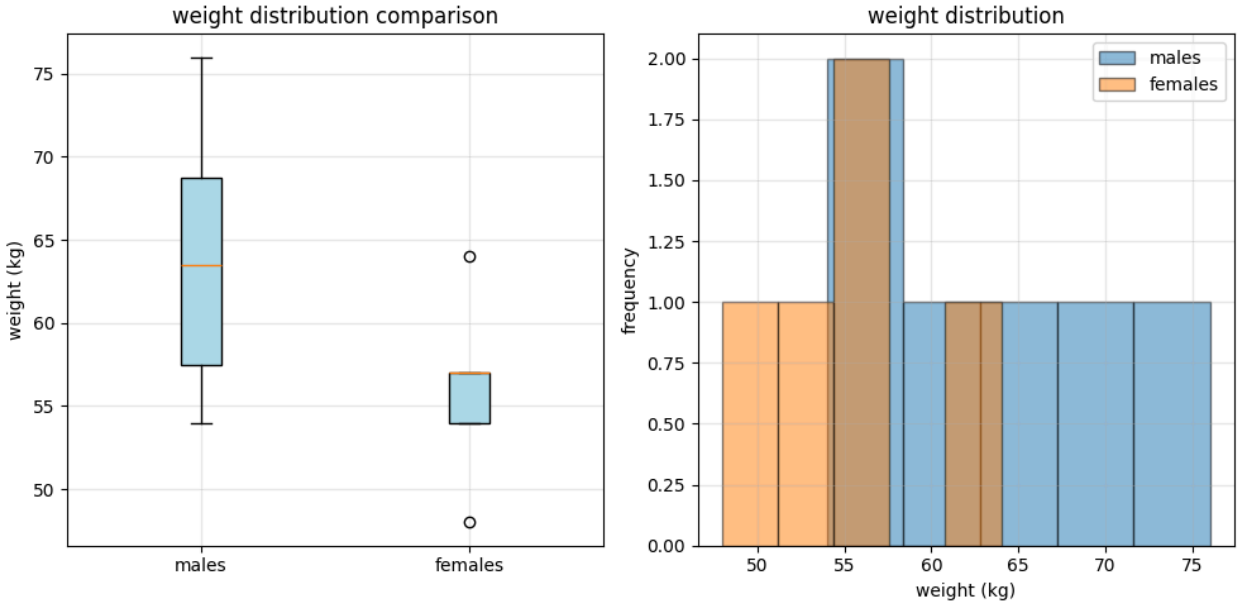
In [37]:
```
# actual difference in sample means
diff = np.mean(males) - np.mean(females)
print(f'observed difference = {diff:.2f} kg')
print(f'claimed difference = 7 kg')
print(f'is {diff:.2f} significantly less than 7?')
```

```
observed difference = 7.83 kg
claimed difference = 7 kg
is 7.83 significantly less than 7?
```

In [38]:
```
# visualization of two-sample comparison
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 5))

# boxplot comparison
plt.subplot(1, 2, 1)
plt.boxplot([males, females], labels=['males', 'females'], patch_artist=True,
            boxprops=dict(facecolor='lightblue'))
plt.ylabel('weight (kg)')
plt.title('weight distribution comparison')
plt.grid(True, alpha=0.3)
```

```
# histogram comparison
plt.subplot(1, 2, 2)
plt.hist(males, alpha=0.5, label='males', bins=5, edgecolor='black')
plt.hist(females, alpha=0.5, label='females', bins=5, edgecolor='black')
plt.xlabel('weight (kg)')
plt.ylabel('frequency')
plt.title('weight distribution')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f'\nmales: mean={np.mean(males):.2f}, std={np.std(males, ddof=1):.2f}')
print(f'females: mean={np.mean(females):.2f}, std={np.std(females, ddof=1):.2f
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_15027/1351096302.p
y:9: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dr
opped in 3.11.
  plt.boxplot([males, females], labels=['males', 'females'], patch_artist=True,
```




```
males: mean=63.83, std=8.35
females: mean=56.00, std=5.79
```

In [39]:

```
### practice problems — hypothesis testing

# q1. msrtc claims max time to reach kolhapur is 5 hours

# a. null hypothesis:
#    H₀: μ <= 5 (claim — travel time is at most 5 hours)
#    H₁: μ > 5 (right tail test)

# example data
```

```python
trip = [4.8, 5.2, 6.1, 4.9, 5.5, 5.8, 4.7, 5.3, 6.0, 5.1]

# b. python command and calculation
z_stat_q1, p_val_q1 = sswa.ztest(trip, value=5, alternative='larger')
print('q1: msrtc travel time test')
print(f'sample mean = {np.mean(trip):.2f} hours')
print(f'sample std = {np.std(trip, ddof=1):.2f}')
print(f'z-statistic = {z_stat_q1:.4f}')
print(f'p-value = {p_val_q1:.4f}')

# c. lower confidence bound (one-sided 95%)
alpha = 0.05
t_crit_q1 = t.ppf(1 - alpha, len(trip) - 1)
lower_bound_q1 = np.mean(trip) - t_crit_q1 * np.std(trip, ddof=1) / np.sqrt(le
print(f'95% lower bound = {lower_bound_q1:.2f} hours')
print(f'decision: lower_bound ({lower_bound_q1:.2f}) > 5? {lower_bound_q1 > 5}
if p_val_q1 < 0.05:
    print('reject H₀: evidence that mean > 5 hours')
else:
    print('fail to reject H₀: insufficient evidence')
print()

# q2. car manufacturer claims minimum mileage is 18 kmpl

# a. null hypothesis:
#    H₀: μ >= 18 (claim – mileage at least 18 kmpl)
#    H₁: μ < 18 (left tail test)

# example data
mileage = [17.6, 18.2, 17.9, 18.0, 17.4, 17.8, 18.1, 17.5, 17.7, 18.3]

# b. python command and calculation
z_stat_q2, p_val_q2 = sswa.ztest(mileage, value=18, alternative='smaller')
print('q2: car mileage test')
print(f'sample mean = {np.mean(mileage):.2f} kmpl')
print(f'sample std = {np.std(mileage, ddof=1):.2f}')
print(f'z-statistic = {z_stat_q2:.4f}')
print(f'p-value = {p_val_q2:.4f}')

# c. upper confidence bound (one-sided 95%)
t_crit_q2 = t.ppf(1 - alpha, len(mileage) - 1)
upper_bound_q2 = np.mean(mileage) + t_crit_q2 * np.std(mileage, ddof=1) / np.s
print(f'95% upper bound = {upper_bound_q2:.2f} kmpl')
print(f'decision: upper_bound ({upper_bound_q2:.2f}) < 18? {upper_bound_q2 < 1
if p_val_q2 < 0.05:
    print('reject H₀: evidence that mean < 18 kmpl')
else:
    print('fail to reject H₀: insufficient evidence')
print()

# q3. msrtc claims difference with ksrtc is not more than 1 hour

# a. null hypothesis:
```

```
#    H₀: μ_msrtc - μ_ksrtc <= 1 (claim – difference at most 1 hour)
#    H₁: μ_msrtc - μ_ksrtc > 1 (right tail test)

# example data
msrtc = [4.2, 3.8, 5.0, 4.5, 4.1, 4.8]
ksrtc = [3.1, 3.4, 3.6, 3.9, 3.2, 3.5]

# b. python command and calculation
z_stat_q3, p_val_q3 = sswa.ztest(msrtc, ksrtc, value=1, alternative='larger')
diff_q3 = np.mean(msrtc) - np.mean(ksrtc)
print('q3: msrtc vs ksrtc travel time difference')
print(f'msrtc mean = {np.mean(msrtc):.2f} hours')
print(f'ksrtc mean = {np.mean(ksrtc):.2f} hours')
print(f'observed difference = {diff_q3:.2f} hours')
print(f'claimed max difference = 1 hour')
print(f'z-statistic = {z_stat_q3:.4f}')
print(f'p-value = {p_val_q3:.4f}')
if p_val_q3 < 0.05:
    print('reject H₀: evidence that difference > 1 hour')
else:
    print('fail to reject H₀: insufficient evidence')
print()

# q4. i claim my students get minimum 10k more salary than ruchi's

# a. null hypothesis:
#    H₀: μ_sudeep - μ_ruchi >= 10 (claim – difference at least 10k)
#    H₁: μ_sudeep - μ_ruchi < 10 (left tail test)

# example data (salaries in thousands)
sudeep = [52, 54, 56, 55, 53, 57, 54, 56]
ruchi = [50, 51, 49, 50, 48, 51, 50, 49]

# b. python command and calculation
z_stat_q4, p_val_q4 = sswa.ztest(sudeep, ruchi, value=10, alternative='smaller
diff_q4 = np.mean(sudeep) - np.mean(ruchi)
print('q4: sudeep vs ruchi salary difference')
print(f'sudeep mean = {np.mean(sudeep):.2f}k')
print(f'ruchi mean = {np.mean(ruchi):.2f}k')
print(f'observed difference = {diff_q4:.2f}k')
print(f'claimed min difference = 10k')
print(f'z-statistic = {z_stat_q4:.4f}')
print(f'p-value = {p_val_q4:.4f}')
if p_val_q4 < 0.05:
    print('reject H₀: evidence that difference < 10k (claim not supported)')
else:
    print('fail to reject H₀: claim could be true')
```

```
q1: msrtc travel time test
sample mean = 5.34 hours
sample std = 0.50
z-statistic = 2.1629
p-value = 0.0153
95% lower bound = 5.05 hours
decision: lower_bound (5.05) > 5? True
reject H₀: evidence that mean > 5 hours

q2: car mileage test
sample mean = 17.85 kmpl
sample std = 0.30
z-statistic = -1.5667
p-value = 0.0586
95% upper bound = 18.03 kmpl
decision: upper_bound (18.03) < 18? False
fail to reject H₀: insufficient evidence

q3: msrtc vs ksrtc travel time difference
msrtc mean = 4.40 hours
ksrtc mean = 3.45 hours
observed difference = 0.95 hours
claimed max difference = 1 hour
z-statistic = -0.2286
p-value = 0.5904
fail to reject H₀: insufficient evidence

q4: sudeep vs ruchi salary difference
sudeep mean = 54.62k
ruchi mean = 49.75k
observed difference = 4.88k
claimed min difference = 10k
z-statistic = -7.3301
p-value = 0.0000
reject H₀: evidence that difference < 10k (claim not supported)
```

In [40]:
```
# paired t-test
```

In [41]:
```
# annova is used when we have more than 1 samples
# this test is used to compare the means of all the pairs of the samples and e
```

In [42]:
```
from statsmodels import stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
import statsmodels.stats.multicomp
import pandas as pd
```

In [43]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='faithful')
```

In [44]:
```
df.head()
```

Out[44]:

	eruptions	waiting
0	3.600	79
1	1.800	54
2	3.333	74
3	2.283	62
4	4.533	85

In [45]:
```
erupt = df.eruptions
```

In [46]:
```
ts = (len(erupt)-1) * np.var(erupt, ddof = 1)/1.42
```

In [47]:
```
ts
```

Out[47]:
```
np.float64(248.61928042408863)
```

In [49]:
```
p_value = chi.cdf(ts, len(erupt))
```

In [50]:
```
# ratios of variances of two samples will be compared aagainst two values
# continuity correction
```

In [ ]:
```
# you and establish weather the r difference is statistically significant or n
```

In [ ]:
```

```

In [13]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

paired t-test

paired t-test is used to compare the average of paired differences between two related samples (typically before and after measurements on the same subjects).

use cases:

- before and after treatment
- same subjects measured twice
- matched pairs experiments

formula: $t = (\bar{d} - d_0) / (s\_d / \sqrt{n})$

where:

- $\bar{d}$ = mean of differences
- d_0 = claimed difference (often 0)
- s_d = standard deviation of differences
- n = number of pairs

In [14]:
```
import numpy as np
import pandas as pd
import scipy
from scipy import stats
from scipy.stats import norm, t, binom, poisson, expon, chi2, f
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels
import statsmodels.api as sm
from statsmodels import stats as sm_stats
from statsmodels.stats import weightstats as sswa
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd
```

```
from statsmodels.stats import proportion as ssp
from statsmodels.stats.proportion import proportions_ztest
import os
import warnings
warnings.filterwarnings("ignore")
```

In [15]:
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

example: process improvement study

claim: after process revamp, turnaround time (tat) has come down by average of at least 0.7 minutes

hypotheses:

- H_0: μ_before - μ_after ≥ 0.7 (claim)
- H_1: μ_before - μ_after < 0.7 (left tail test)

In [16]:
```
# data: same subjects measured before and after
before = np.array([5, 8, 7, 6, 9, 8, 7])
after = np.array([4, 9, 5, 5, 9, 8, 6])

# calculate differences
diff = before - after
print('paired data:')
print(f'before: {before}')
print(f'after:  {after}')
print(f'diff:   {diff}')
print(f'\nmean difference = {np.mean(diff):.2f} minutes')
print(f'std of differences = {np.std(diff, ddof=1):.2f}')
```

```
paired data:
before: [5 8 7 6 9 8 7]
after:  [4 9 5 5 9 8 6]
diff:   [ 1 -1  2  1  0  0  1]

mean difference = 0.57 minutes
std of differences = 0.98
```

In [17]:
```
# method 1: using ttost_paired (two one-sided test)
result = sswa.ttost_paired(before, after, low=0.7, upp=0.7)


print(result)
# extract numeric p-values from the returned tuples
t_stat = result[0]
pval_left = result[1][1]
pval_right = result[2][1]

print('paired t-test results (ttost_paired):')
print(f'test statistic = {t_stat:.4f}')
```

```
print(f'p-value (left tail) = {pval_left:.4f}')
print(f'p-value (right tail) = {pval_right:.4f}')
print(f'\nfor left tail test, use left p-value: {pval_left:.4f}')
if pval_left < 0.05:
    print('reject H₀')
else:
    print('fail to reject H₀')
```

```
(np.float64(0.6303407448408251), (np.float64(-0.34856850115866744), np.float6
4(0.6303407448408251), np.float64(6.0)), (np.float64(-0.34856850115866744), n
p.float64(0.3696592551591749), np.float64(6.0)))
paired t-test results (ttost_paired):
test statistic = 0.6303
p-value (left tail) = 0.6303
p-value (right tail) = 0.3697

for left tail test, use left p-value: 0.6303
fail to reject H₀
```

In [18]:
```
# method 2: using scipy ttest_1samp on differences
t_stat, p_val = scipy.stats.ttest_1samp(before - after, popmean=0.7, alternati
print('paired t-test results (ttest_1samp):')
print(f't-statistic = {t_stat:.4f}')
print(f'p-value = {p_val:.4f}')
print(f'\ninterpretation:')
if p_val < 0.05:
    print('reject H₀: evidence that improvement is less than 0.7 min')
else:
    print('fail to reject H₀: claim is reasonable')
```

```
paired t-test results (ttest_1samp):
t-statistic = -0.3486
p-value = 0.3697

interpretation:
fail to reject H₀: claim is reasonable
```

one-way anova (analysis of variance)

one-way anova is used when comparing means of three or more independent groups to determine if at least one group mean is different.

hypotheses:

- H_0: all group means are equal ($\mu_1 = \mu_2 = \mu_3 = ...$)
- H_1: at least one group mean is different

key concepts:

- total variance = between-group variance + within-group variance
- f-statistic = (between-group variance) / (within-group variance)

- larger f-statistic indicates more evidence against H_0
- p-value tells us probability of observing this f-statistic if H_0 is true

In [19]:
```
# load marks data
df = pd.read_excel('data/Marks.xlsx')
df.head()
```

Out[19]:

	Subject	Trainer	Marks
0	Calculus	Sudeep	65
1	Calculus	Sudeep	75
2	Calculus	Sudeep	86
3	Calculus	Sudeep	82
4	Calculus	Sudeep	75

In [20]:
```
# check unique subjects
print('subjects in dataset:')
print(np.unique(df.Subject))
print(f'\ntrainers in dataset:')
print(np.unique(df.Trainer))
```

```
subjects in dataset:
['Calculus' 'Statistics' 'Trigno']

trainers in dataset:
['Ruchi' 'Sudeep']
```

example: comparing marks across subjects

question: do students score differently in different subjects?

hypotheses:

- H_0: mean marks are equal across all subjects
- H_1: at least one subject has different mean marks

In [21]:
```
# perform one-way anova for marks by subject
# formula: 'dependent_variable ~ independent_variable'
model = ols('Marks ~ Subject', df).fit()
anova_table = sm.stats.anova_lm(model)
print('one-way anova: marks by subject')
print(anova_table)
print(f'\np-value = {anova_table["PR(>F)"][0]:.4f}')
print(f'\ninterpretation:')
if anova_table['PR(>F)'][0] < 0.05:
    print('reject H₀: at least one subject has different mean marks')
```

```
else:
    print('fail to reject H₀: all subjects have equal mean marks')
```

```
one-way anova: marks by subject
            df       sum_sq    mean_sq         F    PR(>F)
Subject    2.0    15.158906   7.579453  0.082763  0.920735
Residual  38.0  3480.060606  91.580542       NaN       NaN

p-value = 0.9207

interpretation:
fail to reject H₀: all subjects have equal mean marks
```

In [22]:
```
# perform one-way anova for marks by trainer
model_trainer = ols('Marks ~ Trainer', df).fit()
anova_trainer = sm.stats.anova_lm(model_trainer)
print('one-way anova: marks by trainer')
print(anova_trainer)
print(f'\np-value = {anova_trainer["PR(>F)"][0]:.4f}')
print(f'\ninterpretation:')
if anova_trainer['PR(>F)'][0] < 0.05:
    print('reject H₀: trainers have different mean student marks')
else:
    print('fail to reject H₀: trainers have equal mean student marks')
```

```
one-way anova: marks by trainer
            df       sum_sq      mean_sq         F    PR(>F)
Trainer    1.0  1007.318546  1007.318546  15.79059  0.000296
Residual  39.0  2487.900966    63.792332       NaN       NaN

p-value = 0.0003

interpretation:
reject H₀: trainers have different mean student marks
```

anova background calculation - manual f-statistic

understanding how f-statistic is calculated helps interpret anova results better.

steps:

1. calculate grand mean (overall mean)
2. calculate total variance
3. calculate within-group variance
4. calculate between-group variance
5. f-statistic = (between variance / df_between) / (within variance / df_within)

In [23]:
```
# step 1: calculate grand mean
grand_mean = np.mean(df.Marks)
print(f'grand mean = {grand_mean:.2f}')
```

```
grand mean = 74.34
```

In [24]:
```
# step 2: calculate total variance
df['total_var'] = (df['Marks'] - grand_mean)**2
total_variance = np.sum(df.total_var)
print(f'total variance (ss_total) = {total_variance:.2f}')
```

```
total variance (ss_total) = 3495.22
```

In [25]:
```
# step 3: separate data by subject
df_calc = df[df.Subject == 'Calculus'].copy()
df_stats = df[df.Subject == 'Statistics'].copy()
df_trigno = df[df.Subject == 'Trigno'].copy()

# calculate group means
mean_calc = np.mean(df_calc.Marks)
mean_stats = np.mean(df_stats.Marks)
mean_trigno = np.mean(df_trigno.Marks)

print(f'calculus mean = {mean_calc:.2f}')
print(f'statistics mean = {mean_stats:.2f}')
print(f'trigonometry mean = {mean_trigno:.2f}')
```

```
calculus mean = 73.55
statistics mean = 75.07
trigonometry mean = 74.20
```

In [26]:
```
# step 4: calculate within-group variance
df_calc['within_var'] = (df_calc['Marks'] - mean_calc)**2
df_stats['within_var'] = (df_stats['Marks'] - mean_stats)**2
df_trigno['within_var'] = (df_trigno['Marks'] - mean_trigno)**2

ss_within = sum(df_calc['within_var']) + sum(df_stats['within_var']) + sum(df_
print(f'within-group variance (ss_within) = {ss_within:.2f}')
```

```
within-group variance (ss_within) = 3480.06
```

In [27]:
```
# step 5: calculate between-group variance
n_calc = len(df_calc)
n_stats = len(df_stats)
n_trigno = len(df_trigno)

ss_between = (n_calc * (mean_calc - grand_mean)**2 +
              n_stats * (mean_stats - grand_mean)**2 +
              n_trigno * (mean_trigno - grand_mean)**2)

print(f'between-group variance (ss_between) = {ss_between:.2f}')
print(f'\nverification: ss_total = ss_between + ss_within')
print(f'{total_variance:.2f} = {ss_between:.2f} + {ss_within:.2f}')
```

```
between-group variance (ss_between) = 15.16

verification: ss_total = ss_between + ss_within
3495.22 = 15.16 + 3480.06
```

In [28]:
```
# step 6: calculate f-statistic
k = 3  # number of groups
```

```
n = len(df)  # total observations
df_between = k - 1
df_within = n - k

ms_between = ss_between / df_between
ms_within = ss_within / df_within
f_statistic = ms_between / ms_within

print(f'degrees of freedom (between) = {df_between}')
print(f'degrees of freedom (within) = {df_within}')
print(f'mean square (between) = {ms_between:.2f}')
print(f'mean square (within) = {ms_within:.2f}')
print(f'f-statistic = {f_statistic:.4f}')
```

```
degrees of freedom (between) = 2
degrees of freedom (within) = 38
mean square (between) = 7.58
mean square (within) = 91.58
f-statistic = 0.0828
```

In [29]:
```
# step 7: calculate p-value
p_value = 1 - f.cdf(f_statistic, df_between, df_within)
print(f'p-value = {p_value:.4f}')
print(f'\ninterpretation:')
if p_value < 0.05:
    print('reject H₀: group means are different')
else:
    print('fail to reject H₀: group means are equal')
```

```
p-value = 0.9207

interpretation:
fail to reject H₀: group means are equal
```

post-hoc test: tukey hsd

after finding significant anova result, tukey hsd (honestly significant difference) test identifies which specific pairs of groups differ.

it performs pairwise comparisons while controlling for multiple testing error.

In [30]:
```
# tukey hsd for subject comparison
print('tukey hsd: pairwise subject comparison')
print(pairwise_tukeyhsd(df.Marks, df.Subject))
print(f'\ninterpretation:')
print('reject column shows if that pair has significantly different means')
print('false = means are not significantly different')
print('true = means are significantly different')
```

```
tukey hsd: pairwise subject comparison
    Multiple Comparison of Means - Tukey HSD, FWER=0.05
============================================================
  group1     group2   meandiff p-adj   lower   upper  reject
------------------------------------------------------------
  Calculus Statistics   1.5212 0.9156 -7.7434 10.7858  False
  Calculus     Trigno   0.6545 0.9838 -8.6101  9.9192  False
Statistics     Trigno  -0.8667 0.9667 -9.3889  7.6555  False
------------------------------------------------------------

interpretation:
reject column shows if that pair has significantly different means
false = means are not significantly different
true = means are significantly different
    Multiple Comparison of Means - Tukey HSD, FWER=0.05
============================================================
  group1     group2   meandiff p-adj   lower   upper  reject
------------------------------------------------------------
  Calculus Statistics   1.5212 0.9156 -7.7434 10.7858  False
  Calculus     Trigno   0.6545 0.9838 -8.6101  9.9192  False
Statistics     Trigno  -0.8667 0.9667 -9.3889  7.6555  False
------------------------------------------------------------

interpretation:
reject column shows if that pair has significantly different means
false = means are not significantly different
true = means are significantly different
```

In [31]:
```
# tukey hsd for trainer comparison
print('tukey hsd: pairwise trainer comparison')
print(pairwise_tukeyhsd(df.Marks, df.Trainer))
print(f'\ninterpretation:')
print('this shows whether the two trainers have significantly different studer
```

```
tukey hsd: pairwise trainer comparison
Multiple Comparison of Means - Tukey HSD, FWER=0.05
==================================================
group1 group2 meandiff p-adj  lower   upper  reject
--------------------------------------------------
 Ruchi Sudeep   9.9879 0.0003 4.9039 15.0719   True
--------------------------------------------------

interpretation:
this shows whether the two trainers have significantly different student marks
```

two-way anova

two-way anova analyzes the effect of two independent categorical variables on a continuous dependent variable.

advantages:

- tests main effect of each factor
- tests interaction effect between factors
- more efficient than running separate one-way anovas

example: marks depend on both subject and trainer

In [32]:
```
# two-way anova without interaction
model_2way = ols('Marks ~ Trainer + Subject', df).fit()
anova_2way = sm.stats.anova_lm(model_2way)
print('two-way anova (no interaction):')
print(anova_2way)
print(f'\ninterpretation:')
print(f'trainer effect p-value = {anova_2way["PR(>F)"][0]:.4f}')
print(f'subject effect p-value = {anova_2way["PR(>F)"][1]:.4f}')
```

```
two-way anova (no interaction):
            df       sum_sq      mean_sq          F    PR(>F)
Trainer    1.0  1007.318546  1007.318546  15.341424  0.000372
Subject    2.0    58.479354    29.239677   0.445319  0.644008
Residual  37.0  2429.421612    65.660044        NaN       NaN

interpretation:
trainer effect p-value = 0.0004
subject effect p-value = 0.6440
```

interaction effect in two-way anova

interaction effect occurs when the effect of one factor depends on the level of another factor.

example: perhaps certain trainers are better at teaching certain subjects.

hypothesis:

- H_0: there is no interaction between trainer and subject
- H_1: there is interaction between trainer and subject

In [33]:
```
# two-way anova with interaction (use * instead of +)
model_interaction = ols('Marks ~ Trainer * Subject', df).fit()
anova_interaction = sm.stats.anova_lm(model_interaction)
print('two-way anova (with interaction):')
print(anova_interaction)
print(f'\ninterpretation:')
print(f'trainer effect p-value = {anova_interaction["PR(>F)"][0]:.4f}')
print(f'subject effect p-value = {anova_interaction["PR(>F)"][1]:.4f}')
print(f'interaction effect p-value = {anova_interaction["PR(>F)"][2]:.4f}')
```

```
two-way anova (with interaction):
                   df       sum_sq      mean_sq          F    PR(>F)
Trainer           1.0  1007.318546  1007.318546  15.892045  0.000325
Subject           2.0    58.479354    29.239677   0.461302  0.634241
Trainer:Subject   2.0   210.943834   105.471917   1.663987  0.204016
Residual         35.0  2218.477778    63.385079        NaN       NaN

interpretation:
trainer effect p-value = 0.0003
subject effect p-value = 0.6342
interaction effect p-value = 0.2040
```

one-sample variance test (chi-square test)

this test compares sample variance against a claimed value to determine if the difference is statistically significant.

test statistic: $\chi^2 = (n-1) * s^2 / \sigma_0^2$

where:

- n = sample size
- s^2 = sample variance
- σ_0^2 = claimed population variance

the test statistic follows chi-square distribution with df = n-1

In [34]:
```
# load eruption data
df_faithful = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='faithful')
erupt = df_faithful.eruptions

print(f'sample size = {len(erupt)}')
print(f'sample variance = {np.var(erupt, ddof=1):.4f}')
```

```
sample size = 272
sample variance = 1.3027
```

example: left tail variance test

claim: minimum variance is 1.42

hypotheses:

- H_0: $\sigma^2 \geq 1.42$
- H_1: $\sigma^2 < 1.42$ (left tail test)

In [35]:
```
# case 1: left tail test
claim_var = 1.42
test_stat = (len(erupt) - 1) * np.var(erupt, ddof=1) / claim_var
```

```
p_value = chi2.cdf(test_stat, len(erupt) - 1)

print(f'left tail variance test:')
print(f'claimed variance = {claim_var}')
print(f'sample variance = {np.var(erupt, ddof=1):.4f}')
print(f'chi-square statistic = {test_stat:.4f}')
print(f'p-value = {p_value:.4f}')
print(f'\ninterpretation:')
if p_value < 0.05:
    print('reject H₀: evidence that variance < 1.42')
else:
    print('fail to reject H₀: difference can be attributed to chance variation
```

```
left tail variance test:
claimed variance = 1.42
sample variance = 1.3027
chi-square statistic = 248.6193
p-value = 0.1685

interpretation:
fail to reject H₀: difference can be attributed to chance variation
```

example: right tail variance test

claim: maximum variance is 1.1

hypotheses:

- H_0: $\sigma^2 \leq 1.1$
- H_1: $\sigma^2 > 1.1$ (right tail test)

In [36]:
```
# case 2: right tail test
claim_var2 = 1.1
test_stat2 = (len(erupt) - 1) * np.var(erupt, ddof=1) / claim_var2
p_value2 = 1 - chi2.cdf(test_stat2, len(erupt) - 1)

print(f'right tail variance test:')
print(f'claimed variance = {claim_var2}')
print(f'sample variance = {np.var(erupt, ddof=1):.4f}')
print(f'chi-square statistic = {test_stat2:.4f}')
print(f'p-value = {p_value2:.4f}')
print(f'\ninterpretation:')
if p_value2 < 0.05:
    print('reject H₀: evidence that variance > 1.1')
else:
    print('fail to reject H₀: claim is reasonable')
```

```
right tail variance test:
claimed variance = 1.1
sample variance = 1.3027
chi-square statistic = 320.9449
p-value = 0.0200

interpretation:
reject H₀: evidence that variance > 1.1
```

example: two-tail variance test

claim: variance equals 1.5

hypotheses:

- H_0: $\sigma^2 = 1.5$
- H_1: $\sigma^2 \neq 1.5$ (two tail test)

In [37]:
```python
# case 3: two tail test
claim_var3 = 1.5
test_stat3 = (len(erupt) - 1) * np.var(erupt, ddof=1) / claim_var3
p1 = chi2.cdf(test_stat3, len(erupt) - 1)
p2 = 1 - chi2.cdf(test_stat3, len(erupt) - 1)

# for two-tail test, take smaller p-value and multiply by 2
p_value3 = min(p1, p2) * 2

print(f'two tail variance test:')
print(f'claimed variance = {claim_var3}')
print(f'sample variance = {np.var(erupt, ddof=1):.4f}')
print(f'chi-square statistic = {test_stat3:.4f}')
print(f'p-value (left) = {p1:.4f}')
print(f'p-value (right) = {p2:.4f}')
print(f'final p-value = {p_value3:.4f}')
print(f'\ninterpretation:')
if p_value3 < 0.05:
    print('reject H₀: evidence that variance ≠ 1.5')
else:
    print('fail to reject H₀: variance could be 1.5')
```

```
two tail variance test:
claimed variance = 1.5
sample variance = 1.3027
chi-square statistic = 235.3596
p-value (left) = 0.0577
p-value (right) = 0.9423
final p-value = 0.1153

interpretation:
fail to reject H₀: variance could be 1.5
```

two-sample variance test (f-test)

this test compares variances of two independent samples to determine if their ratio is significantly different from a claimed value.

test statistic: $f = (s_1^2 / s_2^2) / \text{claimed\_ratio}$

the test statistic follows f-distribution with $df_1 = n_1\text{-}1$ and $df_2 = n_2\text{-}1$

In [38]:
```
# create two samples from eruption data
sample1 = erupt[30:75]
sample2 = erupt[130:185]

print(f'sample 1:')
print(f'  size = {len(sample1)}')
print(f'  variance = {np.var(sample1, ddof=1):.4f}')
print(f'\nsample 2:')
print(f'  size = {len(sample2)}')
print(f'  variance = {np.var(sample2, ddof=1):.4f}')
print(f'\nvariance ratio = {np.var(sample1, ddof=1) / np.var(sample2, ddof=1):
```

```
sample 1:
  size = 45
  variance = 1.5416

sample 2:
  size = 55
  variance = 1.3054

variance ratio = 1.1809
```

example: two-sample variance comparison

claim: variance of sample1 is at least 1.4 times variance of sample2

hypotheses:

- H_0: $\sigma_1^2 / \sigma_2^2 \geq 1.4$
- H_1: $\sigma_1^2 / \sigma_2^2 < 1.4$ (left tail test)

In [39]:
```
# perform f-test
claimed_ratio = 1.4
f_stat = (np.var(sample1, ddof=1) / np.var(sample2, ddof=1)) / claimed_ratio
p_value_f = f.cdf(f_stat, len(sample1) - 1, len(sample2) - 1)

print(f'two-sample variance test:')
print(f'claimed ratio = {claimed_ratio}')
print(f'observed ratio = {np.var(sample1, ddof=1) / np.var(sample2, ddof=1):.4
print(f'f-statistic = {f_stat:.4f}')
print(f'p-value = {p_value_f:.4f}')
```

```
print(f'\ninterpretation:')
if p_value_f < 0.05:
    print('reject H₀: evidence that ratio < 1.4')
else:
    print('fail to reject H₀: claim is reasonable')
```

```
two-sample variance test:
claimed ratio = 1.4
observed ratio = 1.1809
f-statistic = 0.8435
p-value = 0.2817

interpretation:
fail to reject H₀: claim is reasonable
```

tests for discrete data

when dealing with categorical or count data, we use different tests:

- proportion tests (z-test for proportions)
- chi-square tests (goodness of fit, independence)
- poisson rate tests

note: discrete data can be approximated as continuous for large samples (n > 10)

one-sample proportion test

this test compares sample proportion against a claimed value to determine if the difference is statistically significant.

commonly used for:

- success/failure rates
- approval ratings
- conversion rates
- yes/no survey responses

In [40]:
```
# example: percentage of freshers
# claim: percentage of freshers is limited to 70%
# H₀: p ≤ 0.7
# H₁: p > 0.7 (right tail test)

total_students = 75
freshers = 56
sample_prop = freshers / total_students

print(f'sample proportion = {sample_prop:.3f} or {sample_prop*100:.1f}%')
print(f'claimed proportion = 0.7 or 70%')
```

```
sample proportion = 0.747 or 74.7%
claimed proportion = 0.7 or 70%
```

In [41]:
```python
# perform proportion z-test
z_stat_prop, p_val_prop = ssp.proportions_ztest(freshers, total_students, valu

print('one-sample proportion test:')
print(f'z-statistic = {z_stat_prop:.4f}')
print(f'p-value = {p_val_prop:.4f}')
print(f'\ninterpretation:')
if p_val_prop < 0.05:
    print('reject H₀: evidence that proportion > 70%')
else:
    print('fail to reject H₀: proportion ≤ 70% is reasonable')
```

```
one-sample proportion test:
z-statistic = 0.9292
p-value = 0.1764

interpretation:
fail to reject H₀: proportion ≤ 70% is reasonable
```

two-sample proportion test

this test compares proportions from two independent samples to determine if their difference is statistically significant.

examples:

- comparing conversion rates between two marketing campaigns
- comparing success rates of two treatments
- comparing approval ratings between two groups

approximating discrete as continuous

for large sample sizes (n > 10), discrete distributions can be approximated by continuous distributions:

- binomial → normal
- poisson → normal

this simplifies calculations and provides good approximations.

In [42]:
```python
# example: binomial approximation
# probability of 200 rainy days out of 300, when p(rain) = 0.7

# exact using binomial
prob_binom = binom.cdf(200, 300, 0.7)
print(f'exact probability (binomial) = {prob_binom:.4f}')
```

```
# approximation using normal
mean_days = 300 * 0.7
std_days = np.sqrt(300 * 0.7 * (1 - 0.7))
prob_norm = norm.cdf(200, mean_days, std_days)
print(f'approximate probability (normal) = {prob_norm:.4f}')

# with continuity correction (200.5 instead of 200)
prob_norm_corrected = norm.cdf(200.5, mean_days, std_days)
print(f'approximate with correction = {prob_norm_corrected:.4f}')
print(f'\ncontinuity correction improves accuracy')
```

```
exact probability (binomial) = 0.1163
approximate probability (normal) = 0.1039
approximate with correction = 0.1157

continuity correction improves accuracy
```

In [44]:

```
# probability of incident between 3 to 5 hours
# can only use exponential (poisson gives count, not time intervals)
prob_interval = expon.cdf(5, scale=0.5) - expon.cdf(3, scale=0.5)
print(f'probability of incident between 3-5 hours = {prob_interval:.4f}')
```

```
probability of incident between 3-5 hours = 0.0024
```

In [1]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

interval estimation for mean

interval estimation helps us find a range where the true population parameter likely exists based on sample data. instead of giving a single point estimate, we provide a confidence interval.

key components:

- sample mean ($\bar{x}$) - calculated from sample data
- confidence level (cl) - typically 95% or 99%
- margin of error - depends on standard error and critical value
- critical value - from z or t distribution based on confidence level

formula for confidence interval:

- $ci = \bar{x} \pm z * (\sigma / \sqrt{n})$ when population std is known (z-distribution)
- $ci = \bar{x} \pm t * (s / \sqrt{n})$ when population std is unknown (t-distribution)

interpretation: we are 95% confident that the true population mean lies within this calculated interval.

In [ ]:
```
import numpy as np
import pandas as pd
import scipy
from scipy import stats
from scipy.stats import norm, t, binom, poisson, expon, chi2, f
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels
from statsmodels import stats as sm_stats
from statsmodels.stats import weightstats as sswa
import os
import warnings
```

```
warnings.filterwarnings("ignore")
```

In [ ]:
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

example: confidence interval for mean

given:

- sample size n = 100
- sample mean $\bar{x}$ = 45.7
- population std σ = 6.8
- confidence level = 95%

we want to find the interval estimate where the true population mean likely lies.

In [ ]:
```
# method 1: using z-distribution (manual calculation)
n = 100
sample_mean = 45.7
pop_std = 6.8
cl = 0.95

# critical values for 95% confidence (2.5% in each tail)
z_lower = norm.ppf(0.025)  # -1.96
z_upper = norm.ppf(0.975)  # +1.96

# calculate confidence interval bounds
lower_bound = sample_mean + z_lower * pop_std / np.sqrt(n)
upper_bound = sample_mean + z_upper * pop_std / np.sqrt(n)

print(f'95% confidence interval using z-distribution:')
print(f'lower bound = {lower_bound:.2f}')
print(f'upper bound = {upper_bound:.2f}')
print(f'\ninterpretation: we are 95% confident that true mean is between {lowe
```

In [ ]:
```
# method 2: using t-distribution (more accurate for smaller samples)
# critical values from t-distribution with df = n-1
t_lower = t.ppf((1-cl)/2, n-1)
t_upper = t.ppf(cl + (1-cl)/2, n-1)

lower_t = sample_mean + t_lower * pop_std / np.sqrt(n)
upper_t = sample_mean + t_upper * pop_std / np.sqrt(n)

print(f'95% confidence interval using t-distribution:')
print(f'lower bound = {lower_t:.2f}')
print(f'upper bound = {upper_t:.2f}')
```

In [ ]:
```
# method 3: using built-in scipy function
ci = scipy.stats.t.interval(cl, n-1, sample_mean, pop_std/np.sqrt(n))
print(f'95% confidence interval using scipy:')
print(f'interval = {ci}')
```

```
print(f'\nconclusion: 95% chance that actual population mean lies between {ci[
print(f'2.5% chance it may be less than {ci[0]:.2f}')
print(f'2.5% chance it may be greater than {ci[1]:.2f}')
```

interval estimation for poisson

for discrete count data following poisson distribution, we can also find confidence intervals. poisson distribution is used when counting occurrences of events (like number of calls per hour, defects per product, etc).

In [ ]:
```
# example: average 20 events occur, find 95% confidence interval
lambda_param = 20
poisson_ci = scipy.stats.poisson.interval(0.95, lambda_param)
print(f'95% confidence interval for poisson (λ=20):')
print(f'interval = {poisson_ci}')
print(f'\ninterpretation: we expect between {poisson_ci[0]} and {poisson_ci[1]
```

In [ ]:
```
# finding bounds using ppf (percent point function)
lower_poisson = poisson.ppf(0.025, 20)
upper_poisson = poisson.ppf(0.975, 20)
print(f'using ppf method:')
print(f'lower bound = {lower_poisson}')
print(f'upper bound = {upper_poisson}')
```

interval estimation for proportion

when working with proportions or percentages (like success rate, approval rating, conversion rate), we estimate the confidence interval for the true population proportion.

formula: $p \pm z * \sqrt{(p(1-p)/n)}$

where:

- p = sample proportion
- n = sample size
- z = critical value from standard normal distribution

In [ ]:
```
# example: out of 500 people surveyed, 200 responded positively
n = 500
r = 200  # number of positive responses
p = r / n  # sample proportion
sd = np.sqrt(p * (1 - p))  # standard deviation for proportion

# for 95% confidence level
z_lower = norm.ppf(0.025)
z_upper = norm.ppf(0.975)

# calculate bounds
```

```
lower_prop = p + z_lower * sd / np.sqrt(n)
upper_prop = p + z_upper * sd / np.sqrt(n)

print(f'sample proportion = {p:.3f} or {p*100:.1f}%')
print(f'\n95% confidence interval for proportion:')
print(f'lower bound = {lower_prop:.3f} ({lower_prop*100:.1f}%)')
print(f'upper bound = {upper_prop:.3f} ({upper_prop*100:.1f}%)')
print(f'\ninterpretation: true population proportion is between {lower_prop*10
```

hypothesis testing - introduction

hypothesis testing is a statistical method to make decisions about population parameters based on sample data.

key concepts:

- null hypothesis (H_0): the claim we assume to be true (status quo)
- alternative hypothesis (H_1): what we want to prove
- p-value: probability of observing the data if H_0 is true
- significance level (α): threshold for decision, typically 0.05
- decision rule: if p-value < α, reject H_0; otherwise, fail to reject H_0

types of tests:

- left tail: H_1 states parameter is less than claim
- right tail: H_1 states parameter is greater than claim
- two tail: H_1 states parameter is not equal to claim

one-sample z-test / one-sample t-test

purpose: compare sample mean against a claimed value and determine if the difference is statistically significant.

when to use:

- z-test: large sample ($n \geq 30$) or known population σ
- t-test: small sample ($n < 30$) with unknown population σ

formulas:

- $z = (\bar{x} - \mu_0) / (\sigma / \sqrt{n})$
- $t = (\bar{x} - \mu_0) / (s / \sqrt{n})$

where:

- $\bar{x}$ = sample mean

- μ_0 = claimed population mean
- σ = population standard deviation (z-test)
- s = sample standard deviation (t-test)
- n = sample size

In [ ]:
```
# load data for hypothesis testing examples
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='faithful')
df.head()
```

In [ ]:
```
erupt = df.eruptions
print(f'sample size = {len(erupt)}')
print(f'sample mean = {np.mean(erupt):.2f}')
print(f'sample std = {np.std(erupt, ddof=1):.2f}')
```

example: left tail test

claim: population mean eruption time is at least 3.6 minutes

hypotheses:

- H_0: $\mu \geq 3.6$ (claim - population mean is at least 3.6)
- H_1: $\mu < 3.6$ (alternative - we want to prove mean is less than 3.6)

this is a left tail test because H_1 has < symbol. significance level $\alpha = 0.05$ (confidence level = 95%)

In [ ]:
```
# perform one-sample z-test
z_stat, p_value = sswa.ztest(erupt, value=3.6, alternative='smaller')

print('one-sample z-test results:')
print(f'z-statistic = {z_stat:.4f}')
print(f'p-value = {p_value:.4f}')
print(f'\ndecision:')
if p_value < 0.05:
    print('reject H₀: evidence that mean < 3.6')
else:
    print('fail to reject H₀: insufficient evidence that mean < 3.6')

print(f'\ninterpretation:')
print(f'p-value ({p_value:.4f}) tells us the probability of observing this sam
print(f'mean if true population mean is actually 3.6')
print(f'since p-value > 0.05, difference can be attributed to chance variation
```

In [ ]:
```
# manual calculation of z-statistic
sample_mean = np.mean(erupt)
claim_value = 3.6
sample_std = np.std(erupt, ddof=1)
n = len(erupt)
```

```
# formula: z = (x̄ - μ₀) / (s / √n)
z_manual = (sample_mean - claim_value) / (sample_std / np.sqrt(n))
print(f'manual calculation:')
print(f'z-statistic = ({sample_mean:.2f} - {claim_value}) / ({sample_std:.2f}
print(f'z-statistic = {z_manual:.4f}')
```

In [ ]:
```
# finding critical value for left tail test (α = 0.05)
t_crit = t.ppf(0.05, n-1)
print(f't-critical value (df={n-1}) = {t_crit:.4f}')
print(f'\ndecision rule: if t-statistic < {t_crit:.4f}, reject H₀')
print(f'our t-statistic ({z_manual:.4f}) is not less than {t_crit:.4f}')
print(f'therefore, we fail to reject H₀')
```

In [ ]:
```
# calculating 95% upper confidence bound (one-sided)
# for left tail test, we calculate upper bound
# formula: x̄ + t_critical * (s / √n)

t_crit_positive = t.ppf(0.95, n-1)  # for upper bound
upper_bound = np.mean(erupt) + t_crit_positive * np.std(erupt, ddof=1) / np.sq

print(f'95% upper confidence bound = {upper_bound:.4f}')
print(f'sample mean = {np.mean(erupt):.4f}')
print(f'claim value = 3.6')
print(f'\ninterpretation:')
print(f'we are 95% confident that true mean ≤ {upper_bound:.4f}')
print(f'since {upper_bound:.4f} > 3.6, the claim (mean ≥ 3.6) is reasonable')
```

example: right tail test

claim: maximum eruption time is 3.42 minutes

hypotheses:

- H_0: $\mu \leq 3.42$ (claim - mean is at most 3.42)
- H_1: $\mu > 3.42$ (alternative - mean is greater than 3.42)

this is a right tail test because H_1 has > symbol.

In [ ]:
```
# perform right tail test
z_stat_right, p_val_right = sswa.ztest(erupt, value=3.42, alternative='larger'

print('right tail test results:')
print(f'z-statistic = {z_stat_right:.4f}')
print(f'p-value = {p_val_right:.4f}')
print(f'\ndecision:')
if p_val_right < 0.05:
    print('reject H₀: evidence that mean > 3.42')
else:
    print('fail to reject H₀: insufficient evidence that mean > 3.42')
```

In [ ]:
```
# calculating 95% lower confidence bound (one-sided)
# for right tail test, we calculate lower bound
# formula: x¯ - t_critical * (s / √n)

t_crit_positive = t.ppf(0.95, n-1)
lower_bound = np.mean(erupt) - t_crit_positive * np.std(erupt, ddof=1) / np.sc

print(f'95% lower confidence bound = {lower_bound:.4f}')
print(f'sample mean = {np.mean(erupt):.4f}')
print(f'claim value = 3.42')
print(f'\ninterpretation:')
print(f'we are 95% confident that true mean ≥ {lower_bound:.4f}')
print(f'since {lower_bound:.4f} > 3.42, there is evidence mean > 3.42')
```

practice: hypothesis test with small sample

claim: population mean > 6

hypotheses:

- H_0: $\mu \leq 6$
- H_1: $\mu > 6$ (right tail test)

In [ ]:
```
my_list = [3, 6, 5, 4, 6, 7, 9, 6, 2]
my_array = np.array(my_list)

print(f'sample size = {len(my_list)}')
print(f'sample mean = {np.mean(my_list):.4f}')
print(f'sample std = {np.std(my_list, ddof=1):.4f}')
```

In [ ]:
```
# perform right tail test
z_stat, p_val = sswa.ztest(my_list, value=6, alternative='larger')

print('test results:')
print(f'z-statistic = {z_stat:.4f}')
print(f'p-value = {p_val:.4f}')
print(f'\ndecision: fail to reject H₀ (p-value > 0.05)')
print(f'conclusion: insufficient evidence that mean > 6')
```

In [ ]:
```
# manual t-statistic calculation
t_manual = (np.mean(my_list) - 6) / (np.std(my_list, ddof=1) / np.sqrt(len(my_
print(f't-statistic = {t_manual:.4f}')
print(f'\ninterpretation: sample mean is {t_manual:.4f} standard errors away f
```

In [ ]:
```
# calculate p-value using t-distribution
p_t = 1 - t.cdf(t_manual, len(my_list) - 1)  # right tail
print(f'p-value from t-distribution = {p_t:.4f}')
print(f'\nsince p-value > 0.05, we fail to reject H₀')
print(f'the sample does not provide strong evidence that mean > 6')
```

two-sample z-test / two-sample t-test

purpose: compare means of two independent samples and determine if their difference is statistically significant.

formula for test statistic:

- $z = (\bar{x}_1 - \bar{x}_2 - d_0) / SE$
- where $SE = \sqrt{(\sigma_1^2/n_1 + \sigma_2^2/n_2)}$

for t-test:

- $t = (\bar{x}_1 - \bar{x}_2 - d_0) / SE$
- where $SE = \sqrt{(s_1^2/n_1 + s_2^2/n_2)}$

d_0 is the claimed difference (often 0)

use cases:

- compare treatment vs control groups
- compare before vs after
- compare two populations

example: comparing two groups

claim: average weight difference between males and females is at least 7 kg

hypotheses:

- H_0: μ_males - μ_females ≥ 7
- H_1: μ_males - μ_females < 7 (left tail test)

In [ ]:
```
males = [76, 56, 62, 65, 54, 70]
females = [54, 57, 48, 64, 57]

print(f'males:')
print(f'  n = {len(males)}')
print(f'  mean = {np.mean(males):.2f} kg')
print(f'  std = {np.std(males, ddof=1):.2f} kg')
print(f'\nfemales:')
print(f'  n = {len(females)}')
print(f'  mean = {np.mean(females):.2f} kg')
print(f'  std = {np.std(females, ddof=1):.2f} kg')
print(f'\nobserved difference = {np.mean(males) - np.mean(females):.2f} kg')
```

In [ ]:
```
# perform two-sample z-test
z_stat_2samp, p_val_2samp = sswa.ztest(males, females, value=7, alternative='s
```

```
print('two-sample z-test results:')
print(f'z-statistic = {z_stat_2samp:.4f}')
print(f'p-value = {p_val_2samp:.4f}')
print(f'\ndecision:')
if p_val_2samp < 0.05:
    print('reject H₀: evidence that difference < 7 kg')
else:
    print('fail to reject H₀: claim that difference ≥ 7 kg is reasonable')
```

In [ ]:
```
# visualization: comparing two groups
plt.figure(figsize=(12, 5))

# boxplot comparison
plt.subplot(1, 2, 1)
plt.boxplot([males, females], labels=['males', 'females'],
            patch_artist=True, boxprops=dict(facecolor='lightblue'))
plt.ylabel('weight (kg)')
plt.title('weight distribution comparison')
plt.grid(True, alpha=0.3)

# histogram comparison
plt.subplot(1, 2, 2)
plt.hist(males, alpha=0.5, label='males', bins=5, edgecolor='black', color='bl
plt.hist(females, alpha=0.5, label='females', bins=5, edgecolor='black', color
plt.xlabel('weight (kg)')
plt.ylabel('frequency')
plt.title('weight distribution')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

practice problems - hypothesis testing

solve the following hypothesis testing problems using appropriate tests.

In [ ]:
```
# q1. msrtc claims max time to reach kolhapur is 5 hours

# hypotheses:
# H₀: μ ≤ 5 (claim - travel time is at most 5 hours)
# H₁: μ > 5 (right tail test)

# example data (hours)
trip = [4.8, 5.2, 6.1, 4.9, 5.5, 5.8, 4.7, 5.3, 6.0, 5.1]

# perform test
z_stat_q1, p_val_q1 = sswa.ztest(trip, value=5, alternative='larger')
print('q1: msrtc travel time test')
print(f'sample mean = {np.mean(trip):.2f} hours')
print(f'sample std = {np.std(trip, ddof=1):.2f}')
```

```
print(f'z-statistic = {z_stat_q1:.4f}')
print(f'p-value = {p_val_q1:.4f}')

# calculate lower bound (one-sided 95%)
alpha = 0.05
t_crit_q1 = t.ppf(1 - alpha, len(trip) - 1)
lower_bound_q1 = np.mean(trip) - t_crit_q1 * np.std(trip, ddof=1) / np.sqrt(le
print(f'95% lower bound = {lower_bound_q1:.2f} hours')
print(f'decision: lower_bound ({lower_bound_q1:.2f}) > 5? {lower_bound_q1 > 5}

if p_val_q1 < 0.05:
    print('reject H₀: evidence that mean > 5 hours')
else:
    print('fail to reject H₀: claim is reasonable')
print()
```

In [ ]:
```
# q2. car manufacturer claims minimum mileage is 18 kmpl

# hypotheses:
# H₀: μ ≥ 18 (claim - mileage at least 18 kmpl)
# H₁: μ < 18 (left tail test)

# example data (kmpl)
mileage = [17.6, 18.2, 17.9, 18.0, 17.4, 17.8, 18.1, 17.5, 17.7, 18.3]

# perform test
z_stat_q2, p_val_q2 = sswa.ztest(mileage, value=18, alternative='smaller')
print('q2: car mileage test')
print(f'sample mean = {np.mean(mileage):.2f} kmpl')
print(f'sample std = {np.std(mileage, ddof=1):.2f}')
print(f'z-statistic = {z_stat_q2:.4f}')
print(f'p-value = {p_val_q2:.4f}')

# calculate upper bound (one-sided 95%)
t_crit_q2 = t.ppf(1 - alpha, len(mileage) - 1)
upper_bound_q2 = np.mean(mileage) + t_crit_q2 * np.std(mileage, ddof=1) / np.s
print(f'95% upper bound = {upper_bound_q2:.2f} kmpl')
print(f'decision: upper_bound ({upper_bound_q2:.2f}) < 18? {upper_bound_q2 < 1

if p_val_q2 < 0.05:
    print('reject H₀: evidence that mean < 18 kmpl')
else:
    print('fail to reject H₀: claim is reasonable')
print()
```

In [ ]:
```
# q3. msrtc claims difference with ksrtc is not more than 1 hour

# hypotheses:
# H₀: μ_msrtc - μ_ksrtc ≤ 1 (claim - difference at most 1 hour)
# H₁: μ_msrtc - μ_ksrtc > 1 (right tail test)

# example data (hours)
msrtc = [4.2, 3.8, 5.0, 4.5, 4.1, 4.8]
```

```
ksrtc = [3.1, 3.4, 3.6, 3.9, 3.2, 3.5]

# perform test
z_stat_q3, p_val_q3 = sswa.ztest(msrtc, ksrtc, value=1, alternative='larger')
diff_q3 = np.mean(msrtc) - np.mean(ksrtc)
print('q3: msrtc vs ksrtc travel time difference')
print(f'msrtc mean = {np.mean(msrtc):.2f} hours')
print(f'ksrtc mean = {np.mean(ksrtc):.2f} hours')
print(f'observed difference = {diff_q3:.2f} hours')
print(f'claimed max difference = 1 hour')
print(f'z-statistic = {z_stat_q3:.4f}')
print(f'p-value = {p_val_q3:.4f}')

if p_val_q3 < 0.05:
    print('reject H₀: evidence that difference > 1 hour')
else:
    print('fail to reject H₀: claim is reasonable')
print()
```

In [ ]:

```
# q4. instructor claims students get minimum 10k more salary than another batc

# hypotheses:
# H₀: μ_batch1 - μ_batch2 ≥ 10 (claim - difference at least 10k)
# H₁: μ_batch1 - μ_batch2 < 10 (left tail test)

# example data (salaries in thousands)
batch1 = [52, 54, 56, 55, 53, 57, 54, 56]
batch2 = [50, 51, 49, 50, 48, 51, 50, 49]

# perform test
z_stat_q4, p_val_q4 = sswa.ztest(batch1, batch2, value=10, alternative='smalle
diff_q4 = np.mean(batch1) - np.mean(batch2)
print('q4: batch salary comparison')
print(f'batch1 mean = {np.mean(batch1):.2f}k')
print(f'batch2 mean = {np.mean(batch2):.2f}k')
print(f'observed difference = {diff_q4:.2f}k')
print(f'claimed min difference = 10k')
print(f'z-statistic = {z_stat_q4:.4f}')
print(f'p-value = {p_val_q4:.4f}')

if p_val_q4 < 0.05:
    print('reject H₀: evidence that difference < 10k (claim not supported)')
else:
    print('fail to reject H₀: claim could be true')
```

In [1]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

advanced statistical tests

this notebook covers chi-square tests, proportion tests, poisson rate tests, and variance tests used in statistical analysis.

In [151…
```
import numpy as np
import pandas as pd
import scipy
from scipy import stats
from scipy.stats import norm, t, binom, poisson, expon, chi2, f
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels
import statsmodels.api as sm
from statsmodels import stats as sm_stats
from statsmodels.stats import weightstats as sswa
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd
from statsmodels.stats import proportion as ssp
from statsmodels.stats.proportion import proportions_ztest
import os
import warnings
warnings.filterwarnings("ignore")
from scipy.stats import chi2_contingency
```

In [152…
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

In [153…
```
# example: cdac mumbai vs pune engineers
# claim: mumbai has at least 12% more engineers than pune

# hypotheses:
# H₀: prop(mumbai) - prop(pune) >= 0.12 (claim)
# H₁: prop(mumbai) - prop(pune) < 0.12 (left tail test)

# data
```

```python
# mumbai: 45 engineers out of 70 students
# pune: 60 engineers out of 100 students

engg = np.array([45, 60])
counts = np.array([70, 100])

# perform two-sample proportion test
z_stat, p_value = ssp.proportions_ztest(engg, counts, value=0.12, alternative=
print(f'z-statistic: {z_stat:.4f}')
print(f'p-value: {p_value:.4f}')

# calculate actual difference
actual_diff = (45/70) - (60/100)
print(f'\nactual difference: {actual_diff:.4f} ({actual_diff*100:.2f}%)')
print(f'claimed difference: 0.12 (12%)')

print(f'\ndecision:')
if p_value > 0.05:
    print('p-value > 0.05: fail to reject H₀')
    print('conclusion: claim that difference >= 12% is reasonable')
else:
    print('p-value < 0.05: reject H₀')
    print('conclusion: difference is less than 12%')
```

```
z-statistic: -1.0186
p-value: 0.1542

actual difference: 0.0429 (4.29%)
claimed difference: 0.12 (12%)

decision:
p-value > 0.05: fail to reject H₀
conclusion: claim that difference >= 12% is reasonable
```

---

two-sample proportion test

what it does: compares the proportions (percentages) between two groups to see if the difference is real or just by chance.

when to use:

- comparing success rates between two groups
- testing if proportion difference matches a claimed value
- example: comparing pass rates between two colleges

how it works:

- takes count of successes and total count from both groups
- tests if the difference between proportions is statistically significant

In [154…
```
# example: car brand market share
# claimed pattern: 35%, 30%, 15%, 12%, 8% for 5 brands
# observed counts from sample

pattern = np.array([0.35, 0.30, 0.15, 0.12, 0.08])
obs_counts = np.array([120, 105, 80, 50, 30])

print(f'total observations: {sum(obs_counts)}')
```

```
total observations: 385
```

---

chi-square goodness of fit test

what it does: checks if observed data matches an expected pattern or distribution.

when to use:

- testing if data follows a claimed distribution
- comparing observed frequencies vs expected frequencies
- single categorical variable with multiple categories

example: market share of car brands - does actual sales match claimed percentages?

In [155…
```
# calculate expected counts based on pattern
exp_counts = pattern * sum(obs_counts)
print('expected counts based on claimed pattern:')
print(exp_counts)
```

```
expected counts based on claimed pattern:
[134.75 115.5   57.75  46.2   30.8 ]
```

In [156…
```
# perform chi-square goodness of fit test
chi_stat, p_value = scipy.stats.chisquare(obs_counts, f_exp=exp_counts)

print(f'chi-square statistic: {chi_stat:.4f}')
print(f'p-value: {p_value:.4f}')
print(f'degrees of freedom: {len(obs_counts) - 1}')

print(f'\ndecision:')
if p_value < 0.05:
    print('p-value < 0.05: reject H₀')
    print('conclusion: observed counts do NOT match the claimed pattern')
else:
    print('p-value >= 0.05: fail to reject H₀')
    print('conclusion: observed counts match the claimed pattern')
```

```
chi-square statistic: 11.4750
p-value: 0.0217
degrees of freedom: 4

decision:
p-value < 0.05: reject H₀
conclusion: observed counts do NOT match the claimed pattern
```

In [157…
```python
# manual calculation: chi-square contribution from each category
# formula: (observed - expected)² / expected
ts_array = (obs_counts - exp_counts) ** 2 / exp_counts

print('chi-square contribution by category:')
for i, contribution in enumerate(ts_array):
    print(f'  category {i+1}: {contribution:.4f}')

print(f'\ntotal chi-square statistic: {sum(ts_array):.4f}')
print(f'category with highest contribution: {np.argmax(ts_array) + 1} (value:
```

```
chi-square contribution by category:
  category 1: 1.6146
  category 2: 0.9545
  category 3: 8.5725
  category 4: 0.3126
  category 5: 0.0208

total chi-square statistic: 11.4750
category with highest contribution: 3 (value: 8.5725)
```

In [158…
```python
# example: equal preference test
# testing if all categories have equal preference

pattern = np.array([0.25, 0.25, 0.25, 0.25])
obs_count = np.array([5, 5, 5, 5])

exp_counts = sum(obs_count) * pattern

chi_stat, p_value = scipy.stats.chisquare(obs_count, exp_counts)
print(f'chi-square statistic: {chi_stat:.4f}')
print(f'p-value: {p_value:.4f}')
print(f'\nwith perfect equal distribution, chi-square = 0')
```

```
chi-square statistic: 0.0000
p-value: 1.0000

with perfect equal distribution, chi-square = 0
```

---

chi-square test of independence

what it does: tests if two categorical variables are related or independent of each other.

when to use:

- checking if choice in one variable depends on another variable
- testing association between two categorical variables
- example: does food preference depend on gender?

how it works:

- creates contingency table (cross-tabulation)
- compares observed frequencies with what we'd expect if variables were independent

In [159…
```
# example: food preference vs gender
# H₀: food preference is independent of gender (no association)
# H₁: food preference depends on gender (association exists)

# contingency table:
#              veg    nonveg   fastfood
# male         110     150       40
# female       120      80       90

data = np.array([[110, 150, 40],   # male
                 [120, 80, 90]])    # female

print('observed data (contingency table):')
print('        veg  nonveg  fastfood')
print(f'male    {data[0,0]:3d}   {data[0,1]:3d}     {data[0,2]:3d}')
print(f'female  {data[1,0]:3d}    {data[1,1]:2d}      {data[1,2]:2d}')
```

```
observed data (contingency table):
        veg  nonveg  fastfood
male    110   150      40
female  120    80      90
```

In [160…
```
# perform chi-square test of independence
chi_stat, p_value, dof, expected_freq = chi2_contingency(data)

print(f'\nchi-square statistic: {chi_stat:.4f}')
print(f'p-value: {p_value:.4f}')
print(f'degrees of freedom: {dof}')
print(f'  formula: (rows - 1) × (columns - 1) = (2-1) × (3-1) = {dof}')

print(f'\ndecision:')
if p_value < 0.05:
    print('p-value < 0.05: reject H₀')
    print('conclusion: food preference DEPENDS on gender (variables are associ
else:
    print('p-value >= 0.05: fail to reject H₀')
    print('conclusion: food preference is independent of gender')
```

```
chi-square statistic: 40.8121
p-value: 0.0000
degrees of freedom: 2
  formula: (rows - 1) × (columns - 1) = (2-1) × (3-1) = 2

decision:
p-value < 0.05: reject H₀
conclusion: food preference DEPENDS on gender (variables are associated)
```

In [161…
```python
# extract expected frequencies (what we'd expect if variables were independent
exp_values = chi2_contingency(data)[3]

print('expected frequencies (if independent):')
print(exp_values)
print('\nthese are the counts we would expect if gender had no effect on food
```

```
expected frequencies (if independent):
[[116.94915254 116.94915254  66.10169492]
 [113.05084746 113.05084746  63.89830508]]

these are the counts we would expect if gender had no effect on food choice
```

In [162…
```python
# manual calculation: chi-square contributions
# shows which cells contribute most to the association
ts = (exp_values - data)**2 / exp_values

print('chi-square contribution by cell:')
print(ts)
print(f'\ntotal chi-square statistic: {sum(sum(ts)):.4f}')
print('\nhigher values indicate cells where observed differs most from expecte
```

```
chi-square contribution by cell:
[[ 0.41292066  9.34045689 10.30682312]
 [ 0.4271593   9.66254161 10.66223081]]

total chi-square statistic: 40.8121

higher values indicate cells where observed differs most from expected
```

In [163…
```python
# example: accident rate test
# claim: accidents are limited to maximum 12 per day
# H₀: rate <= 12 per day
# H₁: rate > 12 per day (right tail test)

# data: 135 accidents in 10 days
# observed rate: 135/10 = 13.5 per day

observed_accidents = 135
time_period = 10  # days
claimed_rate = 12  # per day

observed_rate = observed_accidents / time_period
print(f'observed rate: {observed_rate:.2f} accidents per day')
print(f'claimed maximum rate: {claimed_rate} accidents per day')
```

```
observed rate: 13.50 accidents per day
claimed maximum rate: 12 accidents per day
```

---

one-sample poisson rate test

what it does: tests if the rate at which events occur matches a claimed rate.

when to use:

- testing event rates (accidents per day, calls per hour, defects per batch)
- count data where events occur randomly over time/space
- example: testing if accident rate exceeds safety limit

key concept: rate = total events / time period

In [164…
```python
# perform poisson rate test
from statsmodels.stats.rates import test_poisson

z_stat, p_value = test_poisson(135, 10, 12, method='score', alternative='large

print(f'\nz-statistic: {z_stat:.4f}')
print(f'p-value: {p_value:.4f}')

print(f'\ndecision:')
if p_value < 0.05:
    print('p-value < 0.05: reject H₀')
    print('conclusion: accident rate EXCEEDS 12 per day (claim violated)')
else:
    print('p-value >= 0.05: fail to reject H₀')
    print('conclusion: accident rate does not exceed 12 per day (claim holds)'
```

```
z-statistic: 1.3693
p-value: 0.0855

decision:
p-value >= 0.05: fail to reject H₀
conclusion: accident rate does not exceed 12 per day (claim holds)
```

In [165…
```python
# example: mumbai vs bangalore accident rates
# claim: mumbai has at least 4 more accidents per day than bangalore
# H₀: rate(mumbai) - rate(bangalore) >= 4
# H₁: rate(mumbai) - rate(bangalore) < 4 (left tail test)

# data:
# mumbai: 135 accidents in 10 days
# bangalore: 130 accidents in 15 days

mumbai_accidents = 135
mumbai_days = 10
bangalore_accidents = 130
```

```
bangalore_days = 15

mumbai_rate = mumbai_accidents / mumbai_days
bangalore_rate = bangalore_accidents / bangalore_days
actual_diff = mumbai_rate - bangalore_rate

print(f'mumbai rate: {mumbai_rate:.2f} accidents/day')
print(f'bangalore rate: {bangalore_rate:.2f} accidents/day')
print(f'actual difference: {actual_diff:.2f} accidents/day')
print(f'claimed difference: 4.0 accidents/day')
```

```
mumbai rate: 13.50 accidents/day
bangalore rate: 8.67 accidents/day
actual difference: 4.83 accidents/day
claimed difference: 4.0 accidents/day
```

---

two-sample poisson rate test

what it does: compares event rates between two groups to see if the difference is significant.

when to use:

- comparing accident rates between two cities
- comparing defect rates between two production lines
- testing if rate difference matches a claimed value

formula: rate difference = $(\text{events}_1 / \text{time}_1) - (\text{events}_2 / \text{time}_2)$

In [166…

```
# perform two-sample poisson rate test
from statsmodels.stats.rates import test_poisson_2indep

z_stat, p_value = test_poisson_2indep(135, 10, 130, 15,
                                      value=4,
                                      compare='diff',
                                      method='score',
                                      alternative='smaller')

print(f'\nz-statistic: {z_stat:.4f}')
print(f'p-value: {p_value:.4f}')

print(f'\ndecision:')
if p_value < 0.05:
    print('p-value < 0.05: reject H₀')
    print('conclusion: difference is LESS than 4 accidents/day')
else:
    print('p-value >= 0.05: fail to reject H₀')
    print('conclusion: mumbai has at least 4 more accidents/day than bangalore
```

```
z-statistic: 0.6064
p-value: 0.7279

decision:
p-value >= 0.05: fail to reject H₀
conclusion: mumbai has at least 4 more accidents/day than bangalore
```

---

one-sample variance test (chi-square test)

what it does: tests if population variance matches a claimed value using sample data.

when to use:

- testing consistency/variability of a process
- quality control - checking if variation is within acceptable limits
- example: testing if eruption time variability is as claimed

key point: uses chi-square distribution because variance involves squared deviations.

In [167…
```python
# load geyser eruption data
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='faithful')
erupt = df.eruptions

print(f'sample size: {len(erupt)}')
print(f'sample mean: {np.mean(erupt):.4f} minutes')
print(f'sample std: {np.std(erupt, ddof=1):.4f} minutes')
```

```
sample size: 272
sample mean: 3.4878 minutes
sample std: 1.1414 minutes
```

In [168…
```python
# calculate sample variance
sample_var = np.var(erupt, ddof=1)
print(f'sample variance: {sample_var:.4f}')
```

```
sample variance: 1.3027
```

In [169…
```python
# for 95% confidence interval on variance (two-tailed)
# need chi-square critical values at 2.5% and 97.5%

n = len(erupt)
df_chi = n - 1

# lower critical value (2.5% tail)
chi_lower = scipy.stats.chi2.ppf(0.025, df_chi)
print(f'chi-square lower critical value (2.5%): {chi_lower:.4f}')
print(f'degrees of freedom: {df_chi}')
```

```
chi-square lower critical value (2.5%): 227.2931
degrees of freedom: 271
```

In [170…
```
# upper critical value (97.5% percentile)
chi_upper = scipy.stats.chi2.ppf(0.975, df_chi)
print(f'chi-square upper critical value (97.5%): {chi_upper:.4f}')
```

```
chi-square upper critical value (97.5%): 318.4935
```

In [171…
```
# calculate 95% confidence interval for variance
# formula: [(n-1)×s² / chi_upper, (n-1)×s² / chi_lower]

var_lower = (df_chi * sample_var) / chi_upper
var_upper = (df_chi * sample_var) / chi_lower

print(f'\n95% confidence interval for variance:')
print(f'lower bound: {var_lower:.4f}')
print(f'upper bound: {var_upper:.4f}')
print(f'sample variance: {sample_var:.4f}')

print(f'\ninterpretation:')
print(f'we are 95% confident that true population variance')
print(f'lies between {var_lower:.4f} and {var_upper:.4f}')
```

```
95% confidence interval for variance:
lower bound: 1.1085
upper bound: 1.5532
sample variance: 1.3027

interpretation:
we are 95% confident that true population variance
lies between 1.1085 and 1.5532
```

In [172…
```
# convert to standard deviation confidence interval
std_lower = np.sqrt(var_lower)
std_upper = np.sqrt(var_upper)

print(f'95% confidence interval for standard deviation:')
print(f'lower bound: {std_lower:.4f} minutes')
print(f'upper bound: {std_upper:.4f} minutes')
print(f'sample std: {np.std(erupt, ddof=1):.4f} minutes')
```

```
95% confidence interval for standard deviation:
lower bound: 1.0528 minutes
upper bound: 1.2463 minutes
sample std: 1.1414 minutes
```

---

summary: when to use each test

proportion tests

- **one-sample:** testing if single proportion matches claimed value
- **two-sample:** comparing proportions between two groups

chi-square tests

- **goodness of fit:** testing if data matches expected distribution pattern
- **independence:** testing if two categorical variables are related

poisson rate tests

- **one-sample:** testing if event rate matches claimed rate
- **two-sample:** comparing event rates between two groups

variance tests

- **chi-square:** testing if variance/std deviation matches claimed value
- used for quality control and process consistency

---

1. Do Urban and Rural stores differ in the variability of average customer spend?
2. Does the distribution of UPI usage across the four bins (<40%, 40–50%, 50–60%, >60%) appear uniform?
3. Is there a difference in UPI payment percentage between Urban and Rural stores?
4. Do region and store location jointly influence footfall levels?
5. Do Urban and Rural stores differ in the proportion of stores offering discounts?

In [173…
```
df = pd.read_excel('data/MetroMart.xlsx')
```

In [174…
```
pd.set_option('display.max_columns', None)    # showing every column
```

In [175…
```
df.head()
```

Out[175…

	Location	Region	Footfall	Avg_Spend	BillingTime_Before	BillingTime_After	Monthly
0	Rural	East	788	500.05	5.36	5.19	
1	Urban	South	978	580.24	5.06	4.58	
2	Urban	East	1018	565.95	4.94	4.21	
3	Urban	North	952	560.86	5.36	4.68	
4	Rural	North	803	508.40	5.30	4.96	

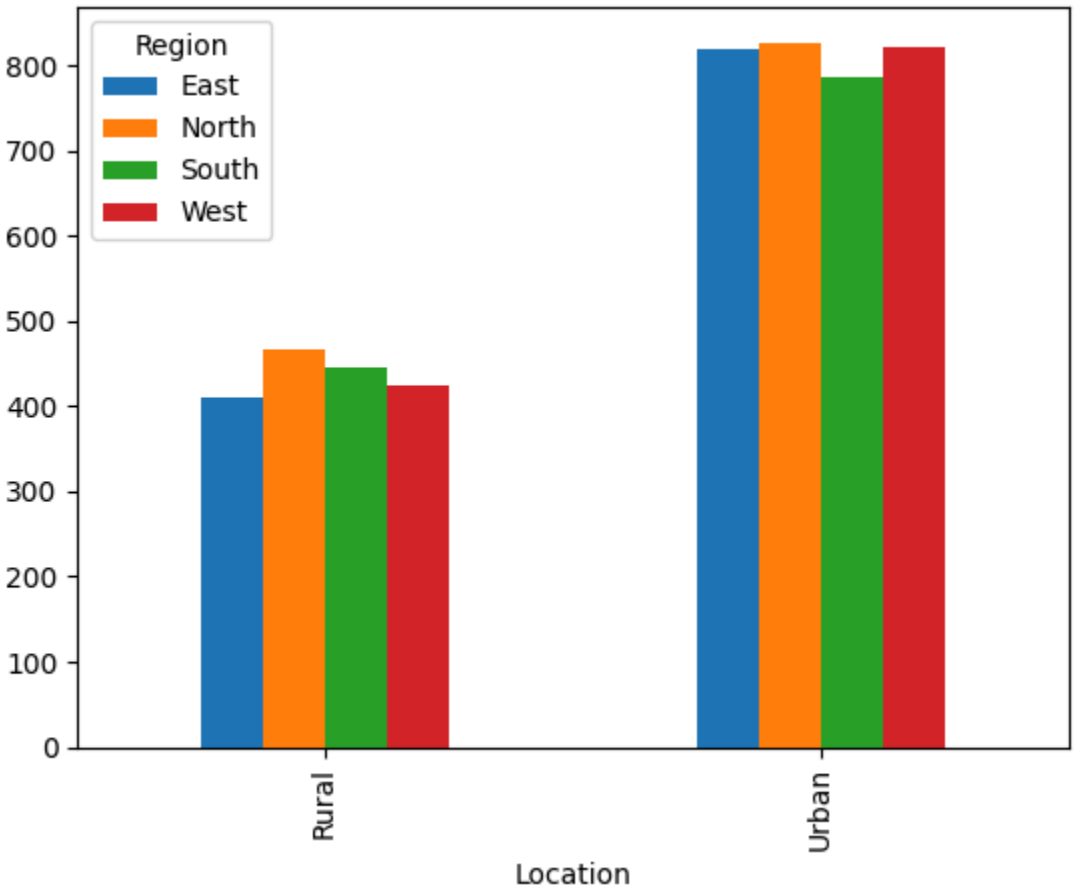
In [176…
```
df.Location.value_counts().reset_index() # couting values from location
```

Out[176…

	Location	count
0	Urban	3254
1	Rural	1746

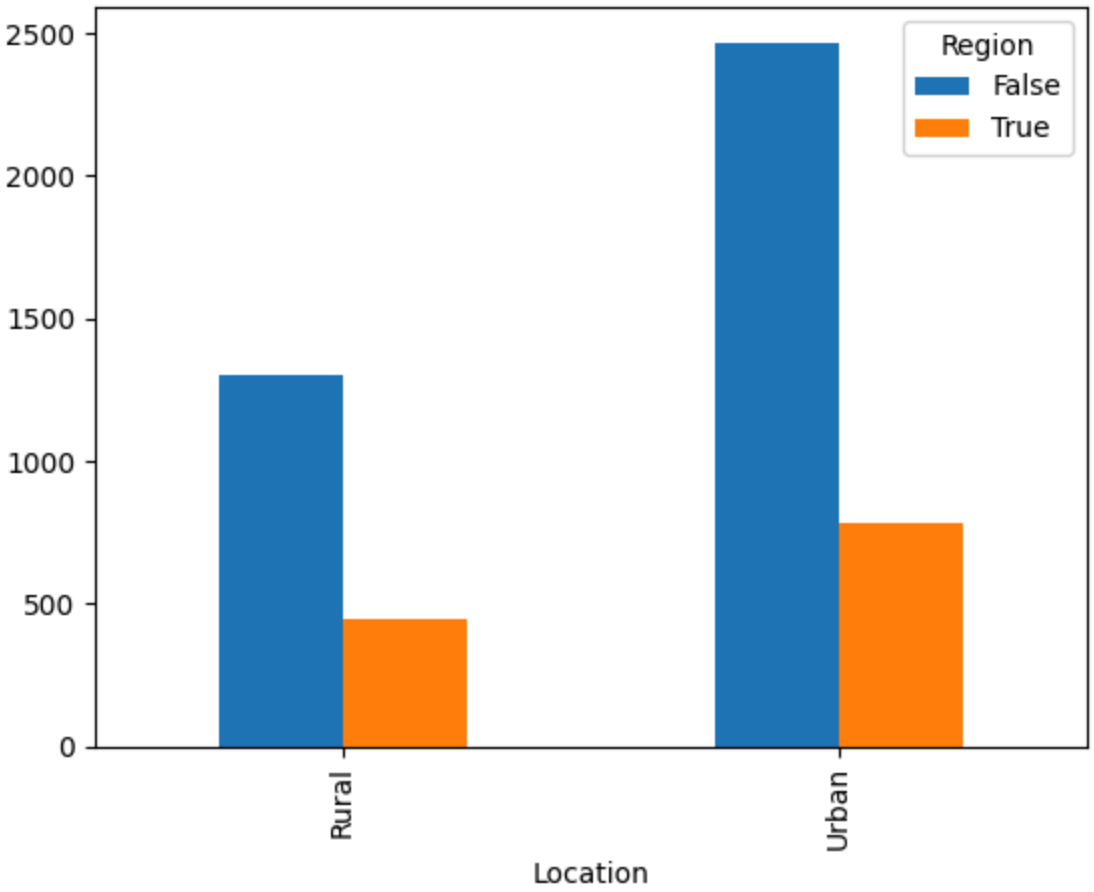
In [177…

```
pd.crosstab(df.Location, df.Region).plot(kind='bar')
plt.show()
```



In [178…

```
pd.crosstab(df.Location, df.Region=='South').plot(kind='bar')
plt.show()
```



In [179…

```
df.iloc[np.where((df.Location == 'Urban') & (df.Region=='South') & (df.Promoti
```

Out[179…

	Location	Region	Footfall	Avg_Spend	BillingTime_Before	BillingTime_After	Mon
9	Urban	South	889	552.81	5.18	4.65	
27	Urban	South	914	569.61	5.15	4.79	
29	Urban	South	976	555.28	5.54	4.97	
36	Urban	South	954	535.53	5.53	5.03	
39	Urban	South	889	599.80	5.25	4.69	
…	…	…	…	…	…	…	
4912	Urban	South	943	593.49	4.99	4.55	
4954	Urban	South	894	577.95	5.09	4.36	
4971	Urban	South	958	591.91	5.30	4.92	
4983	Urban	South	942	578.32	4.69	4.07	
4995	Urban	South	962	610.29	5.11	4.72	

379 rows × 12 columns

In [180…
```
df.iloc[(df.Location.value_counts())]
```

Out[180…

	Location	Region	Footfall	Avg_Spend	BillingTime_Before	BillingTime_After	Mon
3254	Rural	North	738	514.16	5.04	4.44	
1746	Rural	North	748	493.42	5.59	5.19	

In [181…
```
bins = [0,4,9,14,18]
```

In [182…
```
complaints_count = pd.cut(df.Complaints, bins = bins).value_counts().reset_inc
```

In [183…
```
complaints_count
```

Out[183…

	Complaints	count
0	(0, 4]	2996
1	(4, 9]	1681
2	(9, 14]	143
3	(14, 18]	3

In [184…
```
# 1.        Do Urban and Rural stores differ in the variability of average cus
```

```
rural = df.Avg_Spend.iloc[np.where(df.Location=='Rural')]
urban = df.Avg_Spend.iloc[np.where(df.Location=='Urban' )]
f_ts=np.var(rural,ddof=1)/np.var(urban,ddof=1)
p_val = f.cdf(f_ts,len(rural)-1,len(urban)-1)
p_val*2
```

Out[184…
```
np.float64(0.8021795430179889)
```

In [185…
```
df.head()
```

Out[185…

	Location	Region	Footfall	Avg_Spend	BillingTime_Before	BillingTime_After	Monthly
0	Rural	East	788	500.05	5.36	5.19	
1	Urban	South	978	580.24	5.06	4.58	
2	Urban	East	1018	565.95	4.94	4.21	
3	Urban	North	952	560.86	5.36	4.68	
4	Rural	North	803	508.40	5.30	4.96	

In [186…
```
# 2.        Does the distribution of UPI usage across the four bins (<40%, 40–
```

In [187…
```
upi = df.Payment_UPI_pct
upi_cats = pd.cut(df.Payment_UPI_pct, bins = [0,40,50,60,100]).value_counts()
```

In [188…
```
upi_cats.values
```

Out[188…
```
array([1958, 1950,  895,  197])
```

In [189…
```
obs_counts = upi_cats.values
```

In [190…
```
exp_counts = np.array([1250,1250,1150,1352])
```

In [191…
```
scipy.stats.chisquare(obs_counts, f_exp=exp_counts)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_1941/2577874924.py i
n ?()
----> 1 scipy.stats.chisquare(obs_counts, f_exp=exp_counts)

/opt/homebrew/Caskroom/miniconda/base/lib/python3.13/site-packages/scipy/stat
s/_axis_nan_policy.py in ?(***failed resolving arguments***)
    575                     res = np.full(n_out, NaN)
    576                     res = _add_reduced_axes(res, reduced_axes, keepdim
s)
    577                     return tuple_to_result(*res)
    578
--> 579                 res = hypotest_fun_out(*samples, **kwds)
    580                 res = result_to_tuple(res, n_out)
    581                 res = _add_reduced_axes(res, reduced_axes, keepdims)
    582                 return tuple_to_result(*res)

/opt/homebrew/Caskroom/miniconda/base/lib/python3.13/site-packages/scipy/stat
s/_stats_py.py in ?(f_obs, f_exp, ddof, axis, sum_check)
   7570     Power_divergenceResult(statistic=array([3.5 , 9.25]), pvalue=arra
y([0.62338763, 0.09949846]))
   7571
   7572     For a more detailed example, see :ref:`hypothesis_chisquare`.
   7573     """  # noqa: E501
-> 7574     return _power_divergence(f_obs, f_exp=f_exp, ddof=ddof, axis=axis,
   7575                              lambda_="pearson", sum_check=sum_check)

/opt/homebrew/Caskroom/miniconda/base/lib/python3.13/site-packages/scipy/stat
s/_stats_py.py in ?(f_obs, f_exp, ddof, axis, lambda_, sum_check)
   7383                         f"frequencies must agree with the sum of the "
   7384                         f"expected frequencies to a relative tolerance "
   7385                         f"of {rtol}, but the percent differences are:\n"
   7386                         f"{relative_diff}")
-> 7387                 raise ValueError(msg)
   7388
   7389     else:
   7390         # Avoid warnings with the edge case of a data set with length 0

ValueError: For each axis slice, the sum of the observed frequencies must agree
with the sum of the expected frequencies to a relative tolerance of 1.490116119
3847656e-08, but the percent differences are:
0.0004
```

In [ ]:
```
# 3. Is there a difference in UPI payment percentage between Urban and Rural s
```

In [ ]:
```
rural = df.Payment_UPI_pct.iloc[np.where(df.Location=='Rural')]
urban = df.Payment_UPI_pct.iloc[np.where(df.Location=='Urban')]
```

In [ ]:
```
np.mean(rural)
```

In [ ]:
```
np.mean(urban)
```

In [ ]:
```
counts = np.array([6003, 4806])
obs = np.array([10000,10000])
```

In [ ]:
```
ssp.proportions_ztest(counts, obs, value = 0, alternative='two-sided')
```

In [ ]:
```
# 4. Do region and store location jointly influence footfall levels?
```

In [ ]:
```
model = ols('df.Footfall~df.Location*df.Region', df).fit()
```

In [ ]:
```
res = sm.stats.anova_lm(model)
```

In [ ]:
```
res
```

In [ ]:
```
len(np.where((df.Location=='Urban')& (df.Promotion=='Discount'))[0])
```

In [ ]:
```
sswa.ztest(df.MonthlySales, value=95, alternative = 'smaller')
```

In [ ]:
```
np.mean(df.MonthlySales)
```

In [ ]:
```
sswa.ztest(df.MonthlySales, value = 115, alternative = 'smaller')
```

In [ ]:
```
sswa.ztest(df.MonthlySales, value = 105, alternative = 'smaller')
```

In [ ]:
```
# corrected: use sqrt(n) for standard error and fix column name typo
n = len(df.MonthlySales)
se = np.std(df.MonthlySales, ddof=1) / np.sqrt(n)
upper_95_one_sided = np.mean(df.MonthlySales) + 1.645 * se
upper_95_one_sided
```

In [ ]:
```
norm.ppf(0.05)
```

In [ ]:
```
# do urban and rural stores differ in their complaint rates?
```

In [ ]:
```
# 1.        Do Urban and Rural stores differ in the variability of average cus
# 2.        Does the distribution of UPI usage across the four bins (<40%, 40–
# 3.        Is there a difference in UPI payment percentage between Urban and
# 4.        Do region and store location jointly influence footfall levels?
# 5.        Do Urban and Rural stores differ in the proportion of stores offer
# 6.        Is the average monthly sales (in lakhs) across stores equal to ₹95
# 7.        Has the new billing software reduced billing time per customer?
# 8.        Do stores in the North and South regions differ in their average m
# 9.        Is the overall average complaint rate across stores equal to 4 com
# 10.        Do Urban and Rural stores differ in their complaint rates?
# 11.        Is the proportion of stores offering discounts equal to 52%?
# 12.        Do Urban and Rural stores differ in the variability of card payme
# 13.        Is customer satisfaction category (Low/Medium/High) related to st
```

In [ ]:
```
# tasks 1-13: hypothesis tests on metromart data
results = {}
```

In [ ]:
```python
# q1: variance difference in average spend (urban vs rural)
urban_spend = df.loc[df['Location'] == 'Urban', 'Avg_Spend'].dropna()
rural_spend = df.loc[df['Location'] == 'Rural', 'Avg_Spend'].dropna()
f_stat = np.var(urban_spend, ddof=1) / np.var(rural_spend, ddof=1)
df1, df2 = len(urban_spend) - 1, len(rural_spend) - 1
p_val = 2 * min(stats.f.cdf(f_stat, df1, df2), stats.f.sf(f_stat, df1, df2))
results['q1_variance_avg_spend'] = {'f_stat': f_stat, 'p_value': p_val}
```

In [ ]:
```python
# q2: uniformity of upi usage bins
upi_bins = pd.cut(df['Payment_UPI_pct'], bins=[0, 40, 50, 60, 100], include_lc
upi_counts = upi_bins.value_counts().sort_index()
exp_counts = np.full(len(upi_counts), upi_counts.sum() / len(upi_counts))
chi_stat, chi_p = stats.chisquare(upi_counts, f_exp=exp_counts)
results['q2_upi_uniformity'] = {'chi2': chi_stat, 'p_value': chi_p}
```

In [ ]:
```python
# q3: difference in upi percentage means (urban vs rural)
urban_upi = df.loc[df['Location'] == 'Urban', 'Payment_UPI_pct'].dropna()
rural_upi = df.loc[df['Location'] == 'Rural', 'Payment_UPI_pct'].dropna()
t_stat, t_p = stats.ttest_ind(urban_upi, rural_upi, equal_var=False)
results['q3_upi_mean_diff'] = {'t_stat': t_stat, 'p_value': t_p}
```

In [ ]:
```python
# q4: two-way anova on footfall by region and location
anova_model = ols('Footfall ~ C(Location) * C(Region)', data=df).fit()
anova_table = sm.stats.anova_lm(anova_model, typ=2)
results['q4_region_location_footfall'] = {
    'p_location': float(anova_table.loc['C(Location)', 'PR(>F)']),
    'p_region': float(anova_table.loc['C(Region)', 'PR(>F)']),
    'p_interaction': float(anova_table.loc['C(Location):C(Region)', 'PR(>F)'])
}
```

In [ ]:
```python
# q5: discount proportion difference between urban and rural
is_discount = df['Promotion'].str.lower() == 'discount'
disc_counts = df.groupby('Location')['Promotion'].apply(lambda s: (s.str.lower
loc_totals = df['Location'].value_counts()
count_arr = np.array([disc_counts.get('Urban', 0), disc_counts.get('Rural', 0)
nobs_arr = np.array([loc_totals.get('Urban', 0), loc_totals.get('Rural', 0)])
z_stat, z_p = proportions_ztest(count_arr, nobs_arr)
results['q5_discount_prop_diff'] = {'z_stat': float(z_stat), 'p_value': float(
```

In [ ]:
```python
# q6: monthly sales mean equals 95
sales = df['MonthlySales'].dropna()
t_stat, t_p = stats.ttest_1samp(sales, 95)
results['q6_monthly_sales_mean'] = {'t_stat': t_stat, 'p_value': t_p}
```

In [ ]:
```python
# q7: billing time reduction after software (paired one-sided test)
before = df['BillingTime_Before'].dropna()
after = df.loc[before.index, 'BillingTime_After']
pair_stat, pair_p_two = stats.ttest_rel(before, after)
pair_p_one = pair_p_two / 2 if pair_stat > 0 else 1 - pair_p_two / 2
results['q7_billing_time_reduction'] = {'t_stat': pair_stat, 'p_value_one_side
```

In [ ]:
```
# q8: north vs south monthly sales difference
north_sales = df.loc[df['Region'] == 'North', 'MonthlySales'].dropna()
south_sales = df.loc[df['Region'] == 'South', 'MonthlySales'].dropna()
t_stat, t_p = stats.ttest_ind(north_sales, south_sales, equal_var=False)
results['q8_north_south_sales'] = {'t_stat': t_stat, 'p_value': t_p}
```

In [ ]:
```
# q9: complaints mean equals 4
complaints = df['Complaints'].dropna()
t_stat, t_p = stats.ttest_1samp(complaints, 4)
results['q9_complaints_mean'] = {'t_stat': t_stat, 'p_value': t_p}
```

In [ ]:
```
# q10: complaints difference between urban and rural
urban_complaints = df.loc[df['Location'] == 'Urban', 'Complaints'].dropna()
rural_complaints = df.loc[df['Location'] == 'Rural', 'Complaints'].dropna()
t_stat, t_p = stats.ttest_ind(urban_complaints, rural_complaints, equal_var=Fa
results['q10_complaints_diff'] = {'t_stat': t_stat, 'p_value': t_p}
```

In [ ]:
```
# q11: overall discount proportion equals 52%
is_discount = df['Promotion'].str.lower() == 'discount'
total_discounts = is_discount.sum()
n_total = len(is_discount)
z_stat, z_p = proportions_ztest(total_discounts, n_total, value=0.52)
results['q11_discount_rate_overall'] = {'z_stat': float(z_stat), 'p_value': fl
```

In [ ]:
```
# q12: variance difference in card payment percentage
urban_card = df.loc[df['Location'] == 'Urban', 'Payment_Card_pct'].dropna()
rural_card = df.loc[df['Location'] == 'Rural', 'Payment_Card_pct'].dropna()
f_stat = np.var(urban_card, ddof=1) / np.var(rural_card, ddof=1)
df1, df2 = len(urban_card) - 1, len(rural_card) - 1
p_val = 2 * min(stats.f.cdf(f_stat, df1, df2), stats.f.sf(f_stat, df1, df2))
results['q12_card_variance'] = {'f_stat': f_stat, 'p_value': p_val}
```

In [ ]:
```
# q13: satisfaction category vs location (chi-square)
satisfaction_bins = pd.cut(df['SatisfactionScore'], bins=[0, 2.5, 3.5, 5.1], l
cont_table = pd.crosstab(df['Location'], satisfaction_bins)
chi_stat, chi_p, _, _ = chi2_contingency(cont_table)
results['q13_location_vs_satisfaction'] = {'chi2': chi_stat, 'p_value': chi_p}
```

In [ ]:
```
results
```

In [2]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

We'll use:

- `pandas` for data manipulation
- `numpy` for numerical operations
- `scipy.stats` for statistical functions and probability distributions

In [3]:
```
# Import all required libraries
import scipy
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib
from matplotlib import pyplot as plt
import seaborn as sns
import os
from scipy.stats import *
from scipy.stats import norm, t, binom, poisson, expon
```

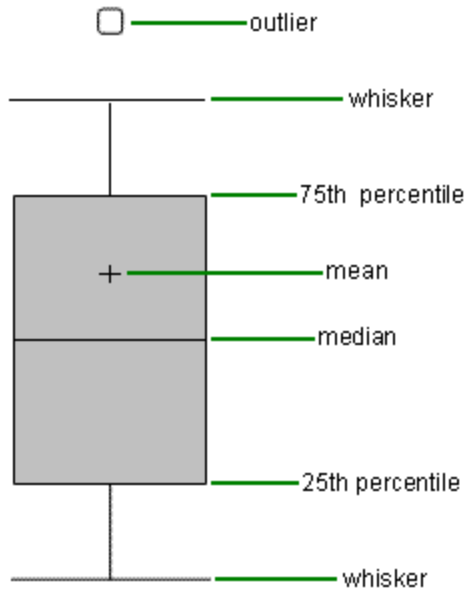
In [4]:
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

---

1. Descriptive Statistics: Boxplot

Boxplot?

A **boxplot** (box-and-whisker plot) is a standardized way of displaying the distribution of data based on five key summary statistics:



Key Components:

1. **Median (Q2)**: The middle value of the dataset (50th percentile)
2. **First Quartile (Q1)**: 25th percentile
3. **Third Quartile (Q3)**: 75th percentile
4. **Interquartile Range (IQR)**: The range between Q1 and Q3 (middle 50% of data)

$$\text{IQR} = Q_3 - Q_1$$

5. **Whiskers**: Lines extending from the box to the smallest and largest values within acceptable range
6. **Outliers**: Data points outside the whiskers (shown as individual dots)

Outlier Detection Formula:

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR}$$

$$\text{Upper Bound} = Q_3 + 1.5 \times \text{IQR}$$

- **Lower Whisker** = Smallest data point ≥ Lower Bound, minimum / 0th Percentile
- **Upper Whisker** = Largest data point ≤ Upper Bound, maximum / 100th Percentile
- **Outliers** = Any value < Lower Bound OR > Upper Bound [`usually dots`]

Use Cases:

- Compare distributions across multiple groups
- Identify skewness and spread
- Detect outliers quickly

In [5]:
```
df = pd.read_excel('data/Values.xlsx')
df.head()
```

Out[5]:

	Values
0	32
1	38
2	128
3	49
4	44

In [6]:
```
q1 = np.percentile(df.Values, 25)  # First quartile (25th percentile)
q2 = np.percentile(df.Values, 50)  # Median (50th percentile)
q3 = np.percentile(df.Values, 75)  # Third quartile (75th percentile)

# IQR (Interquartile Range)
iqr = q3 - q1


print(f'q1: {q1}')
print(f'q1: {q2}')
print(f'q1: {q3}')
print(f'iqr: {iqr}')
```

```
q1: 43.0
q1: 50.0
q1: 56.25
iqr: 13.25
```

In [7]:
```
# calculate outlier boundaries
uf = q3 + 1.5 * iqr  # Upper fence (upper bound)
lf = q1 - 1.5 * iqr  # Lower fence (lower bound)

print(f"lf: {lf}")
print(f"uf: {uf}")
```

```
lf: 23.125
uf: 76.125
```

In [8]:
```
# count how many outliers are above the upper bound
upper_outliers_count = (df.Values > uf).sum()
```

```
print(upper_outliers_count)
```

```
7
```

In [9]:
```
# count how many outliers are below the lower bound
lower_outliers_count = (df.Values < lf).sum()
print(lower_outliers_count)
```

```
6
```

In [10]:
```
# Find the UPPER WHISKER:
# The upper whisker is the maximum value that is still ≤ upper bound (uf)
# We find all values between Q3 and the upper bound, then take the maximum
ind = np.where((df.Values > q3) & (df.Values <= uf))
upper_whisker = max(df.Values.iloc[ind])
print(upper_whisker)
```

```
75
```

In [11]:
```
# Find the LOWER WHISKER:
# The lower whisker is the minimum value that is still ≥ lower bound (lf)
# We find all values between the lower bound and Q1, then take the minimum
ind = np.where((df.Values < q1) & (df.Values >= lf))
lower_whisker = min(df.Values.iloc[ind])
print(lower_whisker)
```

```
24
```

In [12]:
```
# Extract all UPPER OUTLIERS (values > upper bound) at particular index
ind = np.where(df.Values > uf)
upper_outliers = df.Values.iloc[ind]
print(f"Upper outliers:\n{upper_outliers}")
```

```
Upper outliers:
2      128
119    116
251     77
288     89
394     81
445    120
467    127
Name: Values, dtype: int64
```

In [13]:
```
# Extract all LOWER OUTLIERS (values < lower bound)
ind = np.where(df.Values < lf)
lower_outliers = df.Values.iloc[ind]
print(f"Lower outliers:\n{lower_outliers}")
```

```
Lower outliers:
32     18
61    -21
284   -25
302   -27
321   -17
427   -24
Name: Values, dtype: int64
```

In [14]:
```
df = pd.read_excel('data/Travel.xlsx')
df.head()
```

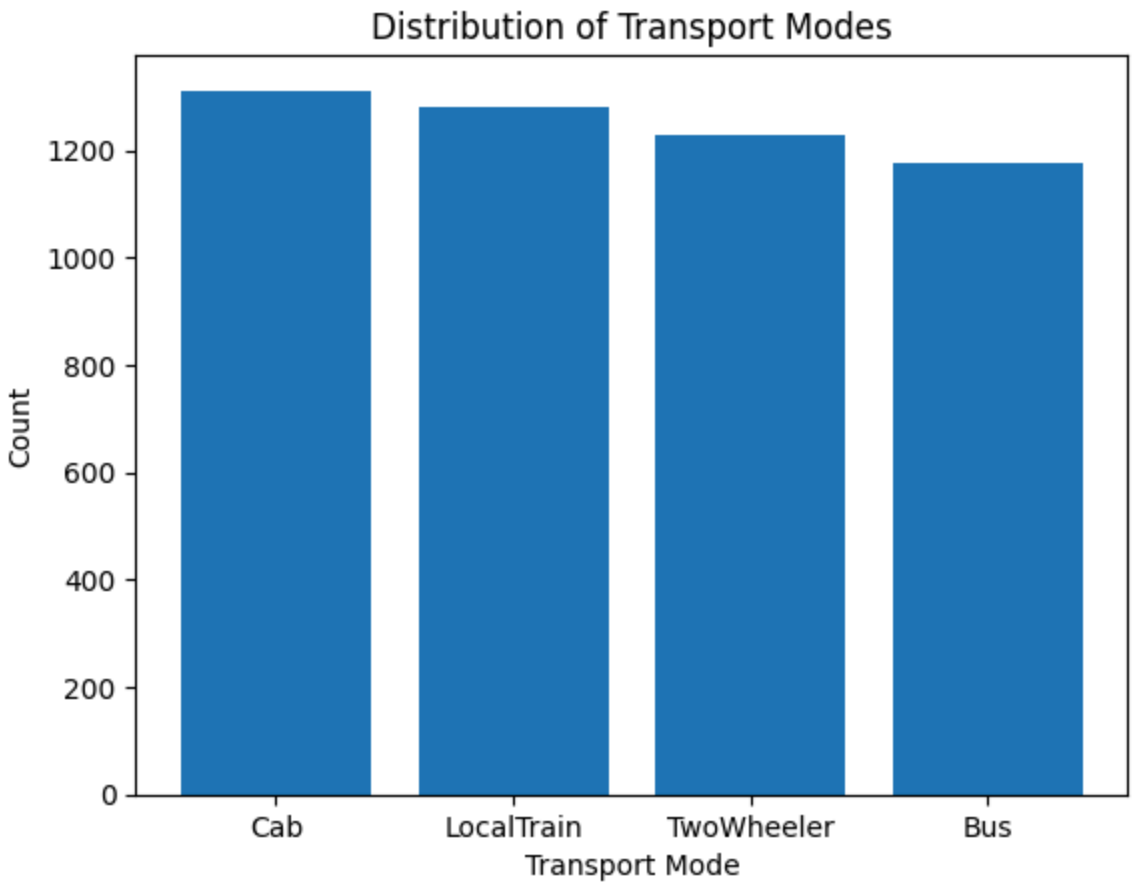
Out[14]:

	Gender	Location	TransportMode
0	Female	Suburbs	Cab
1	Male	City	Cab
2	Female	Towns	Bus
3	Female	Suburbs	LocalTrain
4	Male	Suburbs	Bus

In [15]:
```
#unique locations
np.unique(df.Location)
```

Out[15]:
```
array(['City', 'Suburbs', 'Towns'], dtype=object)
```

In [16]:
```
# Count occurrences of each TransportMode and visualize with a bar chart
# value_counts() returns a Series with unique values and their frequencies
s1 = df.TransportMode.value_counts()

# Create bar chart: x-axis = transport modes, y-axis = counts
plt.bar(s1.index, s1.values)
plt.xlabel('Transport Mode')
plt.ylabel('Count')
plt.title('Distribution of Transport Modes')
plt.show()
```



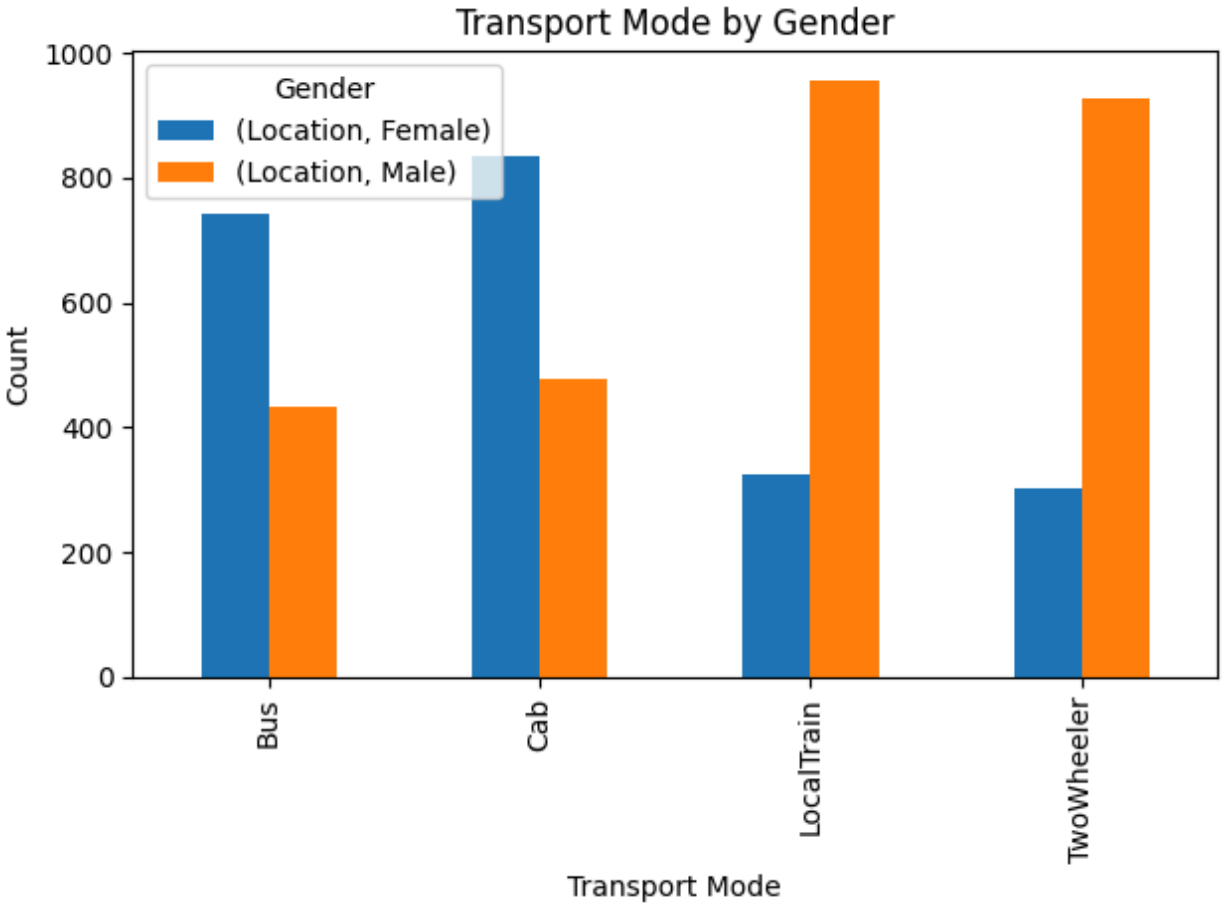
In [17]:
```
s1
```

Out[17]:
```
TransportMode
Cab           1312
LocalTrain    1281
TwoWheeler    1230
Bus           1177
Name: count, dtype: int64
```

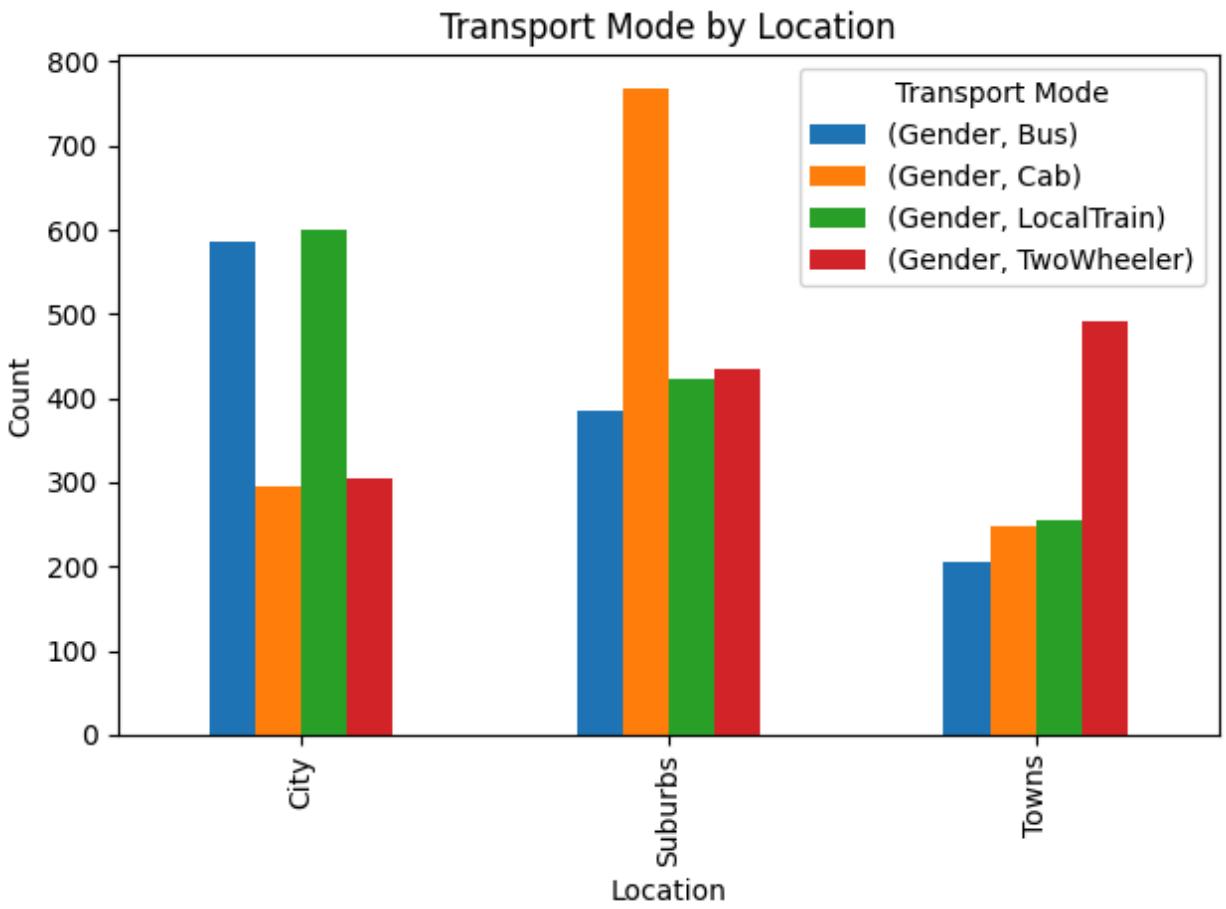
In [18]:
```
# calculate proportions (relative frequencies) for each TransportMode
# reset index to convert Series to DataFrame for easier manipulation
df2 = df.TransportMode.value_counts().reset_index()

# Sort by count in descending order
df2 = df2.sort_values('count', ascending=False)

# Calculate proportion: count / total count
df2['prop'] = df2['count'] / sum(df2['count'])

print(df2)
```

```
  TransportMode  count    prop
0           Cab   1312  0.2624
1    LocalTrain   1281  0.2562
2    TwoWheeler   1230  0.2460
3           Bus   1177  0.2354
```

In [19]:
```
# pivot_table creates a cross-tabulation (counts) of TransportMode vs Gender

df.pivot_table(columns='Gender', index='TransportMode', aggfunc=np.size).plot(
plt.ylabel('Count')
plt.title('Transport Mode by Gender')
plt.xlabel('Transport Mode')
plt.legend(title='Gender')
plt.tight_layout()
plt.show()
```



In [20]:
```
# Create a grouped bar chart: Location by TransportMode
# This shows which transport modes are used in each location
df.pivot_table(columns='TransportMode', index='Location', aggfunc=np.size).plo
plt.ylabel('Count')
plt.title('Transport Mode by Location')
plt.xlabel('Location')
plt.legend(title='Transport Mode')
plt.tight_layout()
plt.show()
```



---

3. Handling Missing Data

Strategies for Missing Data:

1. **Numeric columns**: Fill with mean, median, or mode
2. **Categorical columns**: Fill with mode OR maintain original proportions

We'll demonstrate **proportion-based filling** for categorical data.

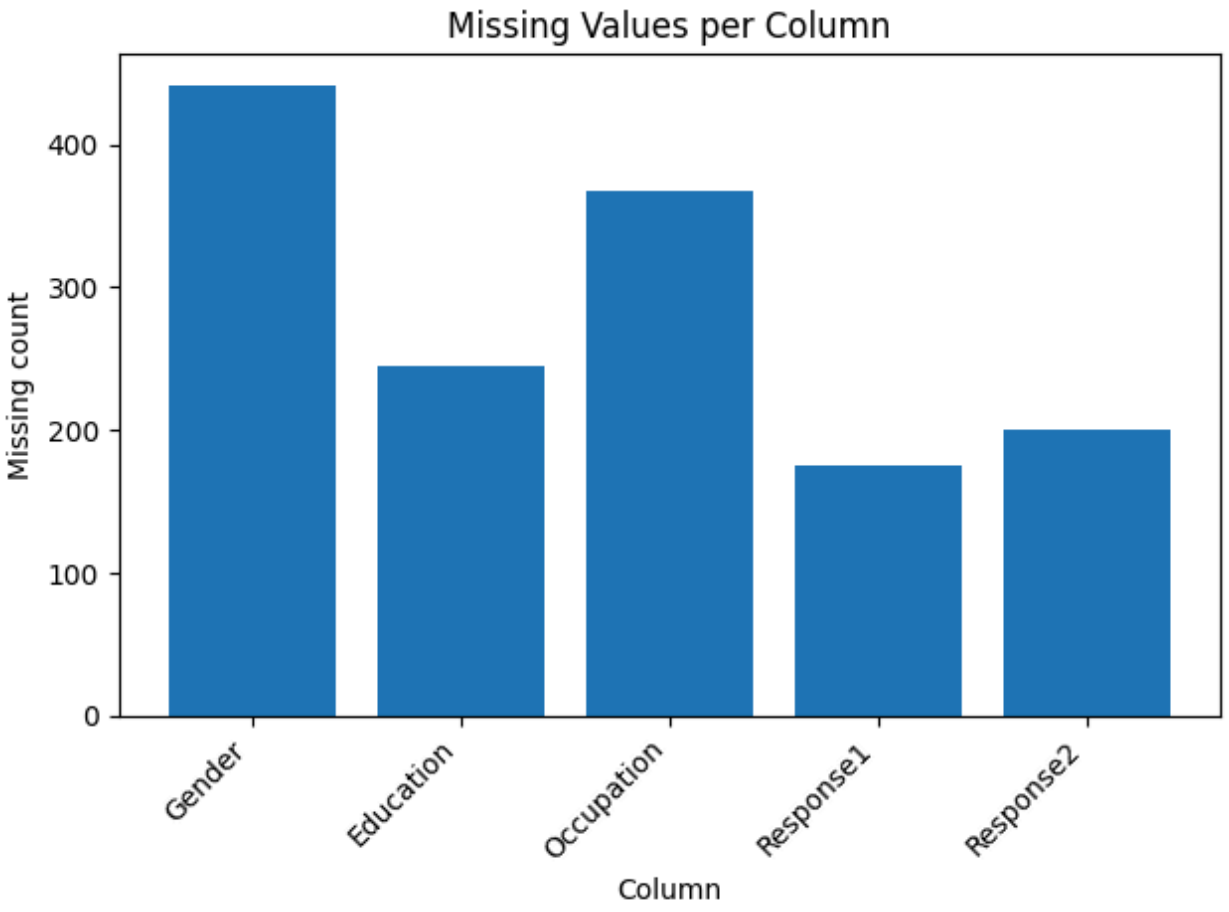
In [21]:
```
# Load dataset with missing values
df = pd.read_excel('data/Responses.xlsx')
df.head()
```

Out[21]:

	Gender	Education	Occupation	Response1	Response2
0	Male	HighSchool	Banking	115.4	23.0
1	NaN	NaN	Software	38.0	26.0
2	NaN	College	Manufacturing	56.7	36.0
3	Male	PostGrad	Manufacturing	52.6	38.0
4	Male	PostGrad	Banking	62.7	55.0

In [22]:

```
# Visualize missing values count for each column
# isnull().sum() counts NaN values per column
n = df.isnull().sum().reset_index()
n.columns = ['Column', 'Missing_Count']

# Create bar chart showing missing values
plt.bar(n['Column'], n['Missing_Count'])
plt.xticks(rotation=45, ha='right')
plt.ylabel('Missing count')
plt.xlabel('Column')
plt.title('Missing Values per Column')
plt.tight_layout()
plt.show()
```



In [23]:
```
# Check current distribution of Gender (before filling NaNs)
print("Gender distribution (before filling):")
print(df.Gender.value_counts(normalize=True))
```

```
Gender distribution (before filling):
Gender
Male      0.509761
Female    0.490239
Name: proportion, dtype: float64
```

Fill Missing Values with Same Proportions

Instead of filling all NaNs with a single value (like mode), we'll **randomly assign** values based on the existing distribution. This preserves the original proportions in the data.

In [24]:
```
# Calculate proportions for non-NaN values in Gender column
# normalize=True gives proportions instead of counts
probs = df['Gender'].value_counts(normalize=True)
print("Current proportions:")
print(probs)
```

```
Current proportions:
Gender
Male      0.509761
Female    0.490239
Name: proportion, dtype: float64
```

In [25]:
```
# Find all row indices where Gender is NaN
nan_idx = df['Gender'][df['Gender'].isna()].index
print(f"Number of missing Gender values: {len(nan_idx)}")
```

```
Number of missing Gender values: 441
```

In [26]:
```
# Randomly assign 'Male' or 'Female' to NaN values
# based on the observed proportions in the data
# np.random.choice samples from probs.index with probabilities from probs.valu
df.loc[nan_idx, 'Gender'] = np.random.choice(
    probs.index,           # ['Male', 'Female']
    size=len(nan_idx),     # Number of NaNs to fill
    p=probs.values         # Probabilities for each gender
)

print("\nGender distribution (after filling):")
print(df.Gender.value_counts(normalize=True))
```

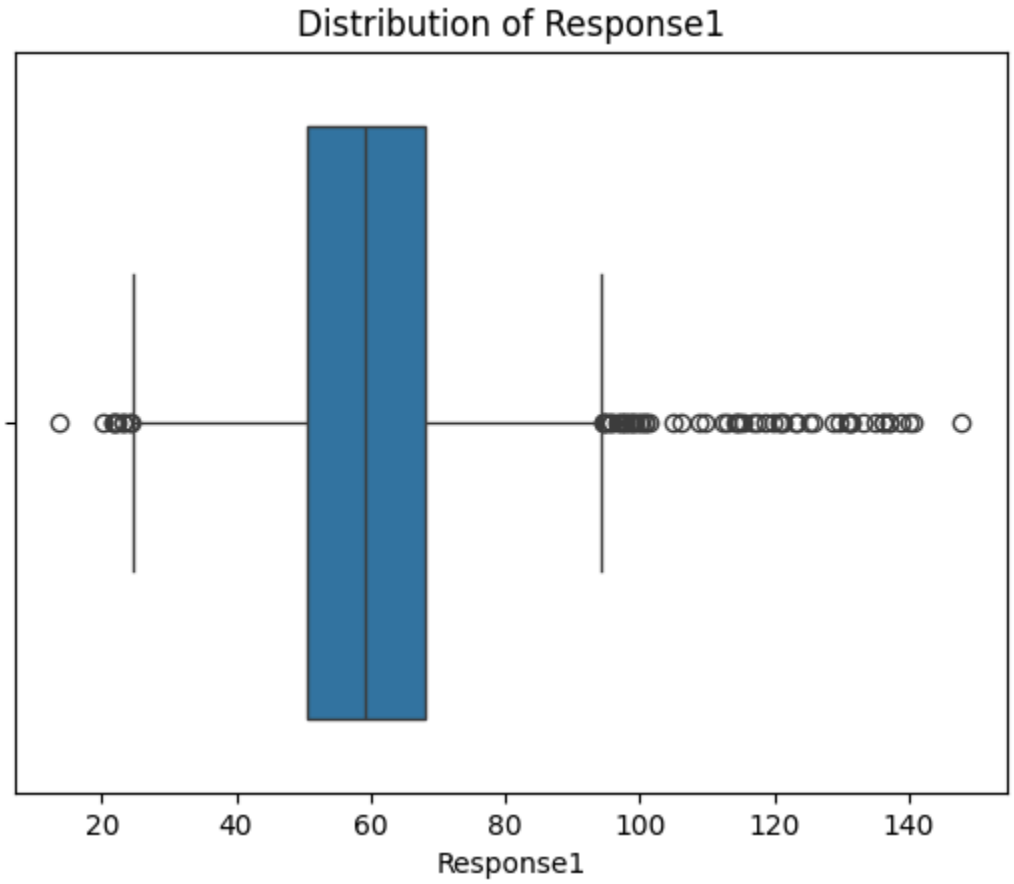
```
Gender distribution (after filling):
Gender
Male      0.5094
Female    0.4906
Name: proportion, dtype: float64
```

---

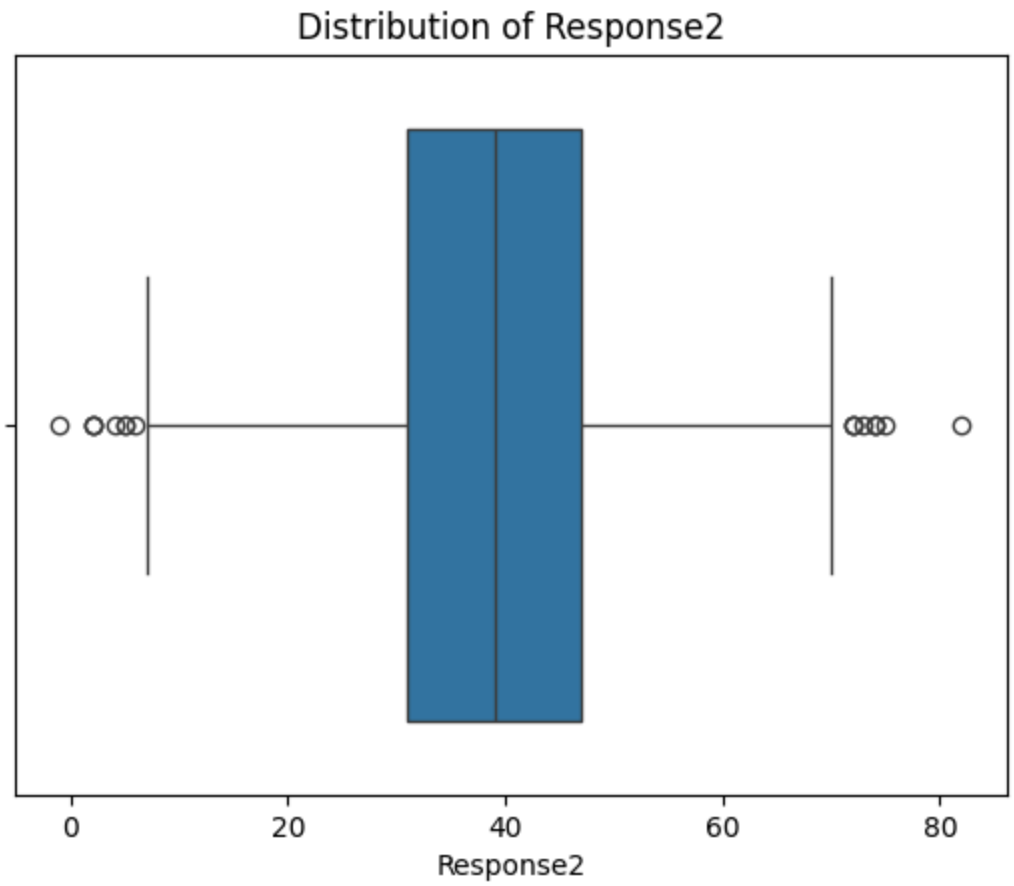
4. Boxplot for Grouped Data

We can use boxplots to compare distributions across different categories.

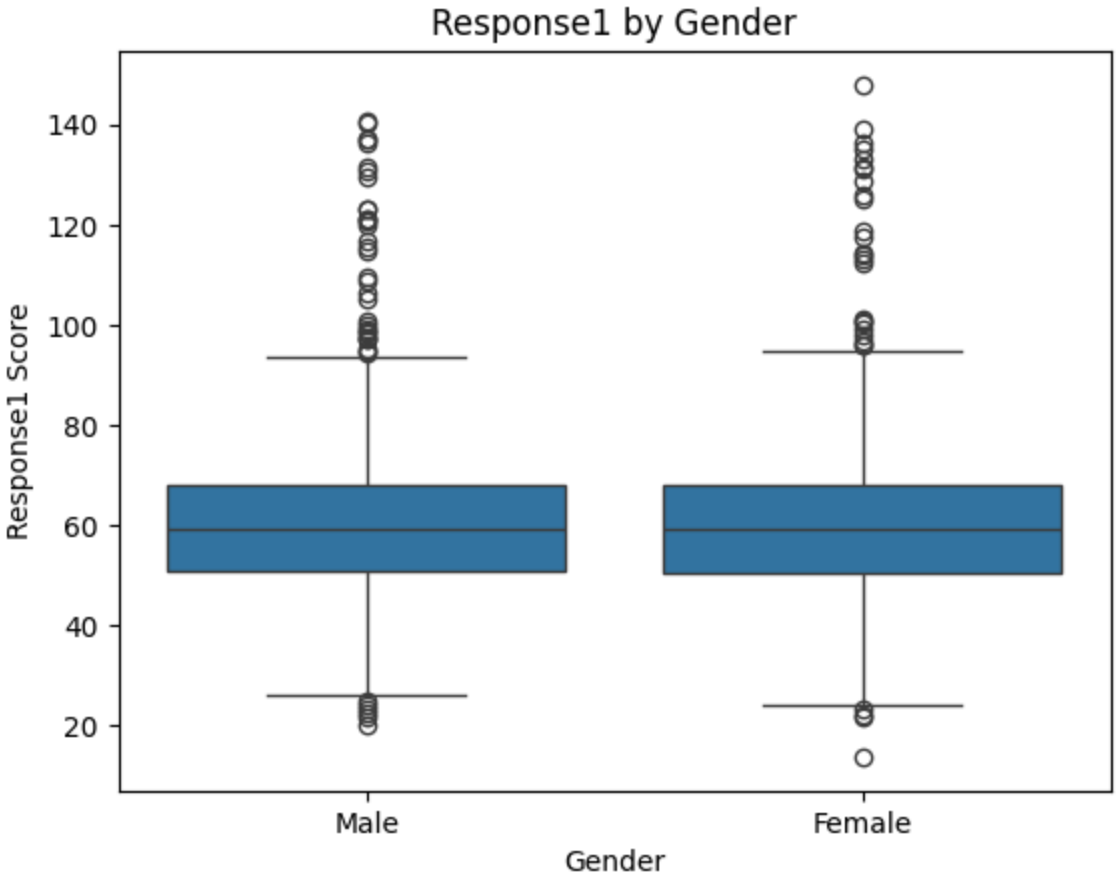
In [27]:
```
# Single boxplot for Response1 (shows overall distribution)
sns.boxplot(x=df.Response1)
plt.title('Distribution of Response1')
plt.show()
```



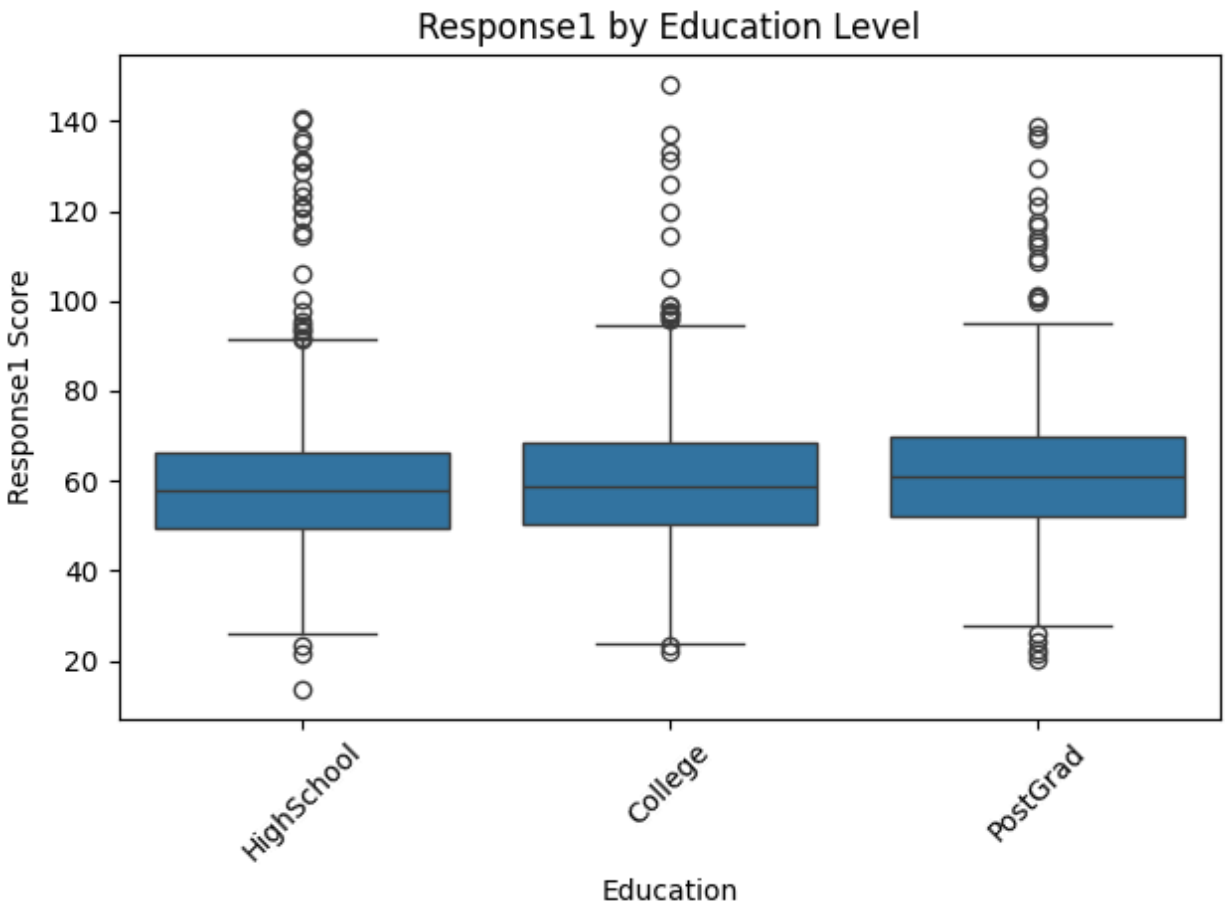
In [28]:
```
# Single boxplot for Response2
sns.boxplot(x=df.Response2)
plt.title('Distribution of Response2')
plt.show()
```



In [29]:
```
# Grouped boxplot: Compare Response1 distribution for Male vs Female
# x = categorical variable (Gender), y = numeric variable (Response1)
# This shows if Response1 differs between genders
sns.boxplot(x=df.Gender, y=df.Response1)
plt.title('Response1 by Gender')
plt.ylabel('Response1 Score')
plt.show()
```



In [30]:
```
# Grouped boxplot: Compare Response1 distribution across Education levels
sns.boxplot(x=df.Education, y=df.Response1)
plt.title('Response1 by Education Level')
plt.ylabel('Response1 Score')
plt.xlabel('Education')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

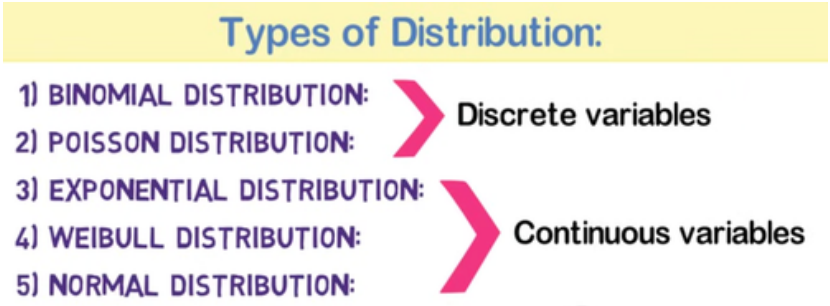


PROBABLITY AND STATS

Discrete vs Continuous Variables

Feature	Discrete Variable	Continuous Variable
Meaning	Countable values	Measurable values
Possible values	Finite or countably infinite	Infinite in any range
Decimals	❌ Not allowed	✔ Allowed
Nature	Whole numbers	Fractions/decimals possible
How obtained	Counting	Measuring
Examples	# of students, # of cars, goals scored, dice outcomes	Height, weight, temperature, time, speed
Typical distributions	Binomial, Poisson	Normal, Exponential, Uniform

Feature	Discrete Variable	Continuous Variable
Statistics impact	Uses frequencies, bar, historgram plots	Uses density curves, line graphs
Memory trick	`"How many?"`	`"How much?"`



---

5. Probability Theory & Distributions

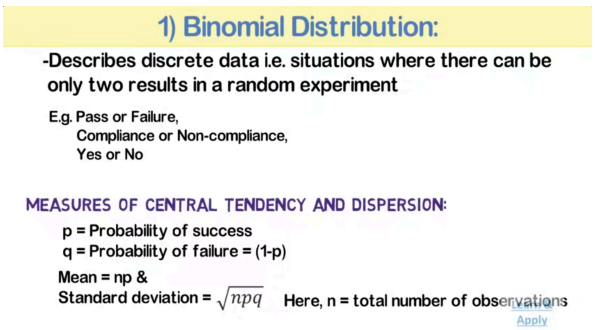
Probability distributions describe how values of a random variable are distributed.

Types:

1. **Discrete Distributions** (for count data):
    - Binomial
    - Poisson
2. **Continuous Distributions** (for measured data):
    - Normal (Gaussian)
    - Exponential

---

6. Binomial Distribution



Example: Probability of Rain

Suppose the probability that it rains on a given day is **p = 0.3** (so **q = 1 - p = 0.7**).

If you observe the weather for **n = 10** days, let **X** be the number of days it rains.

- **p** = Probability of rain on a day = 0.3
- **q** = Probability of no rain on a day = 1 - 0.3 = 0.7

Mean (Expected Value): $$ \text{Mean} = np = 10 \times 0.3 = 3 $$

Standard Deviation: $$ \text{Standard Deviation} = \sqrt{npq} = \sqrt{10 \times 0.3 \times 0.7} \approx 1.449 $$

When to Use:

- **Binary outcomes** (success/failure, yes/no, heads/tails)
- Fixed number of independent trials
- Constant probability of success

Formula:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

Where:

- n = number of trials
- k = number of successes
- p = probability of success

Key Functions:

- `binom.pmf(k, n, p)` : Probability of exactly k successes
- `binom.cdf(k, n, p)` : Probability of k or fewer successes
- `binom.sf(k, n, p)` : Probability of more than k successes (survival function)

Function	What it does	When to use
pmf	Probability Mass Function: Gives probability of a discrete variable taking an exact value	For discrete distributions (e.g., binomial, Poisson) to find P(X = k)

Function	What it does	When to use
pdf	Probability Density Function: Gives relative likelihood of a value for continuous variables (not actual probability at a point)	For continuous distributions (e.g., normal, exponential) to get density at x
cdf	Cumulative Distribution Function: Probability that variable is less than or equal to a value (P(X ≤ x))	To find probability up to a value (area to the left)
sf	Survival Function: Probability that variable is greater than a value (P(X > x)), i.e., 1 - CDF	To find probability above a value (area to the right)
ppf	Percent Point Function (Inverse CDF): Finds the value for a given cumulative probability	To get the value at a given percentile/quantile
isf	Inverse Survival Function: Finds the value for a given upper-tail probability	To get the value at a given upper-tail percentile

Example:

"In 15 coin flips with 60% probability of heads, what's the probability of getting exactly 10 heads?"

In [31]:
```
# Binomial PMF (Probability Mass Function)
# P(X = 10) when n=15, p=0.6
# Question: What's the probability of exactly 10 successes in 15 trials?
prob_exactly_10 = binom.pmf(10, 15, 0.6)
print(f"P(X = 10): {prob_exactly_10:.4f}")
```

```
P(X = 10): 0.1859
```

In [32]:
```
# Binomial CDF (Cumulative Distribution Function)
# P(X ≤ 10) when n=15, p=0.6
# Question: What's the probability of 10 or fewer successes?
prob_up_to_10 = binom.cdf(10, 15, 0.6)
print(f"P(X ≤ 10): {prob_up_to_10:.4f}")
```

```
P(X ≤ 10): 0.7827
```

In [33]:
```
# P(X > 9) = 1 - P(X ≤ 9)
# Question: What's the probability of MORE than 9 successes?
prob_more_than_9 = 1 - binom.cdf(9, 15, 0.6)
print(f"P(X > 9) = 1 - P(X ≤ 9): {prob_more_than_9:.4f}")
```

```
P(X > 9) = 1 - P(X ≤ 9): 0.4032
```

In [34]:
```
# Alternative: Using Survival Function (SF)
# binom.sf(k, n, p) = P(X > k) = 1 - P(X ≤ k)
prob_more_than_9_sf = binom.sf(9, 15, 0.6)
print(f"P(X > 9) using SF: {prob_more_than_9_sf:.4f}")
```

```
P(X > 9) using SF: 0.4032
```

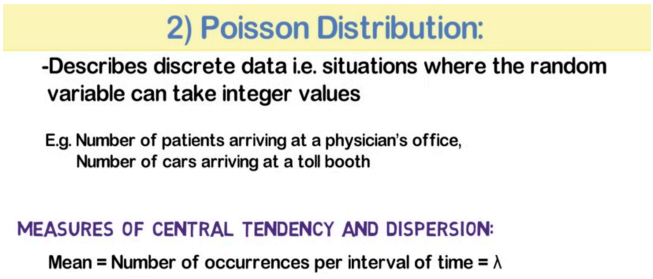
In [35]:
```
# P(3 ≤ X ≤ 8) = P(X ≤ 8) - P(X ≤ 2)
# Question: What's the probability of getting between 3 and 8 successes (inclu
prob_between_3_and_8 = binom.cdf(8, 15, 0.6) - binom.cdf(2, 15, 0.6)
print(f"P(3 ≤ X ≤ 8): {prob_between_3_and_8:.4f}")
```

```
P(3 ≤ X ≤ 8): 0.3899
```

In [36]:
```
prob_between_3_and_8 = binom.cdf(30, 70, 0.2) - binom.cdf(9, 70, 0.2)
prob_between_3_and_8
```

Out[36]:
```
np.float64(0.9154609748998225)
```

Poisson Distribution



When to Use:

- Counting **rare events** in a fixed interval (time, space, volume)
- Events occur independently
- Known average rate (λ)

Formula:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

Where:

- λ = average rate (mean number of events)
- k = number of events
- e ≈ 2.71828

Examples:

- Number of calls to a call center per hour
- Number of defects in a production batch
- Number of arrivals at a service point

Key Functions:

- `poisson.pmf(k, λ)` : Probability of exactly k events
- `poisson.cdf(k, λ)` : Probability of k or fewer events

In [37]:
```
# Poisson PMF = (probablity mass function)

# Question: If average is 50 events, what's the probability of exactly 45 even

prob_exactly_45 = poisson.pmf(45, 50)
print(f"P(X = 45) when λ=50: {prob_exactly_45:.4f}")
```

```
P(X = 45) when λ=50: 0.0458
```

In [38]:
```
# Poisson CDF = (cumulative distribution function)

# Question: What's the probability of 45 or fewer events when average is 50?

prob_up_to_45 = poisson.cdf(45, 50)
print(f"P(X ≤ 45) when λ=50: {prob_up_to_45:.4f}")
```

```
P(X ≤ 45) when λ=50: 0.2669
```

In [39]:
```
# P(X > 44) = 1 - P(X ≤ 44)
# Question: What's the probability of MORE than 44 events?
prob_more_than_44 = 1 - poisson.cdf(44, 50)
print(f"P(X > 44) when λ=50: {prob_more_than_44:.4f}")
```

```
P(X > 44) when λ=50: 0.7790
```

Exponential Distribution

When to Use:

- Modeling **time between events** in a Poisson process
- Waiting times, lifetimes, durations

Formula:

$$f(x) = \frac{1}{\beta} e^{-x/\beta}$$

Where:

- β = scale parameter (mean)
- x = time/duration

Examples:

- Time until next customer arrival
- Lifetime of electronic components
- Time between phone calls

Key Functions:

- `expon.cdf(x, scale=β)` : $P(X \leq x)$
- `expon.ppf(q, scale=β)` : Inverse CDF (quantile function)

In [40]:
```
# Exponential CDF
# Question: If average time is 43.8 mins, what's the probability
# that an event occurs within 100 minutes?
prob_within_100 = expon.cdf(100, scale=43.8)
print(f"P(X ≤ 100) when mean=43.8: {prob_within_100:.4f}")
```

```
P(X ≤ 100) when mean=43.8: 0.8980
```

In [41]:
```
# P(X > 100) = 1 - P(X ≤ 100)
# Question: What's the probability that it takes MORE than 100 minutes?
prob_more_than_100 = 1 - expon.cdf(100, scale=43.8)
print(f"P(X > 100) when mean=43.8: {prob_more_than_100:.4f}")
```

```
P(X > 100) when mean=43.8: 0.1020
```

In [42]:
```
# Exponential PPF (Percent Point Function / Inverse CDF)
# Question: What time corresponds to the 60th percentile?
# (60% of events occur before this time)
time_60th_percentile = expon.ppf(0.6, scale=43.8)
print(f"60th percentile time: {time_60th_percentile:.2f} minutes")
```

```
60th percentile time: 40.13 minutes
```

In [43]:
```
# Question: What time corresponds to the 99th percentile?
# (99% of events occur before this time)
time_99th_percentile = expon.ppf(0.99, scale=43.8)
print(f"99th percentile time: {time_99th_percentile:.2f} minutes")
```

```
99th percentile time: 201.71 minutes
```

Normal Distribution (Gaussian)

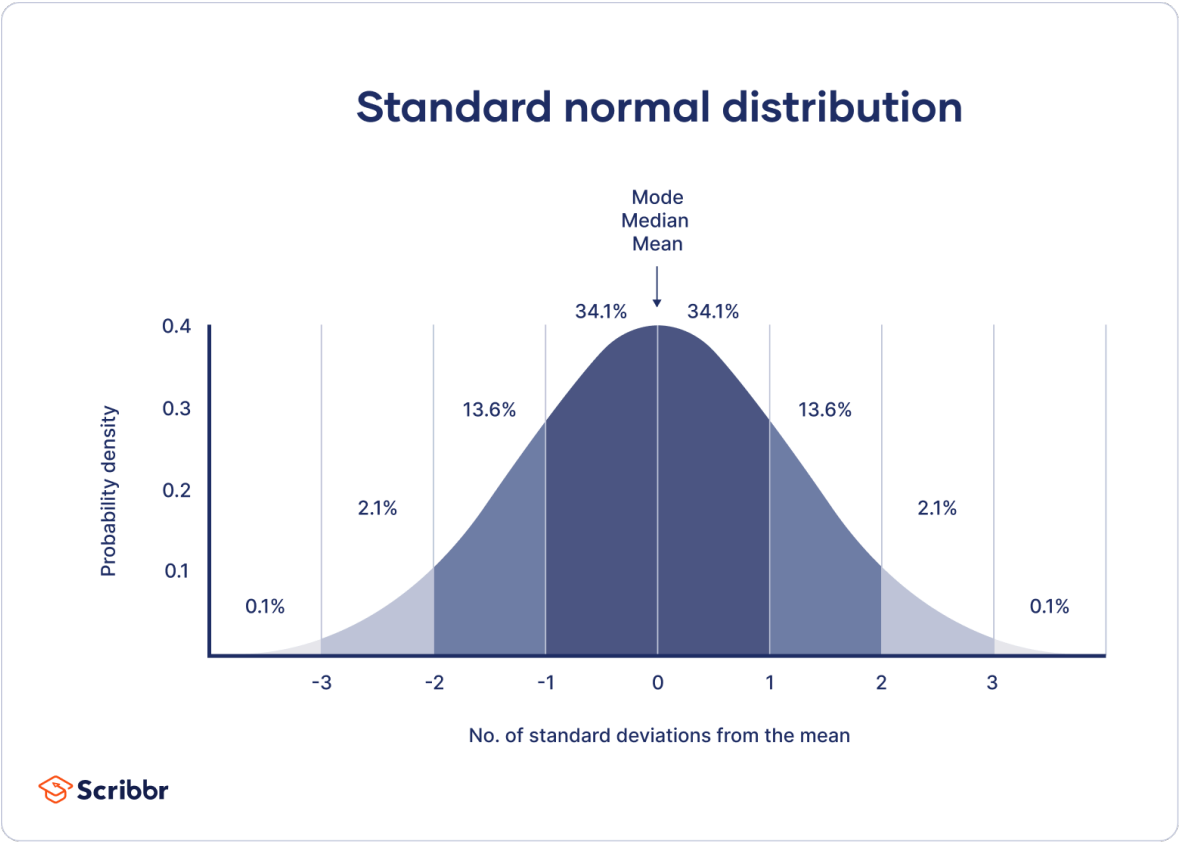
When to Use:

- **Most important** continuous distribution
- Models many natural phenomena (heights, weights, test scores)

- Central Limit Theorem: means of samples tend to be normally distributed

Properties:

- Bell-shaped, symmetric curve
- Completely defined by **mean (μ)** and **standard deviation (σ)**
- 68-95-99.7 rule:
    - 68% of data within μ ± σ
    - 95% of data within μ ± 2σ
    - 99.7% of data within μ ± 3σ



Z-Score:

$$z = \frac{x - \mu}{\sigma}$$

Converts any normal distribution to **standard normal** (μ=0, σ=1)

Key Functions:

- `norm.cdf(x, μ, σ)` : P(X ≤ x)
- `norm.ppf(q, μ, σ)` : Inverse CDF (find x for given probability q)

Example Problem:

Scenario: Travel time from Mumbai to Kolhapur

- Average time (μ) = 5 hours
- Standard deviation (σ) = 1.3 hours

In [44]:
```
# Question: What is the probability of completing the trip in LESS than 4.5 ho
# P(X < 4.5) when μ=5, σ=1.3
prob_less_than_4_5 = norm.cdf(4.5, 5, 1.3)
print(f"P(X < 4.5 hours): {prob_less_than_4_5:.4f}")
```

```
P(X < 4.5 hours): 0.3503
```

In [45]:
```
# Question: What is the probability of taking MORE than 4.5 hours?
# P(X > 4.5) = 1 - P(X ≤ 4.5)
prob_more_than_4_5 = 1 - norm.cdf(4.5, 5, 1.3)
print(f"P(X > 4.5 hours): {prob_more_than_4_5:.4f}")
```

```
P(X > 4.5 hours): 0.6497
```

In [46]:
```
# Question: What is the probability of taking between 4.2 and 5.3 hours?
# P(4.2 < X < 5.3) = P(X ≤ 5.3) - P(X ≤ 4.2)
prob_between = norm.cdf(5.3, 5, 1.3) - norm.cdf(4.2, 5, 1.3)
print(f"P(4.2 < X < 5.3 hours): {prob_between:.4f}")
```

```
P(4.2 < X < 5.3 hours): 0.3221
```

In [47]:
```
# Question: Below what time are 65% of trips completed?
# Find the 65th percentile (x such that P(X ≤ x) = 0.65)
time_65th = norm.ppf(0.65, 5, 1.3)
print(f"65% of trips are completed in less than {time_65th:.2f} hours")
```

```
65% of trips are completed in less than 5.50 hours
```

Z-Score

Z-score tells you **how far a value is from the mean**, measured in units of standard deviation.
It answers:
"How unusual or typical is this value compared to the rest of the data?"

What a Z-Score Really Means

- A z-score of **0** means the value is exactly at the mean.
- A **positive** z-score means the value is above the mean.
- A **negative** z-score means the value is below the mean.
- A **large absolute value** (like > 2 or > 3) means the value is far from the mean and may be an outlier.

Z-score converts raw data into a standardized scale so values from different units or ranges can be compared fairly.

Z-Score Formula

$$ z = \frac{x - \mu}{\sigma} $$

- x = the value
- μ = mean of the dataset
- σ = standard deviation

Why Z-Scores Are Used

1. Standardizing different units

Raw values might be in different units, like height (cm) and weight (kg).
Z-scores place them on the same scale.

2. Detecting outliers

Values with very high or very low z-scores indicate unusual or extreme data points.

3. Probability and normal distribution

Z-scores map your data onto the standard normal distribution, allowing probability calculations.

4. Machine learning

Many algorithms expect data to be standardized (mean = 0, std = 1).

Simple Example

Dataset mean = 14
Standard deviation = 2.83
Value = 18

$$ z = \frac{18 - 14}{2.83} \approx 1.41 $$

Interpretation:
The value 18 is **1.41 standard deviations above the mean**.
It is slightly higher than average, but not extreme.

In short

A z-score is a measure that shows **how far a value is from the mean**, using standard

deviation as the ruler.
It tells you whether the value is low, high, or extremely different compared to the rest of the dataset.

In [48]:
```
# Calculate Z-score for x = 4.5 when μ=5, σ=1.3
# Calculate Z-score manually
x = 4.5          # the VALUE
mu = 5           # MEAN of dataset
sigma = 1.3      # STD

z = (x - mu) / sigma

print(f"Z-score for x = {x}: {z:.4f}")
```

```
Z-score for x = 4.5: -0.3846
```

In [49]:
```
# For STANDARD NORMAL distribution (μ=0, σ=1):
# We can use the Z-score directly
# P(Z ≤ z) gives the same probability as P(X ≤ 4.5) in original distribution
prob_standard = norm.cdf(z)  # No need to specify μ and σ (defaults: 0, 1)
print(f"P(Z ≤ {z:.4f}) in standard normal: {prob_standard:.4f}")
print(f"This matches our earlier result: {prob_less_than_4_5:.4f}")
```

```
P(Z ≤ -0.3846) in standard normal: 0.3503
This matches our earlier result: 0.3503
```

Summary

Descriptive Statistics:

- **Boxplot**: Visualize distribution, identify outliers using IQR method
- **Quartiles**: Q1, Q2 (median), Q3 divide data into quarters
- **Outliers**: Values beyond Q1 - 1.5×IQR or Q3 + 1.5×IQR

Probability Distributions:

Distribution	Type	Use Case	Parameters
Binomial	Discrete	Fixed trials, binary outcome	n, p
Poisson	Discrete	Rare events, known rate	λ
Exponential	Continuous	Time between events	β (scale)
Normal	Continuous	Natural phenomena, measurements	μ, σ

Key Functions:

- **PMF/PDF**: Probability at a specific value
- **CDF**: Cumulative probability (P(X ≤ x))
- **PPF**: Inverse CDF (find x for given probability)
- **SF**: Survival function (P(X > x) = 1 - CDF)

---

student's t-distribution

when to use:

- **small sample sizes** (typically n < 30)
- **unknown population standard deviation**
- estimating population mean from sample data
- similar to normal distribution but with heavier tails (accounts for uncertainty)

properties:

- bell-shaped and symmetric like normal distribution
- controlled by **degrees of freedom (df)** = n - 1
- as sample size increases, t-distribution approaches normal distribution
- heavier tails than normal (more probability in extremes)

formula for t-statistic:

$$t = \frac{\bar{x} - \mu}{s / \sqrt{n}}$$

where:

- $\bar{x}$ = sample mean
- μ = hypothesized population mean
- s = sample standard deviation
- n = sample size

key functions:

- `t.cdf(x, df)` : P(T ≤ x) with given degrees of freedom
- `t.ppf(q, df)` : inverse cdf (find t-value for given probability)
- `t.pdf(x, df)` : probability density at x

quick reference: when to use which distribution

binomial

- count successes in fixed number of independent trials with constant probability
- example: 10 coin flips, how many heads?
- **key:** probability is given and expects 2 outcomes (success/failure)

poisson

- count rare events in fixed interval of time/space
- example: number of emails received per hour, typos in entire newspaper
- **key:** when probability is not given (when probability is small, we do frequency counts instead)

exponential

- time between events in a poisson process
- example: wait time until next customer arrives, time between horn blows during 2-hour class
- **key:** monitoring the time between 2 events

normal

- model continuous data that clusters around a mean
- example: heights of adults, measurement errors
- **key:** both mean and std are given, but sample size isn't given

student's t

- estimate population mean with small sample size when standard deviation is unknown
- example: average test scores from 15 students
- **key:** sample size is given (usually under 30), degrees of freedom = sample size - 1

---

quick reference: distribution functions

pmf (probability mass function)

- used to find exact discrete value

- applies to: binomial, poisson
- example: exactly 5 heads in 10 flips

cdf (cumulative distribution function)

- gives the total probability up to a particular number
- applies to: all distributions
- example: at most 5 heads, probability ≤ x

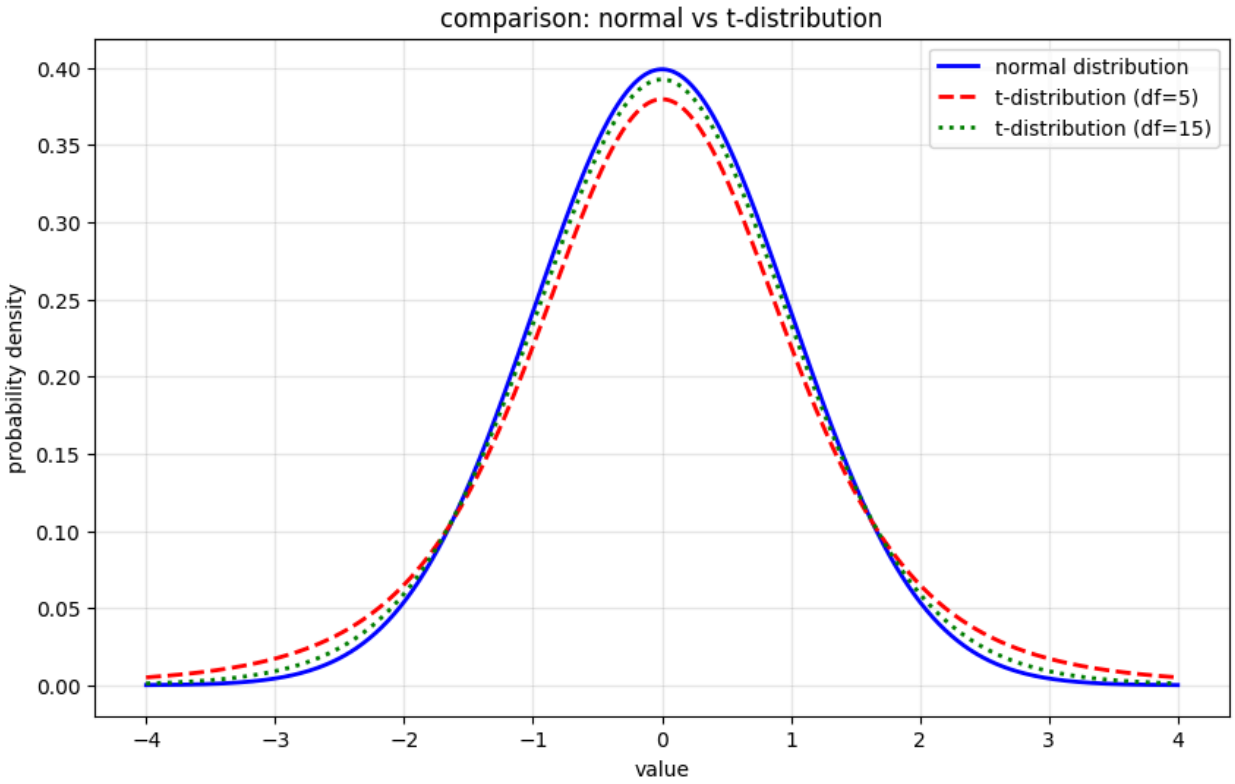
sf (survival function) or 1 - cdf

- probability greater than a particular number
- applies to: all distributions
- example: more than 5 heads, probability > x

ppf (percent point function)

- gives the value at a given percentile (left of the probability)
- inverse of cdf
- applies to: all distributions
- example: what value has 95% of data below it?

In [50]:
```
# visualize t-distribution vs normal distribution
x = np.linspace(-4, 4, 1000)

plt.figure(figsize=(10, 6))
plt.plot(x, norm.pdf(x), 'b-', linewidth=2, label='normal distribution')
plt.plot(x, t.pdf(x, df=5), 'r--', linewidth=2, label='t-distribution (df=5)')
plt.plot(x, t.pdf(x, df=15), 'g:', linewidth=2, label='t-distribution (df=15)'
plt.xlabel('value')
plt.ylabel('probability density')
plt.title('comparison: normal vs t-distribution')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print("notice: as degrees of freedom increase, t-distribution approaches norma
```



notice: as degrees of freedom increase, t-distribution approaches normal distribution

In [51]:
```python
# question: what t-value corresponds to 95% confidence (two-tailed)?
# for 95% confidence, we need 2.5% in each tail
# so we find the 97.5th percentile

t_critical = t.ppf(0.975, df)
print(f"t-critical value (95% confidence, df={df}): {t_critical:.4f}")
print(f"\ninterpretation: for 95% confidence interval with {n} samples,")
print(f"we use ± {t_critical:.4f} standard errors around the mean")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[51], line 5
      1 # question: what t-value corresponds to 95% confidence (two-tailed)?
      2 # for 95% confidence, we need 2.5% in each tail
      3 # so we find the 97.5th percentile
----> 5 t_critical = t.ppf(0.975, df)
      6 print(f"t-critical value (95% confidence, df={df}): {t_critical:.4f}")
      7 print(f"\ninterpretation: for 95% confidence interval with {n} sample
s,")

File /opt/homebrew/Caskroom/miniconda/base/lib/python3.13/site-packages/scipy/s
tats/_distn_infrastructure.py:2316, in rv_continuous.ppf(self, q, *args, **kwd
s)
   2314 args = tuple(map(asarray, args))
   2315 _a, _b = self._get_support(*args)
-> 2316 cond0 = self._argcheck(*args) & (scale > 0) & (loc == loc)
   2317 cond1 = (0 < q) & (q < 1)
   2318 cond2 = cond0 & (q == 0)

File /opt/homebrew/Caskroom/miniconda/base/lib/python3.13/site-packages/scipy/s
tats/_distn_infrastructure.py:1003, in rv_generic._argcheck(self, *args)
   1001 cond = 1
   1002 for arg in args:
-> 1003     cond = logical_and(cond, (asarray(arg) > 0))
   1004 return cond

TypeError: '>' not supported between instances of 'str' and 'int'
```

In [ ]:

```python
# example: test scores from 15 students
# sample mean = 75, sample std = 8, hypothesized mean = 70

n = 15
sample_mean = 75
sample_std = 8
hypothesized_mean = 70
df = n - 1  # degrees of freedom

# calculate t-statistic
t_stat = (sample_mean - hypothesized_mean) / (sample_std / np.sqrt(n))
print(f"sample size: {n}")
print(f"degrees of freedom: {df}")
print(f"t-statistic: {t_stat:.4f}")

# find probability (p-value for two-tailed test)
p_value = 2 * (1 - t.cdf(abs(t_stat), df))
print(f"p-value (two-tailed): {p_value:.4f}")
```

In [1]:
```
%%html
<link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:wght@300;4

<style>
/* markdown + output */
body, .markdown-body, .jp-RenderedHTMLCommon, div.text_cell_render,
.notebook, .notebook * {
  font-family: "Source Serif 4", serif !important;
}

/* code editor (monaco) */
.monaco-editor, .monaco-editor * {
  font-family: "Source Serif 4", serif !important;
}
</style>
```

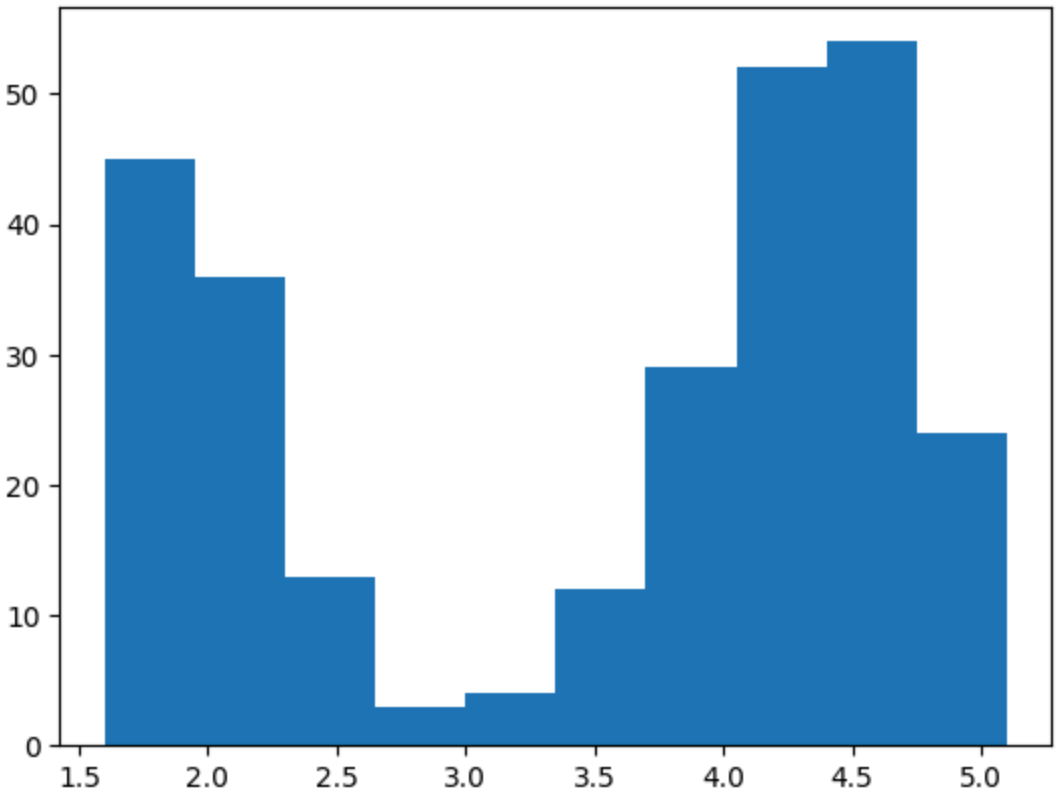
In [1]:
```
import scipy
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib
from matplotlib import pyplot as plt
import seaborn as sns
import os
from scipy.stats import *
from scipy.stats import norm, t
import os
```

In [2]:
```
os.chdir(r'/Users/proxim/Desktop/CDAC-DBDA-coursework/08.advanced-analytics-st
```

In [3]:
```
df = pd.read_excel('data/CDAC_DataBook.xlsx', sheet_name='faithful')
```

In [4]:
```
plt.hist(df.eruptions)
```

Out[4]:
```
(array([45., 36., 13.,  3.,  4., 12., 29., 52., 54., 24.]),
 array([1.6 , 1.95, 2.3 , 2.65, 3.  , 3.35, 3.7 , 4.05, 4.4 , 4.75, 5.1 ]),
 <BarContainer object of 10 artists>)
```




```
In [5]: scipy.stats.skew(df.eruptions)
```

```
Out[5]: np.float64(-0.4158409529189896)
```

```
In [6]: ((df.eruptions - np.mean(df.eruptions)) ** 3).sum()/((len(df.eruptions)-1)* np
```

```
Out[6]: np.float64(-0.41507583552214816)
```

Skewness

What is Skewness?

Skewness measures the **asymmetry** of a distribution.
It indicates whether the data stretches more toward the left or right side.

- Positive skewness → tail on the right (right-skewed)
- Negative skewness → tail on the left (left-skewed)
- Zero skewness → perfectly symmetrical

Skewness Formula (Sample Skewness)

$$ \text{Skewness} = \frac{n}{(n-1)(n-2)} \sum \left(\frac{x_i - \bar{x}}{s} \right)^3 $$

Where:

- $begin:math:text$ n $end:math:text$ = number of observations
- $begin:math:text$ \bar{x} $end:math:text$ = sample mean
- $begin:math:text$ s $end:math:text$ = sample standard deviation

---

Interpretations of Skewness

Skewness Value	Meaning	Shape
0	symmetric distribution	normal-like
> 0	right-skewed	long right tail
< 0	left-skewed	long left tail

Simple Intuition for Skewness

- Positive skewness → few large values pull the tail to the right
- Negative skewness → few small values pull the tail to the left
- Zero skewness → balanced on both sides

Python Notes for Skewness

```
from scipy.stats import skew
skew(data)    # returns sample skewness
```

Ultra-short memory trick for Skewness

- Right tail → positive
- Left tail → negative
- Perfectly balanced → zero

Kurtosis vs Excess Kurtosis

What is Kurtosis?

Kurtosis measures **how heavy or light the tails** of a distribution are compared to a normal distribution.

- High kurtosis → more extreme values (heavy tails)
- Low kurtosis → fewer extreme values (light tails)

For a normal distribution, raw kurtosis = 3.

What is Excess Kurtosis?

Excess kurtosis is defined as:

$$ \text{Excess Kurtosis} = \text{Kurtosis} - 3 $$

This makes the normal distribution easier to compare because its value becomes 0.

- `0` → normal
- `> 0` → heavy tails (leptokurtic)
- `< 0` → light tails (platykurtic)

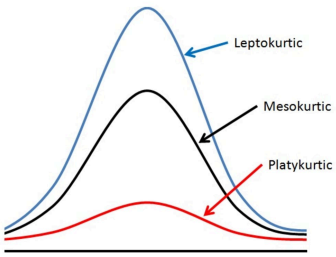
Comparison Table: Kurtosis vs Excess Kurtosis

Concept	Value for Normal	Formula	Interpretation	Notes
Kurtosis (raw)	3	—	>3 heavy tails, <3 light tails	Classical definition
Excess Kurtosis	0	Kurtosis − 3	>0 heavy tails, <0 light tails	SciPy default

---

Types of Kurtosis

Type	Excess Kurtosis	Tail Behavior	Shape
Mesokurtic	0	Normal tails	Bell-shaped
Leptokurtic	> 0	Heavy tails, many extremes	Sharp peak
Platykurtic	< 0	Light tails, few extremes	Flat peak



mesokurtic is normal distribution

Simple Intuition

- Excess Kurtosis = 0 → normal
- Positive → more outliers
- Negative → fewer outliers

Excess kurtosis simply sets the normal curve's baseline to 0.

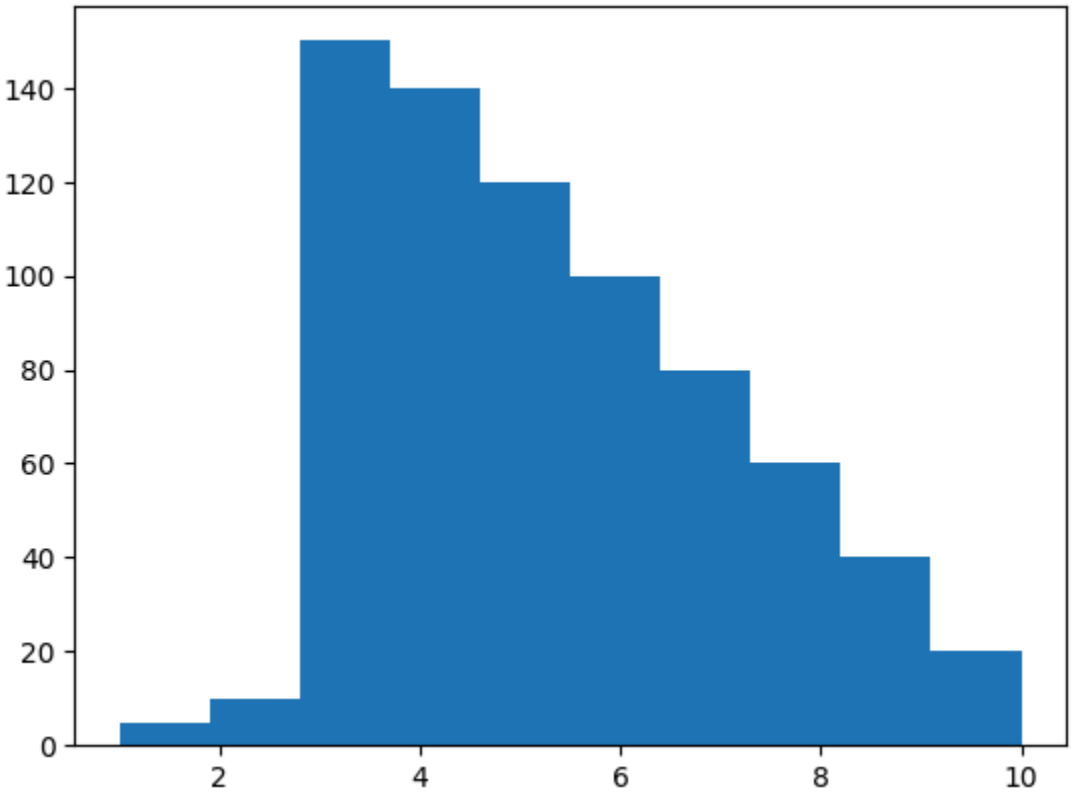
Ultra-short memory trick

- Excess kurtosis = kurtosis − 3
- Normal → 0
- Positive → heavy tails
- Negative → light tails

In [7]:
```
x1 = np.repeat([1,2,3,4,5,6,7,8,9,10],[5,10,150,140,120,100,80,60,40,20])
```

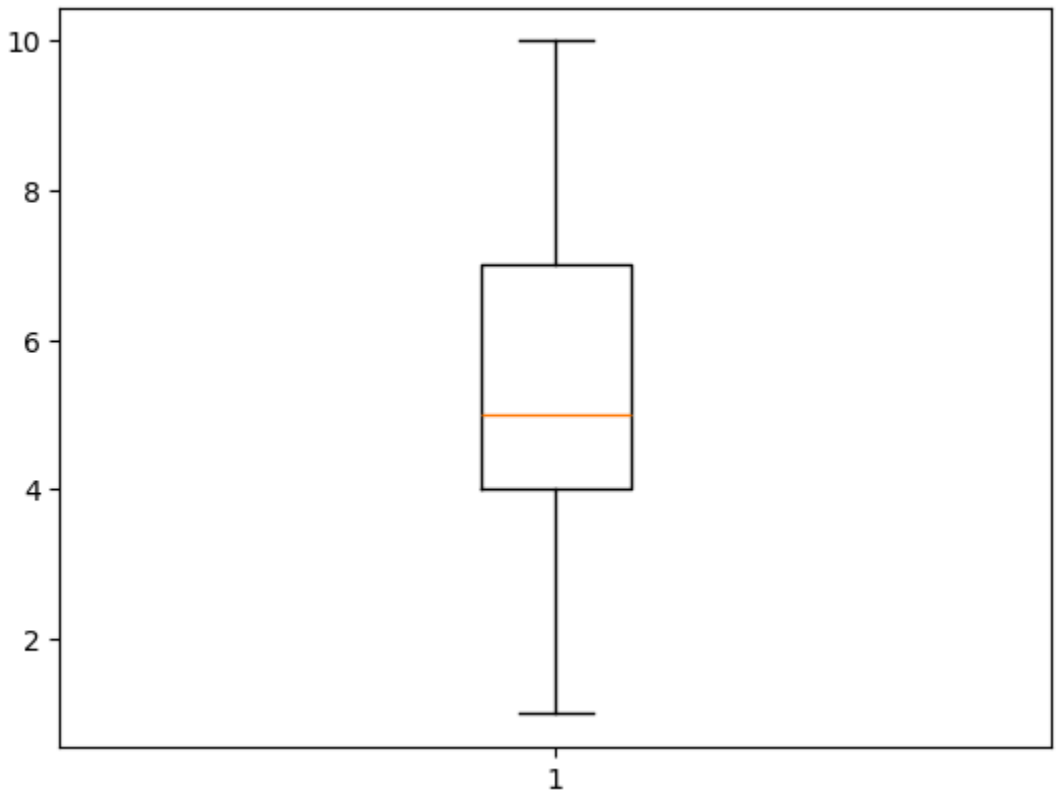
In [8]:
```
plt.hist(x1)
```

Out[8]:
```
(array([  5.,  10., 150., 140., 120., 100.,  80.,  60.,  40.,  20.]),
 array([ 1. ,  1.9,  2.8,  3.7,  4.6,  5.5,  6.4,  7.3,  8.2,  9.1, 10. ]),
 <BarContainer object of 10 artists>)
```



In [9]:
```
plt.boxplot(x1)
```

```
plt.show()
```



In [10]:
```
np.mean(x1)
```

Out[10]:
```
np.float64(5.289655172413793)
```

In [11]:
```
np.median(x1)
```

Out[11]:
```
np.float64(5.0)
```

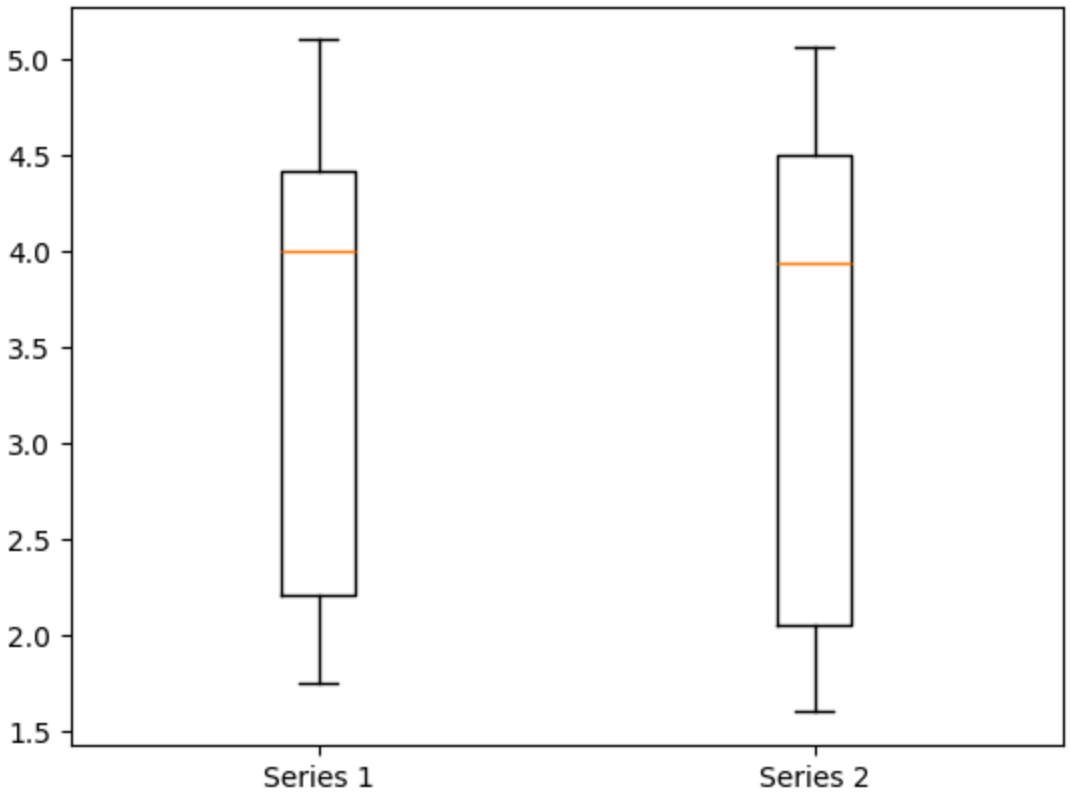
In [12]:
```
len(df.eruptions)
```

Out[12]:
```
272
```

In [13]:
```
# slicing the data into 2 equal parts

e1 = df.eruptions[136:]
e2 = df.eruptions[:136]
```

In [14]:
```
plt.boxplot([e1, e2], labels=['Series 1', 'Series 2'])
plt.show()
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_80669/3390848912.p
y:1: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dr
opped in 3.11.
  plt.boxplot([e1, e2], labels=['Series 1', 'Series 2'])
```



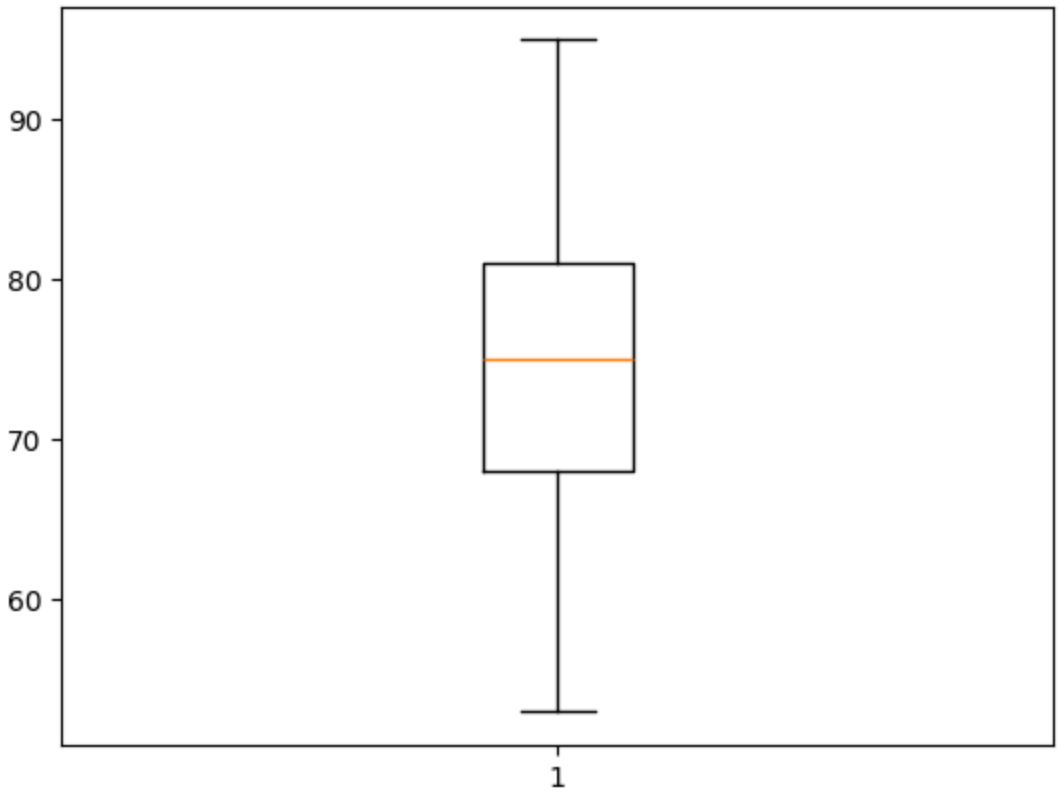
In [15]:
```
df = pd.read_excel('data/Marks.xlsx')
```

In [16]:
```
df.head()
```

Out[16]:

	Subject	Trainer	Marks
0	Calculus	Sudeep	65
1	Calculus	Sudeep	75
2	Calculus	Sudeep	86
3	Calculus	Sudeep	82
4	Calculus	Sudeep	75

In [17]:
```
plt.boxplot(df.Marks)
plt.show()
```



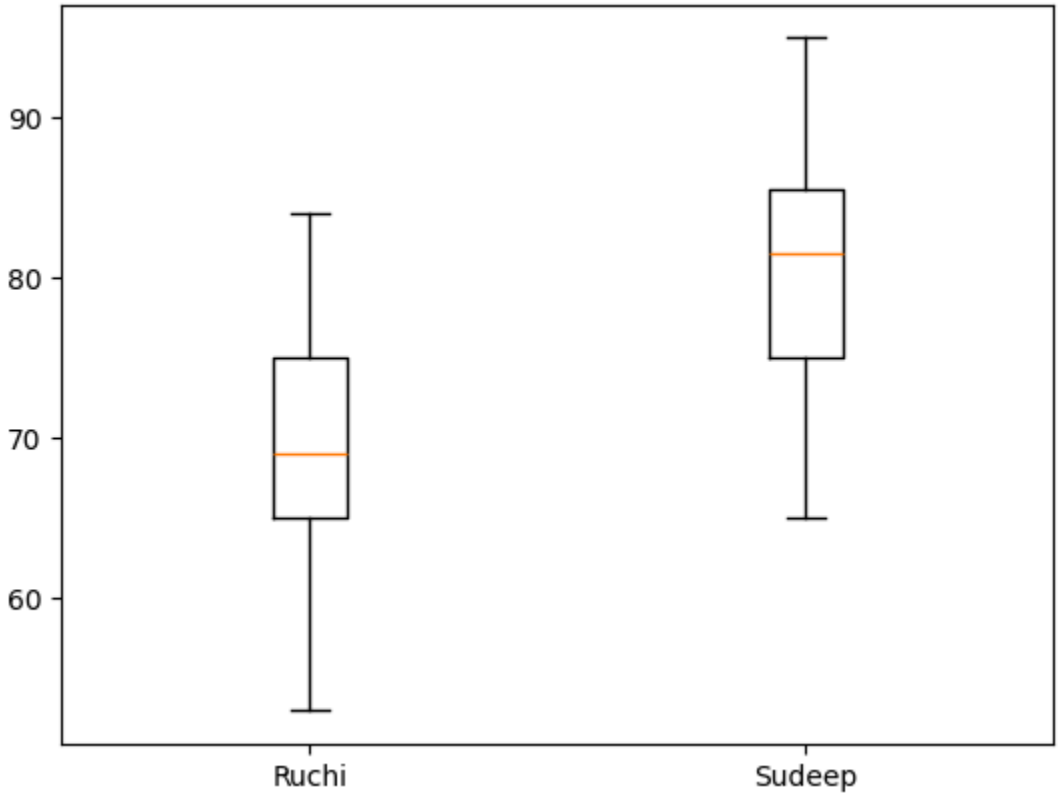
groped boxplots

In [ ]:
```
mylist = []
lab = []
for ctr in np.unique(df.Trainer):
    mylist.append(df.Marks[df.Trainer == ctr])
    lab.append(ctr)
    print(mylist)

plt.boxplot(mylist, labels = lab)
plt.show()
```

```
[6     64
7     67
8     71
9     75
10    81
17    63
18    65
19    75
20    81
21    71
22    65
23    74
24    68
25    76
32    69
33    61
34    57
35    81
36    68
37    53
38    84
39    72
40    68
Name: Marks, dtype: int64]
[6     64
7     67
8     71
9     75
10    81
17    63
18    65
19    75
20    81
21    71
22    65
23    74
24    68
25    76
32    69
33    61
34    57
35    81
36    68
37    53
38    84
39    72
40    68
Name: Marks, dtype: int64, 0     65
1     75
2     86
3     82
4     75
5     68
11    95
```

```
12    87
13    83
14    74
15    68
16    81
26    84
27    84
28    86
29    91
30    78
31    77
Name: Marks, dtype: int64]
```

```
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_80669/4254942189.p
y:8: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dr
opped in 3.11.
  plt.boxplot(mylist, labels = lab)
```



In [19]:
```
df.Subject.unique()
```

Out[19]:
```
array(['Calculus', 'Statistics', 'Trigno'], dtype=object)
```

In [ ]:
```
# groped boxplots
# continious data (label) -> categorize -> subcategorize

mylist = []
lab = []
for ctr1 in np.unique(df.Trainer):
```

```
    for ctr2 in np.unique(df.Subject):
        mylist.append(df.Marks[(df.Trainer==ctr1) & (df.Subject == ctr2)])
        lab.append([ctr1, ctr2])
plt.boxplot(mylist, labels= lab)
plt.xticks(rotation = 90)
plt.show()
```

In [21]:
```
# in the lecture total of 9 hours the train horns for 20 times, there is exam
```

In [22]:
```
mtbf = 9/20
mtbf
```

Out[22]: 0.45

In [23]:
```
from scipy.stats import expon
```

In [24]:
```
expon.cdf(1, scale = 0.45)
```

Out[24]: np.float64(0.8916319767781041)

In [25]:
```
1 - expon.cdf(1, scale = 0.45)
```

Out[25]: np.float64(0.10836802322189587)

In [26]:
```
# avg amount of water in bottle is 990 ml with STD of 20ml, probablity of gett
```

In [27]:
```
1 - norm.cdf(1000, 990, 20)
```

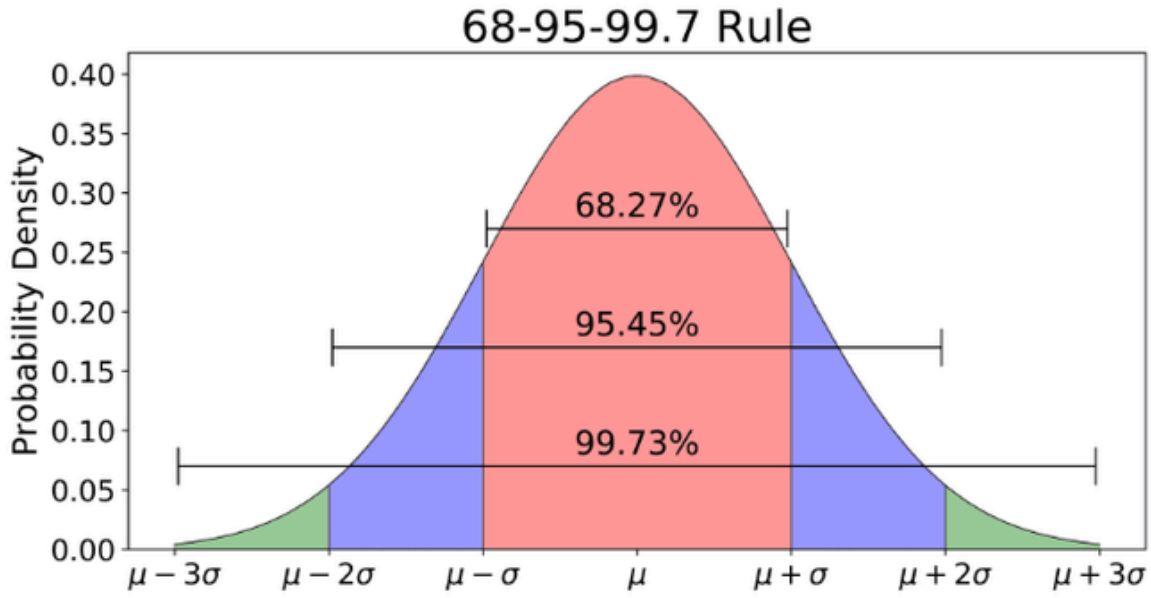
Out[27]: np.float64(0.3085375387259869)

In [28]:
```
# avg = 17 what is the probablity i will reach in 15-19 hours
```

In [29]:
```
prob = norm.cdf(19, 17, 1) - norm.cdf(15, 17, 1)
prob
```

Out[29]: np.float64(0.9544997361036416)

In [30]:
```
# for any normal bell shaped curve 68.26 percent of the area falls between  +1
# for example for this ->  avg = 17 what is the probablity i will reach in 16-
```



The 68-95-99.7 Rule for a Normal Distribution

In [31]:
```
norm.cdf(1) - norm.cdf(-1)
```

Out[31]:
```
np.float64(0.6826894921370859)
```

In [32]:
```
norm.cdf(2) - norm.cdf(-2)
```

Out[32]:
```
np.float64(0.9544997361036416)
```

In [33]:
```
norm.cdf(3) - norm.cdf(-3)
```

Out[33]:
```
np.float64(0.9973002039367398)
```

In [34]:
```
df.head()
```

Out[34]:

	Subject	Trainer	Marks
0	Calculus	Sudeep	65
1	Calculus	Sudeep	75
2	Calculus	Sudeep	86
3	Calculus	Sudeep	82
4	Calculus	Sudeep	75

In [35]:
```
norm.ppf(0.9, 5, 1)
```

Out[35]:
```
np.float64(6.2815515655446)
```

In [36]:
```
from scipy.stats import t
```

In [37]:
```
# normal distribution


# we need to calculate the value of z and give input
# 2nd - degrees of freedom i.e(no(sample)-1)


# in python we need to explicity store the mu sigma in variable, python doesn'
```

In [38]:
```
# time taken from mumbai to kolhapur is 5 hours and sd = 1.4 hours calculated

# so degrees of freedom = sample size -1 = 14
```

In [39]:
```
# Q1. what is the probablity of taking less than 5.6 hours

z = (5.6-5)/1.4
t.cdf(z, 14)
```

Out[39]:
```
np.float64(0.6626221087121706)
```

In [40]:
```
# what is the probablity of taking more than 5.6 hours
t.sf(z,14)
```

Out[40]:
```
np.float64(0.33737789128782925)
```

In [41]:
```
# what is the probablity of taking time b/w 4.5 and 5.6

z1 = (4.5-5)/1.4
z2 = (5.6-5)/1.4
```

In [42]:
```
t.cdf(z2, 14) - t.cdf(z1,14)
```

Out[42]:
```
np.float64(0.29946661799007335)
```

In [ ]:
```
# t distribution can be used with all sample sizes
# normal distribution can be used only when the sample size is > 30 and popula
# as the sample size increases the values that we get from t distribution will
```

In [ ]:
```
# mean = 5 sd = 1.3, what is the probablity of < 5.4
```

In [45]:
```
org = norm.cdf(5.4,5,1.3)
org
```

Out[45]:
```
np.float64(0.6208417632368549)
```

In [46]:
```
z = (5.4-5) / 1.3
```

In [47]:
```
# look how does the t2 gets closer to our org distribution
```

In [48]:
```
t1 = t.cdf(z,14)
t1
```

Out[48]:
```
np.float64(0.6185768199132399)
```

In [49]:
```
t2 = t.cdf(z,500)
t2
```

Out[49]:
```
np.float64(0.6207777003768674)
```

Understanding t Distribution vs Normal Distribution

Normal Distribution

- **Shape:** Bell-shaped, symmetric.
- **When to use:** When population standard deviation (σ) is known and/or sample size is large (n > 30).
- **Formula:** Uses mean (μ) and standard deviation (σ).
- **Example:** Heights of adults, IQ scores.

Example (Normal Distribution)

Suppose travel time from Mumbai to Kolhapur is normally distributed with mean μ = 17 hours and σ = 1 hour.

- **Q:** What is the probability of reaching in 15–19 hours?
- **A:** Use CDF for normal distribution:

In [51]:
```
df.head()
```

Out[51]:

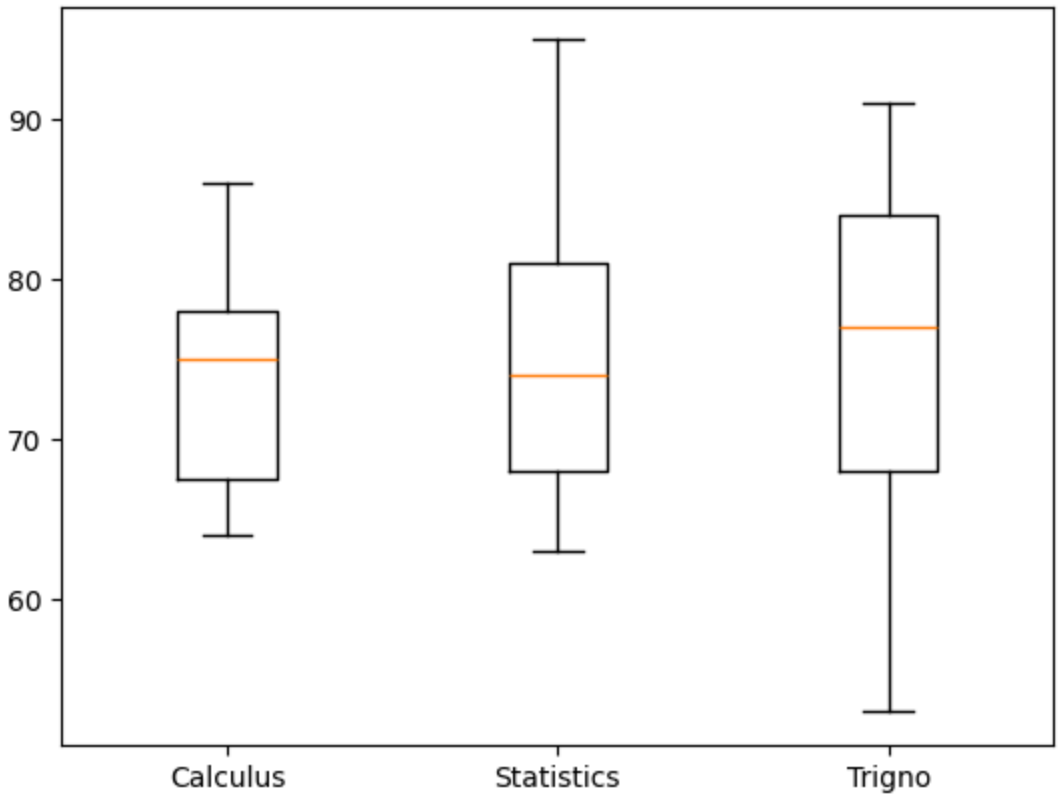
	Subject	Trainer	Marks
0	Calculus	Sudeep	65
1	Calculus	Sudeep	75
2	Calculus	Sudeep	86
3	Calculus	Sudeep	82
4	Calculus	Sudeep	75

In [ ]:
```
mylist = []
lab = []
for ctr in np.unique(df.Trainer):
    mylist.append(df.Marks[df.Trainer == ctr])
    lab.append(ctr)
    print(mylist)
```

```
plt.boxplot(mylist, labels = lab)
plt.show()
```

In [73]:
```
mylist = []
lab = []
for ctr in np.unique(df.Subject):
    mylist.append(df.Marks[df.Subject == ctr])
    lab.append(ctr)
print(mylist)
print(lab)


plt.boxplot(mylist, labels = lab)
plt.show()
```

```
[0     65
1     75
2     86
3     82
4     75
5     68
6     64
7     67
8     71
9     75
10    81
Name: Marks, dtype: int64, 11    95
12    87
13    83
14    74
15    68
16    81
17    63
18    65
19    75
20    81
21    71
22    65
23    74
24    68
25    76
Name: Marks, dtype: int64, 26    84
27    84
28    86
29    91
30    78
31    77
32    69
33    61
34    57
35    81
36    68
37    53
38    84
39    72
40    68
Name: Marks, dtype: int64]
['Calculus', 'Statistics', 'Trigno']
/var/folders/57/62f2rv1j4w94h8tfp_c43b1c0000gn/T/ipykernel_80669/1606882670.p
y:10: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has bee
n renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be
dropped in 3.11.
  plt.boxplot(mylist, labels = lab)
```




```
In [58]: norm.ppf(0.025)
```

```
Out[58]: np.float64(-1.9599639845400545)
```

```
In [59]: norm.ppf(0.975)
```

```
Out[59]: np.float64(1.959963984540054)
```

```
In [61]: x = norm.ppf(0.025, 0.975)
```

```
In [70]: norm.ppf(0.99999999999999)
```

```
Out[70]: np.float64(7.650730905155642)
```